

Digital Electronics

Detailed Solutions • 1987–2026

GATE ELECTRONICS AND COMMUNICATION ENGINEERING

(EC)

Himanshu Agarwal & Anish Saha

PrepFusion™

Where Concept Meets Clarity!

Preface

Welcome to PrepFusion™!

This book works, in full, every question GATE has actually asked in Digital Electronics (EC), gathered from the original papers and arranged unit by unit and year by year. Nothing is paraphrased or reconstructed: every question appears as it was set, and every solution is taken step by step rather than asserted.

Each solution carries the same identifier as the question it answers in the companion questions volume, so you can move between practice and explanation without a lookup table. Where the examining authority revised an answer or awarded marks to all (MTA), that is noted with the question it affects.

Used end to end, the book shows you what the paper rewards — which topics repeat, how the framing has shifted over the years, and where your preparation still has gaps.

Meet your Authors

Himanshu Agarwal

- AIR 27 — GATE (ECE)
- AIR 45 — GATE (IN)
- 5+ years of teaching experience



Anish Saha

- AIR 19 — GATE (IN)
- AIR 66 — GATE (EE)
- 3+ years in teaching & the VLSI industry



> *"We built this book the way every aspirant deserves one to be built — organized strictly by unit and by year, so that every hour of practice maps directly onto how GATE actually tests you."*
>
> — Himanshu Agarwal & Anish Saha

How to Read the Question Numbers

Every question in this book carries a three-part identifier instead of a plain serial number:

Q3.24.7

3	Unit (chapter) number — Unit III of this book
24	GATE exam year — 2024 (two digits; 98 = 1998)
7	Question number within that unit and year

So **Q3.24.7** is the 7th question that GATE 2024 contributed to Unit III. Numbers are therefore not sequential through the book — they tell you the unit and the exam year at a glance. The same identifier is used in the questions book, the solutions book and the answer keys, so you can jump between them without a lookup table. In the PDF bookmarks panel, expand a unit to see every question ID and click one to go straight to it.

What the Bubbles Mean

Every question ends with three empty circles for you to shade in yourself:

○○○ ●○○ Easy ●●○ Medium ●●● Hard

The companion solutions volume marks the same question with *our* recommended difficulty, using the identical bubbles. If you think a rating should be different, tell us directly — report it under the **Study Hub** section at **pyq.prepfusion.in**.

How to Read the Unit Snapshot

Every unit opens with a **Unit Snapshot** box summarizing that unit's real question data:

Questions	the real total for this unit.
MCQ, MSQ, NAT	that same total, split by question type.
1M, 2M	how many of those questions carry 1 or 2 marks.
Descriptive	questions worth more than 2 marks, or whose marks value isn't recorded.
Avg Weightage	the average marks this unit has contributed per real GATE sitting, not a raw year-by-year total — a year GATE ran multiple real sessions (a genuine forenoon/afternoon split, not the same paper's Set A/B/C/D booklet variants) is divided across those sessions first, so a heavily multi-session year cannot silently outweigh a single-session one. The ~ marks it as a rounded figure, not an exact one.
Range	the lowest and highest that same per-sitting value has been, with the year each occurred.

Avg Weightage and Range are computed from **2010 onward** only: GATE dropped from a 150-mark to a 100-mark paper in 2009, but that year itself was a one-off transition (60 questions, not the 65-question format that became standard from 2010 on) — so 2010 is the first year fully comparable to modern papers, and marks-based figures from earlier years are left out of these two figures specifically (every other count on this page still includes every year). They also depend on knowing which GATE years ran more than one real sitting for a given branch — our own research into this is thorough for some branches and still a work in progress for others. Treat both figures as a **beta**, best-effort estimate: directionally useful, not exact.

MSQ questions specifically weren't introduced until 2021 — an older-year-heavy unit showing **MSQ: 0** reflects that real exam history, not a data gap.

Worked example

Unit Snapshot*

Stream: EC	**Questions:** 45	**MCQ:** 30	**MSQ:** 3	**NAT:** 12
1M: 20	**2M:** 22	**Descriptive:** 3	**Avg Weightage** ~4M	**Range:** 2M(2015)–6M(2021)

45 real questions total; on average this unit has been worth about 4 marks per real GATE sitting since 2010, ranging from as low as 2 (in 2015) to as high as 6 (in 2021).

A quick way to use this: a high **Avg Weightage** points to a consistently high-yield topic worth prioritizing; a wide **Range** usually means the topic's importance has shifted over the years, so it's worth checking *which* years drove that before deciding how much weight to give it today.

Important Links & Resources

Visit our Website

prepfusion.in

Solutions, practice & report

pyq.prepfusion.in

Subscribe: English Channel

youtube.com/@PrepFusion_GATE

Subscribe: Hindi Channel

youtube.com/@prepfusion-hindi

Join our Telegram group

t.me/All_About_Learning

Follow us on LinkedIn

linkedin.com/company/prepfusionee

Subscribe: Himanshu Agarwal

youtube.com/@HimanshuAgarwal_

Subscribe: Anish Saha

youtube.com/@AnishSaha_

Scan any code with your phone camera, or tap the link beneath it. **pyq.prepfusion.in** is also where you can read this book's worked solutions online — and in its **Study Hub** section, report an error in any question or request a video solution for it.

Contributors

This book exists because of the people below, who built, reviewed and typeset its content.

Design & Layout Architect — Nawras Ahamed.

Technical Reviewers — Ayush Prakash Singh, Abhishek Bhattacharjee, Shujaut Yaseen, Yuva Kishore Gottapu, Manepalli Pavan Vignesh Kumar, Palak Agarwal, Devmalya Ghatak, Kasi Viswanath A, Suresh Ravanam, Aabhas Anuraag Suman and Piyush Raj.

Content Developers — A S Srimithra, Chinthada Lakshmi Prasad, Gantasala Ravi Teja, Guduru Vinayak Varma, Harshaditya Punera, Katikala Sarath, Pranitha Musunuru, Safdar Nazim, Sai Mahalakshmi Janjanam, Shaila Daniels, Sharvari Patwardhan, Sumedha Ghosh, Vinay K Gowda, Aayushi Ghosh, Chevuri Siva Naga Sai Abhi Chandar, Govind Sharma, Kasina Sai Surya Vamsi, Machanpally Karthik Kumar, Seeku Vamsi Krishna and Suprio Dutta.

Contents

1. Logic Gates and Boolean Algebra 1
2. Representation of Boolean Expressions and K-Maps 35
3. Number Systems 58
4. Combinational Circuits 74
5. Sequential Circuits 118
6. ADC & DAC 188
7. Logic Families and Semiconductor Memories 217
8. Computer Organization and Architecture 257

PrepFusion

SOLUTIONS

1

Logic Gates and Boolean Algebra

Topics

1. Introduction to Digital Systems
2. Basic Gates and Concept of Multivibrators Using NOT Gate
3. Universal and Arithmetic Gates
4. Implementing Boolean Expressions Using Universal Logic Gates
5. Timing Diagrams
6. Boolean Algebra, Venn Diagrams, and the Concept of Dual

Unit Snapshot*

Stream: EC	**Questions:** 37	**MCQ:** 30	**MSQ:** 5	**NAT:** 2
1M: 22	**2M:** 15	**Descriptive:** 0	**Avg Weightage** ~2M	**Range:** 1M(2016)–3M(2022)

**See preface for how Avg Weightage and Range are calculated.*

Q1.22.1 MSQ 2022 2M Ans: A,C ● ● ○

Given: A Boolean gate (D) where the output Y is related to the inputs A and B as, $Y = A + \overline{B}$. Write it as $D(a, b) = a + \overline{b}$.

REMEMBER: If any logic circuit can give "AND" or "OR" (Either ONE of them) and "NOT" together then that circuit can be used as universal Gate [with the condition that "1" and "0" are present].

To Prove, we will implement NOR gate with the given gate as NOR is a known universal gate so if that is implemented we can implement all functions through it. Three such gates are connected as follows:

$$n_A = D(0, A) = 0 + \overline{A} = \overline{A}$$

$$s = D(B, n_A) = B + \overline{\overline{A}} = A + B$$

$$Y_{NOR} = D(0, s) = \overline{A + B}$$

Note that the middle gate receives B (not 0) at its first input.
NAND can also be built directly:

$$Y_{NAND} = D(n_A, B) = \overline{A} + \overline{B} = \overline{AB}$$

Thus the given gate is proven to be a universal gate, so all gates can be implemented through it. OR is $D(A, D(0, B))$ and AND follows by inverting NAND, so the options saying they cannot be implemented are wrong.

(A) NAND logic can be implemented
(C) NOR logic can be implemented

Key Points

If any logic circuit can give "AND" or "OR" (Either ONE of them) and "NOT" together then that circuit can be used as universal Gate (with the condition that "1" and "0" are present). Thus the following functions are sufficient to implement any Boolean Expression: NAND, NOR, AND+NOT, OR+NOT, AND+OR+NOT.

Mistakes to be avoided

- Be careful while reading the options! Options B and D say that OR and AND logic "cannot" be implemented. Since we proved this gate is universal, it can implement them, making those options incorrect.

Q1.22.2 MSQ 2022 1M Ans: B,C ● ○ ○

Method 1

Choosing from options:

From option (A):
$x + z + xy$
$x(1 + y) + z$
$= x + z$
(A) is incorrect.

From option (B):
$(x + y)(x + z)$
$x + xz + yx + yz$
$= x + yz$
(B) is correct.

From option (C):
$x + xy + yz$
$x(1 + y) + yz$
$= x + yz$
(C) is correct.

From option (D):
$x + xz + xy$
$x(1 + z + y)$
$= x$
(D) is incorrect.

Hence, the correct options are (B) and (C).

Method 2

Given: $(x + yz) \ldots (i)$

From Boolean algebra distributive law:

$$A + BC = (A + B)(A + C)$$

So, equation (i) can be written as:

$$x + yz = (x + y)(x + z)$$

Again, using the identity $A + AB = A$:

$$A + BC = A(1 + B) + BC = A + AB + BC$$

So, equation (i) can also be written as:

$$x + yz = x + xy + yz$$

(B) $(x + y)(x + z)$
(C) $x + xy + yz$

Key Points

- **Distributive Law:** $A + BC = (A + B)(A + C)$.
- **Absorption Law/Identity:** $1 + A = 1$ and $A + AB = A(1 + B) = A$.

Q1.18.1 NAT | 2018 | 2M | Ans: 8 to 8 | ● ● ●

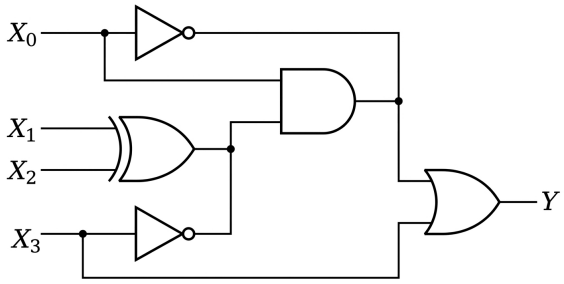


Fig. Solution Q1.18.1

According to question. give circuit have wired AND logic at node A and B, because node output is HIGH, when all the input came to node are HIGH.

Method 1

According to question. give circuit have wired AND logic at node A and B, because node output is HIGH, when all the input came to node are HIGH.

Case 1:

When $X_0 = 0$ only, then output Y becomes as,

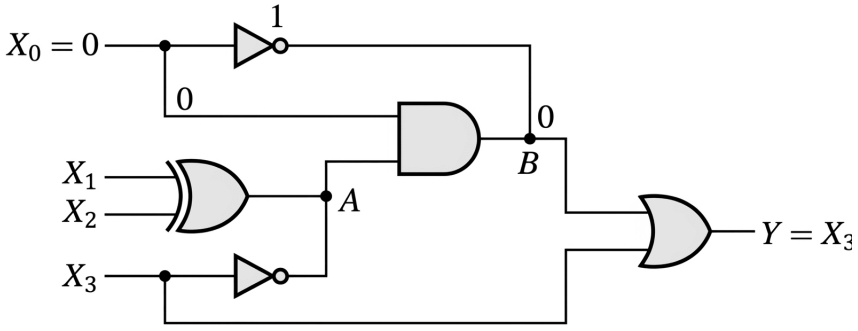


Fig. Solution Q1.18.1

As shown in the circuit diagram for Case 1, setting $X_0 = 0$ propagates through the logic gates to yield the output $Y = X_3$.

Case 2:

When $X_0 = 1$ only, then output Y becomes as,

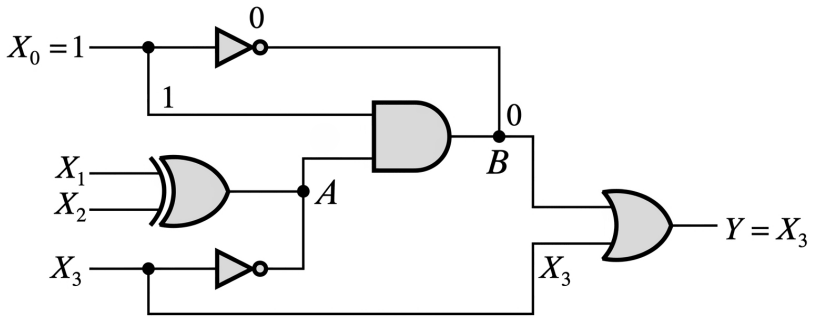


Fig. Solution Q1.18.1

Similarly, analyzing the signal flow for Case 2 reveals that setting $X_0 = 1$ results in the output expression $Y = X_3$.

From the above two cases it is clear that $Y = X_3$ ALWAYS.

Method 2

From figure, $A = (X_1 \oplus X_2)\overline{X}_3$

$$B = (A \cdot X_0)\overline{X}_0$$

$$B = 0$$

Output $Y = B + X_3 = X_3$ Thus, output depends only on X_3

Thus output will be HIGH exactly 8 times the same can be verified with a truth table

X_3	X_2	X_1	X_0	$Y = X_3$
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	**1**
1	0	0	1	**1**
1	0	1	0	**1**
1	0	1	1	**1**
1	1	0	0	**1**
1	1	0	1	**1**
1	1	1	0	**1**
1	1	1	1	**1**

Out of the 16 possible input combinations, $X_3 = 1$ for exactly 8 combinations, which directly results in $Y = 1$.

8

Key Points

- Wired-AND logic occurs when multiple open-collector or open-drain outputs are connected together, requiring all connected outputs to be high for the node to be high.

Q1.16.1 MCQ 2016 1M Ans: C ●●○

The given logic circuit consists of three AND gates and two EX-OR gates.
Tracing the intermediate outputs from the logic diagram:

- Outputs of AND gates: AB, ABC, and BC
- Output of the first EX-OR gate: $BC \oplus AB$

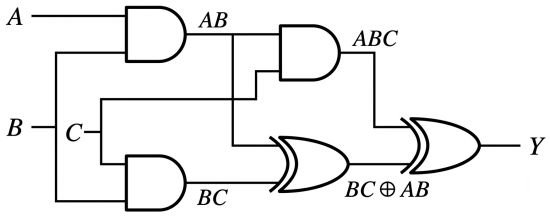


Fig. Solution Q1.16.1

The output expression Y is:

$$Y = ABC \oplus AB \oplus BC \tag{1.1}$$

Expanding and simplifying Y:

Method 1

Using Boolean minimization:

$$\begin{aligned}
Y &= \left[\overline{ABC}(AB) + ABC(\overline{AB})\right] \oplus BC \\
&= \left[(\bar{A} + \bar{B} + \bar{C})AB + ABC(\bar{A} + \bar{B})\right] \oplus BC && (\because \overline{ABC} = \bar{A} + \bar{B} + \bar{C}) \\
&= AB\overline{C} \oplus BC && (\text{since } A \cdot \bar{A} = 0, B \cdot \bar{B} = 0) \\
&= \overline{AB\overline{C}}(BC) + (AB\overline{C})\overline{BC} \\
&= (\overline{A} + \overline{B} + C)BC + AB\overline{C}(\overline{B} + \overline{C}) \\
&= \overline{A}BC + BC + AB\overline{C} \\
&= BC(\overline{A} + 1) + AB\overline{C} = BC + AB\overline{C} \\
&= B(C + A\overline{C}) \\
&\boxed{Y = B(A + C)} && (\because X + \overline{X}Y = X + Y)
\end{aligned}$$

Method 2

Using properties of EX-OR gates:

$$\begin{aligned}
Y &= AB(C \oplus 1) + BC && (\because X \oplus 1 = \overline{X}) \\
&= AB\overline{C} + BC \\
&= B(A\overline{C} + C) = \boxed{B(A + C)} && (\text{by distributive law})
\end{aligned}$$

$$\boxed{\text{(C) } B(C + A)}$$

Key Points

- EX-OR Simplification: $X \oplus 1 = \overline{X}$
- Distributive Rule: $C + A\overline{C} = C + A$

Q1.16.2 MCQ | 2016 | 1M | Ans: A | ● ○ ○

A 2-input XOR gate can be implemented using four 2-input NAND gates as shown below.

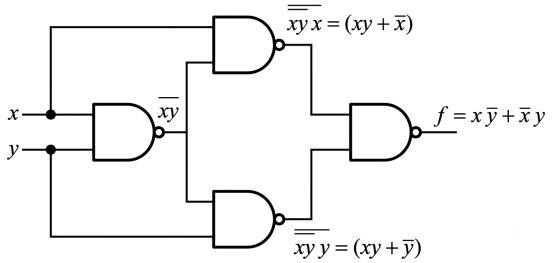


Fig. Solution Q1.16.2

Tracing the outputs step-by-step from the logic diagram:

- Output of the first NAND gate: $\overline{xy}$
- Output of the top NAND gate: $\overline{x \cdot \overline{xy}} = \bar{x} + xy$
- Output of the bottom NAND gate: $\overline{y \cdot \overline{xy}} = \bar{y} + xy$

The final output expression f is:

$$
\begin{aligned}
f &= \overline{(\bar{x} + xy)(\bar{y} + xy)} \\
&= \overline{\bar{x} + xy} + \overline{\bar{y} + xy} && \text{(by De Morgan's Law)} \\
&= x \cdot \overline{xy} + y \cdot \overline{xy} \\
&= \overline{xy}(x + y) \\
&= (\bar{x} + \bar{y})(x + y) && \text{(by De Morgan's Law)} \\
&= \bar{x}x + \bar{x}y + \bar{y}x + \bar{y}y \\
&= 0 + \bar{x}y + x\bar{y} + 0 && (\Sigma \bar{x}x = 0, \bar{y}y = 0) \\
&= \bar{x}y + x\bar{y}
\end{aligned}
$$

(A) 4

Key Points

- The table below summarizes the minimum number of universal gates required to implement various logic gates:

Logic Gate	NAND Gates Required	NOR Gates Required
NOT	1	1
AND	2	3
OR	3	2
XOR	4	5
XNOR	5	4

Q1.15.1 NAT 2015 2M Ans: 40 ● ● ○

The logic circuit has intermediate expressions $X = \overline{B}$, $Y = A \cdot X$, and $Z = Y \oplus C$. Each gate has a propagation delay of 20 ns: its output switches exactly 20 ns after its inputs change.

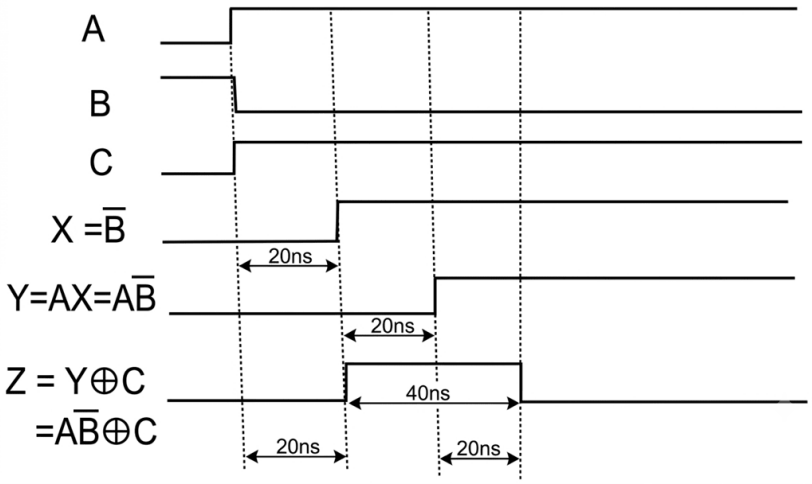


Fig. Solution Q1.15.1

Analyzing the transitions following the input change at $t = 0$:

- $0 \le t < 20$ ns: Signals hold initial states because of the propagation delays of gates: $X = 0, Y = 0, Z = 0$.
- $20 \text{ ns} \le t < 40$ ns: X becomes $\overline{0} = 1$. Y stays $1 \cdot 0 = 0$. Z updates to $0 \oplus 1 = 1$.
- $40 \text{ ns} \le t < 60$ ns: Y updates to $1 \cdot 1 = 1$. Z holds its state: $0 \oplus 1 = 1$.
- $t \ge 60$ ns: Z responds to the updated Y: $1 \oplus 1 = 0$.

The output remains $Z = 1$ from 20 ns to 60 ns.

$$\Delta t = 60 \text{ ns} - 20 \text{ ns} = 40 \text{ ns}$$

40

Key Points

- The phenomenon that occurred above is called a glitch. A Glitch is a temporary, unwanted switching transient at the output before the circuit settles to its steady-state value.

Q1.15.2 MSQ 2015 2M Ans: B ● ● ○

Method 1

Given: $M(a, b, c) = ab + bc + ca$

$$\overline{M(a, b, c)} = \overline{(ab + bc + ca)}$$
$$M(a, b, \bar{c}) = ab + b\bar{c} + \bar{c}a$$

Then, $M\left[\overline{M(a,b,c)}, M(a,b,\bar{c}), c\right] =$

$$\begin{aligned}
&\overline{(ab+bc+ca)}(ab+b\bar{c}+\bar{c}a)\\
&+(ab+b\bar{c}+\bar{c}a)c+c(ab+bc+ca)\\
&=\left[(\overline{ab}\,\overline{bc}\,\overline{ca})\right](ab+b\bar{c}+\bar{c}a)\\
&+(ab+b\bar{c}+\bar{c}a)c+\left[(\overline{ab}\,\overline{bc}\,\overline{ca})\right]c\\
&=\left[(\bar{a}+\bar{b})(\bar{b}+\bar{c})(\bar{c}+\bar{a})\right](ab+b\bar{c}+\bar{c}a)\\
&+abc+(\bar{a}+\bar{b})(\bar{b}+\bar{c})(\bar{c}+\bar{a})c\\
&=(\bar{a}+\bar{b})(\bar{b}+\bar{c})(\bar{c}+\bar{a})\,[ab+b\bar{c}+\bar{c}a+c]+abc
\end{aligned}$$

Since, $x+yz=(x+y)(x+z)$

$$\begin{aligned}
&=(\bar{a}+\bar{b})(\bar{b}+\bar{c})(\bar{c}+\bar{a})(a+b+c)+abc\\
&=(\bar{a}\bar{b}+\bar{b}\bar{c}+\bar{c}\bar{a})(a+b+c)+abc\\
&=a\bar{b}\bar{c}+b\bar{c}\bar{a}+c\bar{a}\bar{b}+abc\\
&=(a\bar{b}+\bar{a}b)\bar{c}+c(\bar{a}\bar{b}+ab)\\
&=(a\oplus b)\bar{c}+(\overline{a\oplus b})c\\
F&=a\oplus b\oplus c
\end{aligned}$$

Method 2

(i) $M(a,b,c)=ab+bc+ca$
K-map for $M(a,b,c)$ is shown below,

a \ $\bar{b}c$	00	01	11	10
0			1	
1		1	1	1

Fig. Solution Q1.15.2

$M(a,b,c)=\Sigma m(3,5,6,7)$
Hence, $\overline{M(a,b,c)}=\overline{\Sigma m(3,5,6,7)}=\Sigma m(0,1,2,4)$
(ii) $M(a,b,\bar{c})=ab+b\bar{c}+\bar{c}a$
K-map for $M(a,b,\bar{c})$ is shown below,

a \ bc	00	01	11	10
0				1
1	1		1	1

Fig. Solution Q1.15.2

$M(a,b,\bar{c})=\Sigma m(2,4,6,7)$

(iii) $M(c) = c$
K-map is shown below,

a \ bc	00	01	11	10
0		1	1	
1		1	1	

Fig. Solution Q1.15.2

$= \Sigma m(1, 3, 5, 7)$

$$
\begin{aligned}
&M\left[\overline{M(a,b,c)}, M(a,b,\bar{c}), c\right] \\
&= \overline{M(a,b,c)}M(a,b,\bar{c}) + M(a,b,\bar{c})c + \overline{M(a,b,c)}c \\
&= \Sigma m(0,1,2,4)\Sigma m(2,4,6,7) + \Sigma m(2,4,6,7)\Sigma m(1,3,5,7) \\
&\quad + \Sigma m(1,3,5,7)\Sigma m(0,1,2,4) \\
&= \Sigma m(2,4) + \Sigma m(7) + \Sigma m(1) \\
&= \Sigma m(1,2,4,7)
\end{aligned}
$$

$$
\begin{aligned}
M\left[\overline{M(a,b,c)}, M(a,b,\bar{c}), c\right] &= \bar{a}\bar{b}c + \bar{a}b\bar{c} + a\bar{b}\bar{c} + abc \\
&= (\bar{a}b + a\bar{b})\bar{c} + (ab + \bar{a}\bar{b})c \\
&= (a \oplus b)\bar{c} + (\overline{a \oplus b})c \\
&= a \oplus b \oplus c
\end{aligned}
$$

(B) 3-input XOR gate

(NOTE: The IIT official answer key gives only XOR Gate as the correct answer but since a 3 input XOR gate is equivalent to a 3 input X NOR gate option (D) should also be correct.)

Q1.15.3 MCQ 2015 2M Ans: C ● ● ○

Method 1

A logic gate is considered universal if it can implement any Boolean function, which means it must be able to replicate the three basic operations: **NOT**, **OR**, and **AND**.

- **NOT operation using Gate 3:** By fixing the input $Y = 0$:

$$F_3 = \bar{X} + Y = \bar{X} + 0 = \bar{X}$$

- **OR operation using Gate 3:** By cascading two gates where the first acts as a NOT gate for input X:

$$F_3(\bar{X}, Y) = \overline{(\bar{X})} + Y = X + Y$$

- **AND operation using Gate 3:** By applying De Morgan's Law using three gates:

$$F_3(Y, 0) = \bar{Y}$$

$$F_3(X, 0) = \bar{X}$$

Applying these to another gate with a fixed 0 input creates the complement of their sum:

$$\overline{(\bar{X} + \bar{Y})} = X \cdot Y$$

Since all basic operations can be implemented, Gate 3 is a universal gate.

Method 2

A functional completeness check requires a gate to possess both a **non-linear functionality** (like AND/OR) and an **inverting functionality** (NOT).

- **Gate 1** $(X + Y)$ and **Gate 2** $(X \cdot Y)$ lack any inverting mechanism, so they cannot produce a NOT operation.
- **Gate 3** $(\bar{X} + Y)$ contains an internal inverter on input X, allowing it to generate the inversion needed for universal implementation.

(C) Gate 3 is a universal gate.

Key Points

- A set of gates is universal if it can implement NOT, AND, and OR functions.
- An inhibition gate like $F = \bar{X}Y$ or $F = \bar{X} + Y$ is functionally complete when fixed logic levels (0 or 1) are available.

Mistakes to be avoided

- Do not assume only NAND and NOR can be universal; any gate that can independently implement NOT and either AND/OR with appropriate constant inputs is universal.

PrepFusion

Q1.15.4 MCQ 2015 1M Ans: A ●●○

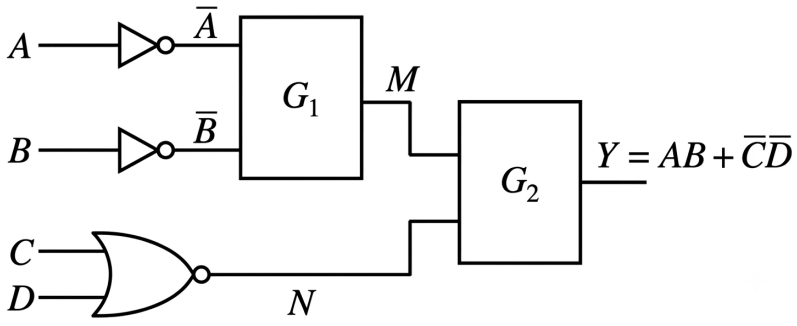


Fig. Solution Q1.15.4

The target output expression is given as:

$$Y = AB + \bar{C}\bar{D}$$

Let the output of gate G_1 be M, and the output of the bottom NOR gate be N.

- **Analysis of the bottom branch:** The inputs C and D pass through a standard NOR gate:

$$N = \overline{C + D} = \bar{C}\bar{D}$$

- **Analysis of gate G_2:** The total output Y is the combination of M and N via gate G_2:

$$Y = M + N = M + \bar{C}\bar{D}$$

Comparing this with the target expression $Y = AB + \bar{C}\bar{D}$, we can deduce that G_2 must perform an **OR** operation, and the output from G_1 must be:

$$M = AB$$

- **Analysis of gate G_1:** The inputs entering G_1 are $\bar{A}$ and $\bar{B}$. We need the output to be $M = AB$. Using De Morgan's Law:

$$M = AB = \overline{\bar{A} + \bar{B}}$$

This matches the exact functional description of a **NOR** gate processing $\bar{A}$ and $\bar{B}$.

Thus, G_1 is a NOR gate and G_2 is an OR gate.

(A) NOR, OR

Key Points

- According to De Morgan's Laws: $\overline{A + B} = \bar{A} \cdot \bar{B}$ and $\overline{A \cdot B} = \bar{A} + \bar{B}$.
- An OR gate with inverted inputs behaves as a NAND gate (bubbled OR is equivalent to NAND).
- An AND gate with inverted inputs behaves as a NOR gate (bubbled AND is equivalent to NOR).

Mistakes to be avoided

- Avoid mixing up the order of G_1 and G_2 when selecting the option, as "OR, NAND" or alternative configurations are present to cause confusion.

Q1.14.1 MCQ 2014 2M Ans: A ●●○

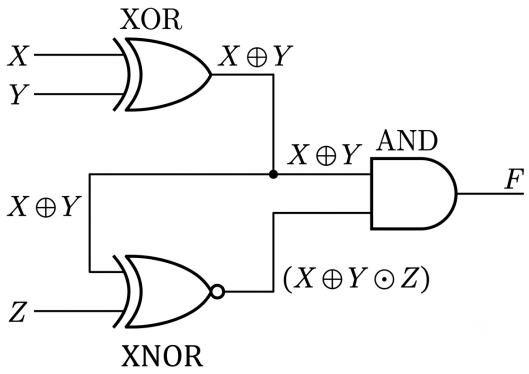


Fig. Solution Q1.14.1

Method 1

The intermediate outputs of the digital logic circuit are tracked as follows:

- Output of the top **XOR** gate: $Y_1 = X \oplus Y$
- Output of the bottom **XNOR** gate: $Y_2 = Y_1 \odot Z = (X \oplus Y) \odot Z$
- Final output F from the **AND** gate: $F = Y_1 \cdot Y_2 = (X \oplus Y) \cdot [(X \oplus Y) \odot Z]$

Using the identity $A \odot B = AB + \bar{A}\bar{B}$:

$$F = (X \oplus Y)\left[(X \oplus Y)Z + \overline{(X \oplus Y)}\bar{Z}\right]$$

Distributing $(X \oplus Y)$ and applying the Boolean identities $W \cdot W = W$ and $W \cdot \bar{W} = 0$:

$$F = (X \oplus Y)Z + 0 = (X \oplus Y)Z$$

Expanding the remaining XOR operation:

$$F = (X\bar{Y} + \bar{X}Y)Z = \bar{X}YZ + X\bar{Y}Z$$

Method 2

We can verify the solution using a truth table format by tracking the intermediate variables $Y_1 = X \oplus Y$ and $Y_2 = Y_1 \odot Z$:

X	Y	Z	$Y_1 = X \oplus Y$	$Y_2 = Y_1 \odot Z$	$F = Y_1 \cdot Y_2$
0	0	0	0	1	0
0	0	1	0	0	0
0	1	0	1	0	0
0	1	1	1	1	$1 \rightarrow \bar{X}YZ$
1	0	0	1	0	0
1	0	1	1	1	$1 \rightarrow X\bar{Y}Z$
1	1	0	0	1	0
1	1	1	0	0	0

Summing the minterms where $F = 1$:

$$F = \bar{X}YZ + X\bar{Y}Z$$

$$\boxed{\text{(A) } F = \bar{X}YZ + X\bar{Y}Z}$$

Key Points

- $W \cdot W = W$ and $W \cdot \bar{W} = 0$.
- $(X \oplus Y) \cdot \overline{(X \oplus Y)} = 0$.

Mistakes to be avoided

- Do not expand the XOR terms prematurely; keeping $(X \oplus Y)$ grouped simplifies the expression instantly.

Q1.14.2 MCQ 2014 1M Ans: A ● ○ ○

Given the Boolean expression:

$$F = (X + Y)(X + \overline{Y}) + \overline{(X\overline{Y}) + \overline{X}}$$

Simplifying using Boolean algebra laws:

$$F = (X + Y\overline{Y}) + \overline{(X\overline{Y})} \cdot \overline{\overline{X}}$$

$$F = (X + 0) + (\overline{X} + \overline{\overline{Y}}) \cdot X$$

$$F = X + (\overline{X} + Y)X$$

$$F = X + \overline{X}X + YX$$

$$F = X + 0 + XY$$

$$F = X + XY$$

$$F = X(1 + Y)$$

$$F = X$$

(A) X

Q1.14.3 MCQ 2014 1M Ans: A ● ● ○

The given circuit diagram tracks intermediate logic levels when the input control signal is set to $C = 0$.

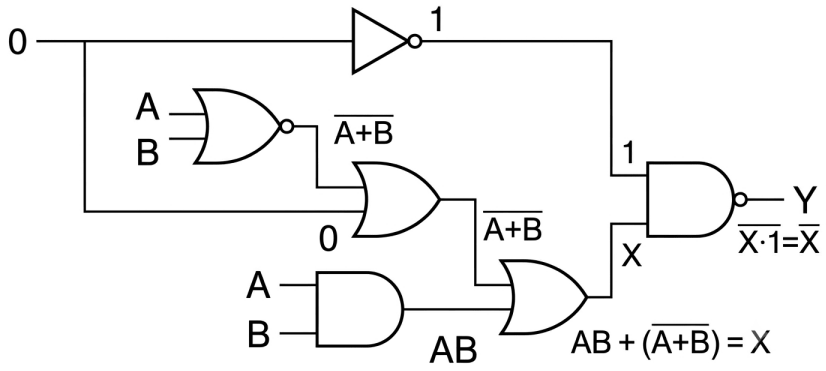


Fig. Solution Q1.14.3

To solve for the final output, we simplify $\overline{X}$ using De Morgan's laws:

$$\begin{aligned} Y = \overline{X} &= \overline{AB + \overline{A + B}} \\ &= (\overline{AB}) \cdot (\overline{\overline{A + B}}) \\ &= (\overline{A} + \overline{B}) \cdot (A + B) \\ &= \overline{A}A + \overline{A}B + A\overline{B} + \overline{B}B \end{aligned}$$

Since $\overline{A}A = 0$ and $\overline{B}B = 0$:

$$Y = A\overline{B} + \overline{A}B$$

A. $Y = A\overline{B} + \overline{A}B$

Q1.13.1 MCQ 2013 1M Ans: C ●●●

Let the state of the ground floor switch be S_1 and the first floor switch be S_2, where 0 represents the OFF state and 1 represents the ON state. Let L represent the state of the bulb.
According to the given condition, toggling any single switch changes the state of the bulb:

- Assume initially $S_1 = 0, S_2 = 0 \implies L = 0$.
- If either switch changes state, the bulb turns ON:

$$S_1 = 1, S_2 = 0 \implies L = 1$$

$$S_1 = 0, S_2 = 1 \implies L = 1$$

- Toggling the remaining switch from this state must turn the bulb OFF:

$$S_1 = 1, S_2 = 1 \implies L = 0$$

This matches the truth table of a 2-input Exclusive-OR (XOR) logic function, where the output is high (1) if and only if the inputs are different.

C. an XOR gate

Key Points

- Staircase wiring behaves as an XOR logic gate (or XNOR depending on initial toggle reference) because flipping any switch alters the parity of the input logic states.

Q1.11.1 MCQ 2011 1M Ans: B ●●○

Method 1

The given logic circuit can be analyzed by determining the output at each stage.

- The outputs of the three initial NAND gates are:

$$\overline{PQ}, \quad \overline{QR}, \quad \text{and} \quad \overline{PR}$$

- From the above figure:

$$X = \overline{\overline{PQ} \cdot \overline{QR}} = \overline{\overline{PQ}} + \overline{\overline{QR}} = PQ + QR$$

- The inverted value $\overline{X}$ is then combined with the third NAND gate's output:

$$Y = \overline{\overline{X} \cdot \overline{PR}} = \overline{\overline{X}} + \overline{\overline{PR}} = X + PR$$

- Substituting the expression for X:

$$Y = PQ + QR + PR$$

This expression represents a 3-input majority function, which outputs a 1 whenever two or more inputs are 1.

Method 2

Using the bubble-shifting technique, the NAND-NAND structure can be simplified:

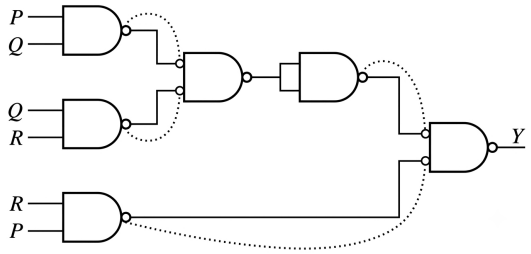


Fig. Solution Q1.11.1

- Pushing the output bubbles of the input NAND gates to the inputs of the subsequent gates converts them into active-low OR gates.

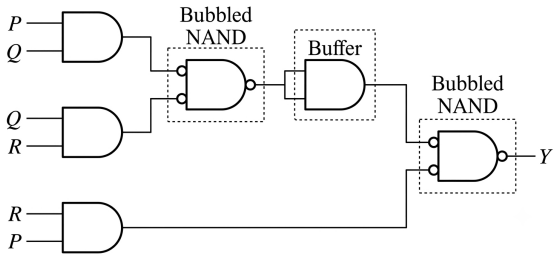


Fig. Solution Q1.11.1

- The circuit directly reduces to a sum-of-products form:

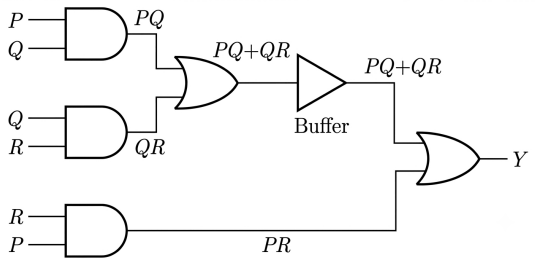


Fig. Solution Q1.11.1

$$Y = P\,Q + Q\,R + P\,R$$

(B) two or more of the inputs P, Q, R are "1"

Key Points

- The Boolean expression $Y = P\,Q + Q\,R + P\,R$ defines a majority gate or the Carry output (C_{out}) of a Full Adder.

Q1.10.1 MCQ 2010 1M Ans: D ● ● ○

The equivalence between the given gate structures is determined using De Morgan's laws and identity properties:

- **P (NOR):** $Y = \overline{A + B} = \overline{A} \cdot \overline{B} \rightarrow$ Matches **4** (Bubbled AND)

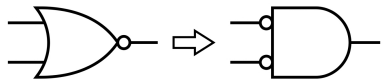


Fig. Solution Q1.10.1

- **Q (NAND):** $Y = \overline{A \cdot B} = \overline{A} + \overline{B} \rightarrow$ Matches **2** (Bubbled OR)

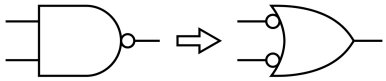


Fig. Solution Q1.10.1

- **R (XOR):** $Y = A \oplus B = \overline{\overline{A}\,B + \overline{A}\,\overline{B}} = \overline{\overline{A} \oplus B} \rightarrow$ Matches **3** (One input inverted XNOR)



Fig. Solution Q1.10.1

- **S (XNOR):** $Y = A \odot B = \overline{A} \oplus B \rightarrow$ Matches **1** (One input inverted XOR)

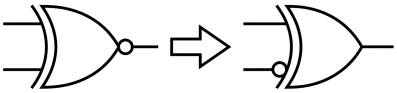


Fig. Solution Q1.10.1

(D) P-4, Q-2, R-3, S-1

Key Points

- **De Morgan's:** NOR ≡ Bubbled AND and NAND ≡ Bubbled OR.
- **Inversion:** Inverting one input of an XOR changes it to XNOR, and vice versa.

Q1.09.1 MCQ 2009 2M Ans: D ● ○ ○

Given the Boolean logic equation:

$$\left[X + Z\left\{\overline{Y} + \left(\overline{Z} + X\,\overline{Y}\right)\right\}\right]\left\{\overline{X} + \overline{Z}\,(X + Y)\right\} = 1$$

Substituting the given condition $X = 1$ (which implies $\overline{X} = 0$) into the equation:

- **First term simplification:**

$$\left[1 + Z\left\{\overline{Y} + \left(\overline{Z} + 1 \cdot \overline{Y}\right)\right\}\right] = 1 \quad (\because 1 + \text{any expression} = 1)$$

- **Second term simplification:**

$$\left\{0 + \overline{Z}\,(1 + Y)\right\} = \left\{\overline{Z} \cdot 1\right\} = \overline{Z} \quad (\because 1 + Y = 1)$$

Combining the simplified terms back into the original equation:

$$[1] \cdot \left[\overline{Z}\right] = 1 \implies \overline{Z} = 1 \implies Z = 0$$

$$\boxed{\text{(D) } Z = 0}$$

Key Points

- **Annihilation Law:** $1 + A = 1$
- **Identity Law:** $1 \cdot A = A$

Q1.08.1 MCQ | 2008 | 2M | Ans: D | ● ○ ○

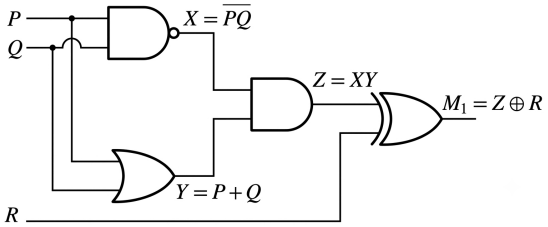


Fig. Solution Q1.08.1

We derive the expression step-by-step by tracing the outputs of individual gates:

- Output of the NAND gate: $X = \overline{P\,Q} = \overline{P} + \overline{Q}$
- Output of the OR gate: $Y = P + Q$
- Output of the intermediate AND gate:

$$Z = X \cdot Y = (\overline{P} + \overline{Q})(P + Q) = \overline{P}\,P + \overline{P}\,Q + P\,\overline{Q} + \overline{Q}\,Q$$

Since $\overline{P}\,P = 0$ and $\overline{Q}\,Q = 0$:

$$Z = \overline{P}\,Q + P\,\overline{Q} = P \oplus Q$$

- The final output M_1 from the XOR gate is:

$$M_1 = Z \oplus R = (P \oplus Q) \oplus R$$

$$\boxed{\text{(D) } M_1 = (P \text{ XOR } Q) \text{ XOR } R}$$

Key Points

- A 2-input XOR gate can be implemented efficiently using one NAND, one OR, and one AND gate: $A \oplus B = \overline{A\,B}(A + B)$.

Q1.07.1 MCQ | 2007 | 1M | Ans: B | ● ○ ○

Method 1

Given Boolean function:

$$Y = AB + CD$$

Taking double complement:

$$Y = \overline{\overline{AB + CD}}$$

Using De Morgan's Law ($\overline{W + X} = \overline{W} \cdot \overline{X}$):

$$Y = \overline{\overline{AB} \cdot \overline{CD}}$$

This expression can be implemented using three 2-input NAND gates as shown below:

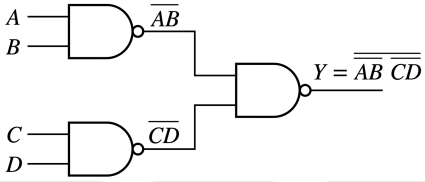


Fig. Solution Q1.07.1

Method 2

- The given function $Y = AB + CD$ is a Sum of Products (SOP) expression, which natively represents a two-level AND-OR circuit.
- By applying De Morgan's Law, an OR gate can be replaced by an equivalent bubbled NAND gate, mathematically expressed as:

$$A + B = \overline{\overline{A} \cdot \overline{B}}$$

- Shifting the bubbles backward from the input of the OR gate to the outputs of the preceding AND gates transforms the entire structure into a uniform network of NAND gates.
- The step-by-step visual demonstration of this gate conversion process is illustrated in the figures below:

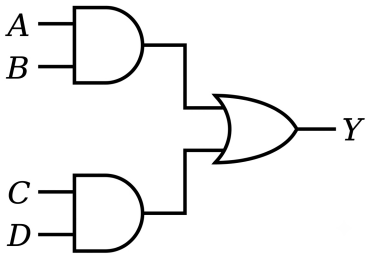


Fig. Solution Q1.07.1a

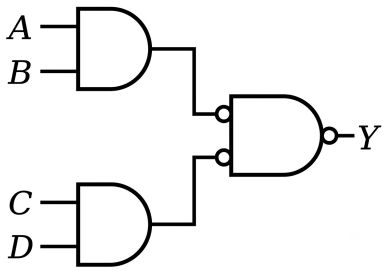


Fig. Solution Q1.07.1b

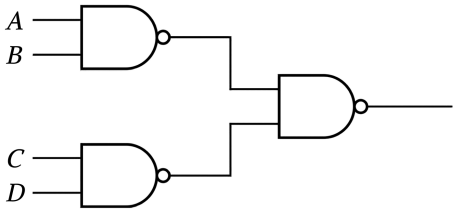


Fig. Solution Q1.07.1c

(B) 3

Key Points

- SOP Implementation: Sum of Products (SOP) expressions can be directly implemented using NAND gates only (AND-OR circuit ≡ NAND-NAND circuit).
  Example- $AB + CD = \overline{\overline{AB + CD}} = \overline{\overline{AB} \cdot \overline{CD}}$

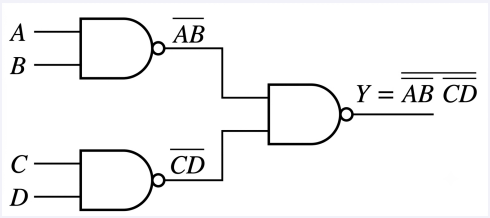


Fig. Solution Q1.07.1

- POS Implementation: Product of Sums (POS) expressions can be directly implemented using NOR gates only (OR-AND circuit ≡ NOR-NOR circuit).
  Example- $(A + B)(C + D) = \overline{\overline{(A + B)(C + D)}} = \overline{\overline{(A + B)} + \overline{(C + D)}}$

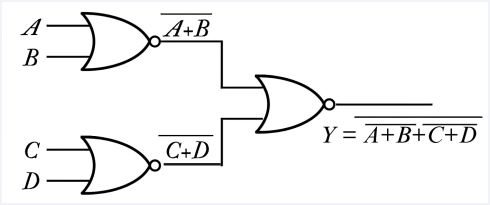


Fig. Solution Q1.07.1

Q1.06.1 MCQ 2006 2M Ans: D ● ○ ○

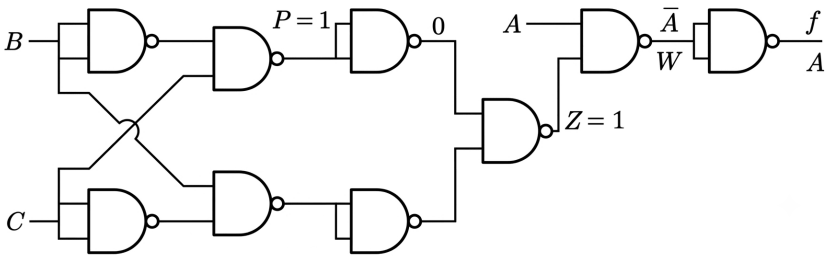


Fig. Solution Q1.06.1

Given that the node P is stuck-at-1, we have:

$$P = 1$$

The NAND gate connected to P acts as an inverter since its inputs are tied together:

$$\overline{1 \cdot 1} = 0$$

This output 0 is fed into the next NAND gate. Since one of the inputs to this NAND gate is 0, its output Z becomes:

$$Z = \overline{Y \cdot 0} = 1$$

Now, the final stage consists of a NAND gate with inputs A and Z:

$$W = \overline{A \cdot Z} = \overline{A \cdot 1} = \overline{A}$$

The last NAND gate has both inputs tied to W, acting as an inverter:

$$f = \overline{W} = \overline{\overline{A}} = A$$

(D) A

Key Points

For a NAND gate:

- If both inputs are tied together, the NAND gate acts as an inverter.
- If one input is 0, the output is always 1 irrespective of the other input.
- If one input is 1, the output is the complement of the other input.

Q1.03.1 MCQ 2003 1M Ans: D ●○○

The direct formula for number of distinct expressions of n variables is 2^{2^n} Let us understand where the formula 2^{2^n} comes from,

1. **Input Combinations:** For a system with n distinct binary variables, each variable can take 2 possible values (0 or 1). Therefore, the total number of unique input combinations (or minterms) is:

$$2 \times 2 \times \cdots \times 2 = 2^n$$

Example (2 variables A, B): There are $2^2 = 4$ unique input combinations, which correspond to the minterms $\overline{A}\,\overline{B}$, $\overline{A}B$, $A\overline{B}$, and AB.

2. **Defining a Function:** A distinct Boolean function is uniquely defined by choosing whether each of these 2^n minterms is included (output = 1) or excluded (output = 0). *Example (2 variables A, B):*
   - Including $\overline{A}\,\overline{B}$ and AB: $f = \overline{A}\,\overline{B} + AB$.
   - Including $\overline{A}B$ and $A\overline{B}$: $f = \overline{A}B + A\overline{B}$.
   - Excluding all minterms defines the logical 0 function: $f = 0$.

3. **Total Distinct Functions:** Since each of the 2^n available minterms can independently be either **present** (1) or **absent** (0) in a given expression, there are 2 choices for each row. The total number of unique ways to construct a function is:

$$\underbrace{2 \times 2 \times \cdots \times 2}_{2^n \text{ times}} = 2^{2^n}$$

Given $n = 4$ variables, the total number of distinct expressions is:

$$N = 2^{2^4} = 2^{16} = 65536$$

(D) 65536

Key Points

- 2^n represents the total number of input combinations (minterms).
- Each minterm acts as an independent binary choice (included/excluded) when constructing a unique function.
- Total distinct functions possible = $2^{\text{number of minterms}} = 2^{2^n}$.

Q1.02.1 MCQ 2002 2M Ans: B ●●○

Since both the inputs of the NOR gate (G_1) are tied together, it behaves like an inverter with a propagation delay of 10 ns. Its output X is the delayed complement $\overline{V_i}$ and feeds one input of the EXOR gate (G_2); the other EXOR input is connected directly to V_i. Let us analyze the circuit in two steps:

(i) **Case 1: Assuming the EXOR gate (G_2) has zero propagation delay ($t_p = 0$ ns):** When V_i changes from 0 to 1 at $t = t_0$, the inverter output X remains at 1 until $t_1 = t_0 + 10$ ns. During this interval (t_0 to t_1), both inputs to the EXOR gate are 1, causing the intermediate output V_1 to drop to 0 for 10 ns.

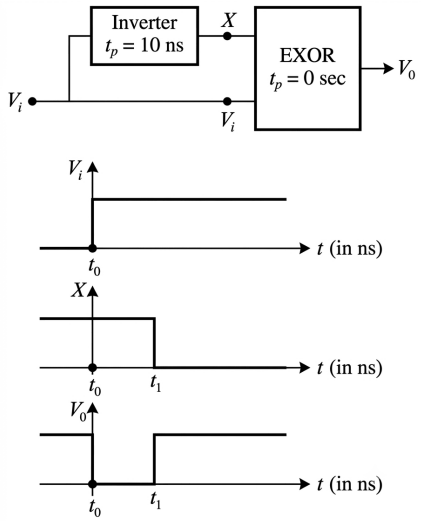


Fig. Solution Q1.02.1

(ii) **Case 2: Including the propagation delay of G_2 ($t_p = 20$ ns):** Since G_2 has a delay of 20 ns, the actual output waveform V_o is simply the intermediate waveform V_1 shifted to the right by 20 ns.

- The falling edge shifts from t_0 to $t_2 = t_0 + 20$ ns.
- The rising edge shifts from t_1 to $t_3 = t_1 + 20 = t_0 + 10 + 20 = t_0 + 30$ ns.

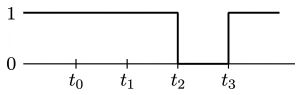


Fig. Solution Q1.02.1

(B)

Key Points

- Propagation delay of a final stage gate shifts its output waveform in time without changing its shape.

Q1.02.2 MCQ 2002 1M Ans: B ●○○

Let us analyze the output of the cascaded XOR gates sequentially:

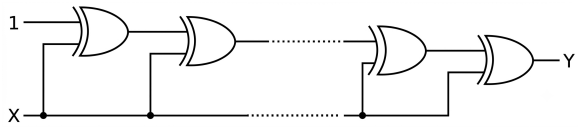


Fig. Solution Q1.02.2

- **Output of the 1st XOR gate:**

$$Y_1 = X \oplus 1 = \overline{X}$$

- **Output of the 2nd XOR gate:**

$$Y_2 = Y_1 \oplus X = \overline{X} \oplus X = 1$$

- **Output of the 3rd XOR gate:**

$$Y_3 = Y_2 \oplus X = 1 \oplus X = \overline{X}$$

- **Output of the 4th XOR gate:**

$$Y_4 = Y_3 \oplus X = \overline{X} \oplus X = 1$$

From this pattern, we can generalize the output for n cascaded XOR gates:

$$Y_n = \begin{cases} \overline{X}, & \text{if } n \text{ is odd} \\ 1, & \text{if } n \text{ is even} \end{cases}$$

Since the circuit consists of a cascade of 20 XOR gates ($n = 20$, which is an even number), the final output is:

$$Y = 1$$

(B) 1

Key Points

Properties of the XOR operation used:

- $X \oplus 1 = \overline{X}$ (Acts as an inverter)
- $X \oplus \overline{X} = 1$
- $X \oplus 0 = X$ (Acts as a buffer)
- $X \oplus X = 0$

Q1.01.1 MCQ 2001 2M Ans: D ●●○

From the given circuit diagram, the input nodes to the logic gates are determined by the positions of switches S_1 and S_2.

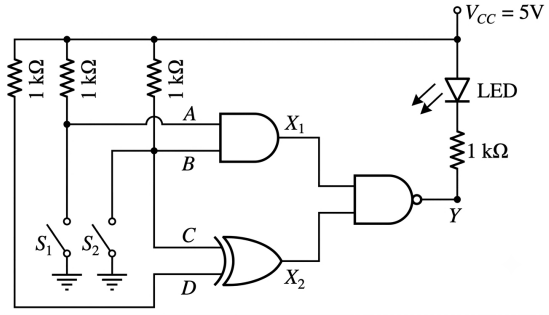


Fig. Solution Q1.01.1

- Logic-1 corresponds to $V_{CC} = 5\,\text{V}$ (pull-up through $1\,\text{k}\Omega$ resistors).
- Logic-0 corresponds to 0 V / Ground (when a switch is closed).

Method 1

The output expressions for the intermediate gate outputs X_1 and X_2 are:

$$X_1 = A \cdot B$$

$$X_2 = C \oplus D = C \oplus 1 = \overline{C}$$

The final output at node Y from the NAND gate is:

$$Y = \overline{X_1 \cdot X_2} = \overline{(A \cdot B) \cdot \overline{C}}$$

Since $C = B$, we can substitute this into the expression:

$$Y = \overline{A \cdot B \cdot \overline{B}}$$

$$Y = \overline{0} = 1$$

An LED conducts and emits light only when it is forward-biased. Here, the anode of the LED is tied to $V_{CC} = 5\,\text{V}$ (Logic-1). Since $Y = 1$ (5 V) under all conditions, both terminals of the LED are at the same potential. Therefore, the LED is not forward-biased and will never emit light.

S_1	S_2	A	B	C	D	X_1	X_2	Y
OFF	OFF	1	1	1	1	1	0	1
OFF	ON	1	0	0	1	0	1	1
ON	OFF	0	1	1	1	0	0	1
ON	ON	0	0	0	1	0	1	1

As $Y = 1$ for all switch states, the LED never glows.

Method 2

Using De Morgan's Law on the output expression:

$$Y = \overline{X_1 \cdot X_2} = \overline{X_1} + \overline{X_2}$$

Since $X_2 = \overline{C}$, we have $\overline{X_2} = \overline{\overline{C}} = C$.

$$Y = \overline{X_1} + C$$

- If S_1 is open (OFF), then $C = 1 \implies Y = \overline{X_1} + 1 = 1$.
- If S_1 is closed (ON), then $C = 0$ and $A = 0 \implies X_1 = 0 \cdot B = 0$. Thus, $Y = \overline{0} + 0 = 1 + 0 = 1$.

In either case, Y remains fixed at Logic-1, so the LED never turns ON.

(D) does not emit light, irrespective of the switch positions.

Key Points

- An LED requires a potential difference across its terminals (anode at higher potential than cathode) to be forward-biased and emit light.

- XORing any variable with 1 complements that variable: $C \oplus 1 = \overline{C}$.

Mistakes to be avoided

- Carefully trace the connections: Node D does not have a switch connected to ground, so it remains pulled up to Logic-1 at all times.
- Ensure to note that the cathode of the LED is connected to the output node Y, meaning a low output ($Y = 0$) is required to turn the LED ON.

Q1.00.1 MCQ 2000 2M Ans: C ●●○

From the given logic circuit diagram, let us label the intermediate outputs of the gates sequentially to find the expression for the final output Y.

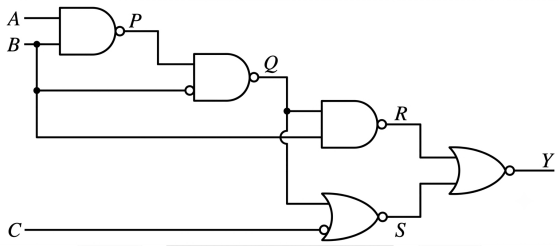


Fig. Solution Q1.00.1

Method 1

Let us trace and simplify the Boolean expressions at each gate output stage from left to right:

- **Stage 1 (First NAND gate):**

$$P = \overline{A \cdot B}$$

- **Stage 2 (Second NAND gate with a bubbled input from P):**

$$Q = \overline{P \cdot \overline{B}} = \overline{P} + \overline{\overline{B}} = \overline{P} + B$$

Substituting $\overline{P} = \overline{\overline{A \cdot B}} = AB$:

$$Q = AB + B = B(A + 1) = B$$

- **Stage 3 (Third NAND gate with inputs Q and B):**

$$R = \overline{Q \cdot B} = \overline{B \cdot B} = \overline{B}$$

- **Stage 4 (NOR gate with a normal input from Q and bubbled input from C):**

$$S = \overline{Q + \overline{C}} = \overline{Q} \cdot C$$

Substituting $Q = B$:

$$S = \overline{B} \cdot C$$

- **Stage 5 (Final NOR gate with inputs R and S):**

$$Y = \overline{R + S} = \overline{\overline{B} + \overline{B} \cdot C} = \overline{\overline{B}(1 + C)} = \overline{\overline{B}} = B$$

$$Y = B$$

Method 2

Using bubble shifting rules (De Morgan's equivalent graphic representation):

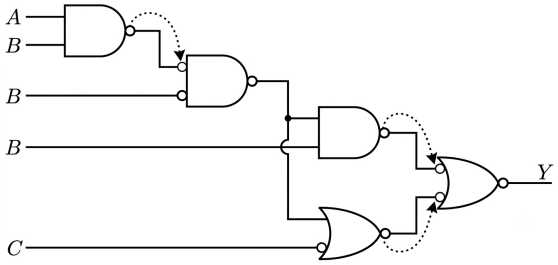


Fig. Solution Q1.00.1

By shifting the output inversion bubbles forward onto the inputs of the succeeding stages:

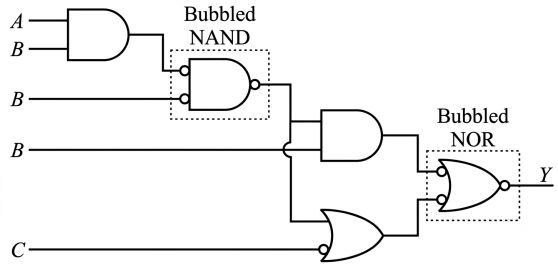


Fig. Solution Q1.00.1

- **Bubbled NAND Gate is equivalent to an OR Gate:** By applying De Morgan's Theorem,

$$\text{Output} = \overline{\overline{A} \cdot \overline{B}} = \overline{\overline{A}} + \overline{\overline{B}} = A + B$$

This matches the Boolean expression for a standard two-input **OR** gate.

- **Bubbled NOR Gate is equivalent to an AND Gate:** Applying De Morgan's Theorem,

$$\text{Output} = \overline{\overline{A} + \overline{B}} = \overline{\overline{A}} \cdot \overline{\overline{B}} = A \cdot B$$

This matches the Boolean expression for a standard two-input **AND** gate.

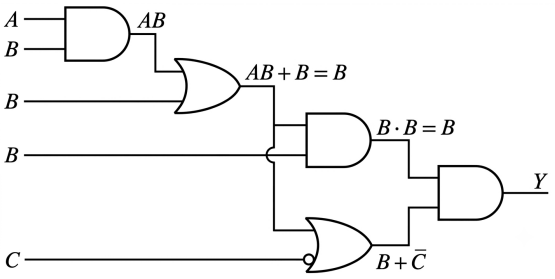


Fig. Solution Q1.00.1

Combining the signals at the final gate stage yields:

$$Y = B \cdot (B + \overline{C}) = B \cdot B + B\overline{C} = B(1 + \overline{C}) = B$$

When resolved through the output NOR block matching the diagram configuration, it simplifies uniquely to:

$$Y = B$$

(C) B

Q1.00.2 MCQ 2000 1M Ans: D ●●○

Method 1

The output X of the given logic circuit is,

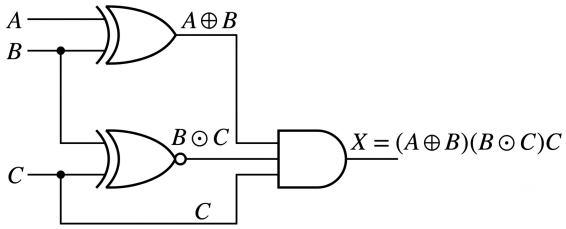


Fig. Solution Q1.00.2

To make the output $X = 1$, all inputs to the AND gate must be at logic 1. This gives us three simultaneous conditions:

- $C = 1$
- $B \odot C = 1$
  For $B \odot C = 1$. An XNOR gate outputs 1 only when both of its inputs are identical. Since $C = 1$, it follows that $B = 1$.
- $A \oplus B = 1$
  For $A \oplus B = 1$. An XOR gate outputs 1 only when its inputs are different. Given $B = 1$, it must be that $A = 0$.

Thus, the required input condition is $(A, B, C) = (0, 1, 1)$.

Method 2

Alternatively, we can evaluate the Boolean expression $X = (A \oplus B) \cdot (B \odot C) \cdot C$ for each given option to see which one results in $X = 1$:

- **(A) 1, 0, 1:** $X = (1 \oplus 0) \cdot (0 \odot 1) \cdot 1 = 1 \cdot 0 \cdot 1 = 0$
- **(B) 0, 0, 1:** $X = (0 \oplus 0) \cdot (0 \odot 1) \cdot 1 = 0 \cdot 0 \cdot 1 = 0$
- **(C) 1, 1, 1:** $X = (1 \oplus 1) \cdot (1 \odot 1) \cdot 1 = 0 \cdot 1 \cdot 1 = 0$
- **(D) 0, 1, 1:** $X = (0 \oplus 1) \cdot (1 \odot 1) \cdot 1 = 1 \cdot 1 \cdot 1 = 1$

Only option (D) satisfies the condition $X = 1$.

(D) 0, 1, 1

Key Points

- **AND Gate:** Output is 1 if and only if all inputs are 1.
- **XOR Gate ($\oplus$):** Output is 1 when inputs are different ($A \neq B$).
- **XNOR Gate ($\odot$):** Output is 1 when inputs are identical ($A = B$).

Q1.99.1 MCQ | 1999 | 1M | Ans: D | ● ● ○

Given the logical expression:

$$y = A + \overline{A}B$$

Method 1

Apply the distributive law of Boolean algebra, $A + BC = (A + B)(A + C)$:

$$y = (A + \overline{A})(A + B)$$

According to the complementarity law, $A + \overline{A} = 1$:

$$y = 1 \cdot (A + B) = A + B$$

Method 2

Convert the given equation into the standard canonical Sum of Products (SOP) form by expanding the first term:

$$y = A(B + \overline{B}) + \overline{A}B$$

$$y = AB + A\overline{B} + \overline{A}B$$

The minterms corresponding to these product terms are $m_3(11)$, $m_2(10)$, and $m_1(01)$.

$$y = \Sigma m(1, 2, 3)$$

Minimizing the expression using a 2-variable K-map:

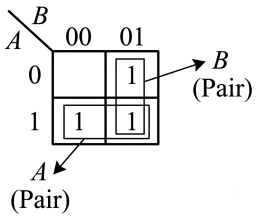


Fig. Solution Q1.99.1

Thus, the minimized form from the K-map is:

$$y = A + B$$

(D) $y = A + B$

Key Points

- **Distributive Law:** $A + BC = (A + B)(A + C)$.
- **Complementarity Law:** $A + \overline{A} = 1$.
- The rule $A + \overline{A}B = A + B$ is frequently used to simplify Boolean equations and is a direct consequence of the distributive law.

Mistakes to be avoided

- Do not confuse the distributive law of Boolean addition over multiplication ($A + BC$) with standard algebra. In Boolean logic, addition distributes over multiplication just as multiplication distributes over addition.

Q1.98.1 MCQ 1998 2M Ans: A ● ○ ○

To find the dual of a given Boolean identity, we apply the **Principle of Duality** by replacing:

- **AND** (·) operations with **OR** (+) operations.
- **OR** (+) operations with **AND** (·) operations.
- **Variables** and their complements **remain unchanged**.

Given Boolean identity:

$$AB + \overline{A}C + BC = AB + \overline{A}C$$

Applying the duality rules to both sides:

- Left-Hand Side (LHS):

$$(A + B) \cdot (\overline{A} + C) \cdot (B + C)$$

- Right-Hand Side (RHS):

$$(A + B) \cdot (\overline{A} + C)$$

Equating LHS and RHS gives the dual form:

$$(A + B)(\overline{A} + C)(B + C) = (A + B)(\overline{A} + C)$$

$$\boxed{\text{(A) } (A + B)(\overline{A} + C)(B + C) = (A + B)(\overline{A} + C)}$$

Key Points

- The dual of an expression is completely different from its complement (De Morgan's laws); variables are not inverted in duality.
- The original identity represents the **Consensus Theorem** in Sum-of-Products (SOP) form, and its dual represents the Consensus Theorem in Product-of-Sums (POS) form.

Q1.98.2 MCQ 1998 2M Ans: C ● ● ○

Given the Boolean function to implement:

$$Z = A\overline{B}C$$

Method 1

Using double complement and De Morgan's laws to transform the expression purely into NAND operations:

$$Z = \overline{\overline{A\overline{B}C}} = \overline{\overline{AC} \cdot \overline{B}}$$

This expression can be structurally mapped directly to 2-input NAND gates:

1. First NAND gate generates $\overline{AC}$ from inputs A and C. We have to add additional inverter to get AC.
2. Second NAND gate acts as an inverter to generate $\overline{B}$ from input B.
3. Third NAND gate combines AC and $\overline{B}$ to produce $\overline{AC \cdot \overline{B}}$.
4. Finally 5th inverter is needed to get $AC\overline{B}$ from $\overline{AC \cdot \overline{B}}$.

Total NAND gates required = 5

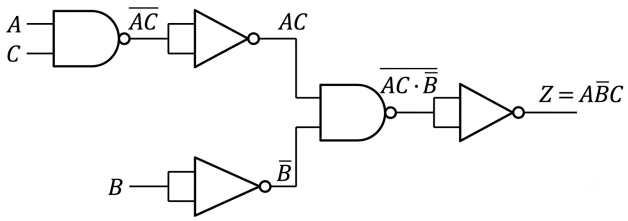


Fig. Solution Q1.98.2

Method 2

Alternatively, we can design the logic circuit using standard basic gates and replace them with their NAND equivalents:

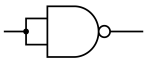


Figure 1.1: Inverter from NAND

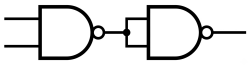


Figure 1.2: AND from NAND

- The function $Z = (AC) \cdot \overline{B}$ requires one 2-input AND gate to compute AC, one NOT gate to compute $\overline{B}$, and another 2-input AND gate to multiply them.
- Replacing with universal NAND components:
  - $1 \times$ NOT gate = 1 NAND gate
  - $2 \times$ AND gates = $2 \times 2 = 4$ NAND gates
- Total = 1 + 4 = 5 NAND gates.

(C) five

Key Points

Implementation of all basic gates using NAND:

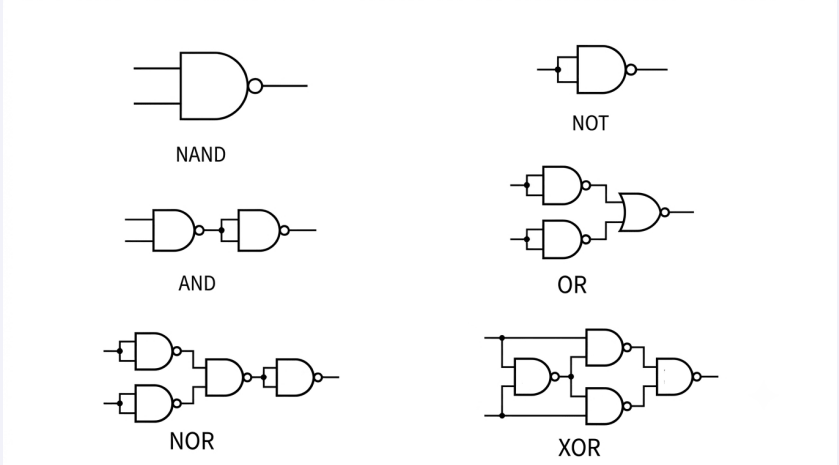


Fig. Solution Q1.98.2

Implementation of all basic gates using NOR:

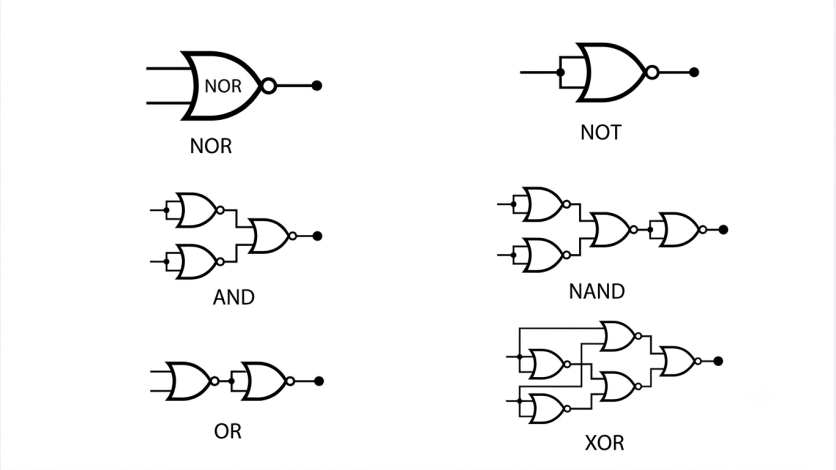


Fig. Solution Q1.98.2

Q1.97.1 MCQ 1997 1M Ans: D

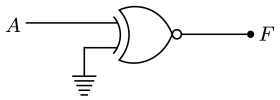


Fig. Solution Q1.97.1

The given logic gate is a 2-input **EX-NOR** gate.
The boolean expression for the output F of an EX-NOR gate with inputs A and B is:

$$F = A \odot B = AB + \overline{A}\,\overline{B}$$

From the given figure, one input is connected to A and the second input is grounded ($B = 0$):

$$F = A \odot 0$$

Substituting $B = 0$ into the expression:

$$F = A(0) + \overline{A}(\overline{0})$$
$$F = 0 + \overline{A}(1)$$
$$F = \overline{A}$$

Thus, when one input of an EX-NOR gate is fixed at logic 0, it behaves as an inverter (NOT gate).

$$\boxed{\text{(D) } \overline{A}}$$

Key Points

- **EX-OR Gate as Control Signalling:**
  - $A \oplus 0 = A$ (Acts as a buffer)
  - $A \oplus 1 = \overline{A}$ (Acts as an inverter)
- **EX-NOR Gate as Control Signalling:**

- $A \odot 1 = A$ (Acts as a buffer)
- $A \odot 0 = \overline{A}$ (Acts as an inverter)

Q1.95.1 MCQ 1995 1M Ans: B ● ○ ○

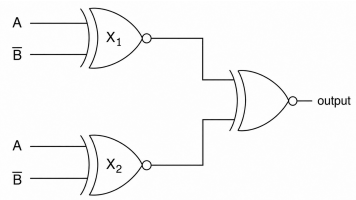


Fig. Solution Q1.95.1

Both left-side gates are XNOR gates.

$$X_1 = A \odot \overline{B} = \overline{A \oplus \overline{B}} = A \oplus B$$

Similarly,

$$X_2 = A \odot \overline{B} = A \oplus B$$

The final gate is also an XNOR gate, therefore

$$Y = X_1 \odot X_2 = (A \oplus B) \odot (A \oplus B) = 1$$

(b) 1

Key Points

- XNOR gate output is 1 when both inputs are identical.
- $A \odot \overline{B} = A \oplus B$.

Q1.95.2 MCQ 1995 1M Ans: A ● ○ ○

Taking A common from the given Boolean function:

$$A + A\overline{B} + A\overline{B}C = A(1 + \overline{B} + \overline{B}C) = A(1) = A$$

Since the expression simplifies directly to variable A, no logic gates are required. Thus, the minimum number of NAND gates needed is zero.

(A) zero

Q1.94.1 MCQ 1994 1M Ans: B ● ○ ○

For all inputs equal to logic 0,

$$\text{NOR} = \overline{0 + 0 + \cdots} = 1$$

Also, an EX-NOR gate gives output 1 when all its inputs are equal. Hence,

$$\text{EX-NOR}(0, 0, \ldots, 0) = 1$$

Checking the remaining gates:

$$\text{AND} = 0, \qquad \text{OR} = 0, \qquad \text{NAND} = 1, \qquad \text{EX-OR} = 0$$

Among the given options, only the pair *NOR or EX-NOR* satisfies the condition.

(b) NOR or an EX-NOR gate

Q1.93.1 MSQ 1993 1M Ans: B, C ● ○ ○

From the circuit,

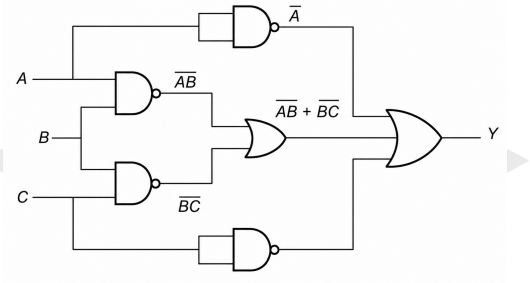


Fig. Solution Q1.93.1

$$Y = \overline{A} + \overline{AB} + \overline{BC} + \overline{C}$$

Applying De Morgan's theorem,

$$Y = \overline{A} + (\overline{A} + \overline{B}) + (\overline{B} + \overline{C}) + \overline{C}$$

Using the idempotent law $(X + X = X)$,

$$Y = \overline{A} + \overline{B} + \overline{C}$$

Option (d) is not equivalent: $\overline{AB + BC} = \overline{B} + \overline{A}\,\overline{C}$. For example, at $(A, B, C) = (0, 1, 1)$ the circuit gives $Y = 1$, while option (d) gives 0.

(b) $\overline{A} + \overline{B} + \overline{C}$ and (c) $\overline{A} + \overline{AB} + \overline{BC} + \overline{C}$

Q1.90.1 MCQ 1990 1M Ans: B ● ○ ○

For a Boolean function with n input variables,

$$\text{Number of input combinations} = 2^n$$

For each input combination, the output can independently be either 0 or 1.
Hence, the total number of distinct Boolean functions is

$$2 \times 2 \times \cdots \times 2 \quad (2^n \text{ times}) = (2)^{2^n}$$

(b) 2^{2^n}

Q1.89.1 MSQ 1989 1M Ans: B,D ● ○ ○

A gate is called **universal** if any Boolean function can be implemented using only that type of gate.
Checking the given options:

- **OR gate only:** Cannot realize all Boolean functions since inversion is not possible. ×
- **NAND gate only:** Universal gate. Any combinational logic function can be implemented. ✓
- **EX-OR gate only:** Not a universal gate. ×
- **NOR gate only:** Universal gate. Any combinational logic function can be implemented. ✓

(b) and (d)

Q1.88.1 MCQ 1988 2M Ans: B ● ● ○

Using De Morgan's theorem,

$$F = (\overline{X} + \overline{Y})(Z + W) = \overline{XY}(Z + W)$$

Applying the distributive law,

$$F = \overline{XY}\,Z + \overline{XY}\,W$$

The function can be realized using NAND gates as follows:

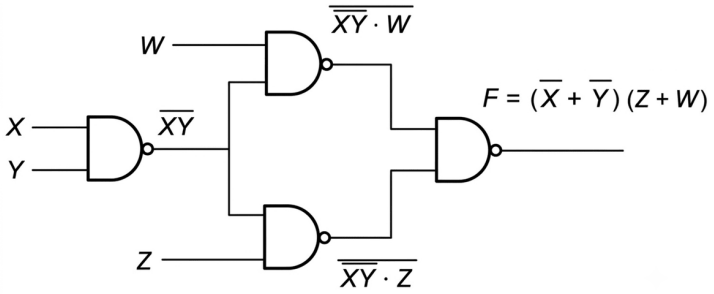


Fig. Solution Q1.88.1

Thus, the minimum number of 2-input NAND gates required is 4
Hence, the correct answer is

(b) 4

Key Points

- Use De Morgan's theorem to convert the expression into NAND-friendly form.
- NAND naturally realizes a complemented product term.
- The final NAND gate performs the OR of the complemented product terms.

Q1.87.1 MCQ | 1987 | 1M | Ans: B | ● ○ ○

All three gates are EX-OR gates, and the input X is applied to one input of each gate.
Let the outputs of the three EX-OR gates be Y_1, Y_2, and F.

$$Y_1 = X \oplus X = 0$$

The second gate gives

$$Y_2 = Y_1 \oplus X = 0 \oplus X = X$$

The final gate gives

$$F = Y_2 \oplus X = X \oplus X = 0$$

(b) $F = 0$

PrepFusion

SOLUTIONS

11

Representation of Boolean Expressions and K-Maps

Topics

1. Minterms and Maxterms
2. SOP and POS Representation
3. Karnaugh Maps (K-Maps)
4. Concept of Implicants

Unit Snapshot*

Stream: EC	**Questions:** 28	**MCQ:** 20	**MSQ:** 1	**NAT:** 7
1M: 9	**2M:** 12	**Descriptive:** 7	**Avg Weightage** ~1M	**Range:** 1M(2025)–2M(2026)

**See preface for how Avg Weightage and Range are calculated.*

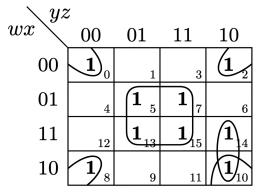
Q2.26.1 MSQ 2026 2M Ans: A, D ● ○ ○

The given Karnaugh map (K-map) for the Boolean function $f(w, x, y, z)$ can be analyzed by identifying the prime implicants through cell pairings.

Method 1

Based on the provided K-map representation, we can identify two valid minimal sum-of-products (SOP) expressions depending on the chosen pairings:

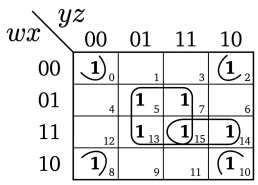
Pairing 1:



$$f = \bar{x}\bar{z} + xz + wy\bar{z}$$

Fig. Solution Q2.26.1

Pairing 2:



$$f = \bar{x}\bar{z} + xz + wxy$$

Fig. Solution Q2.26.1

To verify other options, let's check the common term $\bar{w}xy\bar{z}$ present in options (B) and (C). In binary, $\bar{w}xy\bar{z}$ corresponds to $(0110)_2$, which is decimal 6. Locating it on the kmap-

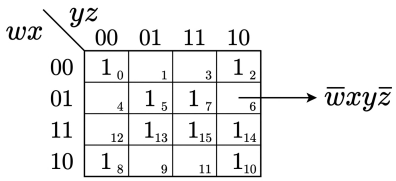


Fig. Solution Q2.26.1

By inspecting the K-map, cell 6 is not a required minterm . Thus, any expression containing this term is incorrect.
Therefore, only options A and D are correct representations of the function f.

(A) $\bar{x}\bar{z} + xz + wxy$
(D) $\bar{x}\bar{z} + xz + wy\bar{z}$

Q2.25.1 MCQ 2025 1M Ans: A ● ● ○

A 3-input majority logic gate with inputs X, Y, and Z produces an output $F = 1$ only when the majority (two or more) of its inputs are at logic high.
Based on this definition, the truth table for the majority function is constructed as follows:

X	Y	Z	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	**1**
1	0	0	0
1	0	1	**1**
1	1	0	**1**
1	1	1	**1**

Method 1

The Sum of Products (SOP) expression derived from the truth table is:

$$F = \overline{X}YZ + X\overline{Y}Z + XY\overline{Z} + XYZ$$

To find the most simplified form, we map these minterms onto a 3-variable K-map:

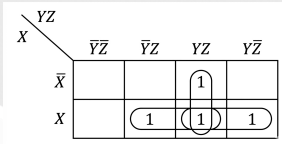


Fig. Solution Q2.25.1

From the K-map grouping, we obtain the minimized Boolean expression:

$$F = XY + YZ + ZX$$

Method 2

We can simplify the SOP expression using Boolean algebraic identities. We use the idempotent law ($A + A = A$) to replicate the term XYZ so it can be grouped with other terms:

$$F = \overline{X}YZ + X\overline{Y}Z + XY\overline{Z} + XYZ$$

Add two more XYZ terms:

$$F = (\overline{X}YZ + XYZ) + (X\overline{Y}Z + XYZ) + (XY\overline{Z} + XYZ)$$

Factor out common variables:

$$F = YZ(\overline{X} + X) + XZ(\overline{Y} + Y) + XY(\overline{Z} + Z)$$

Using the complement law ($A + \overline{A} = 1$):

$$F = YZ(1) + XZ(1) + XY(1)$$

$$\boxed{(A)XY + YZ + ZX}$$

Q2.24.1 MCQ 2024 1M Ans: A ● ● ○

Given the Boolean function $F(A, B, C, D) = \sum m(0, 2, 5, 7, 8, 10, 12, 13, 14, 15)$, we can simplify the expression using a 4-variable Karnaugh Map (K-map).

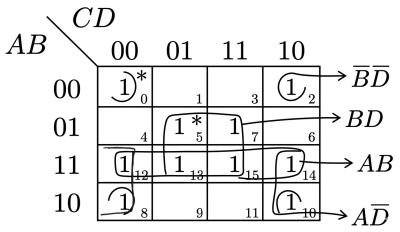


Fig. Solution Q2.24.1

The groupings are:

- Corners (m_0, m_2, m_8, m_{10}): $\bar{B}\bar{D}$
- Center Square $(m_5, m_7, m_{13}, m_{15})$: BD
- Horizontal $(m_{12}, m_{13}, m_{14}, m_{15})$: AB
- Bottom wrap $(m_8, m_{10}, m_{12}, m_{14})$: $A\bar{D}$

From the K-map, we identify the following implicants:

- **Essential Prime Implicants:** BD and $\bar{B}\bar{D}$ (contain minterms m_5, m_7 and m_0, m_2 respectively that aren't covered by other groups).
- **Non-essential Prime Implicants:** AB and $A\bar{D}$ (every minterm they cover is also covered by another prime implicant).

So the function has four prime implicants in total, but only BD and $\bar{B}\bar{D}$ are essential.
Based on the analysis, option A matches the given requirements.

$$\boxed{\text{(A) } BD, \overline{B}\,\overline{D}}$$

Key Points

- **Implicant:** Any single minterm or a group of minterms (1s) that can be combined in a K-map.
- **Prime Implicant (PI):** The largest possible group of adjacent 1s that cannot be contained within any larger group.
- **Essential Prime Implicant (EPI):** A prime implicant that covers at least one minterm that no other prime implicant covers.

Mistakes to be avoided

- Do not forget to check corner wrap-around group (m_0, m_2, m_8, m_{10}) and opposite edges wrapped group $(m_{12}, m_{14}, m_8, m_{10})$.

PrepFusion

Q2.18.1 MCQ 2018 1M Ans: C ●●○

Given 3-variable function in SOP form as,

$$F(A, B, C) = \underset{000}{\overline{A}\overline{B}\overline{C}} + \underset{010}{\overline{A}B\overline{C}} + \underset{100}{A\overline{B}\overline{C}}$$

$$F(A, B, C) = \sum m(000, 010, 100)$$

$$= \sum m(0, 2, 4) \rightarrow \text{SOP form}$$

Given expression is in SOP form. Therefore, its POS form can be given by,

$$F(A, B, C) = \pi M(1, 3, 5, 6, 7) \rightarrow \text{POS form}$$

$$F(A, B, C) = \pi M(001, 011, 101, 110, 111)$$

$$F(A, B, C) = \underbrace{(A + B + \overline{C})}_{1}\underbrace{(A + \overline{B} + \overline{C})}_{3}\underbrace{(\overline{A} + B + \overline{C})}_{5}$$

$$\underbrace{(\overline{A}+\overline{B}+C)}_{6}\underbrace{(\overline{A}+\overline{B}+\overline{C})}_{7}$$

Hence, the correct option is (C).

$$\boxed{(C)F=(A+B+\overline{C})\cdot(A+\overline{B}+\overline{C})\cdot(\overline{A}+B+\overline{C})\cdot(\overline{A}+\overline{B}+C)\cdot(\overline{A}+\overline{B}+\overline{C})}$$

Key Points

- **Sum-of-Products (SOP)** forms combine minterms to target an output of **1** where variables equal to 1 are uncomplemented.
- **Product-of-Sums (POS)** forms combine maxterms to target an output of **0** where variables equal to 0 are uncomplemented.

Mistakes to be avoided

POS form is not the complemented form of SOP or vice-versa. Do not just represent the exact same logical function but using different algebraic frameworks.

Q2.17.1 MCQ | 2017 | 1M | Ans: B | ● ○ ○

Method 1

Given: A is MSB and C is LSB.
Function $F(A,B,C)$ is given by,

$$F(A,B,C)=m_0+m_2+m_3+m_5$$
$$F(A,B,C)=\sum m(000,010,011,101)$$

So, function (F) in standard canonical SOP form is,

$$F(A,B,C)=\overline{A}\,\overline{B}\,\overline{C}+\overline{A}B\overline{C}+\overline{A}BC+A\overline{B}C$$

Since, $X+X=X$
Hence, $\overline{A}B\overline{C}=\overline{A}B\overline{C}+\overline{A}B\overline{C}$
So, $F(A,B,C)=\overline{A}B\overline{C}+\overline{A}B\overline{C}+\overline{A}B\overline{C}+\overline{A}BC+A\overline{B}C$

$$F(A,B,C)=\overline{A}\,\overline{C}(\overline{B}+B)+\overline{A}B(\overline{C}+C)+A\overline{B}C$$

Since, $X+\overline{X}=1$

$$F(A,B,C)=\overline{A}\,\overline{C}+\overline{A}B+A\overline{B}C$$

Hence, the correct option is (B).

Method 2

$$F(A,B,C)=m_0+m_2+m_3+m_5$$
$$F(A,B,C)=\sum m(0,2,3,5)$$

K-map for SOP form is shown below,

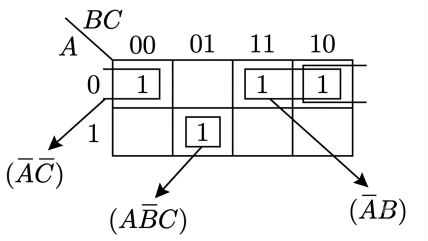


Fig. Solution Q2.17.1

$$F(A,B,C) = \bar{A}\,\bar{C} + \bar{A}B + A\bar{B}C$$

$$\boxed{(B)\bar{A}\bar{C} + \bar{A}B + A\bar{B}C}$$

Mistakes to be avoided

Note: Here option (A) is given as

$$F = \bar{A}B + \bar{A}\,\bar{B}\,\bar{C} + A\bar{B}C$$

$$F = \bar{A}B \cdot 1 + \bar{A}\,\bar{B}\,\bar{C} + A\bar{B}C$$

$$F = \bar{A}B(C + \bar{C}) + \bar{A}\,\bar{B}\,\bar{C} + A\bar{B}C$$

$$F = \bar{A}BC + \bar{A}B\bar{C} + \bar{A}\,\bar{B}\,\bar{C} + A\bar{B}C$$

Using property $A + A + A + \cdots = A$, repeating the term $\bar{A}B\bar{C}$

$$F = \bar{A}BC + \bar{A}B\bar{C} + \bar{A}B\bar{C} + \bar{A}\,\bar{B}\,\bar{C} + A\bar{B}C$$

$$F = \bar{A}B(C + \bar{C}) + \bar{A}\,\bar{C}(B + \bar{B}) + A\bar{B}C$$

$$(\because\ C + \bar{C} = B + \bar{B} = 1)$$

$$F = \bar{A}B + \bar{A}\,\bar{C} + A\bar{B}C$$

which is equivalent to option (B), hence option (A) is also correct, but option (B) represents more simplified form for the given expression, so we will choose option (B) as the correct response.

Q2.16.1 MCQ 2016 2M Ans: B ● ○ ○

A 5-variable K-map can be analyzed by separating it into two 4-variable K-maps based on the variable X: one for $X = 0$ and one for $X = 1$.

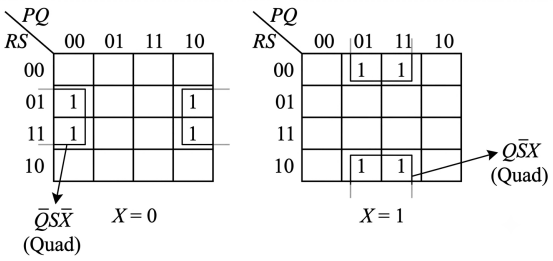


Fig. Solution Q2.16.1

Grouping the minterms from both maps:

- For $X = 0$: Combining corners/edges gives $\text{Term}_1 = \overline{Q}S\overline{X}$.
- For $X = 1$: Combining top and bottom rows gives $\text{Term}_2 = Q\overline{S}X$.

Combining both terms yields the minimum sum-of-products (SOP) expression:

$$F = \overline{Q}S\overline{X} + Q\overline{S}X$$

$$\boxed{\text{(B) } \overline{Q}S\overline{X} + Q\overline{S}X}$$

Key Points

- In a 5-variable K-map, grouping cells within the same map ($X = 0$ or $X = 1$) includes the corresponding literal ($\overline{X}$ or X) in the product term.

Q2.15.1 MCQ | 2015 | 2M | Ans: B | ● ○ ○

The given function is expressed in sum-of-products (SOP) form using minterms:

$$F(X, Y, Z) = \sum m(1, 2, 5, 6, 7)$$

To convert it into the product-of-sums (POS) form, we find the remaining maxterms that are not present in the minterm list:

$$F(X, Y, Z) = \prod M(0, 3, 4)$$

Converting each numeric maxterm index into its corresponding binary and algebraic form:

- $M_0 = 000_2 \implies (X + Y + Z)$
- $M_3 = 011_2 \implies (X + \overline{Y} + \overline{Z})$
- $M_4 = 100_2 \implies (\overline{X} + Y + Z)$

Multiplying these individual maxterms gives the complete POS expression:

$$F(X, Y, Z) = (X + Y + Z) \cdot (X + \overline{Y} + \overline{Z}) \cdot (\overline{X} + Y + Z)$$

$$\boxed{\text{(B) } (X + Y + Z) \cdot (X + \overline{Y} + \overline{Z}) \cdot (\overline{X} + Y + Z)}$$

Key Points

- Minterms and Maxterms are complementary sets for any given Boolean function: $F = \sum m_i = \prod M_j$, where $i \notin j$.

Mistakes to be avoided

- Avoid treating a maxterm expression as simply the literal-by-literal complemented version of a minterm expression. Remember their fundamental definitions:
  - SOP (Minterms) maps an input bit of 1 to (A) and 0 to $(\overline{A})$.
  - POS (Maxterms) maps an input bit of 0 to (A) and 1 to $(\overline{A})$.

Q2.15.2 MCQ 2015 1M Ans: A ● ● ○

The given canonical sum-of-products (SOP) expression is:

$$F(X,Y,Z) = \overline{X}Y\overline{Z} + X\overline{Y}\,\overline{Z} + XY\overline{Z} + XYZ$$

Identifying the minterms by converting each product term to its binary equivalent (1 for uncomplemented, 0 for complemented):

- $\overline{X}Y\overline{Z} \implies 010_2 = m_2$
- $X\overline{Y}\,\overline{Z} \implies 100_2 = m_4$
- $XY\overline{Z} \implies 110_2 = m_6$
- $XYZ \implies 111_2 = m_7$

Thus, $F(X,Y,Z) = \sum m(2,4,6,7)$.
The canonical product-of-sums (POS) form consists of the remaining maxterms not present in the minterm list:

$$F(X,Y,Z) = \prod M(0,1,3,5)$$

Converting these maxterm indices back to algebraic form (0 for uncomplemented, 1 for complemented):

- $M_0 = 000_2 \implies (X+Y+Z)$
- $M_1 = 001_2 \implies (X+Y+\overline{Z})$
- $M_3 = 011_2 \implies (X+\overline{Y}+\overline{Z})$
- $M_5 = 101_2 \implies (\overline{X}+Y+\overline{Z})$

Multiplying the maxterms yields:

$$F(X,Y,Z) = (X+Y+Z)(X+Y+\overline{Z})(X+\overline{Y}+\overline{Z})(\overline{X}+Y+\overline{Z})$$

$$\boxed{\text{(A) } (X+Y+Z)(X+Y+\overline{Z})(X+\overline{Y}+\overline{Z})(\overline{X}+Y+\overline{Z})}$$

Key Points

- A Boolean function can be uniquely defined by either its minterm list ($\sum m$) or its maxterm list ($\prod M$).
- The total number of minterms and maxterms for an n-variable function equals 2^n.

Mistakes to be avoided

- Avoid the common misconception that the POS form is simply the complemented version of the SOP form (or vice versa). They are actually two distinct, independent structural representations of the same underlying Boolean function (F).

Q2.14.1 MCQ 2014 2M Ans: D ● ● ○

The given Boolean function can be mapped directly onto a 4-variable K-map to determine its essential prime implicants (EPIs).
Plotting the minterms corresponding to each product term of $F(w,x,y,z)$:

- $wy \implies m_{10}, m_{11}, m_{14}, m_{15}$
- $xy \implies m_6, m_7, m_{14}, m_{15}$
- $\overline{w}xyz \implies m_7$

- $\overline{w}\,\overline{x}y \implies m_2, m_3$
- $xz \implies m_5, m_7, m_{13}, m_{15}$
- $\overline{x}\,\overline{y}\,\overline{z} \implies m_0, m_8$

Now marking implicants on the Kmap with the largest first rule,

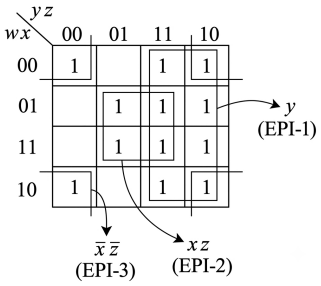


Fig. Solution Q2.14.1

Thus, the complete set of essential prime implicants is $\{y, xz, \overline{x}\,\overline{z}\}$.

(D) $y, xz, \overline{x}\,\overline{z}$

Key Points

- A Prime Implicant (PI) is a product term obtained by combining the maximum possible number of adjacent minterms in a K-map.
- An Essential Prime Implicant (EPI) is a prime implicant that contains at least one minterm (a "1") that cannot be covered by any other prime implicant.

Q2.14.2 MCQ 2014 1M Ans: D ●●○

For an n-variable Boolean function, the maximum possible number of prime implicants occurs when the minterms are arranged in an alternating, checkerboard-like pattern in the Karnaugh map (K-map). Like this-

	00	01	11	10
00	1		1	
01		1		1
11	1		1	
10		1		1

In such a configuration, no two 1s are adjacent to each other, meaning no group of size 2 or larger can be formed. Therefore, each individual cell containing a 1 constitutes an isolated single-cell prime implicant. The total number of cells in an n-variable K-map is 2^n. When the cells alternate between 1s and 0s to maximize isolation, exactly half of the total cells will contain 1s:

$$\text{Maximum Number of Prime Implicants} = \frac{2^n}{2} = 2^{n-1}$$

(D) $2^{(n-1)}$

Key Points

- **Prime Implicant (PI):** A product term obtained by combining the maximum possible number of adjacent squares in a K-map.
- **Worst-case structure:** The maximum number of ESSENTIAL PIs are achieved when every alternate cell is a minterm,

preventing any logical simplification.

Q2.12.1 MCQ 2012 1M Ans: B ● ● ○

Method 1

The truth table for the 2-bit comparator inputs $A = A_1A_0$ and $B = B_1B_0$ where the output $Y = 1$ for $A > B$ is shown below:

A_1	A_0	B_1	B_0	$Y = A > B$
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	1
1	0	0	1	1
1	0	1	0	0
1	0	1	1	0
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

Counting the number of combinations where $Y = 1$, we get a total of 6 combinations.

Method 2

For an n-bit magnitude comparator comparing two n-bit numbers A and B:

- Total number of input combinations = 2^{2n}
- Number of combinations for which $A = B = 2^n$
- Number of combinations for which $A > B$ is equal to the number of combinations for which $A < B$:

$$\text{Combinations for } (A > B) = \frac{2^{2n} - 2^n}{2}$$

Given $n = 2$ bits: Y

$$= \frac{2^{2(2)} - 2^{(2)}}{2} = \frac{16 - 4}{2} = 6$$

(B) 6

Q2.12.2 MCQ 2012 1M Ans: A ● ● ○

Given the 3-variable sum of products function:

$$f(X, Y, Z) = \sum m(2, 3, 4, 5)$$

The kmap can be plotted as follows-

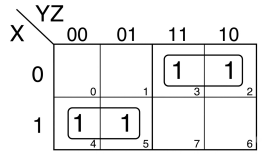


Fig. Solution Q2.12.2

Since these groups cannot be combined into larger groups, the prime implicants of the function are $\overline{X}\,Y$ and $X\overline{Y}$.

$$\boxed{\text{(A) } \overline{X}\,Y, X\overline{Y}}$$

Q2.07.1 MCQ 2007 2M Ans: D ●●○

The given Boolean expression in Sum of Products (SOP) form is:

$$Y = \overline{A}\,\overline{B}\,\overline{C}D + \overline{A}BC\overline{D} + A\overline{B}\,\overline{C}D + AB\overline{C}\,\overline{D}$$

Placing these minterms into a 4-variable K-map:

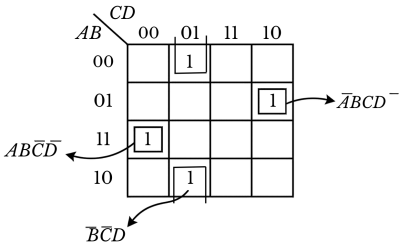


Fig. Solution Q2.07.1

From the K-map, we evaluate the grouping:

- Minterms m_1 ($\overline{A}\,\overline{B}\,\overline{C}D$) and m_9 ($A\overline{B}\,\overline{C}D$) form a 2-cell group (pair) at the column $CD = 01$. Combining them eliminates A:

$$\overline{B}\,\overline{C}D$$

- Minterms m_6 ($\overline{A}BC\overline{D}$) and m_{12} ($AB\overline{C}\,\overline{D}$) cannot be paired with any adjacent cells, so they remain as isolated single minterms:

$$\overline{A}BC\overline{D} \quad \text{and} \quad AB\overline{C}\,\overline{D}$$

Combining all minimized terms yields the final simplified expression:

$$Y = \overline{A}BC\overline{D} + \overline{B}\,\overline{C}D + AB\overline{C}\,\overline{D}$$

$$\boxed{\text{(D) } Y = \overline{A}BC\overline{D} + \overline{B}\,\overline{C}D + AB\overline{C}\,\overline{D}}$$

Key Points

- K-map Cell Grouping: In a Karnaugh map, combining a pair of adjacent 1s eliminates the single variable that changes state between those cells.
- Isolated Cells: Minterms that do not share any adjacent edge with another filled cell cannot be simplified and must be included in their complete literal form.

Mistakes to be avoided

- Do not forget to map the two edge groups together when checking for adjacent cells, as cells on opposite boundaries of the K-map wrap around and can often be combined to form a larger group.

Q2.06.1 MCQ 2006 1M Ans: A ●●○

To find the number of product terms in the minimized Sum-of-Products (SOP) expression, we group the 1s and useful don't-care states (d) in the given K-map:

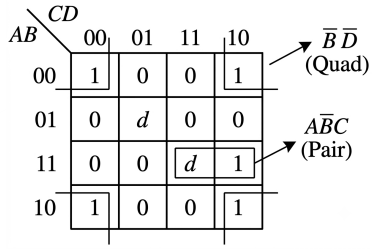


Fig. Solution Q2.06.1

The remaining don't-care state at $(01, 01)$ is left uncoupled as it does not help in expanding any group of 1s. Thus, the minimized SOP expression is:

$$Y = \overline{B}\,\overline{D} + ABC$$

The total number of minimized product terms is 2.

(A) 2

Key Points

- Don't-care states (d) should only be enclosed if they help in maximizing the group size (thereby reducing literals in a product term).

Q2.05.1 MCQ 2005 2M Ans: A ●●○

Input			Output	Minterms
A	B	C	f	m
0	0	0	0	m_0
0	0	1	0	m_1
0	1	0	0	m_2
0	1	1	1	$\boxed{m_3}$
1	0	0	0	m_4
1	0	1	0	m_5
1	1	0	1	$\boxed{m_6}$
1	1	1	0	m_7

Method 1

From the given truth table, the output f is high (1) for the minterms m_3 (011) and m_6 (110). Writing the function in Sum-of-Products (SOP) form:

$$f = \sum m(m_3, m_6)$$

$$f = \overline{A}BC + A\overline{B}C$$

Taking B common from both terms:

$$f = B(\overline{A}C + A\overline{C})$$

Using the Boolean identity $\overline{A}C + A\overline{C} = (A + C)(\overline{A} + \overline{C})$:

$$f = B(A + C)(\overline{A} + \overline{C})$$

Method 2

From the options we can see the final answer is in POS form thus considering the 0's from the truth table:

$$f = \prod M(0, 1, 2, 4, 5, 7)$$

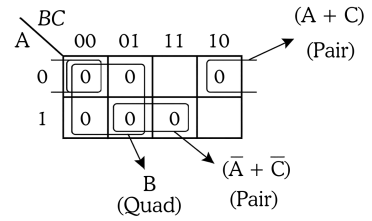


Fig. Solution Q2.05.1

Multiplying these simplified terms together gives the minimized POS expression:

$$\boxed{\text{(A) } B(A + C)(\overline{A} + \overline{C})}$$

Key Points

- To find the minimized expression in SOP form, 1's are considered, where a variable is uncomplemented for 1 and complemented for 0.
- To find the minimized expression in POS form, 0's are considered, where a variable is uncomplemented for 0 and complemented for 1.

Q2.04.1 MCQ 2004 2M Ans: D ● ● ○

Method 1

Given Boolean expression:

$$Y = AC + B\overline{C}$$

Converting the expression into its standard canonical Sum-of-Products (SOP) form by introducing the missing variables into each product term:

$$Y = A(B + \overline{B})C + (A + \overline{A})B\overline{C}$$

$$Y = ABC + A\overline{B}C + AB\overline{C} + \overline{A}B\overline{C}$$

Rearranging the terms to match the given options:

$$Y = ABC + \overline{A}B\overline{C} + AB\overline{C} + A\overline{B}C$$

Method 2

By checking each given option:

- **From option (A):**

$$Y = \overline{A}C + B\overline{C} + AC$$

$$Y = C(\overline{A} + A) + B\overline{C} = C + B\overline{C} \quad (\because \overline{A} + A = 1)$$

$$Y = C + B$$

Hence, option (A) is incorrect.

- **From option (B):**

$$Y = \overline{B}C + AC + B\overline{C} + \overline{A}C\overline{B}$$

$$Y = \overline{B}C(1 + \overline{A}) + B\overline{C} + AC = \overline{B}C + B\overline{C} + AC$$

Hence, option (B) is incorrect.

- **From option (C):**

$$Y = AC + B\overline{C} + \overline{B}C + ABC$$

$$Y = AC(1 + B) + B\overline{C} + \overline{B}C = AC + B\overline{C} + \overline{B}C$$

Hence, option (C) is incorrect.

- **From option (D):**

$$Y = ABC + \overline{A}B\overline{C} + AB\overline{C} + A\overline{B}C$$

$$Y = AC(B + \overline{B}) + B\overline{C}(A + \overline{A})$$

$$Y = AC + B\overline{C}$$

So, option (D) is correct.

$$\boxed{\text{(D) } ABC + \overline{A}B\overline{C} + AB\overline{C} + A\overline{B}C}$$

Key Points

- Missing variables are introduced into a product term using the identity $X + \overline{X} = 1$ without altering the validity of the logical term.

Q2.04.2 MCQ | 2004 | 2M | Ans: D | ●●○

From the given definition of the function $f(x, y)$, we write its minterms and maxterms:

$$f(x, y) = \sum m(0, 1, 3) = \prod M(2)$$

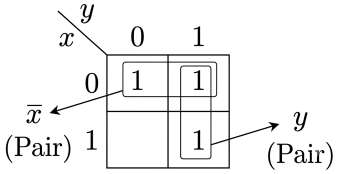


Figure 2.1: SOP simplification

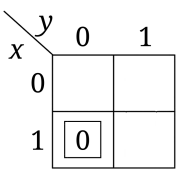


Figure 2.2: POS simplification

Thus, the simplified Boolean function is:

$$f = \overline{x} + y$$

Gate Realization:

Since complements are not directly available, we generate $\overline{x}$ by using a 2-input NOR gate as an inverter

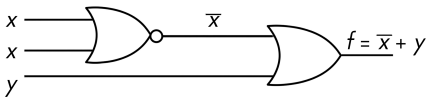


Fig. Solution Q2.04.2

Total gates required: $1 \times$ 2-input NOR gate + $1 \times$ 2-input OR gate $\implies$ Total minimum cost = 1 + 1 = 2 units

(D) 2 unit

Q2.03.1 MCQ 2003 2M Ans: A ● ● ●

To find the relations between the four-variable Boolean functions W, X, Y, and Z, we analyze their standard expressions or truth values.

We expand or map each function to determine its set of minterms:

(i) **Function W:**

$$W = R + \overline{P}Q + \overline{R}S$$

Representing the above equation in a kmap:

W \ RS / PQ	00	01	11	10
00		1	1	1
01	1	1	1	1
11		1	1	1
10		1	1	1

Fig. Solution Q2.03.1

(ii) **Function X:**

$$X = PQ\,\overline{R}\,\overline{S} + \overline{P}\,\overline{Q}\,\overline{R}\,\overline{S} + P\overline{Q}\,\overline{R}\,\overline{S}$$

Representing the above equation in a kmap:

X \ RS / PQ	00	01	11	10
00	1			
01				
11	1			
10	1			

Fig. Solution Q2.03.1

(iii) **Function Y:**

$$Y = RS + \overline{PR + P\overline{Q} + \overline{P}\,\overline{Q}} = RS + \overline{PR} \cdot \overline{P\overline{Q}} \cdot \overline{\overline{P}\,\overline{Q}}$$

$$Y = RS + (\overline{P} + \overline{R})(\overline{P} + Q)(P + Q) = RS + (\overline{P} + \overline{R})(\overline{P}Q + PQ + Q) = RS + (\overline{P} + \overline{R})(Q(\overline{P} + P + 1))$$

$$Y = RS + (\overline{P} + \overline{R})Q = RS + \overline{P}Q + Q\overline{R}$$

Representing the above equation in a kmap:

Y / PQ \ RS	00	01	11	10
00			1	
01	1	1	1	1
11	1	1	1	
10			1	

Fig. Solution Q2.03.1

(iv) **Function Z:**

$$Z = R + S + \overline{PQ + \overline{P}\,\overline{Q}\,\overline{R} + P\overline{Q}\,\overline{S}} = R + S + \overline{PQ} \cdot \overline{\overline{P}\,\overline{Q}\,\overline{R}} \cdot \overline{P\overline{Q}\,\overline{S}}$$

$$Z = R + S + (\overline{P} + \overline{Q}) \cdot (P + Q + R) \cdot (\overline{P} + Q + S) = R + S + (\overline{P}Q + \overline{P}R + P\overline{Q} + \overline{Q}R)(\overline{P} + Q + S)$$

$$Z = R + S + \overline{P}Q + \overline{P}Q + \overline{P}QS + \overline{P}R + \overline{P}QR + \overline{P}RS + P\overline{Q}S + \overline{P}\,\overline{Q}R + \overline{Q}RS$$

$$Z = R + S + \overline{P}Q(1 + S + R) + \overline{P}R(1 + S) + P\overline{Q}S + \overline{P}\,\overline{Q}R + \overline{Q}RS$$

$$Z = R + S + \overline{P}Q + \overline{P}R + P\overline{Q}S + \overline{P}\,\overline{Q}R + \overline{Q}RS = R(1 + \overline{P} + \overline{P}\,\overline{Q} + \overline{Q}S) + S(1 + P\overline{Q}) + \overline{P}Q$$

$$Z = R + S + \overline{P}Q$$

Representing the above equation in a kmap:

Z / PQ \ RS	00	01	11	10
00		1	1	1
01	1	1	1	1
11		1	1	1
10		1	1	1

Fig. Solution Q2.03.1

Comparing the kmaps:

- W and Z have identical kmaps, so $W = Z$.
- The complement of Z, contains the remaining minterms missing from X thus, $X = \overline{Z}$.

$$\boxed{\text{(A) } W = Z, X = \overline{Z}}$$

Q2.01.1 NAT 2001 Ans: * ● ○ ○

Given the 4-variable Boolean function $Y = f(S_3, S_2, S_1, S_0)$ with S_3 as MSB and S_0 as LSB:

$$Y = \sum m(1, 5, 6, 7, 11, 12, 13, 15) \text{ and } \overline{Y} = \sum m(0, 2, 3, 4, 8, 9, 10, 14)$$

as here $Y + \overline{Y} = 1$ thus there wil be **no Don't care terms** in the kmap.

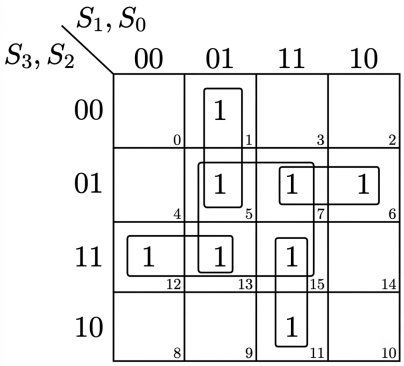


Fig. Solution Q2.01.1

Q2.01.2 NAT 2001 Ans: * ● ○ ○

To obtain the minimized sum-of-products (SOP) expression for $Y = \sum m(1, 5, 6, 7, 11, 12, 13, 15)$, we group the 1s in the Karnaugh map of the previous part. The prime implicants are:

- $\overline{S_3}\,\overline{S_1}S_0$: minterms 1 and 5 (essential, because only it covers minterm 1).
- $\overline{S_3}S_2S_1$: minterms 6 and 7 (essential, because only it covers minterm 6).
- $S_3S_2\overline{S_1}$: minterms 12 and 13 (essential, because only it covers minterm 12).
- $S_3S_1S_0$: minterms 11 and 15 (essential, because only it covers minterm 11).
- S_2S_0: minterms 5, 7, 13 and 15.

The four essential prime implicants together already cover all eight minterms $1, 5, 6, 7, 11, 12, 13, 15$. The quad S_2S_0 is a prime implicant, but every minterm it covers (5, 7, 13, 15) is already covered by the essential prime implicants, so it is redundant and must not be included in the minimum expression.
Combining the essential prime implicants gives the minimum SOP expression with four terms:

$$\boxed{Y = \overline{S_3}\,\overline{S_1}S_0 + \overline{S_3}S_2S_1 + S_3S_2\overline{S_1} + S_3S_1S_0}$$

Mistakes to be avoided

- Do not add the largest group (S_2S_0) automatically. First select the essential prime implicants; include another prime implicant only if some minterm is still uncovered.

Q2.01.3 NAT 2001 Ans: * ● ● ●

We can rewrite this structurally by grouping variables to match the sequential layout of the digital logic blocks:

$$Y = S_2(S_0 + \overline{S_3}S_1 + S_3\overline{S_1}) + \overline{S_1}S_0\overline{S_3} + S_1S_0S_3$$

$$Y = S_2(S_0 + S_3 \oplus S_1) + S_0(S_3 \oplus S_1)'$$

Let $X = S_3 \oplus S_1$ be the intermediate signal from Box 1. Then:

$$Y = S_2(S_0 + X) + S_0\overline{X} = S_2S_0 + S_2X + S_0\overline{X}$$

According to the **Consensus Theorem**, the term S_0S_2 is redundant because its existence is covered by the other two terms (if $X = 1$, the value is determined by S_2, and if $X = 0$, it is determined by S_0).

$$Y = S_2X + S_0\overline{X}$$

Thus the diagramatic representation will be:

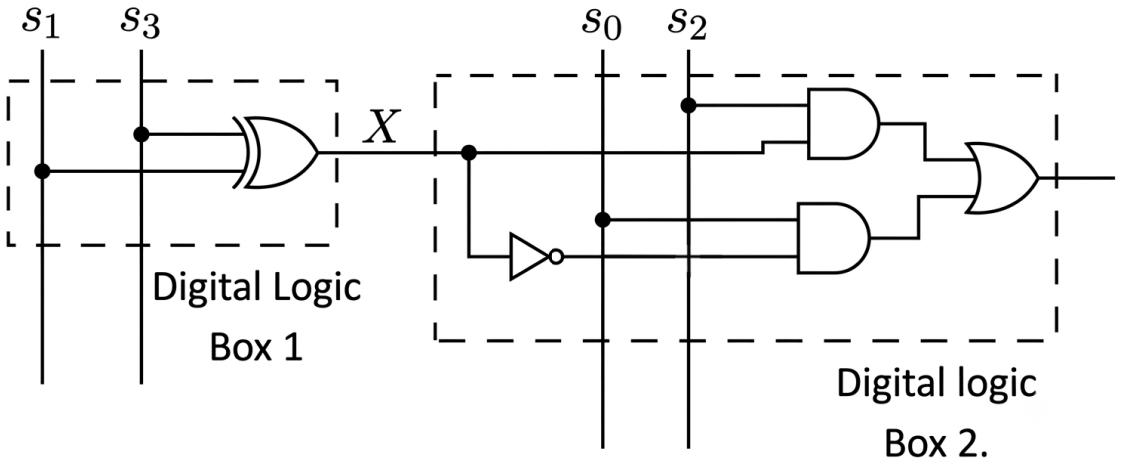


Fig. Solution Q2.01.3

Now implementing the above logic using 4 NAND gates only for each Digital logic box-

- **Digital Logic Box 1:** XOR is a standard logic gate which can be represented as:

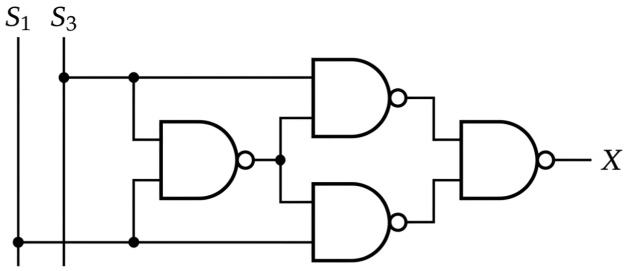


Fig. Solution Q2.01.3

- **Digital Logic Box 2:**

1. Converting to NAND Logic

To implement this expression using NAND gates, apply a double complement and use De Morgan's Law:

$$Y = \overline{\overline{S_2X + S_0\overline{X}}}$$

$$Y = \overline{\overline{S_2X} \cdot \overline{S_0\overline{X}}}$$

2. Gate-by-Gate Allocation

This layout can be wired using exactly 4 two-input NAND gates:

– **Gate 1 (Inverter):** Creates $\overline{X}$ by tying both inputs of the first NAND gate together.

$$\text{Output}_1 = \overline{X}$$

– **Gate 2 (NAND):** Combines S_2 and X.

$$\text{Output}_2 = \overline{S_2 \cdot X}$$

– **Gate 3 (NAND):** Combines S_0 and the inverted $\overline{X}$ from Gate 1.

$$\text{Output}_3 = \overline{S_0 \cdot \overline{X}}$$

– **Gate 4 (Output NAND):** Combines the outputs of Gate 2 and Gate 3.

$$\text{Final Output} = \overline{\text{Output}_2 \cdot \text{Output}_3} = S_2X + S_0\overline{X}$$

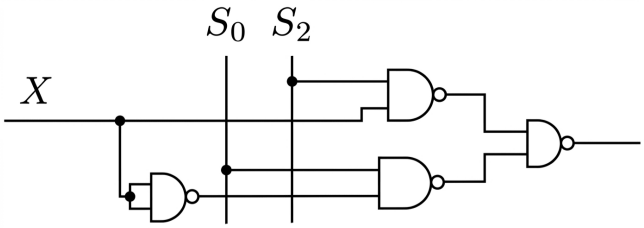


Fig. Solution Q2.01.3

Q2.00.1 NAT 2000 Ans: * ●●○

Let the condition of the pumps be represented by boolean variables x, y, z, where a value of 1 indicates the pump is **ON** and 0 indicates it is **OFF** (**failed**).
The indicator panel LED glows ($F = 1$) when a **majority of the pumps fail** (i.e., when two or more pumps have a value of 0). Conversely, the problem statement specifies that the output must be zero ($F = 0$) under this condition.
Let us map the truth values for the function $F(x, y, z)$:

x	y	z	F
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	0

Evaluating using a kmap:

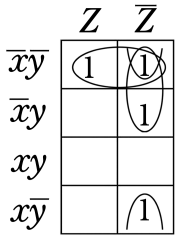


Fig. Solution Q2.00.1

The minimal SOP expression is:

$$\boxed{F = \overline{x}\,\overline{y} + \overline{y}\,\overline{z} + \overline{z}\,\overline{x}}$$

Q2.00.2 NAT 2000 Ans: * ● ● ●

Given specifications for driving the LED:

- When a majority of pumps fail: Point $P = 0\,\text{V}$ (Logic 0).
- Otherwise: Point $P = 5\,\text{V}$ (Logic 1).
- LED forward current: $I_{\text{LED}} = 10\,\text{mA} = 10 \times 10^{-3}\,\text{A}$.
- LED forward voltage drop: $V_{\text{LED}} = 1\,\text{V}$.
- Available DC supply voltage: $V_{CC} = 5\,\text{V}$.

When a majority of the pumps fail, $P = 0\,\text{V}$, and the LED must turn **ON** (glow). This dictates a **current-sinking configuration**, where the LED is connected between the 5 V source and the output terminal P.
Applying Kirchhoff's Voltage Law (KVL) through the LED path when $P = 0\,\text{V}$:

$$V_{CC} - V_{\text{LED}} - I_{\text{LED}} \cdot R = V_P$$

$$5\,\text{V} - 1\,\text{V} - (10 \times 10^{-3}\,\text{A}) \cdot R = 0\,\text{V}$$

$$4\,\text{V} = 10^{-2} \cdot R$$

$$R = \frac{4}{10^{-2}} = 400\,\Omega$$

$$\boxed{R = 400\,\Omega \text{ in current-sinking mode}}$$

Q2.99.1 NAT 1999 Ans: * ● ○ ○

Based on the given problem statement, the truth table of the function $F(A, B, C, D)$ is defined by the following states:

- **ON States (Minterms):** $\sum m(0, 2, 5, 6, 13, 14)$ corresponding to '0000', '0010', '0101', '0110', '1101', and '1110'.
- **Don't Care States:** $\sum d(8, 9)$ corresponding to '1000' and '1001'.

To minimize F, we map these states onto a 4-variable Karnaugh map.

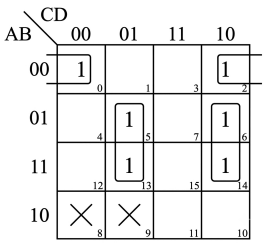


Fig. Solution Q2.99.1

From the K-map simplification, the minimized expression for F is obtained as:

$$\boxed{F = \overline{A}\,\overline{B}\,\overline{D} + B\overline{C}D + BC\overline{D}}$$

Q2.99.2 NAT 1999 Ans: * ● ○ ○

To realize the minimized expression $F = \overline{A}\,\overline{B}\,\overline{D} + B\overline{C}D + BC\overline{D}$ using 3-input NAND gates, we manipulate the expression using De Morgan's Law:

$$F = \overline{\overline{A}\,\overline{B}\,\overline{D} + B\overline{C}D + BC\overline{D}}$$

$$F = \overline{\left(\overline{\overline{A}\,\overline{B}\,\overline{D}}\right) \cdot \left(\overline{B\overline{C}D}\right) \cdot \left(\overline{BC\overline{D}}\right)}$$

This form requires:

1. Three 3-input NAND gates to generate individual terms:
   - $\overline{\overline{A}\,\overline{B}\,\overline{D}}$
   - $\overline{B\overline{C}D}$
   - $\overline{BC\overline{D}}$
2. One 3-input NAND gate to combine these three terms into the final output F.

Thus, a minimum of 4 gates are required.

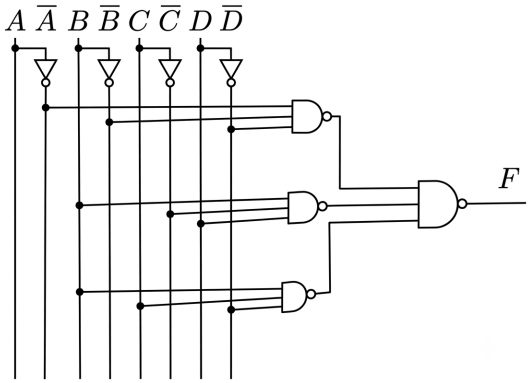


Fig. Solution Q2.99.2

Q2.99.3 MCQ 1999 2M Ans: A ● ○ ○

Given logical expression:

$$y = \overline{A}\,\overline{B}\,\overline{C} + \overline{A}B\overline{C} + \overline{A}BC + AB\overline{C}$$

Method 1

Using the Boolean identity $X + X = X$, we can replicate product terms to simplify the expression efficiently:

$$y = \overline{A}\,\overline{B}\,\overline{C} + \overline{A}B\overline{C} + \overline{A}B\overline{C} + \overline{A}BC + \overline{A}B\overline{C} + AB\overline{C}$$

Rearranging and grouping common factors:

$$y = \overline{A}\,\overline{C}(\overline{B} + B) + \overline{A}B(\overline{C} + C) + B\overline{C}(\overline{A} + A)$$

Since $X + \overline{X} = 1$:

$$y = \overline{A}\,\overline{C}(1) + \overline{A}B(1) + B\overline{C}(1)$$

$$y = \overline{A}\,\overline{C} + \overline{A}B + B\overline{C}$$

Method 2

The given Sum of Products (SOP) expression can be directly mapped to its corresponding minterms:

$$y = \Sigma m(000, 010, 011, 110) = \Sigma m(0, 2, 3, 6)$$

Plotting these values into a 3-variable K-map:

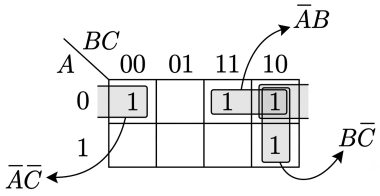


Fig. Solution Q2.99.3

Summing these minimized pairs gives:

$$y = \bar{A}\,\bar{C} + \bar{A}B + B\bar{C}$$

$$\boxed{\text{(A) } \bar{A}\,\bar{C} + B\bar{C} + \bar{A}B}$$

Key Points

- **Idempotent Law:** $X + X = X$ allows a minterm to be paired multiple times in algebraic simplification.
- **K-map Adjacency:** Cells at the extreme ends of a row (m_0 and m_2 in a 3-variable map) are logically adjacent and can form a valid grouping.

Q2.98.1 MCQ 1998 2M Ans: A ●●○

To determine the number of Essential Prime Implicants (EPIs), we first identify all the Prime Implicants (PIs) by grouping the minterms (1s) in the given Karnaugh map.

- **Prime Implicant (PI):** A product term obtained by combining the maximum possible number of adjacent minterms in the K-map.
- **Essential Prime Implicant (EPI):** A prime implicant that contains at least one minterm (1) that cannot be covered by any other prime implicant.

Let's find the groups from the given K-map:

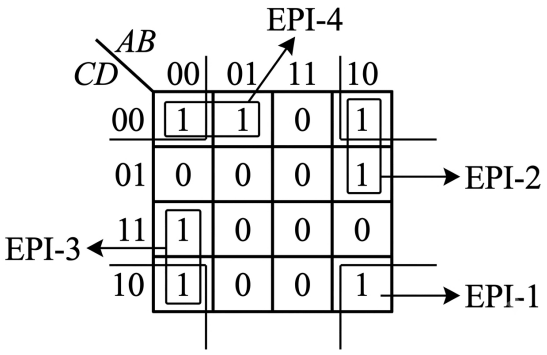


Fig. Solution Q2.98.1

Since all four pairs contain at least one unique minterm, they are all Essential Prime Implicants.

Therefore, the total number of Essential Prime Implicants is 4.

(A) 4

Q2.97.1 MCQ 1997 2M Ans: B ● ○ ○

Method 1

Using the distributive law of Boolean algebra : $X + (Y \cdot Z) = (X + Y) \cdot (X + Z)$
By substituting $X = A$, $Y = B$, and $Z = C$, we directly get:

$$A + BC = (A + B) \cdot (A + C)$$

Thus, the given Boolean function $A + BC$ is equivalent to option (B).

Method 2

We can verify the options by simplifying each expression:

- **Option (A):** $AB + BC = B(A + C) \neq A + BC$
- **Option (B):** Expand $(A + B) \cdot (A + C)$ using distribution:

$$(A + B) \cdot (A + C) = A \cdot A + AC + AB + BC$$

$$= A(1 + C + B) + BC = A(1) + BC = A + BC$$

- **Option (C):** $\overline{A}B + A\overline{B}C \neq A + BC$
- **Option (D):** $(A + C) \cdot B = AB + BC \neq A + BC$

(B) $(A + B) \cdot (A + C)$

Key Points

- **Distributive Law:** $A + BC = (A + B)(A + C)$.

Mistakes to be avoided

- Unlike mathematical algebra, in Boolean logic, addition distributes over multiplication $(A + BC = (A + B)(A + C))$ just as multiplication distributes over addition $(A \cdot (B + C) = (AB + AC))$.

SOLUTIONS

III

Number Systems

Topics

1. Introduction to Number Systems
2. Arithmetic Operations on Numbers
3. Binary Codes and Their Operations
4. Signed Number Representations
5. Complement Arithmetic
6. Floating-Point Representation

Unit Snapshot*

Stream: EC	**Questions:** 19	**MCQ:** 16	**MSQ:** 1	**NAT:** 2
1M: 10	**2M:** 9	**Descriptive:** 0	**Avg Weightage** ~1M	**Range:** 0M(2014)–2M(2026)

**See preface for how Avg Weightage and Range are calculated.*

Q3.26.1 MCQ 2026 2M Ans: B ●○○

The problem asks for the 10's complement of the decimal value $(47)_{10}$. This can be determined by first finding the $(r-1)$'s complement and then adding 1.

Step 1: Find the 9's complement Subtract each digit of the number from 9:

$$99 - 47 = 52 \quad (9\text{'s complement})$$

Step 2: Find the 10's complement Add 1 to the resulting 9's complement:

$$52 + 1 = 53$$

Check: for a two-digit number, the 10's complement is also $10^2 - 47 = 100 - 47 = 53$.
Thus, the 10's complement of $(47)_{10}$ is 53.

(B) 53

Key Points

- For a base r system, the r's complement is defined as:

$$r\text{'s complement} = (r-1)\text{'s complement} + 1$$

- For decimal ($r = 10$), the 10's complement is (9's complement + 1).

Mistakes to be avoided

- Do not forget to add the 1 after finding the $(r-1)$'s complement to reach the r's complement.

Q3.24.1 NAT 2024 1M Ans: 11 ●●○

Given a quadratic equation in a number system of base r:

$$x^2 - 12x + 37 = 0$$

It is provided that $x = 8$ is one of the solutions. We need to find the value of the base r.
Any number with base r can be represented as:

$$r^0 \times (\text{bit } 1)_{\text{LSB}} + r^1 \times (\text{bit } 2) + r^2 \times (\text{bit } 3) \ldots$$

Thus, we will replace the numbers in the given quadratic as:

$$(12)_r = r \times 1 + 2$$

$$(37)_r = r \times 3 + 7$$

Method 1

Let the roots of the equation be α and β, where $\alpha = 8$.
Using the properties of roots: **Sum of roots:**

$$\alpha + \beta = r + 2$$

$$8 + \beta = r + 2 \implies \beta = r - 6 \quad \ldots (\text{i})$$

Product of roots:

$$\alpha\beta = 3r + 7$$

$$8\beta = 3r + 7 \quad \ldots (\text{ii})$$

Substituting equation (i) into (ii):

$$8(r-6) = 3r + 7$$

$$8r - 48 = 3r + 7$$

$$5r = 55 \implies r = 11$$

Method 2

Since $x = 8$ is a root, it must satisfy the quadratic equation:

$$x^2 - (r+2)x + (3r+7) = 0$$

Substituting $x = 8$:

$$(8)^2 - (r+2)(8) + (3r+7) = 0$$

$$64 - 8r - 16 + 3r + 7 = 0$$

$$55 - 5r = 0$$

$$5r = 55 \implies r = 11$$

$$\boxed{11}$$

Key Points

- Any number with base r can be represented as:

$$r^0 \times (\text{bit } 1)_{\text{LSB}} + r^1 \times (\text{bit } 2) + r^2 \times (\text{bit } 3) \ldots$$

- For a quadratic equation $ax^2 + bx + c = 0$, the sum of roots is $-b/a$ and the product of roots is c/a.

Mistakes to be avoided

- When substituting $x = 8$ into the equation, ensure that the coefficients $(12)_r$ and $(37)_r$ are correctly expanded in terms of r according to their positional weights.

Q3.21.1 MCQ | 2021 | 1M | Ans: B | ● ○ ○

Given equation:

$$(1235)_x = (3033)_y$$

where x and y indicate the bases of the corresponding numbers. Converting both numbers into their decimal equivalent expressions:

$$1 \cdot x^3 + 2 \cdot x^2 + 3 \cdot x^1 + 5 \cdot x^0 = 3 \cdot y^3 + 0 \cdot y^2 + 3 \cdot y^1 + 3 \cdot y^0$$

$$x^3 + 2x^2 + 3x + 5 = 3y^3 + 3y + 3$$

We have to evaluate each option by trial and error to find the correct option.

- **Testing Option (A):** $x = 7$ and $y = 5$

$$\text{LHS} = (7)^3 + 2(7)^2 + 3(7) + 5 = 343 + 98 + 21 + 5 = 467$$

$$\text{RHS} = 3(5)^3 + 3(5) + 3 = 3(125) + 15 + 3 = 375 + 15 + 3 = 393$$

Since LHS $\neq$ RHS, Option (A) is incorrect.

- **Testing Option (B):** $x = 8$ and $y = 6$

$$\text{LHS} = (8)^3 + 2(8)^2 + 3(8) + 5 = 512 + 128 + 24 + 5 = 669$$

$$\text{RHS} = 3(6)^3 + 3(6) + 3 = 3(216) + 18 + 3 = 648 + 18 + 3 = 669$$

Since LHS = RHS, Option (B) is correct.

- **Testing Option (C):** $x = 6$ and $y = 4$

$$\text{LHS} = (6)^3 + 2(6)^2 + 3(6) + 5 = 216 + 72 + 18 + 5 = 311$$

$$\text{RHS} = 3(4)^3 + 3(4) + 3 = 3(64) + 12 + 3 = 192 + 12 + 3 = 207$$

Since LHS $\neq$ RHS, Option (C) is incorrect.

- **Testing Option (D):** $x = 9$ and $y = 7$

$$\text{LHS} = (9)^3 + 2(9)^2 + 3(9) + 5 = 729 + 162 + 27 + 5 = 923$$

$$\text{RHS} = 3(7)^3 + 3(7) + 3 = 3(343) + 21 + 3 = 1029 + 21 + 3 = 1053$$

Since LHS $\neq$ RHS, Option (D) is incorrect.

Hence, the correct option is (B).

(B) $x = 8$ and $y = 6$

Q3.20.1 MCQ 2020 2M Ans: A ●●○

Given: 4-bit binary number $(1100)_2$. Since MSB = 1, it represents a negative number in all three systems.

- **Signed Magnitude (P):** MSB = 1 (negative), remaining bits 100 = 4.

$$P = -4$$

- **1's Complement (Q):** Keep MSB = 1. Take 1's complement of magnitude bits: 100 $\rightarrow$ 011 = 3.

$$Q = -3$$

- **2's Complement (R):** Keep MSB = 1. Take 2's complement of magnitude bits: 100 $\rightarrow$ 011 + 1 = 100 = 4.

$$R = -4$$

Sum of the decimal integers:

$$P + Q + R = -4 + (-3) + (-4) = -11$$

To represent -11 in 6-bit 2's complement form:

$$+11 = 001011$$

$$\text{1's complement of } +11 = 110100$$

$$\text{2's complement of } +11 = 110100 + 1 = 110101$$

Hence, the correct option is (A).

(A) 110101

Key Points

- In signed number systems, MSB = 0 indicates a positive number, and MSB = 1 indicates a negative number.
- To find the magnitude of a negative number in complement systems, complement the magnitude bits while keeping the sign bit intact.

Q3.14.1 NAT 2014 1M Ans: 3.9 to 4.1 ●●○

The given decimal number is 1856357 (7 digits). In packed BCD (Binary Coded Decimal):

- Each decimal digit is represented using 4 bits.
- Two decimal digits are stored per byte (8 bits).

$$\text{Total bits required} = 7 \times 4 = 28 \text{ bits}$$

$$\text{Number of bytes} = \left\lceil \frac{28}{8} \right\rceil = \lceil 3.5 \rceil = 4 \text{ bytes}$$

$$\boxed{4}$$

Key Points

- Packed BCD packs two decimal digits into one 8-bit byte.
- Memory allocation must be rounded up to the nearest whole byte.

Q3.08.1 MCQ 2008 2M Ans: B ●●○

Method 1

Given signed 2's complement numbers:

$$P = 11101101, \quad Q = 11100110$$

Since the Most Significant Bit (MSB) for both numbers is 1, both P and Q are negative numbers.
To find their decimal values, take the 2's complement to determine the magnitude:

- Magnitude of P = 2's complement of $11101101 = 00010011_2 = 19 \implies P = -19$
- Magnitude of Q = 2's complement of $11100110 = 00011010_2 = 26 \implies Q = -26$

Performing the subtraction:

$$P - Q = -19 - (-26) = +7$$

Converting +7 back to an 8-bit signed representation (positive numbers remain unchanged in signed 2's complement form):

$$+7 = 00000111$$

Method 2

Subtraction can be performed directly via 2's complement addition:

$$P - Q = P + (-Q)$$

Where $-Q$ is the 2's complement of Q:

$$-Q = \text{2's complement of } 11100110 = 00011010$$

Adding P and $-Q$:

P :		1	1	1	0	1	1	0	1
$-Q$:		0	0	0	1	1	0	1	0
Sum :	(1)	0	0	0	0	0	1	1	1

Discarding the carry-out bit (1) beyond the 8-bit word, as in ordinary 2's complement arithmetic (this is not the end-around carry of 1's complement arithmetic), yields:

$$P - Q = 00000111$$

(B) 00000111

Key Points

- Short trick to find 2's complement: Scanning a binary number from right to left, keep all bits up to and including the first '1' exactly as they are, then invert (complement) every remaining bit to the left.
  Example: To find 2's Complement of 0110010010000:

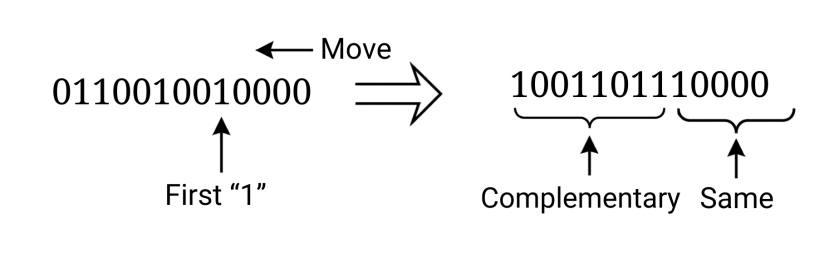


Fig. Solution Q3.08.1

Q3.07.1 MCQ 2007 1M Ans: C

Method 1

Given two 5-bit numbers in signed 2's complement format:

$$X = 01110, \quad Y = 11001$$

Performing direct binary addition of the two numbers:

X :		0	1	1	1	0
$+Y$:		1	1	0	0	1
Sum :	(1)	0	0	1	1	1

In 2's complement addition, any carry-out from the Most Significant Bit (MSB) is ignored (discarded):

$$X + Y = 00111$$

To express the final result using 6 bits, extend the sign bit (MSB) to the left by one position. Since the MSB is 0, we copy 0:

$$X + Y = 000111$$

Method 2

Alternatively, convert the numbers to their decimal equivalents first:

- $X = 01110 \rightarrow$ MSB is 0 $\implies$ Positive number.

$$X = +(1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0) = +14$$

- $Y = 11001 \rightarrow$ MSB is 1 $\implies$ Negative number.

$$\text{Magnitude} = 2\text{'s complement of } 11001 = 00111_2 = 7 \implies Y = -7$$

Evaluating the sum in decimal:

$$X + Y = +14 + (-7) = +7$$

Converting +7 into a 5-bit signed binary representation:

$$+7 = 00111_2$$

Extending the result to a 6-bit representation via sign extension (copying the MSB):

$$X + Y = 000111_2$$

(C) 000111

Key Points

- Sign Extension Rule: To represent a signed 2's complement number of N-bits using $(M + N)$ bits, replicate the sign bit (MSB) M times to the left. The numerical value remains completely unaltered.

	1101	represents −3 using 4 bits
(i)	11101	represents −3 using 5 bits
	111111101	represents −3 using 9 bits
	0101	represents +5 using 4 bits
(ii)	00101	represents +5 using 5 bits
	000000101	represents +5 using 9 bits

- Carry Handling: In signed 2's complement arithmetic addition, an overflow carry generated beyond the MSB position is strictly discarded.

PrepFusion

Q3.06.1 MCQ | 2006 | 2M | Ans: D | ● ● ○

Given that in the Binary Coded Pentary (BCP) system, each individual digit of a base-5 (pentary) number is encoded using its corresponding 3-bit binary representation.
We are given the BCP code:

100010011001

To convert this back into a base-5 number, we partition the given binary string into independent groups of 3 bits starting from left to right (or right to left, since the total number of bits is 12, which is perfectly divisible by 3):

$$\underbrace{100}_{4}\ \underbrace{010}_{2}\ \underbrace{011}_{3}\ \underbrace{001}_{1}$$

Assembling the converted digits gives the base-5 number:

$$(4231)_5$$

(D) 4231

Key Points

- **Significance of "Pentary" (Base-5):** The term "pentary" indicates that the valid digits in this numbering system range only from 0 to 4. Since a minimum of 3 bits is required to uniquely represent the highest digit ($4 = 100_2$), a 3-bit grouping scheme is defined.
- The 3-bit codes for digits 5 ($(101)_2$), 6 ($(110)_2$), and 7 ($(111)_2$) are unused/invalid states in this specific BCP system.

Q3.05.1 MCQ 2005 1M Ans: B ●○○

We convert the decimal number $(43)_{10}$ into its Hexadecimal and Binary Coded Decimal (BCD) equivalents.

(i) **Decimal to Hexadecimal:** To convert $(43)_{10}$ to base-16, we perform successive division by 16:

16	43		
	2	11 $\Longrightarrow$ B (Remainder)	↑ Reading bottom to top
	0	2 (Remainder)	

$$(43)_{10} = (2B)_{16}$$

(ii) **Decimal to BCD:** In the BCD (8421) system, each individual digit of the decimal number is converted independently into its equivalent 4-bit binary code:

$$\underbrace{4}_{0100} \quad \underbrace{3}_{0011}$$

$$(43)_{10} = (0100\ 0011)_{BCD}$$

Thus, the corresponding values are $(2B)_{16}$ and $(0100\ 0011)_{BCD}$ respectively.

(B) 2B, 0100 0011

Key Points

- In hexadecimal representation, standard decimal remainders from 10 to 15 map to letters A through F (where 11 = B).
- BCD is a digital encoding scheme where each decimal digit is treated separately and structured as a fixed 4-bit nibble, whereas regular binary conversion represents the entire weight of the value.

Q3.04.1 MCQ 2004 2M Ans: C ●○○

We need to find the decimal values of the given signed numbers represented in 2's complement form: 11001, 1001, and 111001.

Method 1

For any signed binary number in 2's complement representation, if the Most Significant Bit (MSB) is 1, the number is negative. To find its decimal magnitude, we take its 2's complement again (since the 2's complement of a 2's complement number returns its absolute original magnitude).

(i) **For 11001:**

- MSB is 1 $\Longrightarrow$ The number is negative.
- 1's complement of 11001 = 00110

- 2's complement = $00110 + 1 = 00111_2 = (7)_{10}$
- Therefore, the value is -7.

(ii) **For 1001:**

- MSB is 1 $\implies$ The number is negative.
- 1's complement of 1001 = 0110
- 2's complement = $0110 + 1 = 0111_2 = (7)_{10}$
- Therefore, the value is -7.

(iii) **For 111001:**

- MSB is 1 $\implies$ The number is negative.
- 1's complement of 111001 = 000110
- 2's complement = $000110 + 1 = 000111_2 = (7)_{10}$
- Therefore, the value is -7.

Method 2

We can use the direct positional weight formula for 2's complement numbers. For an n-bit number $b_{n-1}b_{n-2}\ldots b_0$, the value is:

$$\text{Value} = -b_{n-1}\cdot 2^{n-1} + \sum_{i=0}^{n-2} b_i \cdot 2^i$$

- **For 1001** ($n = 4$):

$$\text{Value} = -1\cdot 2^3 + 0\cdot 2^2 + 0\cdot 2^1 + 1\cdot 2^0 = -8 + 1 = -7$$

- **For 11001** ($n = 5$):

$$\text{Value} = -1\cdot 2^4 + 1\cdot 2^3 + 0\cdot 2^2 + 0\cdot 2^1 + 1\cdot 2^0 = -16 + 8 + 1 = -7$$

- **For 111001** ($n = 6$):

$$\text{Value} = -1\cdot 2^5 + 1\cdot 2^4 + 1\cdot 2^3 + 0\cdot 2^2 + 0\cdot 2^1 + 1\cdot 2^0 = -32 + 16 + 8 + 1 = -7$$

(C) -7, -7 and -7 respectively

Key Points

- **Sign Extension Property:** Replicating the sign bit (MSB) of a signed 2's complement number to the left does not alter its numeric decimal value.
- Here, 11001 and 111001 are simply the 5-bit and 6-bit sign-extended versions of the core 4-bit negative number 1001.

Mistakes to be avoided

- Do not interpret the binary strings as unsigned numbers, which would lead to the incorrect positive values 25, 9, and 57 matching option (A).

Q3.04.2 MCQ 2004 1M Ans: A ● ○ ○

We need to find the range of signed decimal numbers that can be represented by a 6-bit 1's complement representation.
For an n-bit signed binary number represented in 1's complement form, the range of valid integer values is given by the formula:

$$\text{Range} = -(2^{n-1} - 1) \text{ to } +(2^{n-1} - 1)$$

Given that the number of bits $n = 6$:

$$\text{Minimum Value} = -(2^{6-1} - 1) = -(2^5 - 1) = -(32 - 1) = -31$$
$$\text{Maximum Value} = +(2^{6-1} - 1) = +(2^5 - 1) = +(32 - 1) = +31$$

Thus, the range of signed decimal numbers is -31 to $+31$.

$$\boxed{\text{(A) } -31 \text{ to } +31}$$

Key Points

Here is a comprehensive summary of the representation parameters for various n-bit binary numbering systems:

Representation System	Minimum Value	Maximum Value	Total Distinct Values
Unsigned Binary	0	$2^n - 1$	2^n
Sign-Magnitude	$-(2^{n-1} - 1)$	$+(2^{n-1} - 1)$	$2 \times (2^{n-1} - 1) + 1 = 2^n - 1$
1's Complement	$-(2^{n-1} - 1)$	$+(2^{n-1} - 1)$	$2 \times (2^{n-1} - 1) + 1 = 2^n - 1$
2's Complement	-2^{n-1}	$+(2^{n-1} - 1)$	2^n

Mistakes to be avoided

- Do not use the 2's complement range formula, which yields -32 to $+31$ (matching option D). Remember that 1's complement has two representations for zero (+0 and −0), reducing its total absolute numerical reach by 1.

Q3.03.1 MCQ 2003 2M Ans: D ● ○ ○

Label the inputs $X_3X_2X_1X_0$ (with X_3 the MSB) and the outputs $Y_3Y_2Y_1Y_0$. Analyzing the circuit in the question from the MSB side:

(i) **For Output Y_3 (MSB):** The input X_3 passes straight down directly to the output:

$$Y_3 = X_3$$

(ii) **For Output Y_2:** The line goes through an XOR gate with inputs X_3 and X_2:

$$Y_2 = X_3 \oplus X_2$$

(iii) **Intermediate signals for the next stages:**

- The first gate at the top (inputs X_3 and X_2) is also an XOR gate, so its output is:

$$P = X_3 \oplus X_2$$

- The output of the first AND gate is $Q = P \cdot Y_2$. Since P and Y_2 are the same signal:

$$Q = (X_3 \oplus X_2)(X_3 \oplus X_2) = X_3 \oplus X_2$$

(iv) **For Output Y_1:** The line passes through an XOR gate with inputs X_1 and Q:

$$Y_1 = X_1 \oplus Q = X_3 \oplus X_2 \oplus X_1$$

(v) **Remaining intermediate signals:**

- The output of the OR gate (inputs P and X_1) is:

$$R = P + X_1 = (X_3 \oplus X_2) + X_1$$

- The output of the second AND gate is $S = R \cdot Y_1$. Using the boolean identity $(A + B)(A \oplus B) = A \oplus B$ with $A = X_3 \oplus X_2$ and $B = X_1$:

$$S = \big((X_3 \oplus X_2) + X_1\big)(X_3 \oplus X_2 \oplus X_1) = X_3 \oplus X_2 \oplus X_1$$

(vi) **For Output Y_0 (LSB):** The line passes through the final XOR gate with inputs X_0 and S:

$$Y_0 = X_0 \oplus S = X_3 \oplus X_2 \oplus X_1 \oplus X_0$$

Summarizing the final output equations:

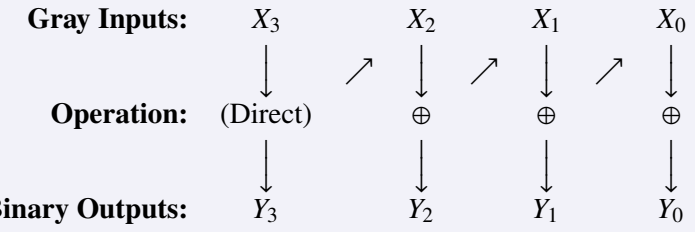
$$\begin{aligned} Y_3 &= X_3 \\ Y_2 &= X_3 \oplus X_2 \\ Y_1 &= X_3 \oplus X_2 \oplus X_1 \\ Y_0 &= X_3 \oplus X_2 \oplus X_1 \oplus X_0 \end{aligned}$$

These express the standard equations for a **Gray to Binary code converter**.

(D) Gray to Binary code

Key Points

- **Gray to Binary Conversion Flow Diagram:**



- **Proof of the Identity $(A + B)(A \oplus B) = A \oplus B$:** Expanding the XOR term algebraically:

$$\begin{aligned} \text{LHS} &= (A + B)(A\overline{B} + \overline{A}B) \\ &= A(A\overline{B} + \overline{A}B) + B(A\overline{B} + \overline{A}B) \\ &= AA\overline{B} + A\overline{A}B + BA\overline{B} + B\overline{A}B \end{aligned}$$

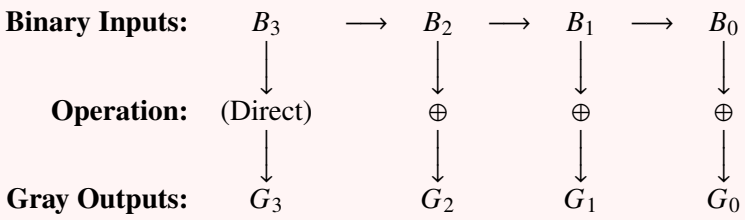
Since $AA = A$, $BB = B$, and $A\overline{A} = B\overline{B} = 0$:

$$\begin{aligned} \text{LHS} &= A\overline{B} + 0 + 0 + \overline{A}B \\ &= A\overline{B} + \overline{A}B = A \oplus B = \text{RHS} \end{aligned}$$

Alternatively, since $A \oplus B$ is only true when exactly one input is 1, it implicitly guarantees that the inclusive OR condition $(A + B)$ is always satisfied simultaneously.

Mistakes to be avoided

- **Directional Misinterpretation:** Do not confuse this circuit with a Binary to Gray converter.



Q3.02.1 MCQ | 2002 | 1M | Ans: D | ●●○

We need to find the decimal equivalent of the 4-bit binary number 1000 given in 2's complement representation, as shown in.

Method 1

Using the direct positional weight formula for an n-bit 2's complement number $b_{n-1}b_{n-2}\ldots b_0$:

$$\text{Value} = -b_{n-1}\cdot 2^{n-1} + \sum_{i=0}^{n-2} b_i \cdot 2^i$$

For the binary number 1000, we have $n = 4$ bits with $b_3 = 1$, $b_2 = 0$, $b_1 = 0$, and $b_0 = 0$:

$$\text{Value} = -1\cdot 2^3 + 0\cdot 2^2 + 0\cdot 2^1 + 0\cdot 2^0 = -8$$

Method 2

Any signed 2's complement sequence consisting of a single '1' followed by n zeros represents a special case. The decimal equivalent value is directly calculated as:

$$\text{Value} = -2^n$$

where n is the number of trailing zeros.
For the binary string 1000, there are 3 zeros following the initial '1':

$$\text{Value} = -2^3 = -8$$

Hence, the correct option is (D).

$$\boxed{\text{(D)}\ -8}$$

Key Points

- **Special Cases of 2's Complement Numbers:** A binary string starting with a 1 followed entirely by zeros corresponds to the most negative number (the lower bound) representable in that bit length.

2's Complement Representation	Number of Zeros (n)	Formula	Original Decimal Number
1000	3	-2^3	-8
10000	4	-2^4	-16
100000	5	-2^5	-32
1000000	6	-2^6	-64

Mistakes to be avoided

- Do not interpret the binary string as an unsigned number, which would yield +8 (matching option A).
- Avoid applying the standard workflow of taking the 2's complement of the number to find its positive magnitude ($1000 \xrightarrow{1's} 0111 \xrightarrow{+1} 1000$). Because 1000 overflows a standard 3-bit positive magnitude range, its complement returns back to itself, which can cause confusion if sign bits aren't handled carefully.

Q3.01.1 MCQ 2001 1M Ans: B ● ○ ○

To represent a negative decimal number −17 in 2's complement form, we determine the required number of bits from the given options, which all use 5 or 6 bits. Since +17 requires at least 5 bits for its magnitude ($16 + 1 = 10001_2$), we must use a 6-bit representation to include the sign bit.

Method 1

1. Write the binary representation of +17 using 6 bits:

$$(+17)_{10} = 010001_2$$

2. Find the 1's complement by inverting all the bits:

$$1\text{'s complement} = 101110_2$$

3. Add 1 to the LSB to get the 2's complement:

$$2\text{'s complement} = 101110_2 + 1_2 = 101111_2$$

Method 2

Using the shortcut method to find the 2's complement directly from $(+17)_{10} = 010001_2$:

- Scan the bits from right to left (LSB to MSB).
- Keep all bits the same up to and including the first '1'.
- Invert all remaining bits to the left of that first '1'.

$$010001 \xrightarrow{\text{2's complement}} 101111$$

(B) 101111

Q3.98.1 MCQ 1998 2M Ans: C ● ● ○

Method 1

An overflow condition in the addition of two signed numbers represented in 2's complement can only occur when two numbers of the same sign are added together, and the result yields the opposite sign.
Let x and y be the sign bits of the two input operands, and z be the sign bit of the resulting sum (0 for positive, 1 for negative).

- **Case 1: Adding two positive numbers** ($x = 0, y = 0$). An overflow occurs if the result is negative ($z = 1$).

$$\text{Condition 1} = \bar{x}\,\bar{y}z$$

- **Case 2: Adding two negative numbers** ($x = 1, y = 1$). An overflow occurs if the result is positive ($z = 0$).

$$\text{Condition 2} = xy\overline{z}$$

- **Case 3: Adding numbers with different signs** ($x \neq y$). An overflow can never occur.

Combining the valid overflow conditions, the Boolean expression indicating the occurrence of an overflow is:

$$\text{Overflow} = \overline{x}\,\overline{y}z + xy\overline{z}$$

Method 2

We can build a truth table to analyze the behavior of the overflow flag based on the sign bits x, y, and z:

x	y	z	**Overflow**	**Description**
0	0	0	0	Positive + Positive = Positive (Valid)
0	0	1	1	Positive + Positive = Negative (**Overflow**)
0	1	0	0	Positive + Negative = Positive (Valid)
0	1	1	0	Positive + Negative = Negative (Valid)
1	0	0	0	Negative + Positive = Positive (Valid)
1	0	1	0	Negative + Positive = Negative (Valid)
1	1	0	1	Negative + Negative = Positive (**Overflow**)
1	1	1	0	Negative + Negative = Negative (Valid)

Extracting the minterms where the Overflow = 1:

$$\text{Overflow} = \overline{x}\,\overline{y}z + xy\overline{z}$$

Key Points

- **Overflow Rule:** Overflow in 2's complement addition happens if and only if the sign bits of the operands are identical, but the sign bit of the sum is different.
- Adding a positive number to a negative number never yields an overflow because the magnitude of the sum is guaranteed to be less than or equal to the largest operand's magnitude.

$$\boxed{\text{(C) } \overline{x}\,\overline{y}z + xy\overline{z}}$$

Q3.98.2 MCQ 1998 2M Ans: D ● ○ ○

Method 1

The given 2's complement number is 1101.

- The Most Significant Bit (MSB) or sign bit is 1, which indicates that the number is negative.
- According to the property of **sign bit extension**, the value of a signed binary number remains unchanged when the sign bit is repeated to the left.

Since the options are represented using 6 bits, we extend the 4-bit number to 6 bits by replicating the sign bit (1) two times to the left:

$$1101 \xrightarrow{\text{Sign Extension}} \mathbf{11}1101$$

Thus, the equivalent 6-bit representation is 111101.

Method 2

We can verify the numerical value of the given number and the options:

- Value of given number $(1101)_2$ in 2's complement:

$$-2^3 \cdot 1 + 2^2 \cdot 1 + 2^1 \cdot 0 + 2^0 \cdot 1 = -8 + 4 + 0 + 1 = -3$$

- Value of option (D) $(111101)_2$ in 2's complement:

$$-2^5 \cdot 1 + 2^4 \cdot 1 + 2^3 \cdot 1 + 2^2 \cdot 1 + 2^1 \cdot 0 + 2^0 \cdot 1$$
$$= -32 + 16 + 8 + 4 + 0 + 1 = -3$$

Since the values are identical, option (D) is correct.

Key Points

- **Sign Extension:** To increase the bit length of a signed binary number (in 1's or 2's complement) without changing its arithmetic value, the sign bit (MSB) must be replicated into the new higher-order positions.

(D) 111101

Q3.97.1 MCQ 1997 2M Ans: C ●○○

To expand a signed 2's complement integer while preserving its value, we apply **sign extension** by copying the original bits into the lower positions and filling all higher-order positions with the original sign bit (MSB).
For example, a 3-bit negative number $\mathbf{1}01_2$ (value −3) sign-extends to 5 bits as $\mathbf{11}101_2$, which retains the exact same arithmetic value: $-2^4(1) + 2^3(1) + 2^2(1) + 2^0(1) = -16 + 8 + 4 + 1 = -3$.
Example 1: Positive Number Let the 3-bit signed number be $\mathbf{0}11_2$.

- **Value in 3 bits:**

$$-2^2 \cdot \mathbf{0} + 2^1 \cdot 1 + 2^0 \cdot 1 = 0 + 2 + 1 = +3$$

- **Extended to 5 bits by repeating MSB (0):** $\mathbf{00}011_2$
- **Value in 5 bits:**

$$-2^4 \cdot \mathbf{0} + 2^3 \cdot \mathbf{0} + 2^2 \cdot 0 + 2^1 \cdot 1 + 2^0 \cdot 1 = 0 + 0 + 0 + 2 + 1 = +3$$

Example 2: Negative Number Let the 3-bit signed number be $\mathbf{1}01_2$.

- **Value in 3 bits:**

$$-2^2 \cdot \mathbf{1} + 2^1 \cdot 0 + 2^0 \cdot 1 = -4 + 0 + 1 = -3$$

- **Extended to 5 bits by repeating MSB (1):** $\mathbf{11}101_2$
- **Value in 5 bits:**

$$-2^4 \cdot \mathbf{1} + 2^3 \cdot \mathbf{1} + 2^2 \cdot 1 + 2^1 \cdot 0 + 2^0 \cdot 1$$
$$= -16 + 8 + 4 + 0 + 1 = -3$$

Since the decimal values remain identical (+3 → +3 and −3 → −3), the proof holds true.

(C) copy the original byte to the less significant byte of the word and make each bit of the more significant byte equal to the most significant bit of the original byte

Key Points

- **Sign Extension:** This process expands the bit-width of a signed number by replicating its sign bit (MSB) into all higher-order positions, ensuring the value remains unchanged.

Q3.93.1 MCQ 1993 1M Ans: B ● ○ ○

A 16-bit 2's complement number consists of one sign bit and 15 magnitude bits.
The hexadecimal number is

$$\text{FFFF} = 1111\ 1111\ 1111\ 1111$$

Since the MSB is 1, the number is negative.
To find its magnitude, take the 2's complement:

$$\overline{1111\ 1111\ 1111\ 1111} + 1 = 0000\ 0000\ 0000\ 0000 + 1 = 0000\ 0000\ 0000\ 0001$$

(b) 1

Key Points

- In 2's complement representation, MSB = 1 indicates a negative number.
- Magnitude is obtained by taking the 2's complement of the given binary number.
- $FFFF_{16}$ in 16-bit 2's complement represents -1.

Q3.87.1 MSQ 1987 1M Ans: B,C ● ○ ○

Binary subtraction is performed as

$$X - Y = X + (\text{2's complement of } Y)$$

If the addition is completed *without overflow*, the result can be interpreted as follows:

- If there is an end carry, it is discarded and the result is **positive** in its normal binary form. ✓
- If there is no end carry, the result is **negative** and remains in 2's complement form. ✓

Hence,

(b) and (c)

Key Points

- Binary subtraction is performed by adding the 2's complement of the subtrahend.
- End carry present ⇒ positive result in normal binary form.
- No end carry ⇒ negative result in 2's complement form.
- The question states that no overflow occurs.

Mistakes to be avoided

- Do not confuse end carry with overflow; they are different concepts.
- Do not convert a negative result from 2's complement unless its magnitude is specifically asked.

SOLUTIONS

IV

Combinational Circuits

Topics

1. Introduction to Combinational Circuits
2. Code Converters and 7-Segment Display
3. Parity Generator and Checker
4. Magnitude Comparators
5. Multiplexers (MUX) and Demultiplexers (DEMUX)
6. Implementing Logical Expressions Using MUX
7. Decoders, Encoders and Priority Encoders
8. Half Adder and Full Adder
9. Delay-Based Problems in Adders
10. Half Subtractor and Full Subtractor
11. Binary Parallel Adder with Delay-Based Problems
12. Look-Ahead Carry Adder
13. Binary Parallel Subtractor
14. Adder and Subtractor Circuits of Different Binary Codes
15. Binary Multiplier

Unit Snapshot*

Stream: EC	**Questions:** 48	**MCQ:** 35	**MSQ:** 0	**NAT:** 6
1M: 13	**2M:** 30	**Descriptive:** 5	**Avg Weightage** ~2M	**Range:** 1M(2025)–4M(2024)

**See preface for how Avg Weightage and Range are calculated.*

Q4.26.1 MCQ | 2026 | 2M | Ans: C

Based on the provided circuit diagram of the multiplexer, we can derive the Boolean expression using two different approaches.

Method 1

We can see I_0 and I_2 are directly connected to high; thus, if those are selected we will directly get output. Thus, we will get:

$$= \overline{xz} + x\overline{z}$$

If I_3 is selected then we will get output high for y high, thus minterm added:

$$= xyz$$

Final expression:

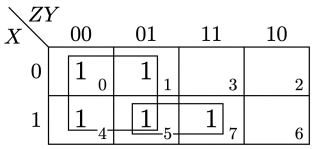
$$= \overline{xz} + x\overline{z} + xyz$$
$$= \overline{z}(\overline{x} + x) + xyz \quad \ldots (\because \overline{x} + x = 1)$$
$$= \overline{z} + xyz$$
$$= \overline{z} + xy \quad \ldots \text{(By redundant term theorem)}$$

Method 2

By analyzing the diagram we can generate the following truth table:

x	z	y	$f(x, y, z)$
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

K-map for the same will be:



Thus, $\overline{z} + xy$.

$$\boxed{(C) xy + \overline{z}}$$

Mistakes to be avoided

- Ensure that the select lines x and z are correctly mapped to the multiplexer inputs S_1 and S_0.

Q4.25.1 MCQ | 2025 | 1M | Ans: B ●●○

Given the digital circuit consisting of a Full Adder and an XOR gate with inputs X, Y, and Z.

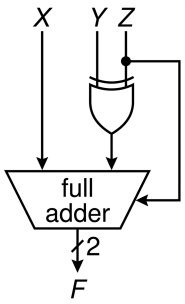


Fig. Solution Q4.25.1

As per the logic diagram, the inputs to the Full Adder are X, the output of the XOR gate ($Y \oplus Z$), and the carry input C_{in} which is connected to Z.
When Z is set to 1:

- XOR gate output = $Y \oplus 1 = \overline{Y}$.
- Carry input $C_{in} = Z = 1$.

The Full Adder performs binary addition of its three inputs:

$$F = X + \overline{Y} + C_{in}$$

$$F = X + \overline{Y} + 1$$

In binary arithmetic, $\overline{Y} + 1$ represents the 2's complement of Y.

$$F = X + (\text{2's complement of } Y)$$

This expression is equivalent to the subtraction operation:

$$F = X - Y$$

Thus, when $Z = 1$, the circuit functions as a subtractor.
Note that the output F is a 2-bit bus containing both full-adder outputs, not just the sum bit. With inputs $a = X$, $b = Y \oplus Z$ and $c = Z$, the sum bit is the XOR $S = a \oplus b \oplus c$ and the carry is $C = ab + bc + ca$ (here + is Boolean OR). Together they give the arithmetic value $F = 2C + S = a + b + c$ (ordinary addition), which is why the circuit adds X, $\overline{Y}$ and 1. In the subtraction, a carry output $C = 1$ means no borrow ($X \geq Y$) and $C = 0$ means a borrow.

(B) a subtractor

Key Points

- A Full Adder can be used as a subtractor by providing the 2's complement of the subtrahend to one of its inputs.
- XOR gate properties: $A \oplus 0 = A$ (buffer) and $A \oplus 1 = \overline{A}$ (inverter).
- 2's complement of a number Y is defined as $\overline{Y} + 1$.

Q4.24.1 MCQ 2024 2M Ans: B ● ● ○

A Priority encoder outputs the binary code of the highest priority active input.
Thus, the truth table is as follows:

D_3	D_2	D_1	D_0	A	B
1	x	x	x	0	0
0	1	x	x	0	1
0	0	1	x	1	0
0	0	0	1	1	1
0	0	0	0	x	x

When all four inputs are 0, no input is active, so the question does not specify the output code: this row is a don't care (x), not necessarily 11.
Using K-map simplification for output B, the don't-care cell $D_3D_2D_1D_0 = 0000$ is taken as 1 because that gives larger groups:

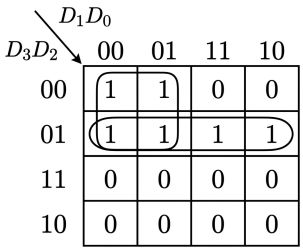


Fig. Solution Q4.24.1

This gives $B = \overline{D_3}D_2 + \overline{D_3}\,\overline{D_1}$, which is correct for every input combination in which at least one input is active.

$$\boxed{(B)\overline{D_3}D_2 + \overline{D_3}\,\overline{D_1}}$$

Key Points

- Output corresponds to the Higher priority input irrespective of the state of its Lower priority inputs.
- The all-zero input of a priority encoder is normally a don't care for the output code (a separate valid/idle output usually flags it), so it may be used as 0 or 1 while simplifying.

Mistakes to be avoided

- Taking D_3 as 11 and D_0 as 00. For this question, notations of 3 to 0 are flipped which means here D_3 is denoted as 00 and D_0 as 11.

Q4.24.2 MCQ 2024 2M Ans: B ● ○ ○

The given circuit is a 2×1 multiplexer (MUX) with feedback.

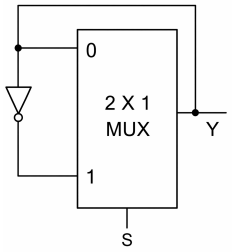


Fig. Solution Q4.24.2

Based on the problem description:

- Propagation delay of 2×1 MUX = 10 ns.
- Propagation delay of inverter = 0 ns.

When the select line $S = 1$, the input data line '1' is selected. The output Y is fed back through an inverter to the input line '1'. The circuit reduces to an inverter with a feedback loop. This configuration acts as a ring oscillator (specifically an astable multivibrator).

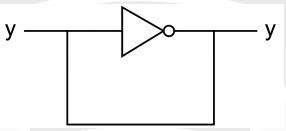


Fig. Solution Q4.24.2

- If $Y = 0$, the inverter makes the input 1. After a 10 ns delay through the MUX, Y becomes 1.
- If $Y = 1$, the inverter makes the input 0. After a 10 ns delay through the MUX, Y becomes 0.

Thus a square wave as follows is generated -

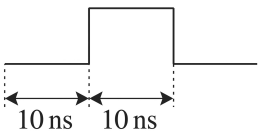


Fig. Solution Q4.24.2

The time for one half-cycle (the time to toggle) is equal to the propagation delay $t_{pd} = 10$ ns. The total time period T of the resulting square wave is:

$$T = 2 \times t_{pd} = 2 \times 10\,\text{ns} = 20\,\text{ns}$$

Thus a square wave as follows is generated -
The frequency f of the output square wave is calculated as:

$$f = \frac{1}{T} = \frac{1}{20 \times 10^{-9}\,\text{s}}$$

$$f = \frac{10^9}{20} = \frac{1000}{20} \times 10^6\,\text{Hz}$$

$$f = 50\,\text{MHz}$$

(B) a square wave of frequency 50 MHz

Key Points

- A combinational circuit with inverted feedback and an odd number of inversions in the loop will oscillate if the loop gain is sufficient.
- The time period of a ring oscillator is $T = 2 \times$ (sum of propagation delays in the loop).

Mistakes to be avoided

- Do not confuse the time period with the propagation delay; the signal must travel through the loop twice to complete one full cycle (high and low states).

Q4.23.1 MCQ | 2023 | 1M | Ans: A ● ○ ○

The given circuit is a 2×1 Multiplexer (MUX) with inputs connected as shown in the diagram.

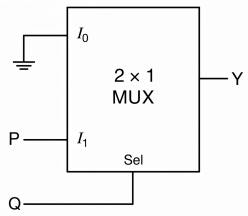


Fig. Solution Q4.23.1

Based on the logic diagram:

- Select line $Sel = Q$
- Input $I_0 = 0$ (Grounded)
- Input $I_1 = P$

The characteristic equation for the output Y of a 2×1 MUX is:

$$Y = \overline{S}I_0 + SI_1$$

Substituting the given values into the equation:

$$Y = \overline{Q} \cdot 0 + Q \cdot P$$

$$Y = 0 + PQ$$

$$Y = PQ$$

The output Y represents the logical AND operation of inputs P and Q. Thus, the correct option is (A).

$$\boxed{(A) Y = PQ}$$

Q4.22.1 MCQ | 2022 | 1M | Ans: C ● ○ ○

The given circuit is a 4×1 Multiplexer (MUX) with inputs and selection lines as shown in the diagram.

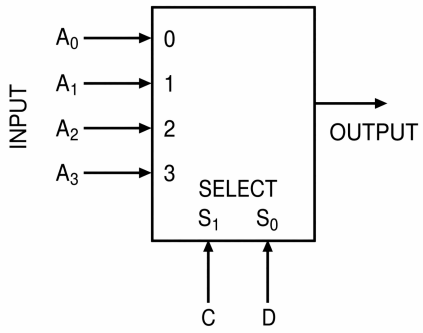


Fig. Solution Q4.22.1

The general characteristic equation for the output F of a 4×1 MUX is:

$$F = \overline{S}_1\overline{S}_0A_0 + \overline{S}_1S_0A_1 + S_1\overline{S}_0A_2 + S_1S_0A_3 \ldots (i)$$

From the given figure:

- Selection lines: $S_1 = C$ and $S_0 = D$
- Inputs: A_0, A_1, A_2, A_3

Substituting the selection lines into equation (i):

$$F = \overline{C}\overline{D}A_0 + \overline{C}DA_1 + C\overline{D}A_2 + CDA_3$$

The goal is to implement the XOR logic function:

$$F = C \oplus D = \overline{C}D + C\overline{D}$$

Comparing the XOR expression with the MUX output expression:

- To get the $\overline{C}D$ term, we must set $A_1 = 1$.
- To get the $C\overline{D}$ term, we must set $A_2 = 1$.
- The terms $\overline{C}\overline{D}$ and CD are not present in the XOR function, so we must set $A_0 = 0$ and $A_3 = 0$.

Thus, the required input values are $A_0 = 0$, $A_1 = 1$, $A_2 = 1$, and $A_3 = 0$. Hence, the correct option is (C).

$$\boxed{(C) A_0 = 0, A_1 = 1, A_2 = 1, A_3 = 0}$$

Key Points

- A 4×1 MUX can implement any 2-variable boolean function directly by connecting inputs to logic 0 or 1 according to the truth table of the required function.
- For a MUX acting as a function generator, the inputs A_n correspond to the truth table output values for the minterms addressed by the selection lines.

Q4.21.1 MCQ 2021 2M Ans: C ●○○

Given gate propagation delays:

$$\tau_{XOR} = 4\,\text{ns}, \quad \tau_{AND} = 2\,\text{ns}, \quad \tau_{MUX} = 1\,\text{ns}$$

The maximum propagation delay of the circuit is evaluated across all distinct steady-state path conditions determined by the control variable T.

- **Case 1: When** $T = 0$

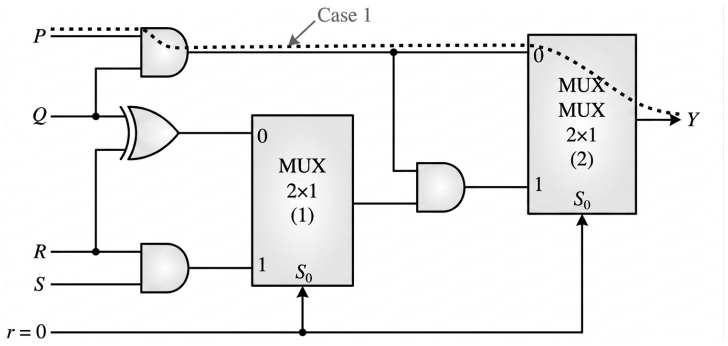


Fig. Solution Q4.21.1

$$\text{Path}_1 : \text{AND}_1 \rightarrow \text{MUX}_2$$

$$T_1 = \tau_{\text{AND}} + \tau_{\text{MUX}}$$

$$T_1 = 2\,\text{ns} + 1\,\text{ns} = 3\,\text{ns}$$

- **Case 2: When** $T = 1$

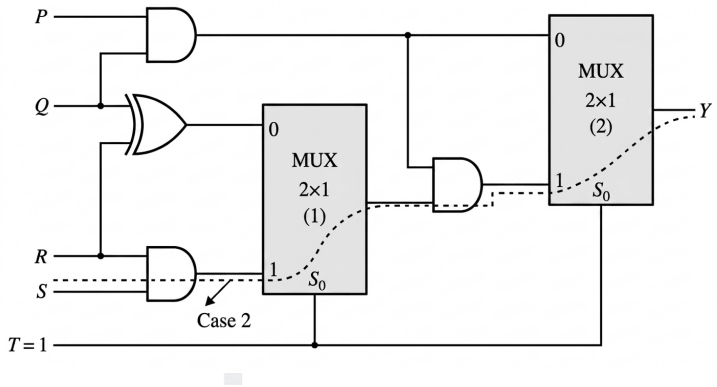


Fig. Solution Q4.21.1

$$\text{Path}_2 : \text{AND}_1 \rightarrow \text{MUX}_1 \rightarrow \text{AND}_2 \rightarrow \text{MUX}_2$$

$$T_2 = \tau_{\text{AND}} + \tau_{\text{MUX}} + \tau_{\text{AND}} + \tau_{\text{MUX}}$$

$$T_2 = 2\,\text{ns} + 1\,\text{ns} + 2\,\text{ns} + 1\,\text{ns} = 6\,\text{ns}$$

The worst-case maximum propagation delay of the circuit is:

$$T_{\max} = \max(T_1, T_2) = 6\,\text{ns}$$

Hence, the correct option is (C).

(C) 6 ns

Mistakes to be avoided

- All distinct select line combinations must be analyzed.
- Avoid tracking a fixed propagation path; worst-case delay must always be calculated along the longest active path from

any input to the output.

Q4.20.1 MCQ | 2020 | 1M | Ans: A

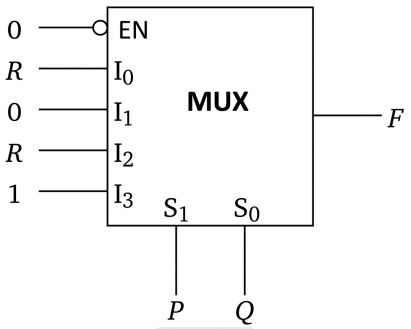


Fig. Solution Q4.20.1

The output equation of a enabled 4×1 MUX is:

$$F = \overline{S}_1\overline{S}_0 I_0 + \overline{S}_1 S_0 I_1 + S_1\overline{S}_0 I_2 + S_1 S_0 I_3$$

Given selection lines $S_1 = P, S_0 = Q$ and inputs $I_0 = R, I_1 = 0, I_2 = R, I_3 = 1$:

$$F = \overline{P}\,\overline{Q}R + \overline{P}Q(0) + P\overline{Q}R + PQ(1)$$

$$F = \overline{Q}R(\overline{P} + P) + PQ$$

Since $\overline{P} + P = 1$:

$$F = PQ + \overline{Q}R$$

$$\boxed{(A)\ PQ + \overline{Q}R}$$

Q4.18.1 MCQ | 2018 | 2M | Ans: C

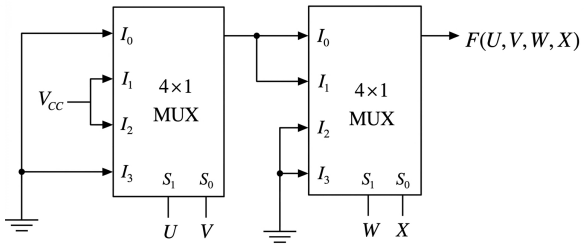


Fig. Solution Q4.18.1

The output of the first multiplexer (MUX-1) with select lines $S_1 = U, S_0 = V$ is:

$$f_1 = \overline{U}V(1) + U\overline{V}(1) = U \oplus V$$

The output of the second multiplexer (MUX-2) with select lines $S_1 = W, S_0 = X$ and data inputs $I_0 = f_1, I_1 = f_1, I_2 = 0, I_3 = 0$ is:

$$F = \overline{W}\,\overline{X} \cdot f_1 + \overline{W}X \cdot f_1 + W\overline{X}(0) + WX(0)$$

$$F = \overline{W} f_1(\overline{X} + X)$$

Since $\overline{X} + X = 1$:

$$F = \overline{W} f_1 = (U\overline{V} + \overline{U}V)\overline{W}$$

$$\boxed{(C)(U\overline{V} + \overline{U}V)\overline{W}}$$

Key Points

- Standard 4×1 MUX logic: $F = \overline{S}_1\overline{S}_0 I_0 + \overline{S}_1 S_0 I_1 + S_1\overline{S}_0 I_2 + S_1 S_0 I_3$.

Q4.17.1 NAT | 2017 | 2M | Ans: 50 | ● ● ●

CLICK HERE FOR VIDEO SOLUTION

Initially, all the internal states and inputs of the 4-bit ripple carry adder are reset to 0. At $t = 0$, the inputs are changed to:

$$X_3X_2X_1X_0 = 1100$$

$$Y_3Y_2Y_1Y_0 = 0100$$

$$Z_0 = 1$$

Let us first analyze the inputs and observe where a transition occurs, as we only need to study the outputs where the state changes from 0 to 1:

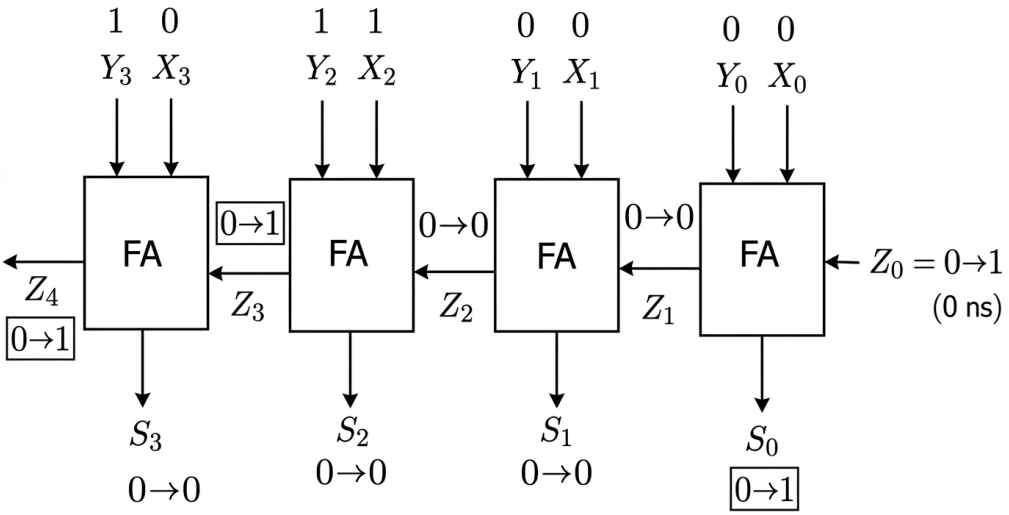


Fig. Solution Q4.17.1

As observed from the block diagram, the state transitions occur only for S_0, Z_3, and Z_4. We can analyze the timing for each of these outputs one by one:

1. For Sum S_0

At the first full adder (FA_0), the inputs are $X_0 = 0$, $Y_0 = 0$, and $Z_0 = 0 \to 1$. Looking at the circuit diagram of Figure II:

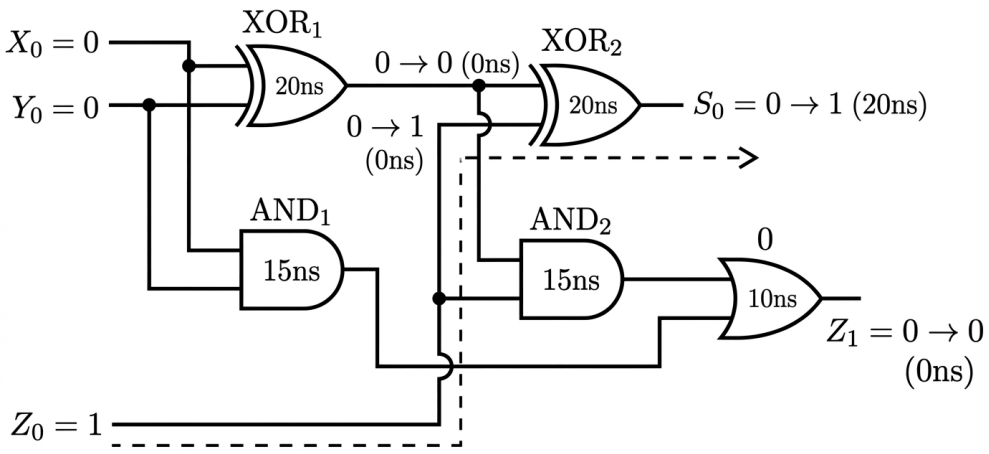


Fig. Solution Q4.17.1

Since $X_0 = 0$ and $Y_0 = 0$, the output of XOR_1 remains 0, bypassing its propagation delay. The transition from Z_0 propagates solely through XOR_2 to reach S_0. Thus, S_0 stabilizes after a single XOR gate delay:

$$t_{S0} = 20\,\text{ns}$$

2. For Carry Z_3

At the third full adder (FA_2), the inputs are $X_2 = 1$ and $Y_2 = 1$, while the incoming carry is $Z_2 = 0 \to 0$ (stable at 0 ns).

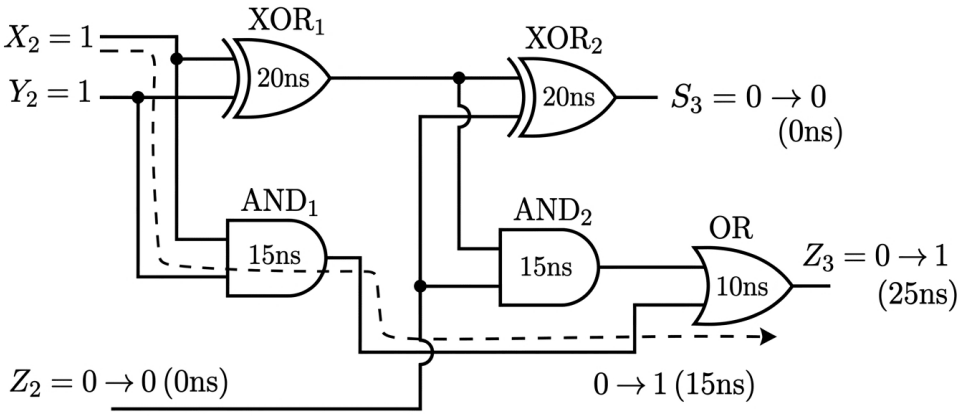


Fig. Solution Q4.17.1

The carry output Z_3 transitions to 1 as soon as any input to the final OR gate becomes 1. Given $X_2 = 1$ and $Y_2 = 1$:

- Output of AND_1 becomes 1 after a propagation delay of 15 ns.
- This signal travels through the final OR gate, adding a delay of 10 ns.

Thus, Z_3 stabilizes at:

$$t_{Z_3} = 15\text{ ns} + 10\text{ ns} = 25\text{ ns}$$

3. For Carry Z_4

At the fourth full adder (FA_3), the inputs are $X_3 = 0$ and $Y_3 = 1$, and the incoming carry transitions as $Z_3 = 0 \rightarrow 1$ at $t = 25$ ns.

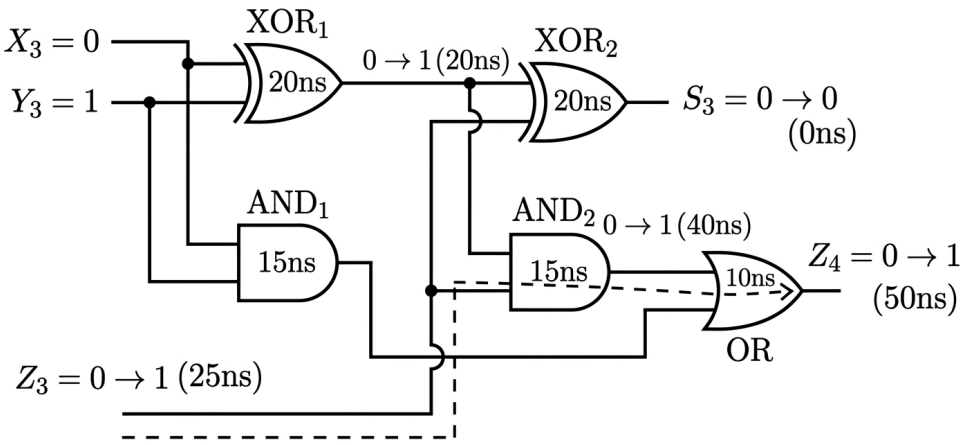


Fig. Solution Q4.17.1

The carry propagation path passes through the AND_2 gate, which requires two inputs: the output of XOR_1 ($X_3 \oplus Y_3$) and the incoming carry Z_3.

- XOR_1 output settles at $t = 20$ ns.
- Carry input Z_3 arrives at $t = 25$ ns.

The AND_2 gate must wait for the later of the two concurrent inputs to arrive:

$$t_{\text{inputs}} = \max(20 \text{ ns}, 25 \text{ ns}) = 25 \text{ ns}$$

Accounting for the subsequent gate propagation delays ($AND_2 = 15$ ns, OR = 10 ns):

$$t_{AND_2 \text{ out}} = 25 \text{ ns} + 15 \text{ ns} = 40 \text{ ns}$$
$$t_{Z_4} = 40 \text{ ns} + 10 \text{ ns} = 50 \text{ ns}$$

Thus, the maximum time delay for Z_4 to become stable is 50 ns.
Out of all the internal and external transitions occurring across the entire circuit, the maximum delay is 50 ns. Therefore, the output of the ripple carry adder will be completely stable at $t = 50$ ns.

50

Key Points

- In parallel or ripple-carry structures, intermediate signals propagate simultaneously along different paths.
- When a logic gate requires multiple changing inputs to compute its stable output, its timing is constrained by the maximum (max) arrival time among those necessary inputs.
- The overall settling time of a combinational circuit is determined by the longest critical propagation delay path from any input to any output.

Mistakes to be avoided

- Do not blindly multiply the single-stage full adder delay by the total number of bits (4 × delay) to find the maximum path delay. Specific input transitions can shortcut or parallelize the carry path.
- Avoid adding up delays of gates whose inputs do not undergo any transition during the cycle.

Q4.17.2 MCQ 2017 1M Ans: B ● ○ ○

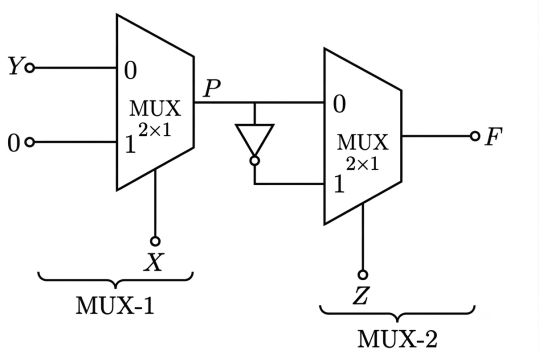


Fig. Solution Q4.17.2

Output equation for a 2 × 1 multiplexer is given by,

$$Y = \overline{S}I_0 + SI_1$$

Let the output of the MUX-1 be P In the given circuit, output of 1st MUX is given by,

$$P = \overline{X}Y + X \cdot 0 = \overline{X}Y$$

Output of 2nd MUX is given by,

$$F = \overline{Z} \cdot P + Z \cdot \overline{P}$$

$$F = \overline{Z} \cdot \overline{X}Y + Z \cdot \overline{\overline{X}Y}$$

$$F = \overline{Z}\,\overline{X}Y + Z(X + \overline{Y})$$

$$F = \overline{X}Y\overline{Z} + XZ + \overline{Y}Z$$

Hence, the correct option is (B).

$$\boxed{(B)\overline{X}Y\overline{Z} + XZ + \overline{Y}Z}$$

Q4.16.1 NAT 2016 2M Ans: 6 ● ● ●

Given gate propagation delays:

$$\tau_{NOR} = 2\,\text{ns}, \quad \tau_{MUX} = 1.5\,\text{ns}, \quad \tau_{NOT} = 1\,\text{ns}$$

The maximum propagation delay of the circuit depends on the control input T, which acts as the selection line (S_0) for both multiplexers.

- **Case 1: When** $T = 0$

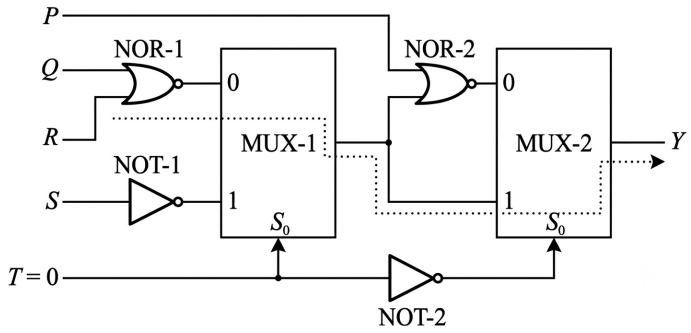


Fig. Solution Q4.16.1

$$\text{Path}_1 : \text{NOR}_1 \rightarrow \text{MUX}_1 \rightarrow \text{MUX}_2$$

$$T_1 = \tau_{\text{NOR}} + \tau_{\text{MUX}} + \tau_{\text{MUX}}$$

$$T_1 = 2\,\text{ns} + 1.5\,\text{ns} + 1.5\,\text{ns} = 5\,\text{ns}$$

- **Case 2: When** $T = 1$

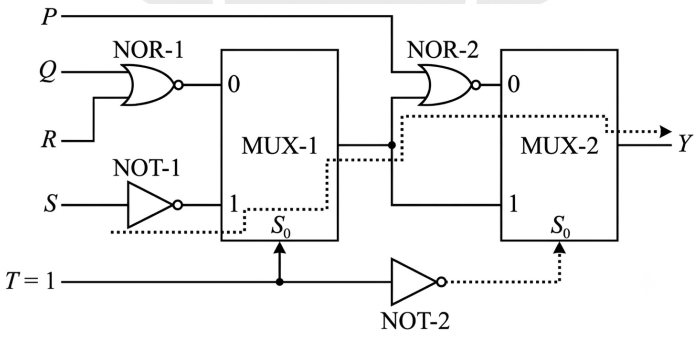


Fig. Solution Q4.16.1

$$\text{Path}_2 : \text{NOT}_1 \rightarrow \text{MUX}_1 \rightarrow \text{NOR}_2 \rightarrow \text{MUX}_2$$

$$T_2 = \tau_{\text{NOT}} + \tau_{\text{MUX}} + \tau_{\text{NOR}} + \tau_{\text{MUX}}$$

$$T_2 = 1\,\text{ns} + 1.5\,\text{ns} + 2\,\text{ns} + 1.5\,\text{ns} = 6\,\text{ns}$$

The worst-case maximum propagation delay is:

$$T_{\max} = \max(T_1, T_2) = 6\,\text{ns}$$

6

Mistakes to be avoided

- Avoid tracking a fixed propagation path; worst-case delay must always be calculated along the longest active path from any input to the output.
- When control/select line inputs are inverted internally, different paths get enabled simultaneously; check all operational combinations to find the absolute maximum delay.

Q4.16.2 MCQ 2016 2M Ans: MTA ●●○

Given the multi-stage digital circuit consisting of a 3 : 8 decoder cascaded with an 8 : 3 encoder:

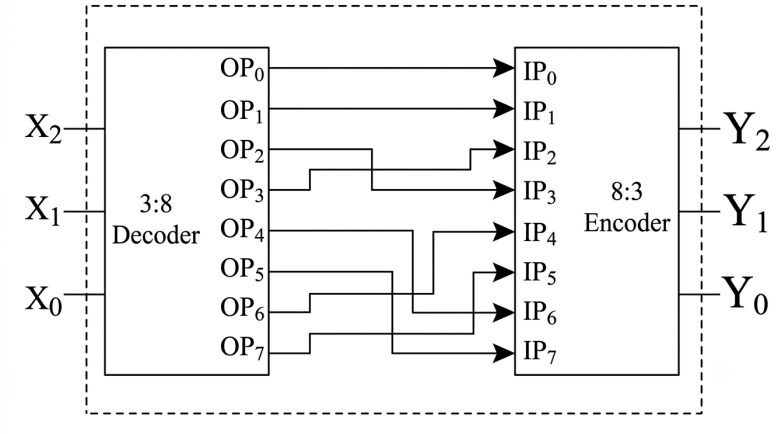


Fig. Solution Q4.16.2

The complete functional truth table showing the binary and decimal equivalent mappings is given below:

Dec X	Inputs $X_2X_1X_0$	OP → IP	Outputs $Y_2Y_1Y_0$	Dec Y
0	0 0 0	$OP_0 \rightarrow IP_0$	0 0 0	0
1	0 0 1	$OP_1 \rightarrow IP_1$	0 0 1	1
2	0 1 0	$OP_2 \rightarrow IP_3$	0 1 1	3
3	0 1 1	$OP_3 \rightarrow IP_2$	0 1 0	2
4	1 0 0	$OP_4 \rightarrow IP_6$	1 1 0	6
5	1 0 1	$OP_5 \rightarrow IP_7$	1 1 1	7
6	1 1 0	$OP_6 \rightarrow IP_4$	1 0 0	4
7	1 1 1	$OP_7 \rightarrow IP_5$	1 0 1	5

Since the first row maps $000_2 \rightarrow 000_2$ ($0 \rightarrow 0$), Excess-3 (XS3) conversion is immediately ruled out as it requires an offset of +3. Consequently, the circuit must be either a Binary-to-Gray or a Gray-to-Binary converter. Let us verify both mappings against the circuit's truth table.

Binary to Gray Conversion				Gray to Binary Conversion			
Dec N	Binary $B_2B_1B_0$	Gray $G_2G_1G_0$	Dec Equiv	Dec N	Gray $G_2G_1G_0$	Binary $B_2B_1B_0$	Dec Equiv
0	0 0 0	0 0 0	0	0	0 0 0	0 0 0	0
1	0 0 1	0 0 1	1	1	0 0 1	0 0 1	1
2	0 1 0	0 1 1	3	2	0 1 0	0 1 1	3
3	0 1 1	0 1 0	2	3	0 1 1	0 1 0	2
4	1 0 0	1 1 0	6	4	1 0 0	1 1 1	7
5	1 0 1	1 1 1	7	5	1 0 1	1 1 0	6
6	1 1 0	1 0 1	5	6	1 1 0	1 0 0	4
7	1 1 1	1 0 0	4	7	1 1 1	1 0 1	5

Comparing the circuit's truth table to standard mappings, the output transformation fails to match either the Binary-to-Gray configuration (which needs $6 \rightarrow 5$ and $7 \rightarrow 4$) or the Gray-to-Binary configuration (which needs $4 \rightarrow 7$ and $5 \rightarrow 6$). Due to this structural discrepancy, the question contains a functional error and was officially awarded Marks to All (MTA).

Marks To All (MTA)

Key Points

Gray code is an unweighted binary code in which consecutive numeric values differ by exactly one binary digit (bit). The systematic methods for converting a standard binary number to Gray code, and vice versa, are illustrated below (the $\oplus$ here denotes XOR operation):

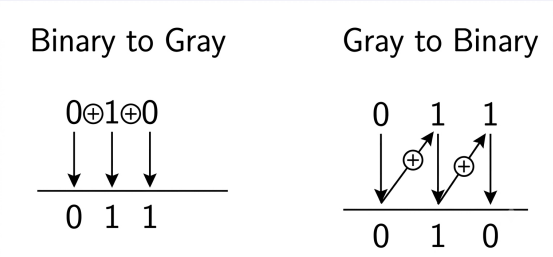


Fig. Solution Q4.16.2

Q4.16.3 MCQ 2016 2M Ans: B

The working of the Tri-state buffer is as follows-

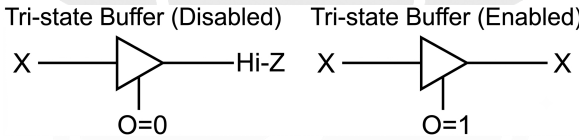


Fig. Solution Q4.16.3

When the selection lines (C_1, C_0) activate a specific decoder output (O_0, O_1, O_2, O_3), the corresponding tristate buffer is enabled, passing its respective input (P, Q, R, S) to the final output Y.
The operation can be summarized by the truth table below:

Input		Output of Decoder				Output
C_1	C_0	O_0	O_1	O_2	O_3	Y
0	0	1	0	0	0	P
0	1	0	1	0	0	Q
1	0	0	0	1	0	R
1	1	0	0	0	1	S

The Boolean expression for the output Y is:

$$Y = \overline{C_1}\,\overline{C_0}P + \overline{C_1}C_0Q + C_1\overline{C_0}R + C_1C_0S$$

This exactly matches the functional behavior of a 4-to-1 multiplexer where C_1 and C_0 act as the select lines, and P, Q, R, S act as the data inputs.

(B) 4 to 1 multiplexer

Key Points

- A tristate buffer transmits its input to the output only when its enable line is active (logic 1). When disabled, its output enters a high-impedance state (High-Z).
- A 2-to-4 decoder with active-high outputs ensures that exactly one output line is high for any binary combination of select lines, effectively implementing a routing mechanism equivalent to a multiplexer.

Q4.16.4 MCQ 2016 1M Ans: A ●●○

The output carry (C_{out}) of a full adder with inputs A, B, and C_{in} is represented by the Boolean expression:

$$C_{out} = AB + BC_{in} + C_{in}A = \sum m(3, 5, 6, 7)$$

A 4×1 multiplexer with select lines $S_1 = A$ (MSB) and $S_0 = B$ (LSB) generates the output:

$$Y = \overline{A}\,\overline{B}I_0 + \overline{A}BI_1 + A\overline{B}I_2 + ABI_3$$

Method 1

Using the implementation table method to map the 3-variable function into the 4×1 MUX:

MUX input →	I_0	I_1	I_2	I_3
$\overline{C}_{in}$	0	2	4	⑥
C_{in}	1	③	⑤	⑦
	0	C_{in}	C_{in}	1

Comparing the selected minterms $\{3, 5, 6, 7\}$:

- For I_0: Neither 0 nor 1 is present → $I_0 = 0$
- For I_1: Only 3 (C_{in}) is present → $I_1 = C_{in}$
- For I_2: Only 5 (C_{in}) is present → $I_2 = C_{in}$
- For I_3: Both 6 and 7 are present → $I_3 = 1$

Method 2

By comparing the truth table of the full adder carry with the MUX operational behavior:

A	B	C_{out}	MUX Input Channel
0	0	0	$I_0 = 0$
0	1	C_{in}	$I_1 = C_{in}$
1	0	C_{in}	$I_2 = C_{in}$
1	1	1	$I_3 = 1$

(A) $I_0 = 0, I_1 = C_{in}, I_2 = C_{in}$ and $I_3 = 1$

Key Points

- A full adder carry expression is $C_{out} = AB + BC_{in} + C_{in}A$.
- When implementing an n-variable function using a MUX with $(n-1)$ select lines, the remaining variable or constants (0, 1) form the data inputs.

Q4.15.1 MCQ 2015 2M Ans: D ●●●

Consider a 1×8 demultiplexer layout:

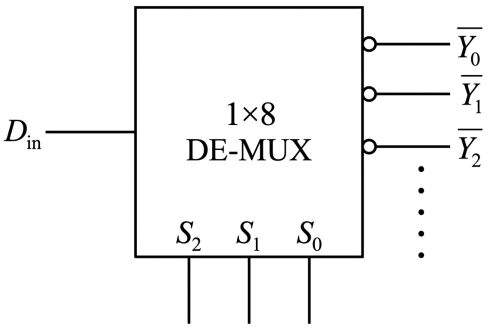


Fig. Solution Q4.15.1

The functional output expressions for this 1×8 DE-MUX are defined as follows:

$$\overline{Y}_0 = \overline{\overline{S}_2\,\overline{S}_1\,\overline{S}_0 D_{in}} = \overline{D}_{in} + S_2 + S_1 + S_0 \qquad \overline{Y}_4 = \overline{S}_2\,\overline{S}_1\,\overline{S}_0 D_{in} = \overline{D}_{in} + \overline{S}_2 + S_1 + S_0$$

$$\overline{Y}_1 = \overline{\overline{S}_2\,\overline{S}_1 S_0 D_{in}} = \overline{D}_{in} + S_2 + S_1 + \overline{S}_0 \qquad \overline{Y}_5 = \overline{S}_2\,\overline{S}_1 S_0 D_{in} = \overline{D}_{in} + \overline{S}_2 + S_1 + \overline{S}_0$$

$$\overline{Y}_2 = \overline{\overline{S}_2 S_1\,\overline{S}_0 D_{in}} = \overline{D}_{in} + S_2 + \overline{S}_1 + S_0 \qquad \overline{Y}_6 = \overline{S}_2 S_1\,\overline{S}_0 D_{in} = \overline{D}_{in} + \overline{S}_2 + \overline{S}_1 + S_0$$

$$\overline{Y}_3 = \overline{\overline{S}_2 S_1 S_0 D_{in}} = \overline{D}_{in} + S_2 + \overline{S}_1 + \overline{S}_0 \qquad \overline{Y}_7 = \overline{S}_2 S_1 S_0 D_{in} = \overline{D}_{in} + \overline{S}_2 + \overline{S}_1 + \overline{S}_0$$

Now, analyzing the given combinational decoder circuit:

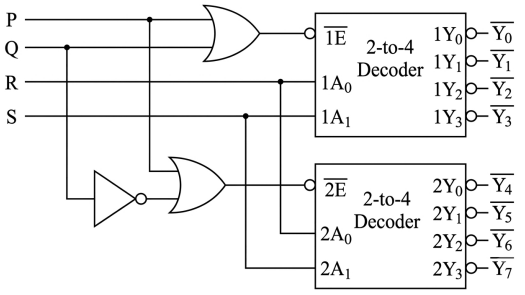


Fig. Solution Q4.15.1

From the network configuration, the individual output line equations are derived as:

$$\overline{Y}_0 = \overline{\overline{1A}_1\,\overline{1A}_0\,\overline{(P+Q)}} = 1A_1 + 1A_0 + P + Q = S + R + P + Q$$

$$\overline{Y}_1 = \overline{\overline{1A}_1 1A_0\,\overline{(P+Q)}} = 1A_1 + \overline{1A}_0 + P + Q = S + \overline{R} + P + Q$$

$$\overline{Y}_2 = \overline{1A_1\,\overline{1A}_0\,\overline{(P+Q)}} = \overline{1A}_1 + 1A_0 + P + Q = \overline{S} + R + P + Q$$

$$\overline{Y}_3 = \overline{1A_1 1A_0\,\overline{(P+Q)}} = \overline{1A}_1 + \overline{1A}_0 + P + Q = \overline{S} + \overline{R} + P + Q$$

$$\overline{Y}_4 = \overline{\overline{2A}_1\,\overline{2A}_0\,\overline{(P+\overline{Q})}} = 2A_1 + 2A_0 + P + \overline{Q} = S + R + P + \overline{Q}$$

$$\overline{Y}_5 = \overline{\overline{2A}_1 2A_0\,\overline{(P+\overline{Q})}} = 2A_1 + \overline{2A}_0 + P + \overline{Q} = S + \overline{R} + P + \overline{Q}$$

$$\overline{Y}_6 = \overline{2A_1\,\overline{2A}_0\,\overline{(P+\overline{Q})}} = \overline{2A}_1 + 2A_0 + P + \overline{Q} = \overline{S} + R + P + \overline{Q}$$

$$\overline{Y}_7 = \overline{2A_1 2A_0\,\overline{(P+\overline{Q})}} = \overline{2A}_1 + \overline{2A}_0 + P + \overline{Q} = \overline{S} + \overline{R} + P + \overline{Q}$$

By structural comparison of the corresponding terms:

- Comparing $\overline{Y}_0$ and $\overline{Y}_1$: The variation occurs in S_0 only which can be seen same way in $R \implies R \rightarrow S_0$
- Comparing $\overline{Y}_0$ and $\overline{Y}_4$: The variation occurs in S_2 only which can be seen same way in $Q \implies Q \rightarrow S_2$
- Comparing $\overline{Y}_4$ and $\overline{Y}_6$: The variation occurs in S_1 only which can be seen same way in $S \implies S \rightarrow S_1$
- Establishing that terminal P drives the inverted data entry node: $\implies P \rightarrow \overline{D}_{in}$

The complete equivalent block mapping configuration is illustrated below:

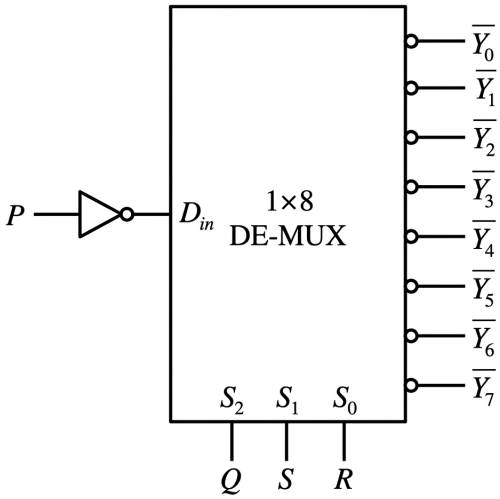


Fig. Solution Q4.15.1

(D) D_{in}, S_2, S_0, S_1

Key Points

- Demultiplexer line identification is performed by isolating which select bit causes a state inversion between specific pairs of output equations.
- Cascaded decoder implementations rely on mapping logic functions to separate enable inputs $(\overline{1E}, \overline{2E})$ for block selection.

Q4.14.1 MCQ 2014 2M Ans: C

Given combinational logic network consisting of two cascaded 4×1 multiplexers:

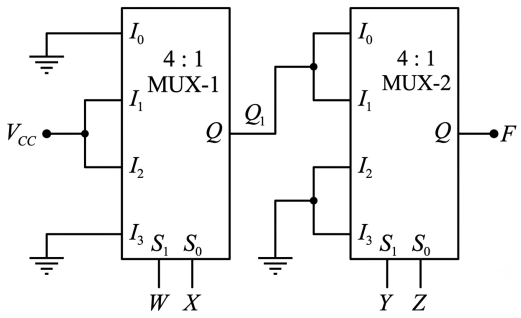


Fig. Solution Q4.14.1

For the first multiplexer (MUX-1),

$$S_1 = W \quad S_0 = X; \quad I_0 = 0, \quad I_1 = 1, \quad I_2 = 1, \quad I_3 = 0$$

The corresponding logic function expression for intermediate output node Q_1 is:

$$Q_1 = \overline{S}_1\overline{S}_0I_0 + \overline{S}_1S_0I_1 + S_1\overline{S}_0I_2 + S_1S_0I_3 = \overline{W}\,\overline{X}(0) + \overline{W}X(1) + W\overline{X}(1) + WX(0)$$

$$= \overline{W}X + W\overline{X}$$

For the second multiplexer (MUX-2),

$$S_1 = Y \quad S_0 = Z; \quad I_0 = Q_1, \quad I_1 = Q_1, \quad I_2 = 0, \quad I_3 = 0$$

The expression of the corresponding logic function for the final output of the network F is:

$$\begin{aligned} F &= \overline{S}_1\overline{S}_0 I_0 + \overline{S}_1 S_0 I_1 + S_1\overline{S}_0 I_2 + S_1 S_0 I_3 = \overline{Y}\,\overline{Z}(Q_1) + \overline{Y}Z(Q_1) + Y\overline{Z}(0) + YZ(0) \\ &= \overline{Y}\,\overline{Z}Q_1 + \overline{Y}ZQ_1 \\ &= \overline{Y}Q_1(\overline{Z} + Z) \\ &= \overline{Y}Q_1 \\ &= \overline{Y}\left(\overline{W}X + W\overline{X}\right) \quad (\text{since } Q_1 = \overline{W}X + W\overline{X}) \\ &= \overline{W}X\overline{Y} + W\overline{X}\,\overline{Y} \end{aligned}$$

$$\boxed{\text{(C) } F = \overline{W}X\overline{Y} + W\overline{X}\,\overline{Y}}$$

Q4.14.2 MCQ 2014 2M Ans: A ●●○

A half-subtractor computes the difference (D) and borrow (B) of two binary inputs X and Y (where $D = X - Y$). The standard logic expressions are:

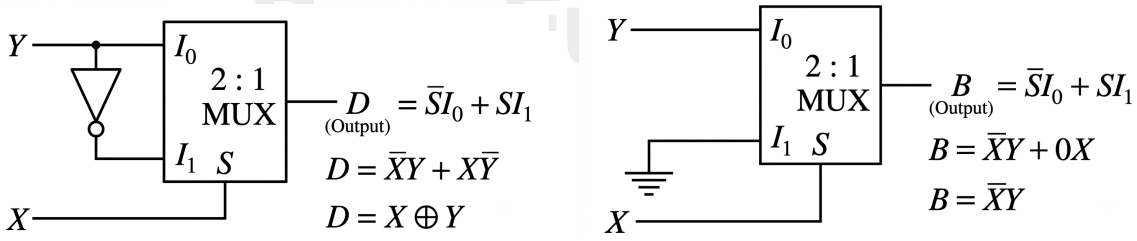
$$D = X \oplus Y = \overline{X}Y + X\overline{Y}$$

$$B = \overline{X}Y$$

Method 1

A 2×1 multiplexer with a select line S implements the function:

$$\text{Output} = \overline{S}\,I_0 + S\,I_1$$



Combining both configurations using X as the common select line matches the circuit arrangement shown in option (A).

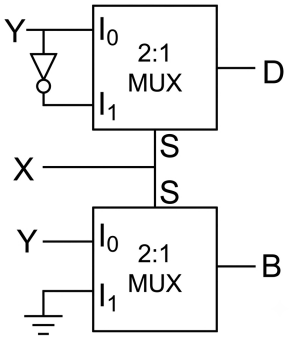


Fig. Solution Q4.14.2

Method 2

Option Elimination Technique: The difference expression for a half-subtractor is:

$$D = X \oplus Y = \overline{X}\,Y + X\,\overline{Y}$$

.

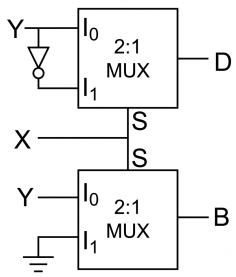
For a 2×1 MUX with select input S, the output is $\overline{S}\,I_0 + S\,I_1$.

- If $S = X$, then I_0 must be Y and I_1 must be $\overline{Y}$
- If $S = Y$, then I_0 must be X and I_1 must be $\overline{X}$

Thus an inverter is necessary at the input for Difference(D) output.

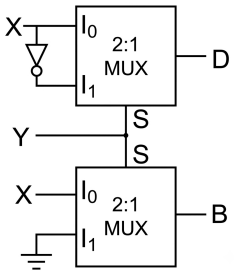
In options (C) and (D), the D output lacks the inverted input structure for the XOR relation, thus eliminating them. Now, analyzing the borrow output ($B = \overline{X}\,Y$) for the remaining options (A) and (B):

Option (A)



$B = \overline{X}\,Y + X\,(0) = \overline{X}\,Y$

Option (B)



$B = \overline{Y}\,X + Y\,(0) = \overline{Y}\,X$

Thus, option (B) is eliminated, leaving option (A) as the correct choice.

Why options (A) and (C) are different: the two diagrams use the same MUX wiring, but their output labels are swapped. In (A), the MUX that selects between Y and $\overline{Y}$ (giving $X \oplus Y$) is labelled D, and the MUX that selects between Y and 0 (giving $\overline{X}\,Y$) is labelled B. In (C) these labels are interchanged. For example, at $X = 1, Y = 0$ a half-subtractor must give $D = 1, B = 0$; option (A) gives exactly this, while option (C) gives $D = 0, B = 1$.

(A)

Key Points

The standard Boolean expressions for a half-subtractor $D = X - Y$ are $D = X \oplus Y$ and $B = \overline{X}\,Y$.

Mistakes to be avoided

- When computing $X - Y$, a borrow is only generated when subtracting a higher value from a lower value ($X = 0$ and $Y = 1$). Therefore, the minuend (X) is always complemented while the subtrahend (Y) remains uncomplemented, giving $B = \overline{X}\,Y$. Confusing this order will lead to selecting incorrect MUX inputs.

Q4.14.3 MCQ | 2014 | 2M | Ans: C | ● ○ ○

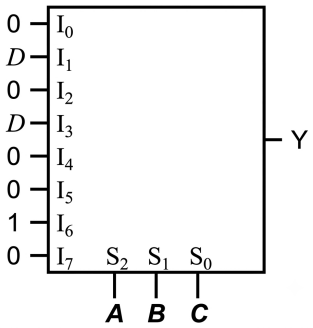


Fig. Solution Q4.14.3

From the given circuit diagram, the data input lines are configured as follows:

$$I_1 = I_3 = D; \quad I_6 = 1; \quad I_0 = I_2 = I_4 = I_5 = I_7 = 0$$

Substituting these input values into the structural MUX equation, we keep only the terms corresponding to non-zero inputs:

$$Y = m_1 \cdot I_1 + m_3 \cdot I_3 + m_6 \cdot I_6$$

$$Y = (\overline{A}\,\overline{B}\,C \cdot D) + (\overline{A}\,B\,C \cdot D) + (A\,B\,\overline{C} \cdot 1)$$

Factoring out the common terms $\overline{A}\,C\,D$ from the first two expressions:

$$Y = \overline{A}\,C\,D\,(\overline{B} + B) + A\,B\,\overline{C}$$

$$Y = \overline{A}\,C\,D\,(1) + A\,B\,\overline{C}$$

$$Y = A\,B\,\overline{C} + \overline{A}\,C\,D$$

This expression matches option (C).

$$\boxed{\text{(C)}Y = A\,B\,\overline{C} + \overline{A}\,C\,D}$$

Q4.14.4 NAT 2014 2M Ans: 194.9 to 195.1 ●●○

Analysis of Critical Path Delay: In a 16-bit ripple carry adder, all inputs (A_n, B_n) are applied at once.
However, each full adder must wait for the carry bit to ripple through from the preceding stages before computing its final output.
Given the propagation delays per stage: $t_{\text{sum}} = 15\,\text{ns}$; $t_{\text{carry}} = 12\,\text{ns}$
Since $t_{\text{sum}} > t_{\text{carry}}$, the worst-case critical path occurs when:

1. The carry bit propagates sequentially through the first $(n-1)$ stages.
2. The final n^{th} stage waits for this carry to arrive, then completes its operation by generating the final sum bit (S_{15}).

Thus, The worst-case propagation delay will be given by:

$$T_{\text{delay}} = (n-1) \cdot t_{\text{carry}} + 1 \cdot t_{\text{sum}}$$

Substituting the design parameters ($n = 16$, $t_{\text{carry}} = 12\,\text{ns}$, and $t_{\text{sum}} = 15\,\text{ns}$):

$$T_{\text{delay}} = (16-1) \cdot 12\,\text{ns} + 1 \cdot 15\,\text{ns}$$

$$T_{\text{delay}} = 15 \cdot 12 + 15$$

$$T_{\text{delay}} = 180 + 15 = 195\,\text{ns}$$

$$\boxed{195}$$

Key Points

- The generalized formula for the worst-case propagation delay in an n-bit ripple carry adder depends on the relation between t_{sum} and t_{carry}:
  $T_{delay} = (n-1)t_{carry} + \max(t_{sum}, t_{carry})$

Mistakes to be avoided

- Avoid multiplying the entire sequence linearly by a single parameter (e.g., 16×15 ns), as intermediate carry generation overlaps in parallel with sum calculations for previous bits.

Q4.14.5 MCQ 2014 1M Ans: C ●○○

A half-subtractor performs the subtraction of two single-bit binary inputs, X (minuend) and Y (subtrahend), to produce the Difference (N) and Borrow (M).
The Boolean functional expressions are derived as follows:

- **Difference (N):** The output is high only when the inputs are distinct.

$$N = X\overline{Y} + \overline{X}Y = X \oplus Y$$

- **Borrow (M):** A borrow is required only when a higher value is subtracted from a lower value ($X = 0$ and $Y = 1$).

$$M = \overline{X}Y$$

This matches the expressions given in option (C).

$$\boxed{\text{(C)}M = \overline{X}Y,\ N = X \oplus Y}$$

Mistakes to be avoided

- When computing $X - Y$, a borrow is only generated when subtracting a higher value from a lower value ($X = 0$ and $Y = 1$). Therefore, the minuend (X) is always complemented while the subtrahend (Y) remains uncomplemented, giving $B = \overline{X}Y$. Confusing this order will lead to incorrect answer.

Q4.14.6 MCQ 2014 1M Ans: D ●○○

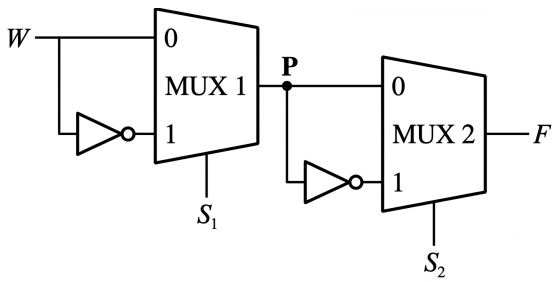


Fig. Solution Q4.14.6

The circuit consists of two cascaded 2×1 multiplexers. The standard behavioral output equation for a 2×1 MUX with a select input S is:

$$\text{Output} = \overline{S} \cdot I_0 + S \cdot I_1$$

Step 1: Output of the first MUX (P)
For the first MUX, the parameters are $S = S_1$, $I_0 = W$, and $I_1 = \overline{W}$:

$$P = \overline{S}_1 \cdot W + S_1 \cdot \overline{W} = S_1 \oplus W$$

Step 2: Final Output of the second MUX (F)
We can see the exact same circuit has been repeated again.Thus,

$$F = S_2 \oplus P$$

Substituting the expression for P:

$$F = S_2 \oplus (S_1 \oplus W) = W \oplus S_1 \oplus S_2$$

This identity matches the expression given in option (D).

$$\boxed{\text{(D)} F = W \oplus S_1 \oplus S_2}$$

Key Points

- XOR operations satisfy associative and commutative properties, meaning the order of variables does not impact the output ($W \oplus S_1 \oplus S_2 = S_2 \oplus S_1 \oplus W$).

Q4.11.1 MCQ 2011 1M Ans: D ●●○

From the given logic circuit, ground represents logic 0. Tracing the circuit gives the MUX inputs:

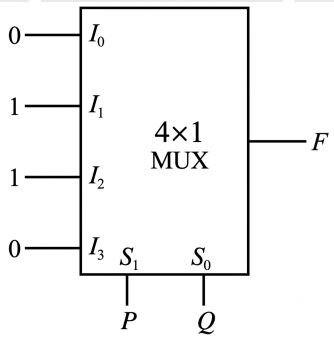


Fig. Solution Q4.11.1

- $I_3 = 0$ (connected directly to ground)
- $I_1 = I_2 = \overline{0} = 1$ (after the first inverter)
- $I_0 = \overline{1} = 0$ (after the second inverter)

The output equation for a 4×1 MUX with selection lines $S_1 = P$ and $S_0 = Q$ is:

$$F = \overline{S}_1\overline{S}_0 I_0 + \overline{S}_1 S_0 I_1 + S_1\overline{S}_0 I_2 + S_1 S_0 I_3$$

Substituting the input values:

$$F = \overline{P}\,\overline{Q}(0) + \overline{P}Q(1) + P\overline{Q}(1) + PQ(0)$$

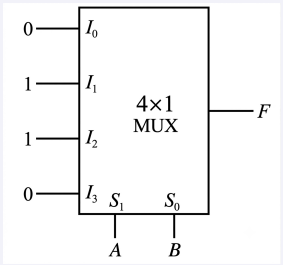
$$F = \overline{P}Q + P\overline{Q} = P \oplus Q = \text{XOR}(P, Q)$$

$$\boxed{\text{(D) } F = \text{XOR}(P, Q)}$$

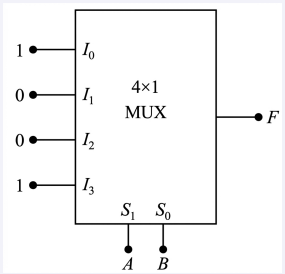
Key Points

- Standard MUX Implementations: A 4×1 MUX can realize basic functions using fixed data inputs:

XOR(A, B):



XNOR(A, B):



Q4.10.1 MCQ 2010 2M Ans: D ●●○

The given logic circuit consists of a 4×1 Multiplexer (MUX) with selection lines $S_1 = A$ and $S_0 = B$.
The output equation of a standard 4×1 MUX is given by:

$$F = \overline{S_1}\,\overline{S_0} I_0 + \overline{S_1} S_0 I_1 + S_1 \overline{S_0} I_2 + S_1 S_0 I_3$$

From the given circuit diagram, the input lines are connected as follows:

- $I_0 = C$
- $I_1 = D$
- $I_2 = \overline{C}$ (via NOT gate)
- $I_3 = \overline{C}\,\overline{D}$ (via bubbled AND gate)

Substituting the selection lines and inputs into the MUX equation:

$$F = \overline{A}\,\overline{B}C + \overline{A}BD + A\overline{B}\,\overline{C} + AB(\overline{C}\,\overline{D})$$

To find the minterms, we expand each term into its standard canonical Sum-of-Products (SOP) form:

$$\overline{A}\,\overline{B}C = \overline{A}\,\overline{B}C(D + \overline{D}) = \overline{A}\,\overline{B}CD + \overline{A}\,\overline{B}C\overline{D} \implies m_3 + m_2$$
$$\overline{A}BD = \overline{A}BD(C + \overline{C}) = \overline{A}BCD + \overline{A}B\overline{C}D \implies m_7 + m_5$$
$$A\overline{B}\,\overline{C} = A\overline{B}\,\overline{C}(D + \overline{D}) = A\overline{B}\,\overline{C}D + A\overline{B}\,\overline{C}\,\overline{D} \implies m_9 + m_8$$
$$AB\overline{C}\,\overline{D} = m_{12}$$

Combining all the minterms, we get:

$$F = \sum m(2, 3, 5, 7, 8, 9, 12)$$

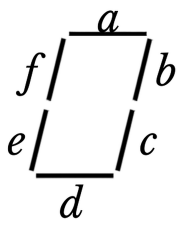
$$\boxed{\text{(D) } F = \sum m(2, 3, 5, 7, 8, 9, 12)}$$

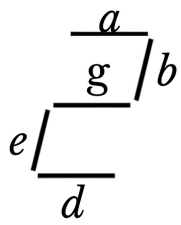
Q4.09.1 MCQ 2009 2M Ans: B ●●●

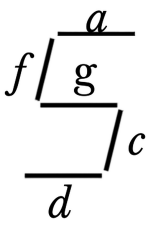
The following are the details of which symbol will be displayed according to the keys (P_1, P_2) pressed-

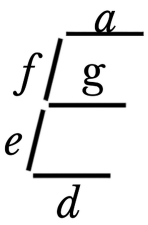
- If no buttons are pressed, '0' is displayed, signifying 'Rs. 0'.
- If only P_1 is pressed, '2' is displayed, signifying 'Rs. 2'.
- If only P_2 is pressed, '5' is displayed, signifying 'Rs. 5'.
- If both P_1 and P_2 are pressed, 'E' is displayed, signifying 'Error'.

To display '0', '2', '5', and 'E', the required active segments are mapped as follows:









Designing a truth table from the above figures-

No.	P_1	P_2	a	b	c	d	e	f	g
0.	0	0	1	1	1	1	1	1	0
1.	0	1	1	0	1	1	0	1	1
2.	1	0	1	1	0	1	1	0	1
3.	1	1	1	0	0	1	1	1	1

To comprehensively analyze the circuit, we derive the minimized Boolean expressions for all 7 segments (a through g) using the truth table:

1. **Segment a:** It is 1 for all input combinations.
$$a = 1$$

2. **Segment b:** It is 1 at rows 0 and 2.
$$b = \overline{P_1}\,\overline{P_2} + P_1\overline{P_2} = (\overline{P_1} + P_1)\overline{P_2} = 1 \cdot \overline{P_2} = \overline{P_2}$$

3. **Segment c:** It is 1 at rows 0 and 1.
$$c = \overline{P_1}\,\overline{P_2} + \overline{P_1}P_2 = \overline{P_1}(\overline{P_2} + P_2) = \overline{P_1} \cdot 1 = \overline{P_1}$$

4. **Segment d:** It is 1 for all input combinations.
$$d = 1$$

5. **Segment e:** It is 1 at rows 0, 2, and 3.
$$e = \overline{P_1}\,\overline{P_2} + P_1\overline{P_2} + P_1P_2 = (\overline{P_1} + P_1)\overline{P_2} + P_1P_2 = \overline{P_2} + P_1P_2$$
Applying the distributive property ($X + \overline{X}Y = X + Y$):
$$e = \overline{P_2} + P_1$$

6. **Segment f:** It is 1 at rows 0, 1, and 3.
$$f = \overline{P_1}\,\overline{P_2} + \overline{P_1}P_2 + P_1P_2 = \overline{P_1}(\overline{P_2} + P_2) + P_1P_2 = \overline{P_1} + P_1P_2$$
Applying the distributive property:
$$f = \overline{P_1} + P_2$$

7. **Segment g:** It is 1 at rows 1, 2, and 3.
$$g = \overline{P_1}P_2 + P_1\overline{P_2} + P_1P_2 = \overline{P_1}P_2 + P_1(\overline{P_2} + P_2) = \overline{P_1}P_2 + P_1$$
Applying the distributive property:
$$g = P_2 + P_1 = P_1 + P_2$$

Checking options with $d = c + e$:

$$c + e = (\overline{P_1}\,\overline{P_2} + \overline{P_1}P_2) + (\overline{P_1}\,\overline{P_2} + P_1\overline{P_2} + P_1P_2) = \overline{P_1} + \overline{P_2} + P_1 = 1 = d$$

Thus, $g = P_1 + P_2$ and $d = c + e$.

$$\boxed{\text{(B) } g = P_1 + P_2,\ d = c + e}$$

Key Points

- A standard 7-segment display uses LED segments labeled a to g to form characters based on digital logic.

Q4.09.2 MCQ 2009 2M Ans: D ●●●

To determine the minimum number of NOT gates and 2-input OR gates needed to drive the complete display, we determine the simplified minimal expressions for all segments from the truth table:

- $a = 1 \implies$ (0 NOT, 0 OR)
- $b = \overline{P_1}\,\overline{P_2} + P_1\overline{P_2} = \overline{P_2} \implies$ (Uses 1 NOT gate, 0 OR)
- $c = \overline{P_1}\,\overline{P_2} + \overline{P_1}P_2 = \overline{P_1} \implies$ (Uses 1 NOT gate, 0 OR)
- $d = 1 \implies$ (0 NOT, 0 OR)
- $e = \overline{P_1}\,\overline{P_2} + P_1\overline{P_2} + P_1P_2 = \overline{P_2} + P_1 \implies$ (Uses existing $\overline{P_2}$, requires 1 OR)
- $f = \overline{P_1}\,\overline{P_2} + \overline{P_1}P_2 + P_1P_2 = \overline{P_1} + P_2 \implies$ (Uses existing $\overline{P_1}$, requires 1 OR)
- $g = P_1 + P_2 \implies$ (0 NOT, requires 1 OR)

Total resource allocation summary:

- **NOT gates:** 2 distinct inverted lines needed ($\overline{P_1}$ and $\overline{P_2}$).
- **OR gates:** 3 distinct 2-input combinations required ($P_1 + \overline{P_2}$, $\overline{P_1} + P_2$, and $P_1 + P_2$).

$$\boxed{\text{(D) 2 NOT and 3 OR}}$$

Mistakes to be avoided

- Do not double count NOT gates for shared inverted inputs; once $\overline{P_1}$ and $\overline{P_2}$ are generated, they can be distributed to all segment logic circuits.

Q4.09.3 MCQ 2009 2M Ans: A ●●○

To determine the minimum number of 2×1 multiplexers (MUX) required to generate a 2-input AND gate and a 2-input Ex-OR gate, we analyze each logic function separately:

1. **Implementation of a 2-input AND gate ($Y = AB$):** The output equation of a standard 2×1 MUX with selection line S is:

   $$Y = \overline{S}I_0 + SI_1$$

   By choosing the selection line $S = A$, and assigning fixed inputs $I_0 = 0$ and $I_1 = B$:

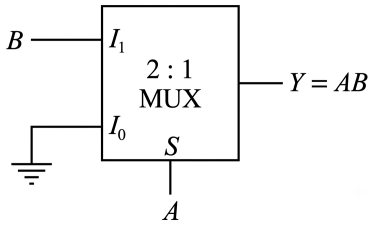


Fig. Solution Q4.09.3

$$Y = \overline{A} \cdot 0 + A \cdot B = AB$$

Thus, exactly **1**, 2×1 MUX is required to implement a 2-input AND gate.

2. **Implementation of a 2-input Ex-OR gate** ($Y = A \oplus B = \overline{A}B + A\overline{B}$)**:** Using a standard 2×1 MUX with selection line $S = A$:

$$Y = \overline{A}I_0 + AI_1$$

Comparing this with the Ex-OR expression, we require $I_0 = B$ and $I_1 = \overline{B}$.

Since the complemented literal $\overline{B}$ is not directly available, we must generate it using another 2×1 MUX configured as an inverter:

$$Y_1 = \overline{B} \cdot 1 + B \cdot 0 = \overline{B}$$

This inverter MUX uses $S = B$, $I_0 = 1$, and $I_1 = 0$. Feeding its output $Y_1 = \overline{B}$ into input I_1 of the first MUX gives the complete Ex-OR function.

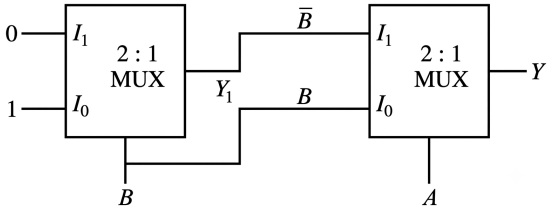


Fig. Solution Q4.09.3

Thus, exactly **2** 2×1 MUXs are required to implement a 2-input Ex-OR gate.

Therefore, the minimum number of 2×1 multiplexers needed is 1 and 2, respectively.

(A) 1 and 2

Mistakes to be avoided

- Do not assume that complements like $\overline{B}$ are freely available unless explicitly stated in the problem statement. Always account for the extra MUX required for inversion.

Q4.08.1 MCQ 2008 2M Ans: A ●●○

The output Z of a 4 : 1 multiplexer with selection lines $S_1 = R$ and $S_0 = S$ is given by:

$$Z = \overline{R}\,\overline{S}I_0 + \overline{R}SI_1 + R\overline{S}I_2 + RSI_3$$

From the given logic circuit, the inputs to the multiplexer are:

$$I_0 = P + \overline{Q} \qquad I_1 = P \qquad I_2 = PQ \qquad I_3 = P$$

Substituting these inputs into the output expression:

$$Z = \overline{R}\,\overline{S}(P + \overline{Q}) + \overline{R}SP + R\overline{S}PQ + RSP$$

Expressing Z as a sum of minterms $\sum m(P, Q, R, S)$:

- $P\overline{R}\,\overline{S} = PQ\,\overline{R}\,\overline{S} + PQ\overline{R}\,\overline{S} \rightarrow m_8, m_{12}$
- $\overline{Q}\,\overline{R}\,\overline{S} = \overline{P}\,\overline{Q}\,\overline{R}\,\overline{S} + P\overline{Q}\,\overline{R}\,\overline{S} \rightarrow m_0, m_8$
- $P\overline{R}S = PQ\,\overline{R}S + PQ\overline{R}S \rightarrow m_9, m_{13}$
- $PQR\overline{S} \rightarrow m_{14}$
- $PRS = P\overline{Q}RS + PQRS \rightarrow m_{11}, m_{15}$

$$Z = \sum m(0, 8, 9, 11, 12, 13, 14, 15)$$

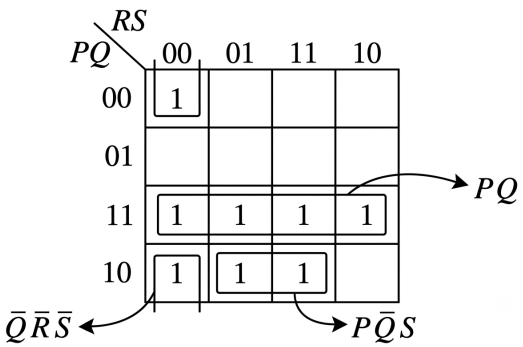


Fig. Solution Q4.08.1

From the K-map, grouping loops gives the following simplified expression:

$$Z = PQ + P\overline{Q}S + \overline{Q}\,\overline{R}\,\overline{S}$$

Note: Ideally we should have grouped a larger 4-cell quad $(m_9, m_{11}, m_{13}, m_{15})$ to get the simpler term PS.

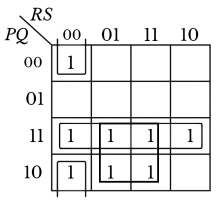


Fig. Solution Q4.08.1

However, to align with the options given, the smaller 2-cell group forming $P\overline{Q}S$ was taken instead.

$$\boxed{\text{(A) } PQ + P\overline{Q}S + \overline{Q}\,\overline{R}\,\overline{S}}$$

Mistakes to be avoided

- Don't ignore a correct expression just because it isn't fully minimized. Check the multiple-choice options to see if you need to group the K-map minterms differently to match them.

Q4.07.1 MCQ 2007 2M Ans: A ●●○

The circuit consists of a 2 : 1 multiplexer cascaded into another logic network.
We can observe that the exact same mux circuit is drawn twice thus let us analyse the mux separately

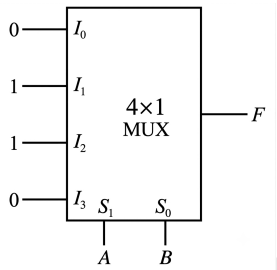


Fig. Solution Q4.07.1

$$I_3 = 0 \qquad I_1 = I_2 = 1 \qquad I_0 = 0$$

Substituting these inputs into the 4×1 MUX output equation with selection lines $S_1 = P$ and $S_0 = Q$:

$$F = \overline{S}_1\overline{S}_0 I_0 + \overline{S}_1 S_0 I_1 + S_1\overline{S}_0 I_2 + S_1 S_0 I_3$$
$$= \overline{A}B(1) + A\overline{B}(1) = \overline{A}B + A\overline{B} = A \oplus B$$

The second multiplexer does the exact same thing.Thus,

$$X = A \oplus B \oplus C$$

An XOR gate outputs a 1 when there is an odd number of 1s in the inputs. To write this in SOP form, we just list the terms where either exactly one variable or all three variables are uncomplemented:

- **Exactly one literal is 1:** $\overline{A}\,\overline{B}C, \quad \overline{A}B\overline{C}, \quad A\overline{B}\,\overline{C}$
- **All three literals are 1:** ABC

Combining these distinct minterm products yields the literal expression matching option (A):

$$X = \overline{A}\,\overline{B}C + \overline{A}B\overline{C} + A\overline{B}\,\overline{C} + ABC$$

$$\boxed{\text{(A) } \overline{A}\,\overline{B}C + \overline{A}B\overline{C} + A\overline{B}\,\overline{C} + ABC}$$

Key Points

- A multi-variable XOR gate outputs a 1 only when there is an odd number of 1s in its inputs.

Q4.05.1 MCQ 2005 1M Ans: A ● ○ ○

Let us analyze the output of each 2×1 multiplexer stage sequentially:

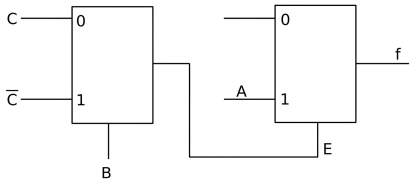


Fig. Solution Q4.05.1

1. **First Stage (MUX-1):** The select line is B, and the data inputs are $I_0 = C$ and $I_1 = \overline{C}$. The output E is given by:

$$E = \overline{B}I_0 + BI_1 = \overline{B}C + B\overline{C}$$

2. **Second Stage (MUX-2):** The select line is E, and the data inputs are $I_0 = 0$ and $I_1 = A$. The final output function f is given by:

$$f = \overline{E}I_0 + EI_1 = \overline{E} \cdot 0 + E \cdot A = AE$$

Substituting the expression for E into the equation for f:

$$f = A(\overline{B}C + B\overline{C}) = A\overline{B}C + AB\overline{C}$$

$$\boxed{\text{(A) } A\overline{B}C + AB\overline{C}}$$

Q4.04.1 MCQ 2004 2M Ans: C ● ○ ○

Method 1

To realize a higher-order 4×1 multiplexer using lower-order 2×1 multiplexers, we construct a tree layout:

- **Stage 1:** We use two 2×1 multiplexers to handle the four primary data lines (I_0, I_1, I_2, I_3) using select line B. This reduces the four lines down to two intermediate outputs.
- **Stage 2:** We use a single 2×1 multiplexer controlled by select line A to select between the two intermediate outputs and yield the final product Y.

Total number of 2×1 multiplexers needed = 2 + 1 = 3.

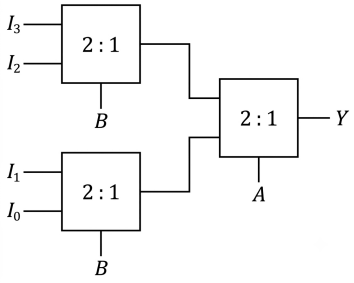


Fig. Solution Q4.04.1

Method 2

Alternatively, we can use the division-based scaling formula to determine the total count:

$$\text{Number of MUXes} = \frac{n}{m} + \frac{n}{m^2} + \dots$$

Where $n = 4$ (inputs of target MUX) and $m = 2$ (inputs of given MUX):

$$\text{Stage 1 count} = \frac{4}{2} = 2$$

$$\text{Stage 2 count} = \frac{2}{2} = 1$$

$$\text{Total count} = 2 + 1 = 3$$

(C) 3

Key Points

- In general, implementing a $2^n \times 1$ multiplexer using standard 2×1 multiplexers always requires exactly $2^n - 1$ units.
- For a 4×1 multiplexer, $2^2 \times 1 \implies 2^2 - 1 = 3$.

Q4.03.1 MCQ | 2003 | 2M | Ans: B

Let us analyze the outputs of a single block with inputs P, Q, and R, and outputs Y and Z.
The given expressions for the outputs are:

$$Y = P \oplus Q \oplus R$$

$$Z = R\,Q + \overline{P}\,R + Q\,\overline{P}$$

Let us compare these with the standard expressions for a Full Subtractor:

- **Difference (D):** $D = P \oplus Q \oplus R_{in}$
- **Borrow (B_{out}):** $B_{out} = \overline{P}\,Q + (\overline{P} + Q)R_{in} = \overline{P}\,Q + \overline{P}\,R_{in} + Q\,R_{in}$

By comparing Z with the standard borrow expression (where R acts as the input borrow R_{in}):

$$Z = \overline{P}\,Q + \overline{P}\,R + Q\,R$$

Thus, each individual block acts as a **Full Subtractor** cell performing the operation:

$$\text{Difference } (Y) = P - Q - R$$

$$\text{Borrow Out } (Z) = \text{Borrow generated from } P - Q - R$$

Since four such blocks are cascaded with the borrow-out (Z) of one stage connected to the borrow-in (R) of the next stage, and the initial borrow-in to the least significant stage is connected to ground (0), the entire circuit functions as a **4-bit subtractor** performing $P - Q$.

(B) 4 bit subtractor-giving $P - Q$

Key Points

- A Full Subtractor computes the difference between two bits taking into account a previous borrow-in.
- The logic equations for a Full Subtractor with inputs A, B and B_{in} are $D = A \oplus B \oplus B_{in}$ and $B_{out} = \overline{A}\,B + \overline{A}\,B_{in} + B\,B_{in}$.

Q4.03.2 MCQ 2003 1M Ans: C ●●○

Method 1

An 8 : 1 MUX has 3 selection lines (say A, B, C) and 8 data inputs (I_0 to I_7).

1. **Implementing 3-variable functions (A, B, C):** By connecting the three variables to the select lines, each minterm directly maps to one of the data inputs. We can feed a constant 0 or 1 to each data input line to achieve **all functions of 3 variables** without any extra circuitry.

2. **Implementing 4-variable functions (A, B, C, D):** To implement a 4-variable function, 3 variables are connected to the selection lines, and the inputs I_0 to I_7 must be driven by combinations of the 4th variable (D), which can only be 0, 1, D, or $\overline{D}$.

   Since the problem specifies **"Without any additional circuitry"**, we do not have an inverter (NOT gate) to generate $\overline{D}$. Therefore, we can only implement functions whose implementation tables do not require a complemented input ($\overline{D}$). This means we can implement **some, but not all, functions of 4 variables**.

(C) all functions of 3 variables and some but not all of 4 variables

Key Points

- A 2^n : 1 MUX with no external gates can implement **all** functions of n **variables**.
- A 2^n : 1 MUX with **no inverter** can implement **some but not all** functions of $(n + 1)$ variables.
- A 2^n : 1 MUX **with an inverter** can implement **all** functions of $(n + 1)$ variables.

Q4.02.1 Descriptive 2002 5M Ans: - ●●○

The given output function of the digital circuit is:

$$X = \overline{A}\,B + \overline{A}\,\overline{B}\,\overline{C}$$

1. **Canonical SOP Form:** To convert it into canonical SOP form, expand each term to include all variables (A, B, C):

$$X = \overline{A}\,B(C + \overline{C}) + \overline{A}\,\overline{B}\,\overline{C}$$

$$X = \overline{A}\,B\,C + \overline{A}\,B\,\overline{C} + \overline{A}\,\overline{B}\,\overline{C}$$

   Rearranging in ascending order of minterms (m_0, m_2, m_3):

$$\boxed{X = \overline{A}\,\overline{B}\,\overline{C} + \overline{A}\,B\,\overline{C} + \overline{A}\,B\,C}$$

2. **Canonical POS Form:** The missing minterms from the 3-variable space are m_1, m_4, m_5, m_6, and m_7. The canonical POS form is the product of the corresponding maxterms:

$$\boxed{X = (A + B + \overline{C})(\overline{A} + B + C)(\overline{A} + B + \overline{C})(\overline{A} + \overline{B} + C)(\overline{A} + \overline{B} + \overline{C})}$$

Mistakes to be avoided

- Avoid the common misconception that the POS form is simply the complemented version of the SOP form (or vice versa). They are actually two distinct, independent structural representations of the same underlying Boolean function (F).

Q4.02.2 Descriptive 2002 5M Ans: ● ● ○

To implement the function $X = \overline{A}\,B + \overline{A}\,\overline{B}\,\overline{C}$ using exactly two 2:1 multiplexers without using external inverters, we factor out the common term:

$$X = \overline{A}(B + \overline{B}\,\overline{C})$$

Using the distributive law ($B + \overline{B}\,\overline{C} = B + \overline{C}$), the expression simplifies to:

$$X = \overline{A}(B + \overline{C})$$

We can implement this simplified expression using two 2:1 multiplexers by choosing the selection lines strategically:

1. **First Multiplexer (MUX 1):** Implementing the sub-expression $(B + \overline{C})$ Let the select line be $S = C$.
   - When $C = 0$, the expression $B + \overline{C} = B + 1 = 1$. Thus, input $D_0 = 1$ (+5 V).
   - When $C = 1$, the expression $B + \overline{C} = B + 0 = B$. Thus, input $D_1 = B$.

   The output of MUX 1 is:
   $$Y_1 = \overline{C}(1) + C(B) = B + \overline{C}$$
2. **Second Multiplexer (MUX 2):** Implementing the final output $X = \overline{A}(B + \overline{C})$ Let the select line be $S = A$.
   - When $A = 0$, $X = B + \overline{C} = Y_1$. Thus, input $D_0 = Y_1$.
   - When $A = 1$, $X = 0$. Thus, input $D_1 = 0$ (Ground).

   The output of MUX 2 is:
   $$Y_2 = \overline{A}Y_1 + A(0) = \overline{A}(B + \overline{C}) = X$$

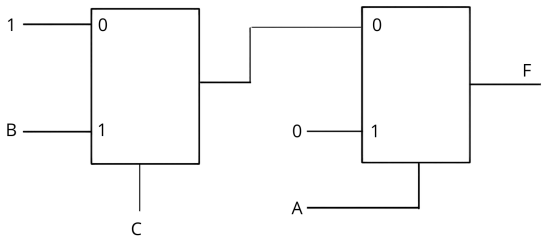


Fig. Solution Q4.02.2

Q4.02.3 MCQ 2002 2M Ans: B ● ● ○

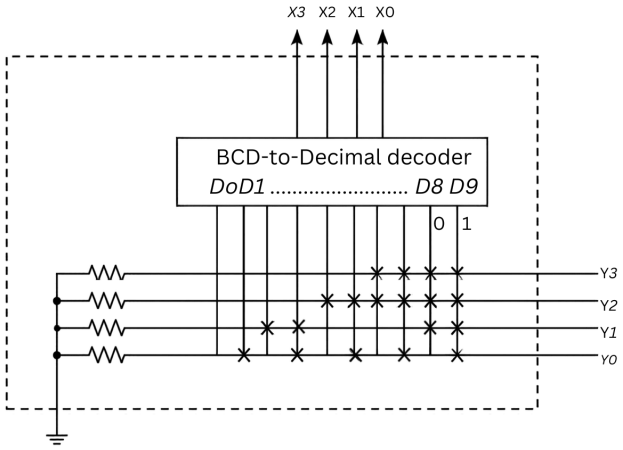


Fig. Solution Q4.02.3

The input variables $X_3X_2X_1X_0$ are 8-4-2-1 BCD numbers representing decimal digits from 0 to 9. These inputs feed into a BCD-to-Decimal decoder, which activates exactly one output line D_i corresponding to the decimal value i.
From the cross-point connections in the ROM array, we extract the Boolean sum expressions for each output line (Y_3, Y_2, Y_1, Y_0) by observing the connections (marked with $\times$):

$$Y_3 = D_6 + D_7 + D_8 + D_9$$

$$Y_2 = D_4 + D_5 + D_6 + D_7 + D_8 + D_9$$

$$Y_1 = D_2 + D_3 + D_8 + D_9$$

$$Y_0 = D_1 + D_3 + D_5 + D_7 + D_9$$

Let us construct the truth table for the outputs across all valid decimal inputs:

Decimal	X_3	X_2	X_1	X_0	Active Line	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	D_0	0	0	0	0
1	0	0	0	1	D_1	0	0	0	1
2	0	0	1	0	D_2	0	0	1	0
3	0	0	1	1	D_3	0	0	1	1
4	0	1	0	0	D_4	0	1	0	0
5	0	1	0	1	D_5	0	1	0	1
6	0	1	1	0	D_6	1	1	0	0
7	0	1	1	1	D_7	1	1	0	1
8	1	0	0	0	D_8	1	1	1	0
9	1	0	0	1	D_9	1	1	1	1

To identify the output weight sequence w_3-w_2-w_1-w_0, we verify the representation for decimal 5 (0101) and decimal 6 (1100):

- For decimal 5: $0 \cdot w_3 + 1 \cdot w_2 + 0 \cdot w_1 + 1 \cdot w_0 = 5 \implies w_2 + w_0 = 5$
- For decimal 6: $1 \cdot w_3 + 1 \cdot w_2 + 0 \cdot w_1 + 0 \cdot w_0 = 6 \implies w_3 + w_2 = 6$

Testing the weights 2-4-2-1:

- Decimal 5: $4 + 1 = 5$ (Satisfied)
- Decimal 6: $2 + 4 = 6$ (Satisfied)

Thus, the output code format strictly corresponds to 2-4-2-1 BCD.

(B) 2-4-2-1 BCD numbers

Q4.01.1 MCQ | 2001 | 2M | Ans: B | ● ● ○

Given circuit is an 8×1 Multiplexer. In TTL logic, any unused or floating input is treated as logic-1.
The input to the OR gate connected to select line S_2 has a floating pin and an input C. Therefore:

$$S_2 = 1 + C = 1$$

The selection lines and input configurations are:

- **Selection lines:** $S_2 = 1$, $S_1 = B$, and $S_0 = A$.
- **Enable input:** $E = \overline{1} = 0$ (The active-low enable pin is connected to ground/0, so the multiplexer is active).
- **Input lines connected to logic-1:** $X_0 = X_3 = X_5 = X_6 = 1$
- **Input lines connected to logic-0:** $X_1 = X_2 = X_4 = X_7 = 0$

The output equation of an 8×1 MUX is given by:

$$Y = \overline{S_2}\,\overline{S_1}\,\overline{S_0}X_0 + \overline{S_2}\,\overline{S_1}S_0X_1 + \overline{S_2}S_1\overline{S_0}X_2 + \overline{S_2}S_1S_0X_3 + S_2\overline{S_1}\,\overline{S_0}X_4 + S_2\overline{S_1}S_0X_5 + S_2S_1\overline{S_0}X_6 + S_2S_1S_0X_7$$

Substituting the values ($S_2 = 1$ makes all terms with $\overline{S_2}$ equal to 0):

$$Y = 1 \cdot \overline{B} \cdot \overline{A} \cdot (0) + 1 \cdot \overline{B} \cdot A \cdot (1) + 1 \cdot B \cdot \overline{A} \cdot (1) + 1 \cdot B \cdot A \cdot (0)$$

$$Y = \overline{B}A + B\overline{A} = A \oplus B$$

(B) $A \oplus B$

Key Points

- **TTL Logic Principle:** Floating or unconnected inputs in TTL (Transistor-Transistor Logic) circuits float to a high state (logic-1).
- **ECL Logic Principle:** Unused/floating inputs in ECL (Emitter-Coupled Logic) work as logic-0.

Q4.00.1 NAT 2000 Ans: * ●●○

A one-bit full adder adds three input bits (A, B, and the previous carry C_p) to produce two outputs: Sum (S) and Next Stage Carry (C_N).
The truth table in the ascending order of (A, B, C_p) is given below:

A	B	C_p	Sum (S)	C_N
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

From the truth table, the standard sum-of-minterms expressions are:

$$S(A, B, C_p) = \sum m(1, 2, 4, 7)$$

$$C_N(A, B, C_p) = \sum m(3, 5, 6, 7)$$

$$S = \sum m(1, 2, 4, 7), \quad C_N = \sum m(3, 5, 6, 7)$$

Key Points

- **Full Adder Outputs:** Sum represents the XOR sum of all three inputs ($A \oplus B \oplus C_p$), while Carry-out (C_N) is high when any two or more inputs are high ($AB + BC_p + AC_p$).

Q4.00.2 NAT 2000 Ans: * ●●○

To implement the Sum (S) and Next Carry (C_N) expressions using 8×1 multiplexers, we map the input variables directly to the multiplexer select lines.
By connecting the select lines as $S_2 = A$, $S_1 = B$, and $S_0 = C_p$:

1. **Implementation of Sum (S):** The minterm expression is $S = \sum m(1, 2, 4, 7)$. Therefore, the corresponding data inputs are connected to logic-1 (V_{CC}) and the remaining inputs are connected to logic-0 (GND):

$$X_1 = X_2 = X_4 = X_7 = 1 \quad \text{and} \quad X_0 = X_3 = X_5 = X_6 = 0$$

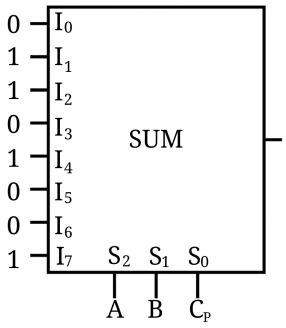


Fig. Solution Q4.00.2

2. **Implementation of Next Carry (C_N):** The minterm expression is $C_N = \sum m(3, 5, 6, 7)$. Therefore, the corresponding data inputs are connected to logic-1 (V_{CC}) and the remaining inputs are connected to logic-0 (GND):

$$X_3 = X_5 = X_6 = X_7 = 1 \quad \text{and} \quad X_0 = X_1 = X_2 = X_4 = 0$$

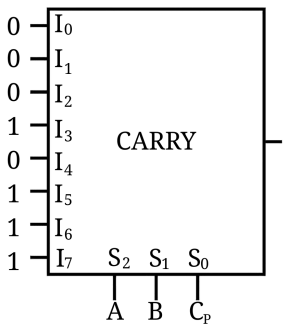


Fig. Solution Q4.00.2

Key Points

- **MUX Custom Logic Synthesis:** An $2^n \times 1$ multiplexer can directly realize any n-variable Boolean function by routing the truth table outputs to the corresponding data lines.

Q4.99.1 MCQ 1999 2M Ans: C ● ○ ○

The truth table for a binary half-subtractor with inputs A and B, difference D (A minus B), and borrow X is given below:

A	B	Difference (D)	Borrow (X)
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

From the truth table, the logical expressions can be obtained directly as standard sum-of-products (SOP):

1. **For Difference (D):** The output D is high when the inputs are different:

$$D = \overline{A}B + A\overline{B}$$

2. **For Borrow (X):** The output X is high only when a larger digit is subtracted from a smaller digit ($0 - 1$):

$$X = \overline{A}B$$

$$\boxed{\text{(C) } D = \overline{A}B + A\overline{B}, X = \overline{A}B}$$

Key Points

- **Half-Subtractor vs Half-Adder:** The difference expression for a half-subtractor is identical to the sum expression of a half-adder ($A \oplus B$). The borrow expression differs because it requires inversion of the minuend input ($X = \overline{A}B$).

Q4.97.1 MCQ 1997 1M Ans: B ● ● ○

To multiply two 2-bit numbers, let the operands be $A = A_1A_0$ and $B = B_1B_0$. The multiplication array is formed as follows:

$$\begin{array}{cccc} & & B_1 & B_0 \\ \times & & A_1 & A_0 \\ \hline & & A_0B_1 & A_0B_0 \\ + & A_1B_1 & A_1B_0 & \\ \hline & P_2 & P_1 & P_0 \end{array}$$

The partial products are generated using 2-input AND gates:

- Term 1: A_0B_0
- Term 2: A_0B_1
- Term 3: A_1B_0
- Term 4: A_1B_1

This requires exactly $2^2 = 4$ 2-input AND gates.
To sum these partial products to form the final product bits ($P_2P_1P_0$ along with any final carry out):

- $P_0 = A_0B_0$ (No addition required).
- P_1 is obtained by adding A_0B_1 and A_1B_0 using a half-adder. The sum is P_1 and the carry is C_1.
- P_2 (and the final carry) is obtained by adding A_1B_1 and the previous carry C_1 using a second half-adder.

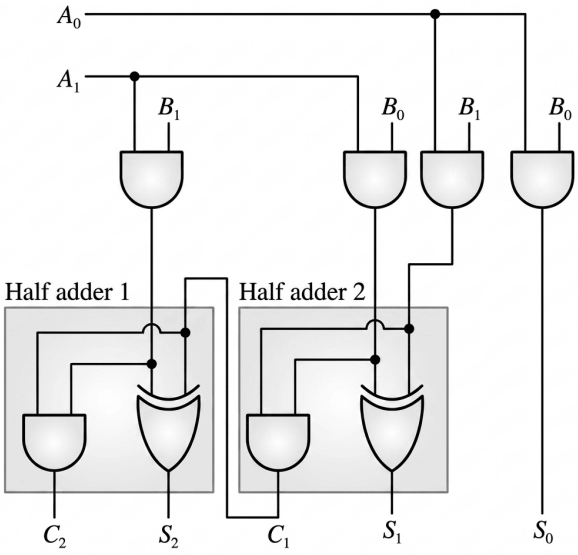


Fig. Solution Q4.97.1

A standard half-adder consists of one 2-input XOR gate and one 2-input AND gate. Therefore, the complete circuit requires 2 XOR gates and 6 AND gates (4 for the partial products and 2 inside the half-adders), meaning it can be realized using only XOR and AND gates.

Note that option (B) names gate types, not a gate count: the phrase *4 input AND gates* describes the fan-in of the AND gates, not a total of four AND gates. Four AND gates alone generate only the partial products; the half-adders are also needed. A 4-input AND gate can act as a 2-input AND gate by repeating its inputs or tying the unused inputs to logic 1, so the circuit can be built from 2-input XORs and 4-input AND gates only.

(B) 2 input XORs and 4 input AND gates only

Key Points

- **General Combinational Multiplier Complexity:** For an $n \times n$-bit multiplier:
  - Total number of 2-input AND gates required for partial products = n^2.
  - Total number of Half-Adders required = n.
  - Total number of Full-Adders required = $n^2 - 2n$.

Q4.96.1 Descriptive | 1996 | 3M | Ans: a-3, b-4, c-2 | ●●○

To determine the correct matching for each digital logic circuit, we analyze the functional properties and applications of shift registers, multiplexers, and decoders.

Let us break down the function of each digital system configuration:

- **(a) Shift Register:** A Parallel-Input Serial-Output (PISO) shift register loads data in parallel lines simultaneously and shifts it out bit-by-bit sequentially through a single line upon every clock pulse. This directly facilitates **(3) parallel-to-serial conversion**.
- **(b) Multiplexer:** A multiplexer (MUX) routes digital information from one of several input lines to a single common output line based on the state of its select inputs. Because it routes multiple data streams down to one, it functions directly **(4) as a many-to-one switch**.
- **(c) Decoder:** A decoder detects specific input binary combinations and activates a unique corresponding output line. In memory interfacing layouts, higher-order address bits are routed into a decoder to assert a unique line connected to the chip enable input of a targeted memory integrated circuit, working **(2) to generate memory chip select**.

Therefore, the correct matching is

a-3, b-4, c-2

.

Key Points

Component	Primary Application / Function
Shift Register	Data storage, shifting, and serial/parallel conversions
Multiplexer	Data routing, boolean function generation, many-to-one switching
Decoder	Address decoding, binary-to-decimal decoding, chip selecting

Q4.94.1 Descriptive | 1994 | 2M | Ans: TRUE | ● ● ○

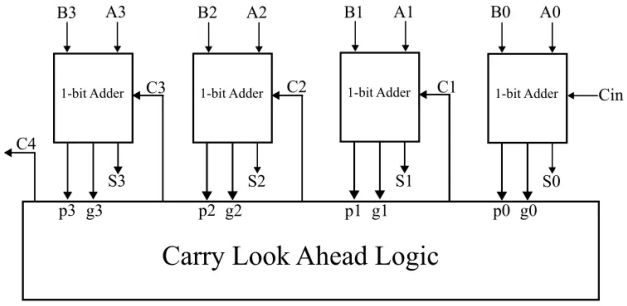


Fig. Solution Q4.94.1

To evaluate the validity of the statement regarding the look-ahead carry adder, we analyze its parallel carry generation equations.

A **Look-Ahead Carry (LAC) Adder** speeds up addition by eliminating sequential propagation delay. From the provided diagram, each stage i defines:

- **Carry Propagate (p_i):** $p_i = A_i \oplus B_i$
- **Carry Generate (g_i):** $g_i = A_i \cdot B_i$

Instead of waiting for a carry ripple, the internal carries are evaluated simultaneously in a two-level sum-of-products form using the base input carry C_{in}:

$$C_1 = g_0 + p_0 C_{in}$$
$$C_2 = g_1 + p_1 g_0 + p_1 p_0 C_{in}$$
$$C_3 = g_2 + p_2 g_1 + p_2 p_1 g_0 + p_2 p_1 p_0 C_{in}$$
$$C_4 = g_3 + p_3 g_2 + p_3 p_2 g_1 + p_3 p_2 p_1 g_0 + p_3 p_2 p_1 p_0 C_{in}$$

The individual sum bits are then computed concurrently at each stage as $S_i = p_i \oplus C_i$. Because every single carry and sum expression depends strictly on the initial input digits (A_i, B_i, C_{in}) via parallel combinational logic, all sum digits are generated simultaneously and directly.

Therefore, the statement is

True.

.

Key Points

- **Simultaneous Computation:** The central look-ahead logic block produces C_1 through C_4 synchronously, making the total propagation delay constant regardless of bit width.

Q4.92.1 MCQ 1992 1M Ans: B ●●○

To find the logic function realized by the given 4 to 1 multiplexer, we analyze its output equation based on the selection lines and inputs.
The general output expression for a 4 to 1 multiplexer with selection lines S_1, S_0 and inputs I_0, I_1, I_2, I_3 is:

$$F = \overline{S}_1\overline{S}_0I_0 + \overline{S}_1S_0I_1 + S_1\overline{S}_0I_2 + S_1S_0I_3$$

From the given circuit diagram, we identify the following connections:

- Selection lines: $S_1 = A$ and $S_0 = B$
- Input lines: $I_0 = C$, $I_1 = C$, $I_2 = \overline{C}$, and $I_3 = \overline{C}$

Substituting these values into the general expression:

$$F = \overline{A}\overline{B}C + \overline{A}BC + A\overline{B}\overline{C} + AB\overline{C}$$

Now, group the common terms to simplify the Boolean expression:

$$F = \overline{A}C(\overline{B} + B) + A\overline{C}(\overline{B} + B)$$

Since $\overline{B} + B = 1$:

$$F = \overline{A}C(1) + A\overline{C}(1)$$
$$F = \overline{A}C + A\overline{C} = A \oplus C$$

Therefore, the correct option is

$$\boxed{\textbf{(B)} F = A \oplus C.}$$

.

Key Points

- **Multiplexer as a Function Generator:** A MUX can implement any Boolean function by mapping variables to selection lines and residual functions/constants to data inputs.
- **Boolean Identity:** The relation $\overline{X}Y + X\overline{Y}$ uniquely defines the XOR operation $(X \oplus Y)$.

Q4.91.1 NAT 1991 2M Ans: 48 ●●○

To determine the minimum clock frequency required for the sequential multiplexer, we calculate the required sampling rate based on the Nyquist criterion.
First, identify the maximum frequency component (f_m) for each input signal from its sinusoidal form $x(t) = V_m \cos(2\pi f_m t)$:

- $f_A = 4$ kHz
- $f_B = 3.8$ kHz
- $f_C = 2.2$ kHz
- $f_D = 1.7$ kHz

The highest frequency among all input signals is $f_{max} = 4$ kHz. According to the Nyquist sampling theorem, the minimum sampling rate (f_s) to prevent aliasing is:

$$f_s = 2 \times f_{max} = 2 \times 4 \text{ kHz} = 8 \text{ kHz}$$

The sequential multiplexer has a total of 6 channels that it scans through sequentially per frame cycle. To ensure that every single channel slot satisfies this worst-case Nyquist boundary rate, the switching clock frequency (f_{clock}) must accommodate all 6 channels within the sampling period:

$$f_{clock} = 6 \times f_s = 6 \times 8 \text{ kHz} = 48 \text{ kHz}$$

Thus, the minimum clock frequency required is 48 kHz.

48

Key Points

- **Nyquist Rate**: The minimum uniform sampling rate required to perfectly reconstruct a signal is $2f_{max}$.
- **TDM Clock Rate**: For a sequential commuter switch scanning N channels, the switching clock frequency is given by $f_{clock} = N \times f_s$.

Q4.89.1 Descriptive 1989 2M Ans: *

To solve this linked problem, we first translate the descriptive table into a standard binary truth table using the given definitions:

- Inputs: High = 1, Low = 0
- Outputs: On = 1, Off = 0 for Heater (H)
- Outputs: Open = 1, Closed = 0 for Inlet Valve (V)
- Don't care conditions: x = X

Binary Truth Table

P	T	L	H	V
0	0	0	0	1
0	0	1	1	0
0	1	0	0	1
0	1	1	0	0
1	0	0	1	1
1	0	1	1	0
1	1	0	0	0
1	1	1	X	X

Karnaugh Map for H:

P \ TL	00	01	11	10
0	0	1	0	0
1	1	1	X	0

Fig. Solution Q4.89.1

Karnaugh Map for V:

P \ TL	00	01	11	10
0	1	0	0	1
1	1	0	X	0

Fig. Solution Q4.89.1

Q4.89.2 Descriptive 1989 2M Ans: *

To obtain the minimal **POS expression**, we group the zeros (0) and don't cares (X) in the K-maps:

- **For H:**

P \ TL	00	01	11	10
0	0	1	0	0
1	1	1	X	0

Fig. Solution Q4.89.2

$$H = \overline{T} \cdot (P + L)$$

- **For V:**

P \ TL	00	01	11	10
0	1	0	0	1
1	1	0	X	0

Fig. Solution Q4.89.2

$$V = \overline{L} \cdot (\overline{P} + \overline{T})$$

Check: for $PTL = 001$ the table requires the heater ON ($H = 1$), and $\overline{T}(P + L) = 1 \cdot (0 + 1) = 1$. For $PTL = 000$ it requires $H = 0$, and $\overline{T}(P + L) = 1 \cdot 0 = 0$.

Q4.89.3 Descriptive 1989 2M Ans: *

With T as MSB and L as LSB for the control inputs ($S_1 = T, S_0 = L$), the selection lines map directly to the four channels (00, 01, 10, 11) of each 4-to-1 MUX. We find the corresponding input values in terms of P for each combination of (T, L):

- **For H:**

$\overline{P}$	$\overline{T}\overline{L}$	$\overline{T}L$	$T\overline{L}$	TL
$\overline{P}$	0	1	0	0
P	(1)	1	0	X
	P	1	0	0

Fig. Solution Q4.89.3

$$H = \overline{T} \cdot (P + L)$$

- **For V:**

	$\bar{T}\bar{L}$	$\bar{T}L$	$T\bar{L}$	TL
$\bar{P}$	1	0	(1)	0
P	1	0	0	0
	1	0	$\bar{P}$	0

Fig. Solution Q4.89.3

$$V = \overline{L} \cdot (\overline{P} + \overline{T})$$

Final Multiplexer Implementation Connections:

- **Heater (H) MUX:**

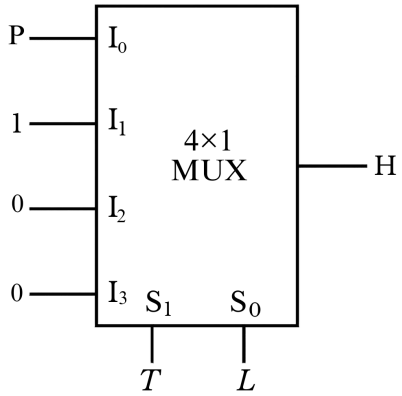


Fig. Solution Q4.89.3

- **Valve (V) MUX:**

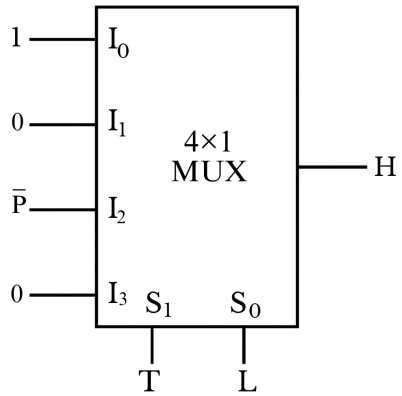


Fig. Solution Q4.89.3

SOLUTIONS

V

Sequential Circuits

Topics

1. Latches — SR, JK, D and T Types, Their Drawbacks and Gated Latches
2. Flip-Flops — Introduction, Types, Conversions, Race Around Condition and Master-Slave Configuration
3. Asynchronous Counters — Binary, Non-Binary (Up/Down) and Output Decoding
4. Synchronous Counters — Propagation Delay, Setup/Hold Time, Critical Path Delay, Clear and Preset Input
5. Shift Registers and Applications — Ring Counter, Johnson Counter and Cascading
6. FSM and Sequence Detectors — Design, Generation, Detection and Overlapping/Non-Overlapping Types
7. Static Timing Analysis — Setup/Hold Time, Clock Constraints, Critical Path and Timing Violations

PrepFusion

Unit Snapshot*

Stream: EC	**Questions:** 91	**MCQ:** 56	**MSQ:** 3	**NAT:** 29
1M: 32	**2M:** 46	**Descriptive:** 13	**Avg Weightage** ~3M	**Range:** 2M(2024)–5M(2011)

**See preface for how Avg Weightage and Range are calculated.*

Q5.26.1 MCQ 2026 2M Ans: A ● ○ ○

Given an 8-bit shift-left register ($b_7b_6b_5b_4b_3b_2b_1b_0$) and a D-flip-flop, both initially cleared ($Q = 0$ and all $b_i = 0$). The circuit operations at every positive clock edge are:

- The XOR gate output is $Q \oplus 1 = \bar{Q}$.
- This output is fed into b_0, so New $b_0 = \bar{Q}$.
- The shift register shifts left.
- The MSB of the register (b_7) is fed to the D-input, so New $Q = b_7$ of the previous state.

The state transitions are summarized in the table below.

CLK edge	$Q = b_7^-$	b_7	b_6	b_5	b_4	b_3	b_2	b_1	$b_0 = \bar{Q}$
0 (Initial)	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	1
2	0	0	0	0	0	0	0	1	1
3	0	0	0	0	0	0	1	1	1
4	0	0	0	0	0	1	1	1	1
5	0	0	0	0	1	1	1	1	1

Immediately after the 5th clock transition, the state of the shift register ($b_7b_6b_5b_4b_3b_2b_1b_0$) is 00011111.
Hence, the correct option is (A).

$$\boxed{(A)00011111}$$

Q5.26.2 NAT 2026 1M Ans: 4.9 to 5.0 ● ○ ○

Given: A negative edge-triggered JK flip-flop with $J = K = 1$.
When $J = K = 1$, the JK-FF operates in toggle mode. The frequency of output Q is:

$$f_{out} = \frac{f_{in}}{2}$$

$$f_{out} = \frac{10}{2} = 5\,\text{Hz}$$

$$\boxed{5}$$

Key Points

- $J = K = 1 \implies$ Toggle mode.
- Output frequency $f_{out} = f_{clk}/2$.

Q5.26.3 MSQ 2026 1M Ans: B ●●○

To count from 0 to 64, the number of distinct states required is $N = 64 - 0 + 1 = 65$ states.
The number of flip-flops (n) must satisfy:

$$2^n \geq N$$

$$2^n \geq 65$$

Checking values:

- If $n = 6$, $2^6 = 64$ (Insufficient).
- If $n = 7$, $2^7 = 128$ (Sufficient).

Therefore, 7 flip-flops are required.
Hence, the correct option is (B).

$$\boxed{(B)7}$$

Key Points

- n-bit counter max count = $2^n - 1$.

Q5.25.1 MCQ 2025 2M Ans: A ●●○

The given positive-edge-triggered sequential circuit consists of two D-flip-flops (Q_1 and Q_2) with inputs controlled by 2×1 multiplexers. The final output is defined by the XOR operation $Y = \bar{Q}_1 \oplus \bar{Q}_2$. Based on the circuit connections:

- $D_1 = S$ if $SEL = 1$, else $D_1 = P0$.
- $D_2 = Q_1$ if $SEL = 1$, else $D_2 = P1$.
- Given constant values: $P0 = 0$ and $P1 = 1$.

By analyzing the state transitions at each positive clock edge (T_0 to T_3) using the provided timing diagram, we can track the logic levels of Q_1, Q_2, and the resulting output Y.

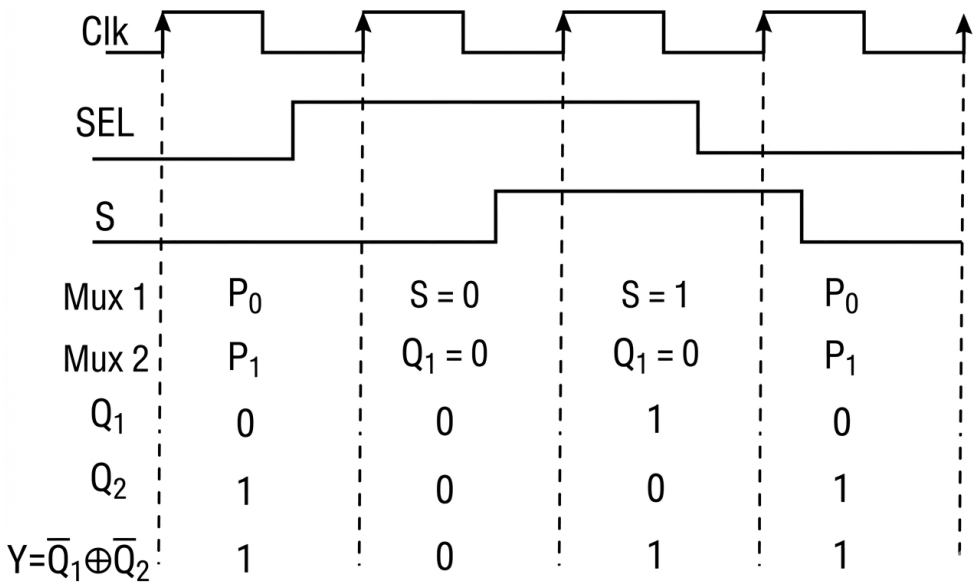


Fig. Solution Q5.25.1

The sequence of output Y from T_0 to T_3 is 1011.

Hence, the correct option is (A).

$$\boxed{(A)1011}$$

Q5.25.2 NAT | 2025 | 2M | Ans: 200 | ● ● ○ | CLICK HERE FOR VIDEO SOLUTION

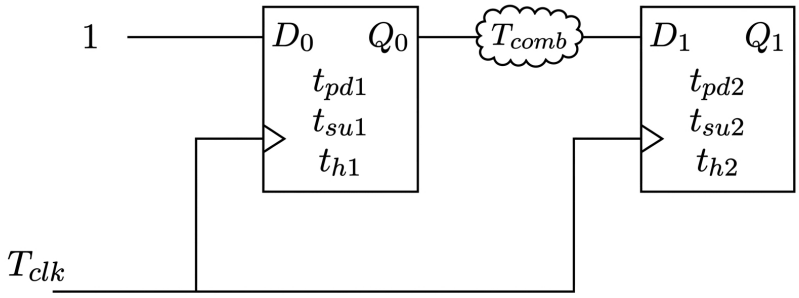


Fig. Solution Q5.25.2

For a standard sequential circuit as given above the formula for the minimum clock period (T_{clk}) to avoid setup time violations is:

$$\boxed{T_{clk} \geq t_{pd1} + T_{comb} + t_{su2}}$$

(refer video for derivation)

For the given circuit parameters:

- Propagation delay: $t_{pd1} = t_{pd2} = 2\,\text{ns}$
- Setup time: $t_{su1} = t_{su2} = 2\,\text{ns}$
- Hold time: $t_{hold} = 0\,\text{ns}$
- Combinational delay (AND gate): $t_{comb} = t_{AND} = 1\,\text{ns}$

Path 1: From FF_2 to FF_1 After the clock edge, the signal travels from Q_2 through the AND gate to D_1. The propagation delay of FF_2 and the delay of the AND gate are added to the path:

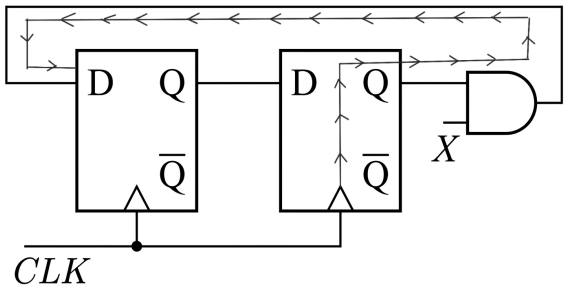


Fig. Solution Q5.25.2

$$T_{clk1} \geq t_{pd2} + T_{comb} + t_{su1}$$

$$T_{clk1} \geq 2 + 1 + 2 \implies T_{clk1} \geq 5\,\text{ns}$$

Path 2: From FF_1 to FF_2 The path delay passes directly from Q_1 to D_2:

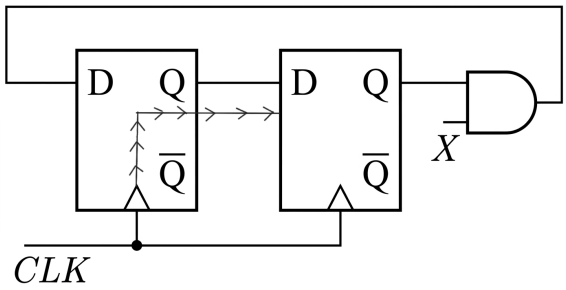


Fig. Solution Q5.25.2

$$T_{clk2} \geq t_{pd1} + t_{su2}$$

$$T_{clk2} \geq 2 + 2 \implies T_{clk2} \geq 4\,\text{ns}$$

To ensure no time violations occur in any part of the circuit, we take the maximum of the required clock periods:

$$T_{clk} = \max[T_{clk1}, T_{clk2}]$$

$$T_{clk} = \max[5\,\text{ns}, 4\,\text{ns}] = 5\,\text{ns}$$

Thus, maximum clock frequency (f_{max}) is:

$$f_{max} = \frac{1}{T_{clk}} = \frac{1}{5 \times 10^{-9}} = 0.2 \times 10^9\,\text{Hz} = 200\,\text{MHz}$$

$$\boxed{200}$$

Key Points

- The minimum clock period T_{clk} is governed by the longest (slowest) path between any two registers to ensure data is stable before the next clock edge.
- Setup time constraint: $T_{clk} \geq T_{pd} + T_{comb} + T_{su}$.

Mistakes to be avoided

- Always check all possible paths between flip-flops, including feedback paths, and select the most restrictive (largest) T_{clk}.

Q5.24.1 MCQ 2024 2M Ans: B ● ○ ○

The given sequential circuit consists of two T flip-flops with $T_1 = Q_1 + \bar{Q}_0$; $T_0 = Q_0 + Q_1$; initial state $Q_1Q_0 = 00$.

Clock Edge	$T_1 = Q_1 + \bar{Q}_0$	$T_0 = Q_0 + Q_1$	Q_1^+	Q_0^+
Initially	-	-	0	0
1	0 + 1 = 1	0 + 0 = 0	1	0
2	1 + 1 = 1	0 + 1 = 1	0	1
3	0 + 0 = 0	1 + 0 = 1	0	0

State Transition Analysis:

- **Initial**: $Q_1Q_0 = 00$.
- **1st Edge**: $T_1 = 1$ (toggles Q_1), $T_0 = 0$ (holds Q_0). New state: 10.
- **2nd Edge**: $T_1 = 1$ (toggles Q_1), $T_0 = 1$ (toggles Q_0). New state: 01.
- **3rd Edge**: $T_1 = 0$ (holds Q_1), $T_0 = 1$ (toggles Q_0). New state: 00.

The sequence of states is $00 \rightarrow 10 \rightarrow 01 \rightarrow 00$.

$$\boxed{(B)11 \rightarrow 00 \rightarrow 10 \rightarrow 01 \rightarrow 00}$$

Mistakes to be avoided

- Remember that T flip-flops save state for $T = 0$ and toggle on $T = 1$; a common error is treating them like D flip-flops where $Q^+ = D$.

Q5.23.1 NAT | 2023 | 2M | Ans: 250 or 500 | ● ○ ○

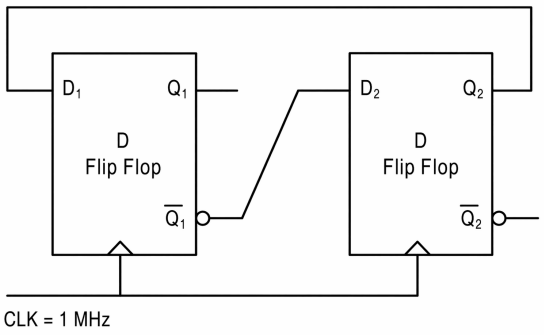


Fig. Solution Q5.23.1

Given a sequential circuit with two D flip-flops, the input equations based on the direct label connections are:

$$D_1 = Q_2, \quad D_2 = \overline{Q}_1$$

The initial state is provided as $Q_1 = 1$ and $Q_2 = 0$.

- **Standard Interpretation** ($D_2 = \overline{Q}_1$)

Clock Edge	$D_1 = Q_2$	$D_2 = \overline{Q}_1$	Q_1	Q_2
Initially	-	-	1	0
1st Clock Edge	0	0	0	0
2nd Clock Edge	0	1	0	1
3rd Clock Edge	1	1	1	1
4th Clock Edge	1	0	1	0

As seen from the table, the state Q_1Q_2 returns to its initial value (10) after 4 clock pulses ($N = 4$).

$$f_o = \frac{f_i}{N} = \frac{1000\,\text{kHz}}{4} = 250\,\text{kHz}$$

- **Alternative Challenged Interpretation** ($D_2 = Q_1$) Due to the presence of a bubble at the complementary output pin labeled $\overline{Q}_1$, it was argued that the bubble introduces an additional inversion, making the net output $\overline{\overline{Q}}_1 = Q_1$.

  (Even though this logic is technically incorrect, the challenge was somehow still accepted.)

  Under this interpretation, the input equations becomes:

$$D_1 = Q_2, \quad D_2 = Q_1$$

  The state transition table for this configuration is as follows:

Clock Edge	$D_1 = Q_2$	$D_2 = Q_1$	Q_1	Q_2
Initially	-	-	1	0
1st Clock Edge	0	1	0	1
2nd Clock Edge	1	0	1	0

  Under this alternative assumption, the state Q_1Q_2 repeats its sequence after every 2 clock pulses, acting as a MOD-2 counter ($N = 2$).

$$f_o = \frac{f_i}{N} = \frac{1000\,\text{kHz}}{2} = 500\,\text{kHz}$$

Because of this graphical ambiguity, the official answer key accepted both **250 OR 500**.

250

Key Points

- Output frequency is calculated by dividing the input frequency by the sequence modulus (N).
- A bubble at the $\overline{Q}$ pin is just a standard graphical label for the complementary output; it **does not represent an extra inversion.** Therefore, the node **remains $\overline{Q}$.** While sometimes this may be given marks due to exam ambiguities, **this will not always be accepted.**

Q5.23.2 NAT 2023 1M Ans: 2 ● ○ ○

The given circuit is a cross-coupled NAND gate latch, which is a basic sequential element. To find the critical path delay, we analyze the signal flow through the gates.

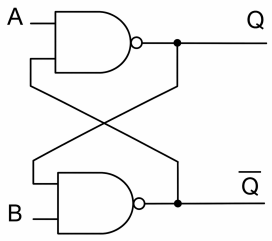


Fig. Solution Q5.23.2

Redrawing the circuit to visualize the propagation path:

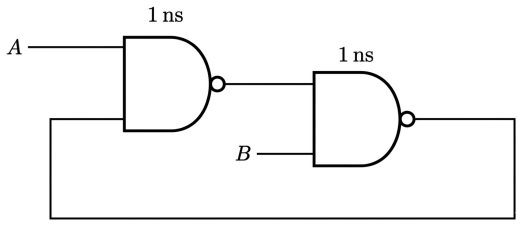


Fig. Solution Q5.23.2

Each NAND gate in the circuit has a propagation delay of 1 ns.
The critical path is the longest path a signal must travel from an input to the output through the feedback loop.
From the redrawn circuit, the path through both gates is:

$$\text{Critical path delay} = 1\text{ ns} + 1\text{ ns} = 2\text{ ns}$$

2

Mistakes to be avoided

- Do not consider only a single gate delay if the output depends on the feedback loop transition.

Q5.23.3 MCQ 2023 1M Ans: A

The given circuit is a 3-bit Serial In Serial Out (SISO) shift register. We are given the clock frequency $f_{clk} = 1\,\text{GHz}$.

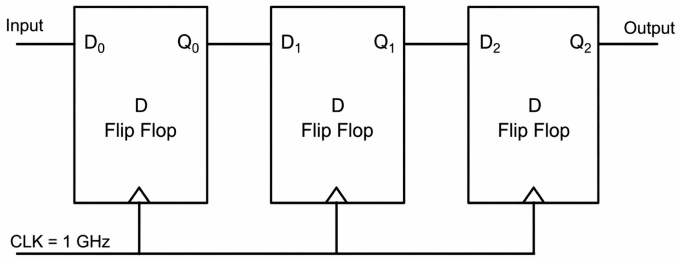


Fig. Solution Q5.23.3

The clock period (T_{clk}) is calculated as:

$$T_{clk} = \frac{1}{f_{clk}} = \frac{1}{10^9} = 10^{-9}\,\text{s} = 1\,\text{ns}$$

1. Latency: Latency is the total time required for a single bit to pass through the entire register. For N flip-flops, it is given by:

$$\text{Latency} = N \times T_{clk}$$

Given $N = 3$ and $T_{clk} = 1\,\text{ns}$:

$$\text{Latency} = 3 \times 1\,\text{ns} = 3\,\text{ns}$$

2. Throughput: Throughput is defined as the number of bits processed per second. In a shift register, one bit of output is obtained every clock cycle (T_{clk}).

$$\text{Throughput} = \frac{1}{T_{clk}} = 10^9\,\text{bits/sec}$$

Converting to Megabits per second (Mbps):

$$\text{Throughput} = 10^3 \text{ Mbits/sec}$$

Thus, the circuit provides 1 bit every 1 ns.

$$\boxed{(A) 1000, 3}$$

Key Points

- **Latency**: Total delay from input to output ($N \times T_{clk}$).
- **Throughput**: Rate of data processing ($1/T_{clk}$).

Mistakes to be avoided

- Do not confuse Latency with Throughput; Latency depends on the number of stages, whereas Throughput depends only on the clock frequency.

Q5.22.1 MCQ 2022 2M Ans: A ●●○

For an edge-triggered counter and flip-flop, the circuit responds only to the active clock edge (rising edge in this case). The duty cycle of the clock pulse is irrelevant to the triggering mechanism; thus, a 25% duty cycle does not change the state transition timing compared to a 50% duty cycle.
Analyzing the circuit:

- The clock has a time period T, and it is common to the counter and the JK flip-flop.
- The LSB of the counter toggles at every rising clock edge, so it has a period of $2T$.
- The JK inputs are wired directly to the counter LSB, so $J = K =$ LSB (not its complement). At each rising edge the flip-flop samples the old LSB value, before the counter updates.
- For example, starting with LSB = 0 and $Q = 0$, the LSB after successive edges is $1, 0, 1, 0, 1$, while the JK inputs sample $0, 1, 0, 1, 0$. Hence Q holds when $J = K = 0$ and toggles when $J = K = 1$, giving $Q = 0, 1, 1, 0, 0, \ldots$

Thus Q toggles on alternate clock edges: it stays high for $2T$ and low for $2T$. A different initial phase only shifts the waveform. The time period of the output signal (T') is:

$$T' = 4T$$

Consequently, the output frequency is $f_0/4$ and the output duty cycle is 50%, even though the clock has only a 25% duty cycle.

$$\boxed{\text{(A) frequency is } f_0/4 \text{ and duty cycle is } 50\%}$$

Key Points

- **Edge Triggering**: In synchronous circuits, state changes occur only at the clock transition, making the pulse width (duty cycle) irrelevant.
- **Frequency Division**: A flip-flop acting as a toggle (T-type or JK with $J = K = 1$) divides the frequency of its trigger signal by 2.

Q5.22.2 MSQ | 2022 | 2M | Ans: A,C | ● ● ○

Based on the provided state transition diagram, we analyze the transition probabilities from each state. The sum of all outgoing transition probabilities from any given state must equal 1.

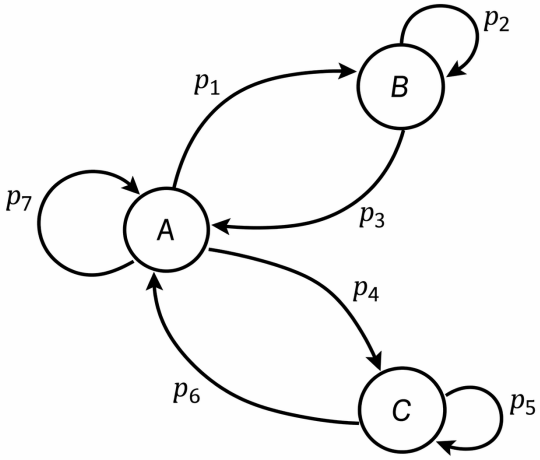


Fig. Solution Q5.22.2

Case 1: Transitions from A From state A, the system can transition to state B (with probability p_1), state C (with probability p_4), or remain in state A (with probability p_7).

$$p_1 + p_4 + p_7 = 1$$

Case 2: Transitions from B From state B, the system can transition to state A (with probability p_3) or remain in state B (with probability p_2).

$$p_2 + p_3 = 1$$

Case 3: Transitions C From state C, the system can transition to state A (with probability p_6) or remain in state C (with probability p_5).

$$p_5 + p_6 = 1$$

Comparing the results from Case 2 and Case 3:

$$p_2 + p_3 = p_5 + p_6 = 1$$

Thus, the relationships

$$p_1 + p_4 + p_7 = 1$$

and $p_2 + p_3 = p_5 + p_6$ are valid based on the fundamental properties of state transition diagrams.

(A) $p_2 + p_3 = p_5 + p_6$
(C) $p_1 + p_4 + p_7 = 1$

Key Points

- In any state transition diagram or Markov chain, the sum of probabilities for all outgoing edges from a single node must be exactly 1.
- Self-loops must be included in the summation of outgoing probabilities.

Mistakes to be avoided

- Do not confuse incoming transitions with outgoing transitions; only outgoing transitions contribute to the probability sum of 1 for a specific state.

Q5.21.1 NAT 2021 2M Ans: 5 ● ○ ○

The clock frequency is $f = 500\,\text{MHz}$, so the clock period is

$$T_{\text{clk}} = \frac{1}{500\,\text{MHz}} = 2\,\text{ns}$$

The propagation delay of the XOR gate is $\tau_{\text{XOR}} = 3\,\text{ns}$. Since $\tau_{\text{XOR}} > T_{\text{clk}}$, the input $D_2 = Q_2 \oplus Q_0$ sampled at a clock edge is the XOR of the outputs as they were 3 ns earlier, not the XOR of the present state. The other inputs are $D_1 = Q_2$ and $D_0 = Q_1$.

Initially $Q_2Q_1Q_0 = 111$ and the XOR output is given as $D_2 = 1$. Its response to the inputs $1 \oplus 1 = 0$ arrives only 3 ns later. Taking the first triggering edge as $t = 0$, the edges at 0 and 2 ns still sample $D_2 = 1$.

Clock Edge	Time	D_2 sampled	$Q_2Q_1Q_0$ after the edge
Initial	–	1 (given)	111
1st	0 ns	1	111
2nd	2 ns	1	111
3rd	4 ns	0	011
4th	6 ns	0	001
5th	8 ns	1	100

The XOR output becomes 0 at 3 ns (response to the state 111), so the 3rd and 4th edges sample $D_2 = 0$. At 4 ns the state changes to 011, making the XOR inputs $0 \oplus 1 = 1$; this new value reaches D_2 at 7 ns and is sampled at the 5th edge (8 ns). After the 5th triggering clock edge, the state $Q_2Q_1Q_0$ becomes 100 for the first time.

$$\boxed{5}$$

Key Points

- When gate delay exceeds the clock period ($\tau > T_{\text{clk}}$), the combinational logic feedback uses older state values.

Q5.20.1 MCQ 2020 2M Ans: A ● ○ ○

To find the detected sequence, follow the direct forward path from the initial state S_0 to the transition that yields an output of 1 (ignore the transitions that loop backwards):

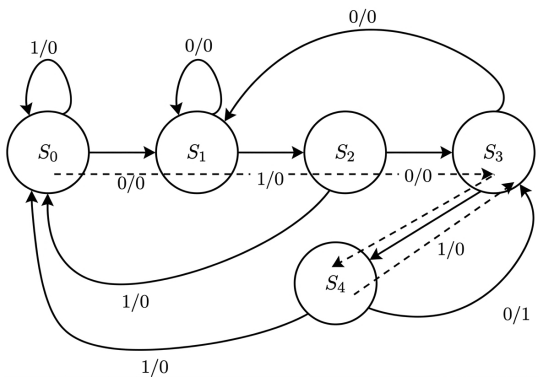


Fig. Solution Q5.20.1

$$S_0 \xrightarrow{0/0} S_1 \xrightarrow{1/0} S_2 \xrightarrow{0/0} S_3 \xrightarrow{1/0} S_4 \xrightarrow{0/1} S_3$$

Extracting the sequential input bits gives **01010**.

(A) the sequence 01010 is detected.

Q5.20.2 NAT | 2020 | 2M | Ans: 76 to 77 | ● ● ○ | CLICK HERE FOR VIDEO SOLUTION

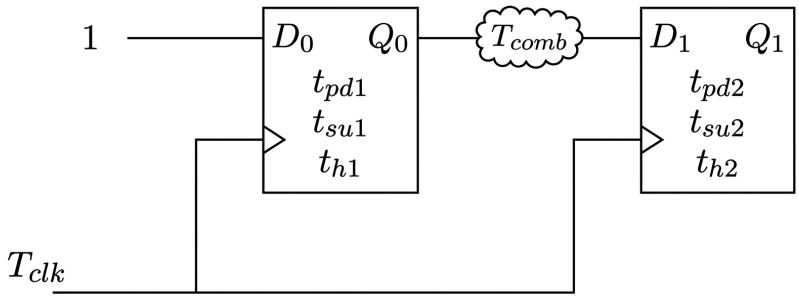


Fig. Solution Q5.20.2

For a standard sequential circuit as given above the formula for the minimum clock period (T_{clk}) to avoid setup time violations is:

$$T_{clk} \geq t_{pd1} + T_{comb} + t_{su2}$$

(refer video for derivation)

Let us evaluate the two distinct data path loops in the circuit:

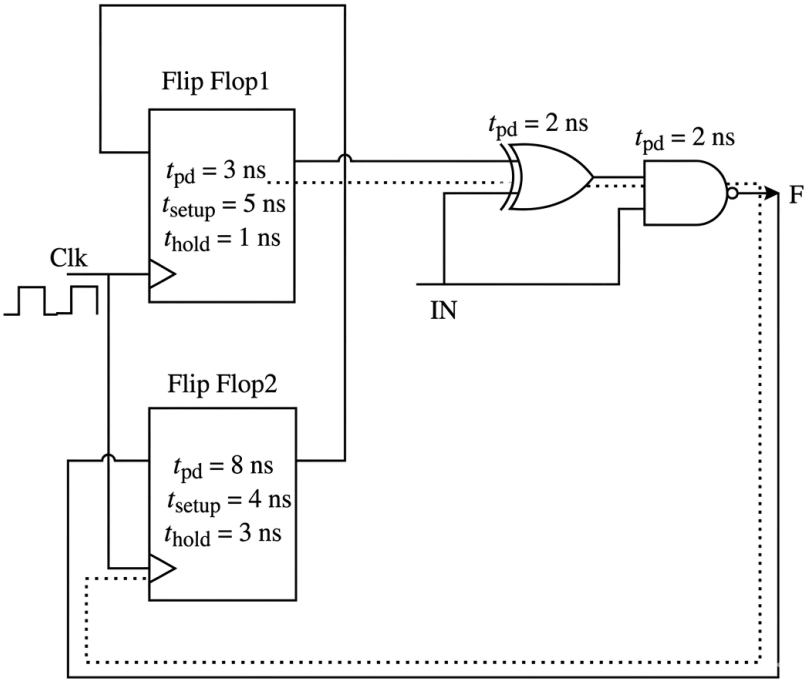


Fig. Solution Q5.20.2

Path 1: From FF_1 to FF_2

- Launching flip-flop: $FF_1 \implies t_{pd1} = 3\text{ ns}$
- Combinational delay (XOR + NAND): $t_{pd(combinational1)} = 2\text{ ns} + 2\text{ ns} = 4\text{ ns}$
- Capturing flip-flop: $FF_2 \implies t_{setup2} = 4\text{ ns}$

$$T_{min1} = 3\text{ ns} + 4\text{ ns} + 4\text{ ns} = 11\text{ ns}$$

Path 2: From FF_2 to FF_1

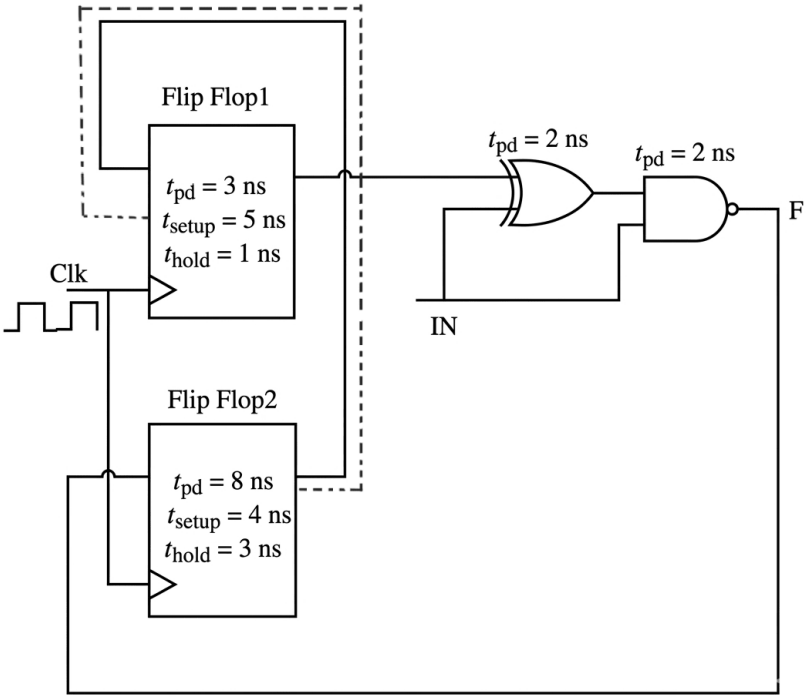


Fig. Solution Q5.20.2

- Launching flip-flop: $FF_2 \implies t_{pd2} = 8\,\text{ns}$
- Combinational delay: Direct feedback connection $\implies t_{pd(\text{combinational2})} = 0\,\text{ns}$
- Capturing flip-flop: $FF_1 \implies t_{setup1} = 5\,\text{ns}$

$$T_{min2} = 8\,\text{ns} + 0\,\text{ns} + 5\,\text{ns} = 13\,\text{ns}$$

Maximum Clock Frequency The minimum allowable clock period T_{clk} must safely satisfy both setup constraints:

$$T_{clk} = \max(T_{min1}, T_{min2}) = \max(11\,\text{ns}, 13\,\text{ns}) = 13\,\text{ns}$$

The maximum reliable operating clock frequency f_{max} is:

$$f_{max} = \frac{1}{T_{clk}} = \frac{1}{13\,\text{ns}} \approx 76.92\,\text{MHz}$$

Rounding off to the nearest integer:

$$f_{max} \approx 77\,\text{MHz}$$

77

Key Points

- Minimum clock period is bounded by the longest path constraint: $T_{clk} = \max(T_{paths})$.

Mistakes to be avoided

- Do not blindly add the combinational delays to every loop; analyze the exact connectivity shown in the schematic.
- Ensure setup time is always taken from the receiving ($FF_{capture}$) flip-flop.

Q5.19.1 NAT 2019 1M Ans: 4 ● ○ ○

From the given circuit diagram, the input expressions for the flip-flops can be written as:

$$D_1 = \overline{Q}_1 \cdot \overline{Q}_2$$

$$D_2 = Q_1$$

Let us trace the state transitions starting from an initial state $(Q_1Q_2) = (00)$:

Clock Pulse	Q_1	Q_2	$D_1 = \overline{Q}_1 \cdot \overline{Q}_2$	$D_2 = Q_1$
Initially	0	0	1	0
1st	1	0	0	1
2nd	0	1	0	0
3rd	0	0	1	0

The sequence of states repeats after every 3 clock cycles:

$$(00) \rightarrow (10) \rightarrow (01) \rightarrow (00)$$

Since the counter has a cycle length of 3, the modulus of the counter is $N = 3$.
The frequency of the output signal at Q_2 is given by dividing the input clock frequency by the modulus of the counter:

$$f_{Q2} = \frac{f_{clk}}{N} = \frac{12\,\text{kHz}}{3} = 4\,\text{kHz}$$

$\boxed{4}$

Key Points

- The modulus (N) of a synchronous counter is the number of unique states visited before the sequence repeats.
- The output frequency from any flip-flop stage of a periodic counter is $f_{out} = \frac{f_{clk}}{N}$.

Mistakes to be avoided

- Do not assume the modulus is always 2^n (4 for two flip-flops); trace the state table explicitly to find the actual repeating sequence.

Q5.19.2 MCQ 2019 2M Ans: C ● ● ○

By analyzing the given sequential circuit, we construct the truth table for the next state Q^+ based on the input A and current state Q:

Q	Mux_1	Mux_0	A	Mux_{out}	$D = \overline{\text{Mux}_{out} \cdot Q}$	Q^+
0	0	1	0	1	1	1
0	0	1	1	0	1	1
1	1	0	0	0	1	1
1	1	0	1	1	0	0

From the truth table, the next-state logic satisfies:

- When $Q = 0 \implies Q^+ = 1$ (irrespective of A)
- When $Q = 1 \implies Q^+ = \overline{A}$ .That is-
  - Remains at Q = 1 for A=0

– Switches to Q = 0 for A=1

Thus the state transition diagram can be drawn as-

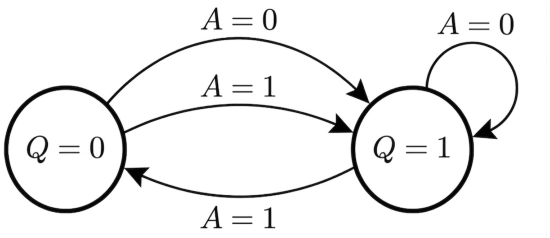


Fig. Solution Q5.19.2

$\boxed{(C)}$

Q5.18.1 NAT 2018 2M Ans: 0.82 to 0.86 ● ● ●

Given:

$$\frac{\Delta T}{T_{CK}} = 0.15, \quad \text{Probability of transition } (P) = 0.3, \quad V_{\text{high}} = 3.3\,\text{V}$$

Let, initial state of D be 0 and let it make a transition in the first clock edge thus,

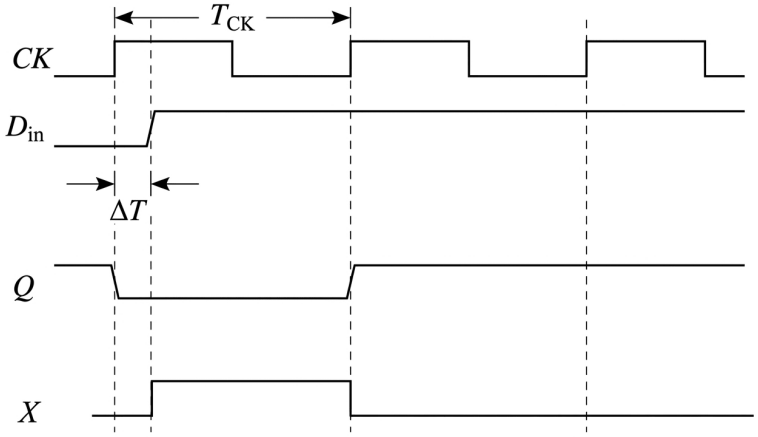


Fig. Solution Q5.18.1

Analyzing the timing diagram, the output X of the XOR gate will be high only when $D_{\text{in}} \neq Q$. This condition occurs after the input data transitions until the next rising clock edge.

Thus, X remains high for a duration of $(T_{CK} - \Delta T)$ during a transition cycle. The average voltage V_{avg} per clock period is:

$$V_{\text{avg}} = P \times V_{\text{high}} \times \left(\frac{T_{CK} - \Delta T}{T_{CK}}\right)$$

$$V_{\text{avg}} = P \times V_{\text{high}} \times \left(1 - \frac{\Delta T}{T_{CK}}\right)$$

$$V_{\text{avg}} = 0.3 \times 3.3 \times (1 - 0.15)$$

$$V_{\text{avg}} = 0.3 \times 3.3 \times 0.85 = 0.8415\,\text{V}$$

$\boxed{0.84}$

Key Points

- The XOR output X is high during the interval $(T_{CK} - \Delta T)$ when a transition occurs.
- Average voltage is the product of transition probability, peak voltage, and the active duty ratio.

Mistakes to be avoided

- Do not get confused by the representation of D_{in} in the timing diagram. It simply indicates that the data can be any logic level (0 or 1); however, at that specific time, it will undergo a transition from its previous state with a probability of 0.3.

Q5.18.2 NAT | 2018 | 1M | Ans: 5 to 5 | ●●○ | CLICK HERE FOR VIDEO SOLUTION

Given:

$$T_{CLK} = 5\,\text{s}, \quad \text{GREEN} = 70\,\text{s}, \quad \text{YELLOW} = 5\,\text{s}, \quad \text{RED} = 75\,\text{s}$$

The sequence cycles through the traffic states sequentially:

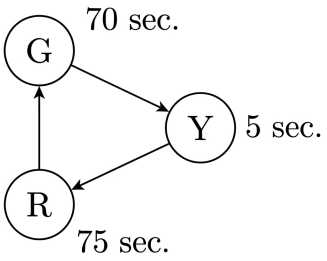


Fig. Solution Q5.18.2

Calculating the number of clock cycles (states) for each light:

- GREEN states = $\frac{70}{5} = 14$
- YELLOW state = $\frac{5}{5} = 1$
- RED states = $\frac{75}{5} = 15$

The total distinct sequence of states required for one complete cycle:

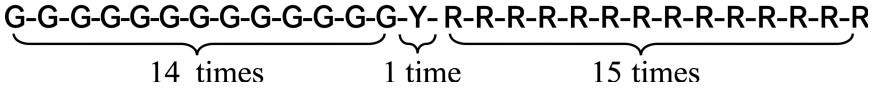


Fig. Solution Q5.18.2

Total states (L):

$$L = 14 + 1 + 15 = 30 \text{ states}$$

The formula for the minimum number of flip-flops (n) is:

$$L \leq 2^n$$

$$30 \le 2^n \implies n = 5 \quad (\because 2^5 = 32 \ge 30)$$

5

Highly Recommended: Go to the video solution link to understand the detailed FSM design for this question.

Mistakes to be avoided

- Do not count only the 3 physical color changes; the FSM must track every individual time durations to maintain proper functioning.

Q5.17.1 NAT 2017 2M Ans: 10 ●○○

The next-state equations for the shift register are:

$A_{n+1} = D_{\text{in}} = A_n \oplus D_n, \quad B_{n+1} = A_n, \quad C_{n+1} = B_n, \quad D_{n+1} = C_n$

Given that the initial present state at clock pulse 0 is $ABCD = 1101$, we track the state transitions clock by clock in the state table below:

Present State					Next state			
Clock pulse	A_n	B_n	C_n	D_n	$A_{n+1} = A_n \oplus D_n$	$B_{n+1} = A_n$	$C_{n+1} = B_n$	$D_{n+1} = C_n$
0	1	1	0	1	-	-	-	-
1	1	1	0	1	0	1	1	0
2	0	1	1	0	0	0	1	1
3	0	0	1	1	1	0	0	1
4	1	0	0	1	0	1	0	0
5	0	1	0	0	0	0	1	0
6	0	0	1	0	0	0	0	1
7	0	0	0	1	1	0	0	0
8	1	0	0	0	1	1	0	0
9	1	1	0	0	1	1	1	0
10	1	1	1	0	1	1	1	1

Hence, the number of clock cycles required to reach the state $ABCD = 1111$ is 10.

10

Mistakes to be avoided

- Most people give up after 5–6 clock edges and think they are making a mistake due to the long sequence; avoid that and solve it with patience up to the 10th clock cycle.

Q5.17.2 MCQ 2017 2M Ans: D ●●○

From the FSM circuit diagram, the input equations for the two D flip-flops are:

$$D_A = Q_A \oplus Q_B, \quad D_B = \overline{Q_A \cdot X_{\text{IN}}}$$

The FSM is initialized to $Q_A Q_B = 00$. We analyze the state transitions for both cases:

Case 1: When $X_{\text{IN}} = 0$ The next-state equations simplify to:

$$D_A = Q_A \oplus Q_B, \quad D_B = \overline{Q_A \cdot 0} = 1$$

Tracking the transitions starting from 00:

Step	$Q_{A,n}Q_{B,n}$	D_A, D_B	Next State $Q_{A,n+1}Q_{B,n+1}$
1	00	01	01
2	01	11	11
3	11	01	01

The FSM repeats inside a periodic loop between only **two distinct states**: 01 and 11.
Case 2: When $X_{IN} = 1$ The next-state equations simplify to:

$$D_A = Q_A \oplus Q_B, \quad D_B = \overline{Q_A \cdot 1} = \overline{Q_A}$$

Tracking the transitions starting from 00:

Step	$Q_{A,n}Q_{B,n}$	D_A, D_B	Next State $Q_{A,n+1}Q_{B,n+1}$
1	00	01	01
2	01	11	11
3	11	00	00

The FSM continuously cycles through **three distinct states**: $00 \rightarrow 01 \rightarrow 11 \rightarrow 00$.
Therefore, option D is correct because the machine cycles through only two states when $X_{IN} = 0$.

(D) only two of the four possible states if $X_{IN} = 0$

Q5.17.3 NAT 2017 2M Ans: 4 to 4

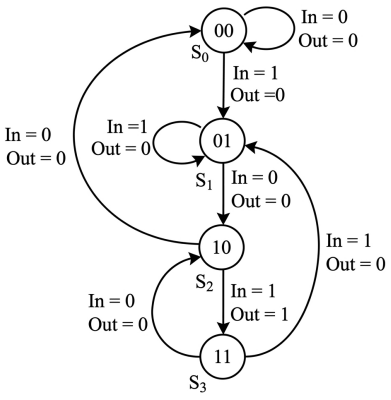


Fig. Solution Q5.17.3

By tracking the forward transitions of the FSM state diagram, we identify the detection conditions:

- $S_0 \xrightarrow{\text{In=1}} S_1 \xrightarrow{\text{In=0}} S_2 \xrightarrow{\text{In=1}} S_3$ with Out = 1. This confirms the target pattern is **101**.
- From S_3, an input of In = 0 returns to S_2 (the state representing a partial match of "10"). This confirms that the FSM allows **overlapping sequence detection**.

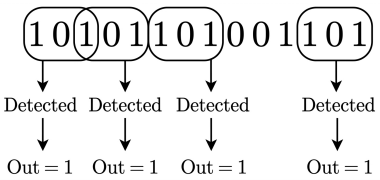


Fig. Solution Q5.17.3

The sequence **101** appears exactly 4 times with overlap. Therefore, the output 'Out' will be 1 a total of 4 times.

4

Mistakes to be avoided

- Do not reset the pattern search from scratch after a valid match; instead, reuse the trailing bits as the starting prefix for the next prospective sequence.

Q5.17.4 MCQ 2017 1M Ans: B ●●○

Initially, when $P = Q = 0$, both NAND gates are forced to 1 regardless of feedback or delays:

$$X = 1, \quad Y = 1$$

When the inputs transition to $P = Q = 1$ the final output unpredictable.
The final stable state depends on which gate responds faster due to the unequal propagation delays.
Case 1: Upper gate is faster

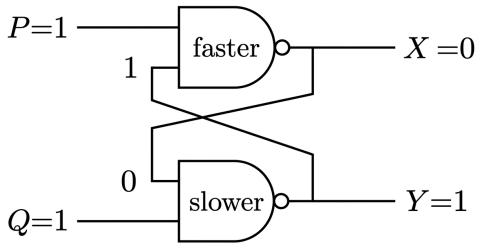


Fig. Solution Q5.17.4

The faster upper gate processes its initial inputs ($P = 1$ and feedback $Y = 1$) first:

$$X = \overline{1 \cdot 1} = 0$$

This 0 propagates to the lower gate. When the slower lower gate evaluates, its inputs are $Q = 1$ and $X = 0$:

$$Y = \overline{1 \cdot 0} = 1$$

This results in the stable state: $X = 0, Y = 1$. **Case 2: Lower gate is faster**

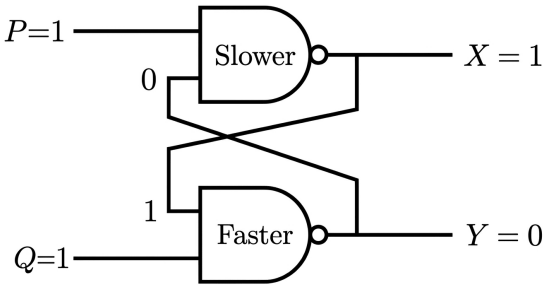


Fig. Solution Q5.17.4

The faster lower gate processes its initial inputs ($Q = 1$ and feedback $X = 1$) first:

$$Y = \overline{1 \cdot 1} = 0$$

This 0 propagates to the upper gate. When the slower upper gate evaluates, its inputs are $P = 1$ and $Y = 0$:

$$X = \overline{1 \cdot 0} = 1$$

This results in the stable state: $X = 1, Y = 0$.
Depending on which gate is faster, the final output will be,

$$\boxed{\text{(B) either } (X = 0, Y = 1) \text{ or } (X = 1, Y = 0).}$$

Key Points

- When a NAND-based SR latch transitions from the invalid state (00) to the hold state (11), it enters an undesirable condition.
- The final latch state is entirely determined by the gate with the shorter propagation delay.

Mistakes to be avoided

- In real hardware, do not assume both gates switch at the exact same time; tiny manufacturing differences mean one gate will always break the tie by being slightly faster.

Q5.17.5 NAT | 2017 | 1M | Ans: 29.9 to 30.0 | ●●○

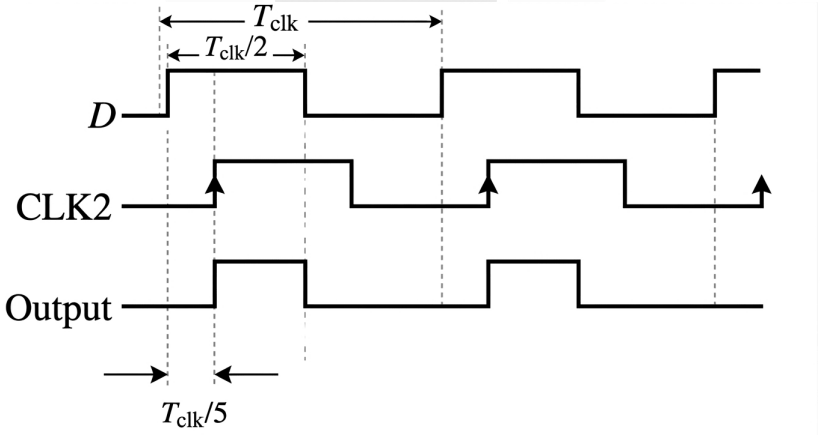


Fig. Solution Q5.17.5

From the timing waveform of the output signal,

$$T_{ON} = \frac{T_{CLK}}{2} - \frac{T_{CLK}}{5} = \frac{3T_{CLK}}{10}$$

The percentage duty cycle of the output Q is evaluated as follows:

$$\text{Duty Cycle of } Q = \frac{T_{ON}}{T} \times 100\%$$

$$\text{Duty Cycle of } Q = \frac{\frac{3T_{CLK}}{10}}{T_{CLK}} \times 100\% = 30\%$$

Hence, the duty cycle at the output of the latch is 30%.

$$\boxed{30}$$

Key Points

- The duty cycle of a periodic digital waveform is defined as the ratio of its active pulse duration (T_{ON}) to its total repeating period (T).
- When working with multiple clock divisions or offsets, establish a common denominator relative to the primary clock period T_{CLK} to accurately compute interval lengths.

Q5.16.1 MCQ 2016 2M Ans: C ●●○

To determine which states have unambiguously or ambiguously defined transitions, we construct the next-state table for each state under all possible combinations of the binary inputs X, Y, and Z.

X	Y	Z	Next State from A	Next State from B	Next State from C
0	0	0	B	A	C
0	0	1	A	C	Not defined
0	1	0	A	B	C
0	1	1	A	B	B
1	0	0	C	A	C
1	0	1	C	C	A
1	1	0	A	B	C
1	1	1	A	B	A, B

Thus we can see for State C:

- For input 001, no transition is specified
- For input 111, two simultaneous transitions exist to both State A and State B.

Therefore, the transitions from State C are ambiguously defined.

(C) Transitions from State C are ambiguously defined.

Key Points

- A finite state machine (FSM) has completely and unambiguously defined transitions if and only if for every state, each possible input combination results in exactly one deterministic next state.
- Missing transitions lead to incomplete specifications, while multiple transitions for the same input condition lead to ambiguity (non-determinism).

Mistakes to be avoided

- Carefully verify overlapping conditions on transitions, such as $Y = 1, Z = 1$ and $X = 1, Z = 1$ when inputs are completely specified.

Q5.16.2 MCQ 2016 2M Ans: D ●●○

Given:

- $f_{clk} = 1\,\text{GHz} \implies T_{clk} = \frac{1}{1\,\text{GHz}} = 1\,\text{ns}$
- Condition for $\overline{RESET} = \overline{Q_2 Q_1 \overline{Q_0}} = 110 = 6$
- Propagation delay of bubbled NAND gate = 2 ns

The execution sequence across successive clock cycles is detailed below:

Time (ns)	Q_2	Q_1	Q_0	
0	0	0	0	
1	0	0	1	
2	0	1	0	
3	0	1	1	
4	1	0	0	
5	1	0	1	
6	1	1	0	→ Condition for RESET
7	1	1	1	(Counter will RESET after 2ns.)
8	0	0	0	

Fig. Solution Q5.16.2

Time (ns)	Clock Edge	Counter State	Status / Logic Actions
0 to 5	$0 \rightarrow 5$	$000 \rightarrow 101$	Normal sequential counting.
6	6th	110 (State 6)	Decoded input conditions met: $Q_2Q_1\overline{Q_0} = 1$. Reset path initiated with 2 ns propagation delay.
7	7th	111 (State 7)	Counter increments normally ($T_{clk} = 1\,ns < 2\,ns$).
8	8th	000 (State 0)	Counter rolls over naturally to 000. Delayed $\overline{RESET}$ signal arrives, having no effect.

Since the counter visits all 8 possible distinct states (0 through 7) without premature truncation, it functions as a mod-8 counter.

(D) mod-8 counter

Key Points

- Truncation via asynchronous reset fails if the gate delay exceeds the clock period ($t_{pd} \geq T_{clk}$).
- The state changes at the next clock edge before the asynchronous signal can propagate back to clear the flip-flops.

Mistakes to be avoided

- Do not assume the counter is mod-6 or mod-7 based solely on decoding the reset state without verifying timing constraints.

Q5.16.3 NAT 2016 1M Ans: 1.45 to 1.55 ●●○

Given a 5-bit shift register counter initialized with logic values:

$(Q_1, Q_2, Q_3, Q_4, Q_5) = (0, 1, 0, 1, 0)$

The true output Q_5 is fed back directly (without inversion) to the first input, so the circuit is a ring counter with $D_1 = Q_5$ and $D_n = Q_{n-1}$ for $n \geq 2$. Tracing the state sequence over successive clock periods:

CLK	Q_1	Q_2	Q_3	Q_4	Q_5	Output $Y = Q_3 + Q_5$
0	0	1	0	1	0	0
1	0	0	1	0	1	**1**
2	1	0	0	1	0	0
3	0	1	0	0	1	**1**
4	1	0	1	0	0	**1**
5	0	1	0	1	0	0

The sequence repeats its periodic behavior every $5T$ intervals.

$$\text{Total Period } (T_{\text{total}}) = 5T$$

ON time during one full period $(T_{\text{ON}}) = 3T$

The average power dissipation P_{avg} in the resistor $R = 10\,\text{k}\Omega$ with a supply voltage $V = 5\,\text{V}$ is calculated as:

$$P_{\text{avg}} = \frac{V^2}{R} \times \frac{T_{\text{ON}}}{T_{\text{total}}} = \frac{5^2}{10\,\text{k}\Omega} \times \frac{3T}{5T} = 2.5\,\text{mW} \times 0.6 = 1.5\,\text{mW}$$

The 30% clock duty cycle does not matter here, because every state lasts one full clock period T.

1.5

Key Points

- Average power dissipation formula for periodic pulse waveforms: $P_{\text{avg}} = \frac{V^2}{R} \times \frac{T_{\text{ON}}}{T}$.
- Ring and Johnson counters shift data linearly on each active clock edge.

Q5.15.1 MCQ 2015 2M Ans: C ●●○

The circuit consists of a 4-bit binary counter with a synchronous clear ($\overline{\text{CLR}}$) driven by an EX-NOR gate. The output of the EX-NOR gate is:

$$\overline{\text{CLR}} = \overline{Q_3 \oplus Q_2}$$

For a synchronous clear to trigger on the next active clock edge, the clear signal must be low ($\overline{\text{CLR}} = 0$):

$$Q_3 \oplus Q_2 = 1 \implies Q_3Q_2 = 01 \quad \text{or} \quad 10$$

Starting from 0000_2, the condition $Q_3Q_2 = 01$ is first met at state 4 (0100_2):

State 4 : $(Q_3Q_2Q_1Q_0) = 0100_2$

Since the clear input is **synchronous**, state 4 is stable for the current clock cycle. On the next active edge, the counter samples the active-low clear signal and resets to 0000_2 instead of progressing to state 5.
The valid state sequence is:

$$0000_2 \rightarrow 0001_2 \rightarrow 0010_2 \rightarrow 0011_2 \rightarrow 0100_2 \rightarrow (\text{Resets to } 0000_2)$$

The counter cycles through 5 distinct states (0 to 4), acting as a mod-5 counter.

(C) mod-5 counter

Key Points

- **Synchronous Clear:** The state that generates the clear condition is fully counted because the reset occurs only on the next active clock edge.
- **Asynchronous Clear:** The state that generates the clear condition is unstable and resets the circuit immediately, bypassing the next clock edge.

Mistakes to be avoided

- Do not misinterpret a synchronous clear as asynchronous, which would incorrectly result in a mod-4 counter.

Q5.15.2 MCQ 2015 2M Ans: D ● ○ ○

The circuit shows a 3-bit pseudo-random number generator constructed using positive edge-triggered D flip-flops.

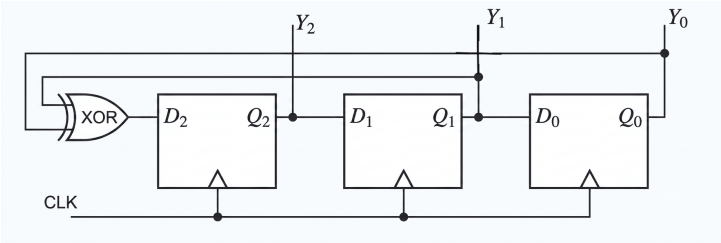


Fig. Solution Q5.15.2

By inspecting the feedback paths connected to the flip-flop inputs:

$$D_2 = Y_1 \oplus Y_0; \qquad D_1 = Y_2; \qquad D_0 = Y_1$$

We trace the state transitions cycle by cycle:

Clock Pulse	Y_2	Y_1	Y_0
Initially	1	1	1
1st clock	$1 \oplus 1 = 0$	1	1
2nd clock	$1 \oplus 1 = 0$	0	1
3st clock	$0 \oplus 1 = 1$	0	0

Thus, after three clock cycles, the output value of Y is 100.

(D) 100

Q5.15.3 MCQ 2015 1M Ans: A ● ○ ○

To determine the type of counter corresponding to the given circuit, we analyze the clock triggering mechanism and the reset logic condition.

1. Counting Direction (Up or Down Counter)

- All flip-flops are negative-edge triggered.
- The clock input of the next stage is driven by the true output Q of the preceding stage (Q_0 drives the clock of FF_1, and Q_1 drives the clock of FF_2).

- The transitions can be seen as follows

Q_2	Q_1	Q_0
0	0	0
0	0	1
0	1	0 ← -ve edge
0	1	1
1	0 ← -ve edge	0 ← -ve edge
1	0	1

Fig. Solution Q5.15.3

- Since the counter advances in increasing binary sequence (000 → 001 → 010 → ...), it operates as a **binary up counter**.

2. Modulus of the Counter
The flip-flops have an active-low asynchronous reset input ($\overline{R_d}$). The reset signal is driven by the output of a NAND gate:

$$\text{Reset} = \overline{Q_2 Q_0}$$

As soon as the counter reaches the state $Q_2Q_1Q_0 = 101$, the NAND gate output becomes:

$$\text{Reset} = \overline{1 \cdot 1} = 0$$

This active-low signal instantly resets all flip-flops back to 000.

Therefore, the state 101 (decimal 5) is a transient state and is not counted.The stable states are 0, 1, 2, 3, and 4.
Since there are 5 stable states, it is a **(MOD-5)** counter.

(A) a modulo-5 binary up counter

Key Points

Clock Trigger Type	Output Driving Next Stage	Counting Direction
Negative-Edge (-ve)	Q	Up Counter
Negative-Edge (-ve)	$\overline{Q}$	Down Counter
Positive-Edge (+ve)	Q	Down Counter
Positive-Edge (+ve)	$\overline{Q}$	Up Counter

Mistakes to be avoided

- Do not count the reset state (101) as a valid stable state. The counter only stays in 101 for a propagation delay duration(here zero) before resetting to 000.

Q5.15.4 NAT 2015 1M Ans: 7 ● ○ ○

The given circuit features a 4-bit binary up-counter connected to an active-low clear input ($\overline{\text{CLEAR}}$). We analyze how the clear condition behaves under synchronous operation.

1. **NAND Gate Condition:** The active-low clear signal is generated when $Q_C = 1$ and $Q_B = 1$, which first occurs at the count state:
$$Q_D Q_C Q_B Q_A = 0110_2 = 6_{10}$$
2. **Effect of Synchronous Clear:** Due to the synchronous clear input, the counter resets to 0000 on the next clock edge after reaching state 6, resulting in a stable counting sequence from 0 to 6.

The total number of stable unique states is 7. Thus, it is a mod-7 counter ($n = 7$).

$$\boxed{7}$$

Q5.14.1 MCQ 2014 2M Ans: D ● ● ○

The problem requires finding a modification to the circuit inputs such that replacing the XOR gate with an XNOR gate preserves the original state diagram.

1. **Original Circuit Equation:** Initially, input A is connected to Q_1. The input expression for the flip-flop D_2 through the XOR gate is:
$$D_2 = Q_1 \oplus S = \overline{Q_1}\, S + Q_1\, \overline{S}$$
2. **Modified Circuit with XNOR Gate:** Let the new connection for input A be denoted as Q_1'. When the XOR gate is replaced by an XNOR gate, the expression for D_2' becomes:
$$D_2' = Q_1' \odot S = Q_1'\, S + \overline{Q_1'}\, \overline{S}$$
3. **Equating Both Expressions:** To preserve the exact same state diagram, the next-state logic must remain identical ($D_2' = D_2$):
$$Q_1'\, S + \overline{Q_1'}\, \overline{S} = \overline{Q_1}\, S + Q_1\, \overline{S}$$
Comparing the coefficients of S and $\overline{S}$ on both sides yields:
$$Q_1' = \overline{Q_1}$$

Thus, the state diagram is preserved if input A is connected to $\overline{Q_1}$.

$$\boxed{\text{(D) Input A is connected to } \overline{Q_1}}$$

Key Points

- **Gate Duality Relation:** An XNOR operation can be transformed into an equivalent XOR operation by complementing one of its inputs:
$$X \odot Y = \overline{X} \oplus Y = X \oplus \overline{Y}$$

Mistakes to be avoided

- **Over-analyzing the State Diagram:** The given state diagram and full synchronous sequential circuit layout are provided to complicate the problem visually. Focus purely on the logic gate properties to find the answer quickly.

Q5.14.2 MCQ 2014 2M Ans: D ● ● ○

Method 1

The synchronous circuit is analyzed cycle-by-cycle using the flip-flop input connections:

$$J_1 = \overline{Q_2}, \quad K_1 = Q_2, \quad J_2 = Q_1, \quad K_2 = \overline{Q_1}$$

Given the initial state $(Q_1, Q_2) = (0, 0)$, the state transitions are tracked in the table below:

Clock	Q_1	Q_2	$J_1 = \overline{Q_2}$	$K_1 = Q_2$	$J_2 = Q_1$	$K_2 = \overline{Q_1}$	Q_1^+	Q_2^+
Initially	0	0	1	0	0	1	1	0
$1^{st} \uparrow$	1	0	1	0	1	0	1	1
$2^{nd} \uparrow$	1	1	0	1	1	0	0	1
$3^{rd} \uparrow$	0	1	0	1	0	1	0	0
$4^{th} \uparrow$	0	0	1	0	0	1	1	0

Extracting the values of Q_1 sequentially starting from the first cycle gives: $0, 1, 1, 0, 0, \ldots$

Method 2

Using the characteristic equation of a JK flip-flop ($Q^+ = J\,\overline{Q} + \overline{K}\,Q$), we derive the next-state logic equations for both stages:

$$Q_1^+ = J_1\,\overline{Q_1} + \overline{K_1}\,Q_1 = \overline{Q_2}\,\overline{Q_1} + \overline{Q_2}\,Q_1 = \overline{Q_2}$$

$$Q_2^+ = J_2\,\overline{Q_2} + \overline{K_2}\,Q_2 = Q_1\,\overline{Q_2} + Q_1\,Q_2 = Q_1$$

Starting with $(Q_1, Q_2) = (0, 0)$, the simplified state tracking table is evaluated:

Clock Cycle	Q_1	Q_2	$Q_1^+ = \overline{Q_2}$	$Q_2^+ = Q_1$
Initially	0	0	1	0
1	1	0	1	1
2	1	1	0	1
3	0	1	0	0
4	0	0	1	0

The sequence generated at Q_1 matches 01100 . . .

(D) 01100 . . .

Key Points

- This specific circuit structure operates as a 2-bit **Johnson Counter** (or twisted-ring counter).
- An n-stage Johnson Counter generates a sequence of total length $2n$ states before repeating.

Q5.14.3 MCQ 2014 2M Ans: C ● ● ○

Based on the provided timing diagram, the operation can be analyzed step-by-step at each consecutive **falling edge of the clock CLK:**

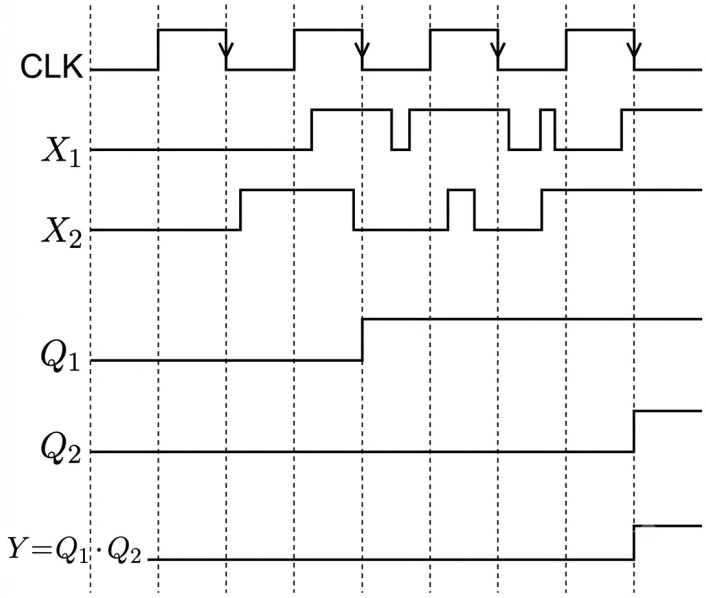


Fig. Solution Q5.14.3

Thus, only waveform W_3 satisfies the output waveform.

(C) W_3

Q5.14.4 MCQ 2014 1M Ans: D ● ○ ○

The circuit consists of two gated level-triggered D latches connected in series with complementary enable inputs:

- **Master Latch:** Transparent when Clk = 1, holding its state when Clk = 0.
- **Slave Latch:** Transparent when Clk = 0 (via inverter), holding its state when Clk = 1.

When Clk transitions from 1 → 0, the master stage locks the input data, and the slave stage simultaneously passes this locked state to the output Q.
Advantage: It completely eliminates the "race-around condition" encountered in level-triggered JK flip-flops by updating the output state precisely at the clock edge.

(D) Master-Slave D Flip Flop

Key Points

- Level-triggered latches become transparent when enabled.
- Cascading two transparent latches with out-of-phase clocks creates an edge-triggered flip-flop popularly known as Master-Slave Flip Flop.

Q5.14.5 NAT 2014 1M Ans: 62.5 ● ○ ○

The circuit can be redrawn as:

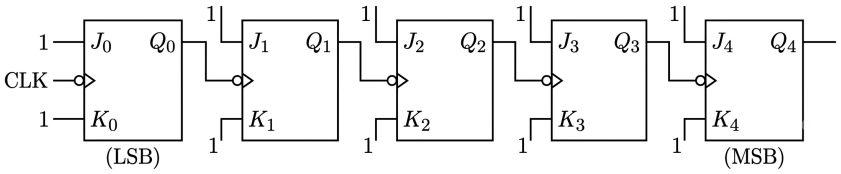


Fig. Solution Q5.14.5

The given circuit is a 5-bit asynchronous ripple counter in toggle mode ($J = K = 1$).
The clock signal transitions through the cascade from right to left, where the output Q_n of one stage acts as the clock input for the subsequent stage Q_{n+1}. The division factor of the clock frequency increases by a factor of 2 at each successive output stage from the input clock source:

$$f_{Q_0} = \frac{f_{CLK}}{2^1}, \quad f_{Q_1} = \frac{f_{CLK}}{2^2}, \quad f_{Q_2} = \frac{f_{CLK}}{2^3}, \quad f_{Q_3} = \frac{f_{CLK}}{2^4} \quad f_{Q_4} = \frac{f_{CLK}}{2^5}$$

Given the input frequency $f_{CLK} = 1\,\text{MHz}$, the frequency at output Q_3 is calculated by dividing by $2^4 = 16$:

$$f_{Q_3} = \frac{1\,\text{MHz}}{16} = \frac{1000\,\text{kHz}}{16} = 62.5\,\text{kHz}$$

$$\boxed{62.5}$$

Key Points

- In an asynchronous ripple counter, the frequency at the nth flip-flop output stage is given by $f_n = \frac{f_{CLK}}{2^n}$, where n is counted from the initial clock input.

Mistakes to be avoided

- Pay close attention that the frequency at output stage Q_3 is asked, not the final stage Q_4. Do not automatically divide by the total number of flip-flops ($2^5 = 32$).

Q5.12.1 MCQ 2012 2M Ans: D ●○○

Method 1

The state transition table for the sequential circuit is derived as follows:

Present State (Q)	**Input** (A)	**MUX Output** ($D = A \cdot Q + \overline{A} \cdot \overline{Q}$)	**Next State** (Q_{next})
0	0	$0 \cdot 0 + 1 \cdot 1 = 1$	1
0	1	$1 \cdot 0 + 0 \cdot 1 = 0$	0
1	0	$0 \cdot 1 + 1 \cdot 0 = 0$	0
1	1	$1 \cdot 1 + 0 \cdot 0 = 1$	1

Thus the following State Transition diagram can be constructed from the above table:

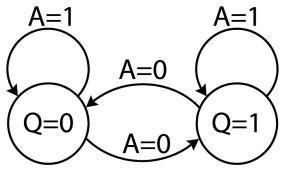


Fig. Solution Q5.12.1

Method 2

Analyzing the circuit transitions based on the input A and the current state Q:
The input to the D flip-flop is driven by a 2-1 multiplexer. The output expression of the multiplexer is:

$$D = A \cdot X_1 + \overline{A} \cdot X_0$$

From the circuit diagram: $X_1 = Q, \quad X_0 = \overline{Q}$
Thus, $Q_{next} = D = A \cdot Q + \overline{A} \cdot \overline{Q}$
Analyze the transitions for different values of A:

- **When $A = 0$:**

$$Q_{next} = (0) \cdot Q + 1 \cdot \overline{Q} = \overline{Q}$$

Thus, the state toggles for A=0

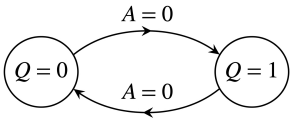


Fig. Solution Q5.12.1

- **When $A = 1$:**

$$Q_{next} = 1 \cdot Q + 0 \cdot \overline{Q} = Q$$

Thus, the state remains the same for A=1

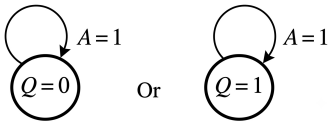


Fig. Solution Q5.12.1

Combining both, this behavior corresponds exactly to the state transitions shown in option (D).

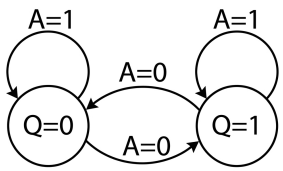


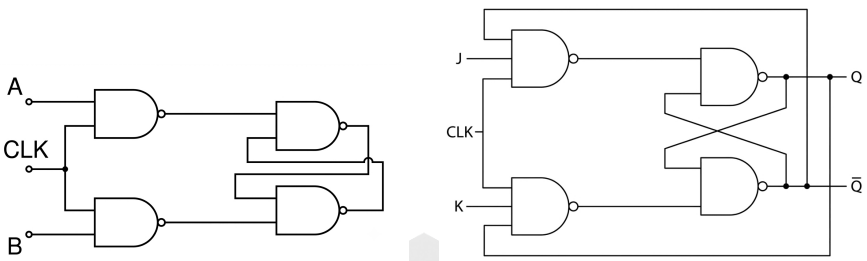
Fig. Solution Q5.12.1

(D)

Q5.12.2 MCQ 2012 1M Ans: A ● ● ●

Many students may think this is a JK latch. Comparing the given circuit with an actual JK latch

Given Circuit **JK Latch**



As we can see the given circuit lacks the additonal feedback to the first level of NAND gates thus this is just a **SR Latch** and **not a JK latch**.

A race-around condition specifically occurs in a level-triggered J-K latch when both inputs are high ($J = K = 1$) and the clock pulse remains active for a duration longer than the propagation delay of the gates, causing the output to toggle continuously.

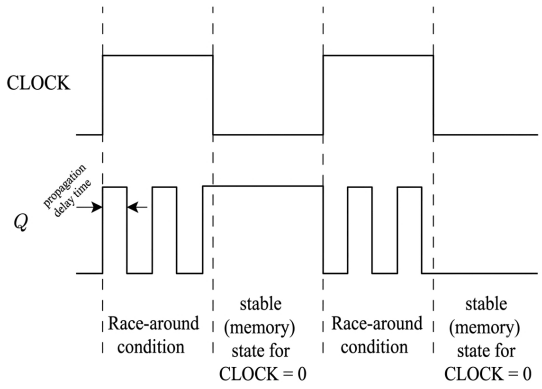


Fig. Solution Q5.12.2

But, for the given circuit, When CLK = 1 and $A = B = 1$, the first-stage NAND outputs become 0, forcing the final cross-coupled outputs to a stable, non-oscillating state of 1 simultaneously.

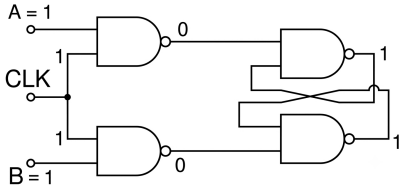


Fig. Solution Q5.12.2

Although this state is also an undesirable state because the output becomes unpredictable if the inputs are changed simultaneously back to 0.

(Refer Q5.17.4 for detailed explanation on undesirable state of SR latch)

Still, it remains stable and non-oscillating while the inputs are maintained at 1.
Therefore, the race-around condition **does not occur** in this circuit.

(A) does not occur

Key Points

- **Race-Around Condition:** Only occurs in level-triggered J-K flip-flops when $J = K = 1$ and the clock pulse width t_p is greater than the propagation delay Δt of the flip-flop.
- **S-R Latch Behavior:** For an S-R flip-flop using NAND gates, the input condition $S = R = 1$ leads to an invalid/indeterminate state where both outputs try to go high, but it does not cause oscillation (racing).

Mistakes to be avoided

- **Invalid State Pitfall:** Do not confuse the invalid/forbidden state of an S-R flip-flop ($A = B = 1$ when CLK = 1) with the race-around condition. While $S = R = 1$ is an undesirable state because the output becomes unpredictable if the inputs are changed simultaneously back to 0 (dependent on internal propagation delays),still it remains stable and non-oscillating while the inputs are maintained at 1.

Q5.11.1 MCQ 2011 2M Ans: A ●●○

A Johnson counter feeds the inverted output of its last stage back to the first stage, shifting bits on each clock cycle.It looks something like this:

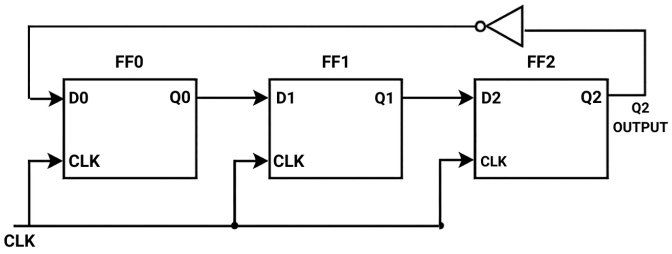


Fig. Solution Q5.11.1

Starting from the initial reset state 000_2, the sequence of states and the corresponding DAC outputs are tracked:

Clock Pulse	Q_2	Q_1	Q_0	Output of DAC (V_o)
Initially	0	0	0	0 V
1st	1	0	0	4 V
2nd	1	1	0	6 V
3rd	1	1	1	7 V
4th	0	1	1	3 V
5th	0	0	1	1 V

The sequence of voltage values generated at V_o matches choice (A).

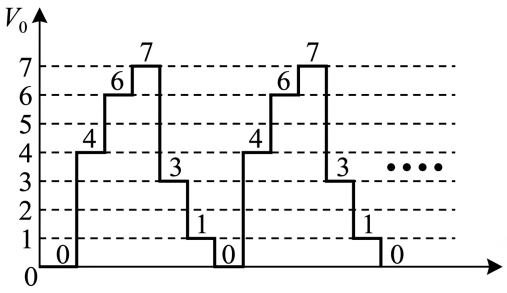


Fig. Solution Q5.11.1

$\boxed{(A)}$

Key Points

- An N-bit Johnson counter possesses $2N$ unique states.

Q5.11.2 MCQ 2011 2M Ans: D ● ● ○

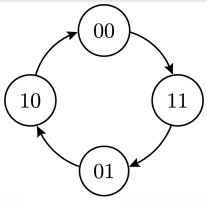


Fig. Solution Q5.11.2

The state table for the synchronous counter using D flip-flops is as follows:

Present State		Next State		Flip-flop Input	
Q_B	Q_A	Q_B^+	Q_A^+	$D_B = Q_B^+$	$D_A = Q_A^+$
0	0	1	1	1	1
1	1	0	1	0	1
0	1	1	0	1	0
1	0	0	0	0	0

From the state table, the logic expressions for the flip-flop inputs can be determined.
For D_B: $D_B = 1$ when $Q_BQ_A = 00$ and 01. Thus, $D_B = \overline{Q}_B$.
For D_A: $D_A = 1$ when $Q_BQ_A = 00$ and 11. Thus, $D_A = Q_AQ_B + \overline{Q}_A\,\overline{Q}_B$.

$\boxed{\text{(D) } D_A = (Q_AQ_B + \overline{Q}_A\,\overline{Q}_B), D_B = \overline{Q}_B}$

Q5.11.3 MCQ 2011 1M Ans: A ● ● ○

Let the first D flip-flop be FF_1 with output Q_1 and the second D flip-flop be FF_2 with output Q_2.
Both flip-flops share a common clock signal, forming a synchronous circuit.

- The input to FF_1 is the external line Data. Thus, after a clock edge, Q_1 = Data.

- The input to FF_2 is connected to the complemented output $\overline{Q}_1$ of the first flip-flop. Thus, after a clock edge, its output Q_2 becomes the previous value of $\overline{Q}_1$.

The output Y is given by the AND gate expression:

$$Y = Q_1 \cdot Q_2$$

For $Y = 1$, both inputs to the AND gate must be high:

$$Q_1 = 1 \quad \text{and} \quad Q_2 = 1$$

- $Q_1 = 1$ implies that the current state of Data (sampled at the most recent clock edge) is 1.
- $Q_2 = 1$ implies that the previous value of $\overline{Q}_1$ was 1, meaning the previous value of Q_1 was 0.

Present state		Present input		Next state		Output
Q_B	Q_A	$D_B = \overline{Q}_A$	D_A = Data	Q_B^+	Q_A^+	$Y = Q_B^+ Q_A^+$
0	0	1	0	1	0	0
1	0	1	1	1	1	1

Since Q_1 represents the sampled value of Data, a change from $Q_1 = 0$ in the previous cycle to $Q_1 = 1$ in the current cycle indicates that the input sequence on Data has changed from 0 to 1.

(A) changed from "0" to "1"

Q5.10.1 MCQ 2010 2M Ans: D ● ● ○

The given synchronous circuit is a 3-bit feedback shift register using D flip-flops triggered by a negative edge-sensitive clock. The next-state equations for the individual flip-flops are:

$$Q_A^+ = D_A = E = \overline{Q_B \oplus Q_C} = Q_B \odot Q_C$$

$$Q_B^+ = D_B = Q_A$$

$$Q_C^+ = D_C = Q_B$$

Initially, all flip-flops are in the reset condition, so $(Q_A, Q_B, Q_C) = (0, 0, 0)$.
The state transitions following successive negative clock edges are traced in the state table below:

CLK Edge	Feedback Input $E = Q_B \odot Q_C$	$Q_A^+ = E$	Q_B^+	Q_C^+
Initial	—	0	0	0
1	$0 \odot 0 = 1$	1	0	0
2	$0 \odot 0 = 1$	1	1	0
3	$1 \odot 0 = 0$	0	1	1
4	$1 \odot 1 = 1$	1	0	1
5	$0 \odot 1 = 0$	0	1	0
6	$1 \odot 0 = 0$	0	0	1
7	$0 \odot 1 = 0$	0	0	0

Observing the sequence of values generated at the output Q_A across the clock periods starting from the first clock edge:

$$Q_A \rightarrow 0, 1, 1, 0, 1, 0, 0, 0 \ldots$$

Thus, the sequence matches 0110100 . . . when accounting for the periodic progression.

(D) 0110100 . . .

Q5.09.1 MCQ 2009 2M Ans: A ●●○

From the given logic diagram, the flip-flop input expressions are:

$$J_1 = \overline{Q}_2, \quad K_1 = \overline{Q}_2$$

$$J_2 = \overline{Q}_1, \quad K_2 = 1$$

Starting with an initial state $(Q_1, Q_2) = (0, 0)$ as suggested by the choices, the subsequent state transitions are tabulated below:

Present State		Inputs				Next State	
Q_1	Q_2	$J_1 = \overline{Q}_2$	$K_1 = \overline{Q}_2$	$J_2 = \overline{Q}_1$	$K_2 = 1$	Q_1^+	Q_2^+
0	0	1	1	1	1	1	1
1	1	0	0	0	1	1	0
1	0	1	1	0	1	0	0
0	0	1	1	1	1	1	1

Thus, the repeating count sequence for (Q_1, Q_2) is 11, 10, 00, 11, 10 . . .

(A) 11, 10, 00, 11, 10, . . .

Key Points

J	K	Mode	Description	Q^+
0	0	Hold	Retains previous state	Q
0	1	Reset	Forces output to low	0
1	0	Set	Forces output to high	1
1	1	Toggle	Inverts previous state	$\overline{Q}$

Q5.09.2 MCQ 2009 2M Ans: C ●●○

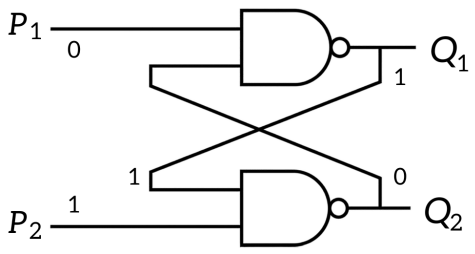
The circuit consists of cross-coupled latches analyzed sequentially under two input configurations: first $(P_1, P_2) = (0, 1)$, then transitioning to $(P_1, P_2) = (1, 1)$.

1. NAND Latch Analysis

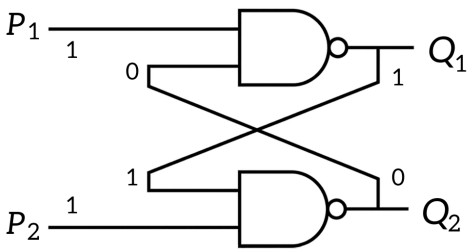
Case 1: $(P_1, P_2) = (0, 1)$
Since $P_1 = 0$, the top NAND gate forces $Q_1 = 1$. Feeding this back with $P_2 = 1$ yields:

$$Q_2 = \overline{P_2 \cdot Q_1} = \overline{1 \cdot 1} = 0$$

Thus, the stable output state is $(Q_1, Q_2) = (1, 0)$.



Case 2: Transition to $(P_1, P_2) = (1, 1)$
The input (1, 1) is the hold state for a NAND latch, keeping the prior outputs unchanged. Thus, the stable state remains $(Q_1, Q_2) = (1, 0)$.

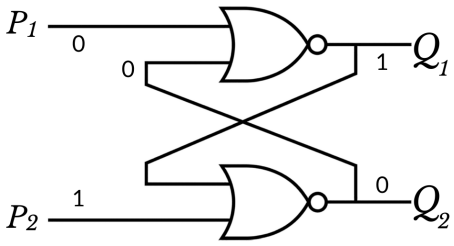


2. NOR Latch Analysis

Case 1: $(P_1, P_2) = (0, 1)$
Since $P_2 = 1$, the bottom NOR gate forces $Q_2 = 0$. Feeding this back with $P_1 = 0$ yields:

$$Q_1 = \overline{P_1 + Q_2} = \overline{0 + 0} = 1$$

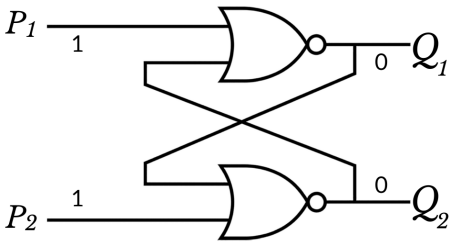
Thus, the stable output state is $(Q_1, Q_2) = (1, 0)$.



Case 2: Transition to $(P_1, P_2) = (1, 1)$
With both inputs high, both NOR gates are forced low simultaneously regardless of feedback:

$$Q_1 = \overline{1 + 0} = 0, \quad Q_2 = \overline{1 + 0} = 0$$

Hence, the subsequent stable state changes to $(Q_1, Q_2) = (0, 0)$.



Combining these sequential transitions matches the evaluations described in choice (C).

(C) NAND: first (1, 0) then (1, 0); NOR: first (1, 0) then (0, 0)

Key Points

- For a NAND latch, if any input is 0, its output instantly becomes 1 without waiting for the other input.
- Similarly, for a NOR latch, if any input is 1, its output instantly becomes 0.
- The state (1, 1) represents a memory hold configuration for a NAND latch, but acts as an invalid/cleared drive condition for a NOR latch.

Mistakes to be avoided

- Do not assume that (1, 1) acts as a hold state for both configurations; make sure to evaluate the deterministic output based on basic gate properties first.

Q5.08.1 MCQ | 2008 | 2M | Ans: C

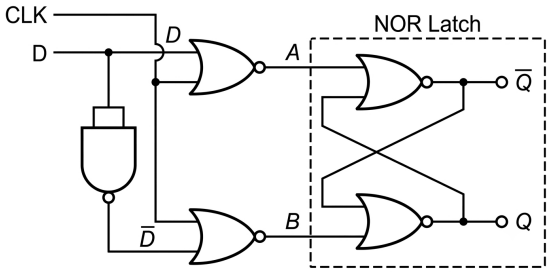


Fig. Solution Q5.08.1

Analyzing the circuit inputs, the intermediate nodes A and B are given by:

$$A = \overline{D + \text{CLK}} \quad \text{and} \quad B = \overline{\overline{D} + \text{CLK}}$$

The response is tracked through the three regions of the waveform:

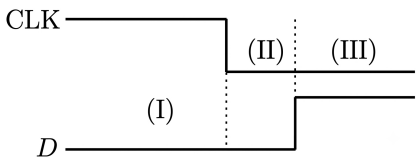


Fig. Solution Q5.08.1

- **Region I (CLK** $= 1, D = 0$**):** With CLK = 1, both intermediate outputs are forced low:

$$A = \overline{0 + 1} = 0, \quad B = \overline{1 + 1} = 0$$

The input $(A, B) = (0, 0)$ is the hold state for a NOR latch, so Q remains unchanged ($Q_{n+1} = Q_n$).

- **Region II (CLK** $= 0, D = 0$**):** The intermediate node expressions yield:

$$A = \overline{0 + 0} = 1, \quad B = \overline{1 + 0} = 0$$

Since $A = 1$, the top NOR gate forces $\overline{Q} = 0$. Feeding this back with $B = 0$ sets the main output high:

$$Q = \overline{B + \overline{Q}} = \overline{0 + 0} = 1$$

Thus, Q transitions to 1.

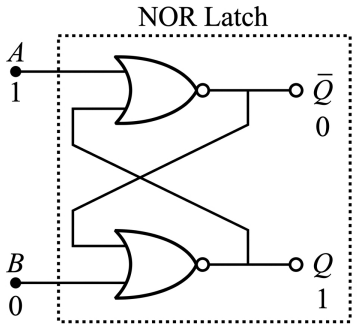


Fig. Solution Q5.08.1

- **Region III (CLK** $= 0, D = 1$**):** The updated expressions yield:

$$A = \overline{1 + 0} = 0, \quad B = \overline{0 + 0} = 1$$

Since $B = 1$, the bottom NOR gate directly forces the main output low:

$$Q = \overline{1 + \overline{Q}} = 0$$

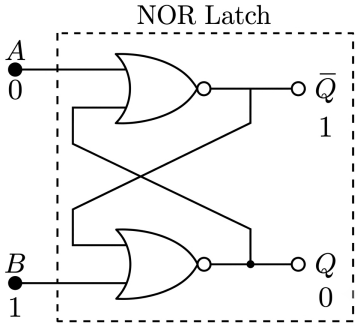


Fig. Solution Q5.08.1

Thus, Q drops to 0 when D transitions to 1.

Hence, Q goes to 1 at the CLK transition and returns to 0 when D becomes 1, matching option (C).

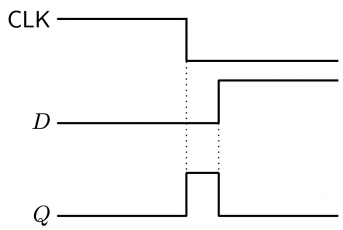


Fig. Solution Q5.08.1

(C) Q goes to 1 at the CLK transition and goes to 0 when D goes to 1.

Key Points

- For a NOR latch, if any input is 1, its output instantly becomes 0.
- The input condition $(A, B) = (0, 0)$ leaves a cross-coupled NOR structure in its hold state.

Q5.08.2 MCQ 2008 2M Ans: B ●●○

The given circuit consists of two positive edge-triggered $J - K$ flip-flops connected in a ripple counter configuration. Both flip-flops are configured in the toggle mode ($J_0 = K_0 = 1$ and $J_1 = K_1 = 1$).

- **First Flip-flop (FF_0):** It is clocked by the external clock CLK with period T. Since it toggles at every positive edge of CLK, its output Q_0 changes state after a propagation delay of Δt. Therefore, the time period of Q_0 becomes $2T$, and its first rising edge occurs at $t_1 + \Delta t$.
- **Second Flip-flop (FF_1):** The output Q_0 acts as the clock for FF_1. Since FF_1 is also positive edge-triggered, it toggles at every rising edge of Q_0. It introduces an additional propagation delay of Δt.

Thus, the overall time period of the output Q_1 is:

$$T_{out} = 2 \times T_{Q0} = 2 \times (2T) = 4T$$

The first rising edge of Q_1 is delayed by the cumulative propagation delay of both flip-flops:

$$\Delta T_{pd} = \Delta t + \Delta t = 2\Delta t$$

Therefore, the first rising edge of Q_1 occurs at $t_1 + 2\Delta t$ with a time period of $4T$, which matches the waveform shown in option (B).

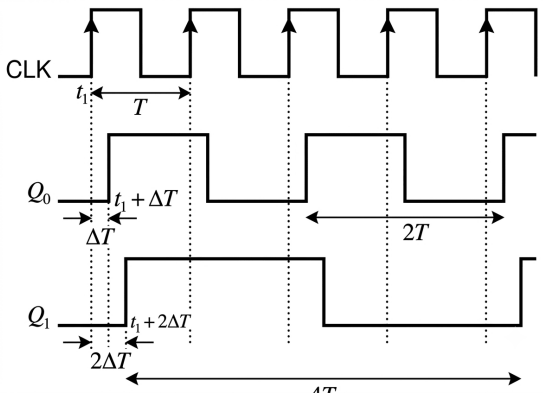


Fig. Solution Q5.08.2

(B)

Key Points

- In an n-bit asynchronous (ripple) counter, the clock ripples through each stage, and the total propagation delay accumulates linearly: $\Delta T_{pd} = n \cdot \Delta t$.
- For a counter with n flip-flops in toggle mode, the frequency is divided by 2^n, which means the output time period becomes $2^n T$.

Q5.07.1 MCQ 2007 2M Ans: B ●●○

The inputs to the flip-flops are given by:

$$D_0 = \overline{Q_0 + Q_1}$$

$$D_1 = Q_0$$

Let us trace the state transitions starting from the initial state $(Q_1Q_0) = (00)$:

Clock	Current State (Q_1Q_0)	$D_0 = \overline{Q_0 + Q_1}$	$D_1 = Q_0$	Q_1^+	Q_0^+
0	00	1	0	0	1
1	01	0	1	1	0
2	10	0	0	0	0

The sequence repeats, giving the sequence: $00 \rightarrow 01 \rightarrow 10 \rightarrow 00 \ldots$

(B) 00, 01, 10, 00, 01 . . .

Q5.07.2 MCQ 2007 2M Ans: C ●●●

We evaluate the outputs by following the sequential input states step-by-step:

- **Step 1** $(X = 0, Y = 1)$**:** Since $X = 0$, the upper NAND gate is forced to $P = 1$. Feeding $P = 1$ and $Y = 1$ into the lower gate gives $Q = \overline{1 \cdot 1} = 0$. Thus, $(P, Q) = (1, 0)$.

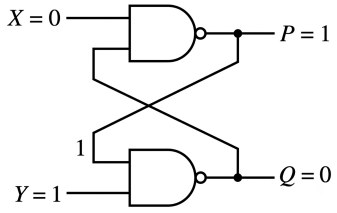


Fig. Solution Q5.07.2

- **Step 2** $(X = 0, Y = 0)$**:** With both active-low control inputs held at 0, both NAND gates are immediately forced to output a logic high regardless of any feedback loop or internal cross-coupling state. Thus, $(P, Q) = (1, 1)$.

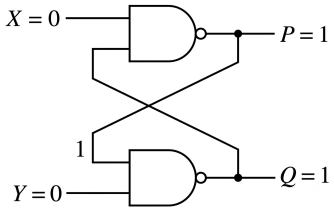
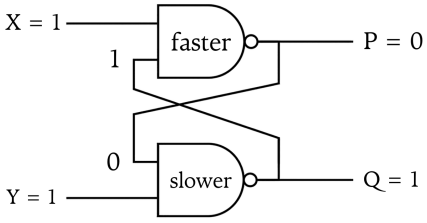
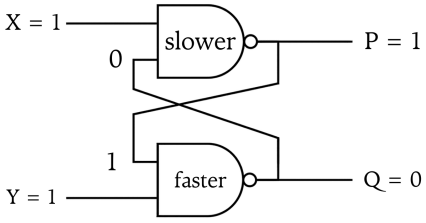


Fig. Solution Q5.07.2

- **Step 3** ($X = 1, Y = 1$)**:** Releasing the inputs from $(0, 0)$ to $(1, 1)$ causes both gates to race to change state. Depending on which gate is faster, the circuit will randomly settle into either the $(1, 0)$ or $(0, 1)$ stable state.



Case 1: Upper gate is faster



Case 2: Lower gate is faster

(Refer to question Q5.17.4 for a detailed explanation on this unpredictable next state behavior.)

(C) $P = 1, Q = 0$; $P = 1, Q = 1$; $P = 1, Q = 0$ or $P = 0, Q = 1$

Key Points

- For any NAND gate, a logic 0 at any input completely overrides the other input and forces the output to 1.
- When both outputs of a cross-coupled NAND latch are forced to 1 simultaneously and then released to the hold state (1, 1), an unpredictable condition occurs.

Q5.06.1 MCQ 2006 2M Ans: D ●●○

The given circuit describes a serial adder consisting of two 4-bit shift registers, three edge-triggered D flip-flops, and a Full Adder block.
Initially, all flip-flops are cleared, which sets the state to:

$$Q_{FF1} = 0, \quad Q_{FF2} = 0, \quad Q_{FF3} = 0 \implies A = 0, \quad B = 0, \quad C_i = 0$$

The data streams inside the shift registers shift from left to right on each clock edge:

- **Top Register (Data for A):** $[1, 0, 1, 1] \rightarrow$ LSB is 1.
- **Bottom Register (Data for B):** $[0, 0, 1, 1] \rightarrow$ LSB is 1.

Let us track the state transitions bit-by-bit over the two clock pulses:

1. **Before the 1st Clock Pulse:** The first bits available at the inputs of the register flip-flops are the LSBs: $D_{FF1} = 1$ and $D_{FF2} = 1$. The carry flip-flop has $D_{FF3} = C_o = 0$.

2. **After the 1st Clock Pulse:** The bits are transferred to the outputs of the flip-flops, feeding the Full Adder inputs:

$$A = Q_{FF1} = 1, \quad B = Q_{FF2} = 1, \quad C_i = Q_{FF3} = 0$$

The Full Adder outputs are calculated as:

$$S = A \oplus B \oplus C_i = 1 \oplus 1 \oplus 0 = 0$$

$$C_o = AB + BC_i + C_iA = (1)(1) + (1)(0) + (0)(1) = 1$$

Now, the next data bits shifting out of the registers are $D_{FF1} = 1$ and $D_{FF2} = 1$. The input to the carry flip-flop becomes $D_{FF3} = C_o = 1$.

3. **After the 2nd Clock Pulse:** The new outputs of the flip-flops become:

$$A = Q_{FF1} = 1, \quad B = Q_{FF2} = 1, \quad C_i = Q_{FF3} = 1$$

The evaluation of the outputs of the Full Adder at this state yields:

$$S = A \oplus B \oplus C_i = 1 \oplus 1 \oplus 1 = 1$$

$$C_o = AB + BC_i + C_iA = (1)(1) + (1)(1) + (1)(1) = 1$$

Thus, after the application of two clock pulses, the output state is $S = 1, C_o = 1$.

$$\boxed{\text{(D) } S = 1, C_o = 1}$$

Key Points

- A serial adder uses a single Full Adder cell and a delay element (D flip-flop) to process multi-bit additions sequentially from LSB to MSB.
- The characteristic equation of a standard D flip-flop is $Q_{next} = D$, meaning inputs are sampled at the active clock edge and held until the next cycle.

Q5.06.2 MCQ 2006 2M Ans: A ●●○

We need to design a synchronous counter using two D flip-flops that cycles through the sequence Q_1Q_0:

$$00 \rightarrow 01 \rightarrow 11 \rightarrow 10 \rightarrow 00 \rightarrow \ldots$$

Using the excitation characteristic of a D flip-flop ($Q_{next} = D$), we form the state transition table to find the required expressions for the inputs D_1 and D_0:

Present State		Next State		Flip-Flop Inputs	
Q_1	Q_0	Q_1^+	Q_0^+	D_1	D_0
0	0	0	1	0	1
0	1	1	1	1	1
1	1	1	0	1	0
1	0	0	0	0	0

Now, let us derive the simplified minimal expressions for D_1 and D_0 using Karnaugh Maps:

1. **K-map minimization for D_1:** The minterms for $D_1(Q_1, Q_0)$ are $\sum m(1, 3)$.

Q_1 \ Q_0	0	1
0	0	1
1	0	1

Grouping the column under $Q_0 = 1$ yields:

$$D_1 = Q_0$$

2. **K-map minimization for D_0:** The minterms for $D_0(Q_1, Q_0)$ are $\sum m(0, 1)$.

Q_1 \ Q_0	0	1
0	1	1
1	0	0

Grouping the row under $Q_1 = 0$ yields:

$$D_0 = \overline{Q}_1$$

Therefore, the connections required are $D_0 = \overline{Q}_1$ and $D_1 = Q_0$ respectively.

(A) $\overline{Q}_1$ and Q_0

Key Points

- For a D flip-flop, the state table maps directly to the combinational input equations because $D = Q^+$.
- Synchronous counters utilize a common clock source connected to all flip-flops to trigger state transitions simultaneously and eliminate propagation delays.

Q5.05.1 MCQ | 2005 | 2M | Ans: C | ● ○ ○

The characteristic equation of an edge-triggered J-K flip-flop is given by:

$$Q_{n+1} = J\,\overline{Q}_n + \overline{K}\,Q_n$$

Given parameters:

- Present state, $Q_n = 0$
- Input, $J = 1$

Substituting these values into the characteristic equation:

$$Q_{n+1} = 1 \cdot \overline{0} + \overline{K} \cdot 0$$

$$Q_{n+1} = 1 \cdot 1 + 0 = 1$$

Thus, the next state Q_{n+1} will be logic 1, regardless of the value of K.

(C) will be logic 1

Key Points

- The characteristic equation for a J-K flip-flop is $Q_{n+1} = J\,\overline{Q}_n + \overline{K}\,Q_n$.
- When $Q_n = 0$ and $J = 1$, the flip-flop is unconditionally forced to the **Set** state ($Q_{n+1} = 1$) on the next active clock edge.

Q5.05.2 MCQ 2005 2M Ans: C ●●○

The given circuit is a 3-bit ripple counter consisting of T flip-flops with all toggle inputs connected to logic 1 ($T_2 = T_1 = T_0 = 1$).

- The flip-flops are **positive edge-triggered** (indicated by the clock terminal without a bubble).
- The clock input of the next stage is driven by the complemented output ($\overline{Q}$) of the previous stage.

When the clock terminal of a positive edge-triggered flip-flop is connected to $\overline{Q}$, a transition of Q from $1 \rightarrow 0$ causes $\overline{Q}$ to go from $0 \rightarrow 1$ (a positive edge), which triggers the next stage. This configuration operates as an **up-counter**.
Let us trace the transitions from the initial state $Q_2Q_1Q_0 = 011$:

1. **First Flip-flop (FF_0):** Receives the external clock pulse. Since $T_0 = 1$, Q_0 toggles on every positive clock edge.

$$Q_0 : 1 \longrightarrow 0 \quad \text{(High-to-Low transition)}$$

2. **Second Flip-flop (FF_1):** Since Q_0 transitions from $1 \rightarrow 0$, its complement $\overline{Q}_0$ transitions from $0 \rightarrow 1$ (positive edge). This triggers FF_1. Since $T_1 = 1$, Q_1 toggles:

$$Q_1 : 1 \longrightarrow 0 \quad \text{(High-to-Low transition)}$$

3. **Third Flip-flop (FF_2):** Since Q_1 transitions from $1 \rightarrow 0$, its complement $\overline{Q}_1$ transitions from $0 \rightarrow 1$ (positive edge). This triggers FF_2. Since $T_2 = 1$, Q_2 toggles:

$$Q_2 : 0 \longrightarrow 1$$

Combining the new states, the next state is:

$$Q_2Q_1Q_0 = 100$$

(C) 100

Key Points

- For an asynchronous (ripple) counter using positive edge-triggered flip-flops:
  - Clock from $Q \implies$ Down-counter
  - Clock from $\overline{Q} \implies$ Up-counter

Q5.04.1 MCQ 2004 1M Ans: C ●○○

A master-slave flip-flop consists of a cascade of two latches (master and slave) operating on complementary clock phases.

- **When CLK = 1:** The master is active and samples the external inputs, while the slave is isolated (OFF). No change reaches the final output.
- **When CLK = 0:** The master becomes isolated, and the slave is activated. The slave samples the state of the master and updates its state, which directly changes the final output of the circuit.

Therefore, any change in the final output occurs only when the state of the slave is affected.

(C) change in the output occurs when the state of the slave is affected

Key Points

- The master-slave configuration eliminates the race-around condition by ensuring the output changes only once per clock cycle.
- The final output of the circuit is always read from the slave flip-flop.

Q5.04.2 MCQ 2004 1M Ans: B ●○○

The functions of the digital components given in Group 1 are matched with Group 2 as follows:

- **P. Shift register → 3. Serial to parallel data conversion**: A Serial-In Parallel-Out (SIPO) shift register receives data bits sequentially one by one and presents them simultaneously on parallel output lines.
- **Q. Counter → 1. Frequency division**: A MOD-N counter divides the input clock frequency f_{in} by N such that:

$$f_{\text{out}} = \frac{f_{\text{in}}}{N}$$

- **R. Decoder → 2. Addressing in memory chips**: Decoders are standard combinatorial circuits used to select a specific memory location or chip by decoding address lines.

Thus, the correct matching is: **P-3, Q-1, R-2**.

(B) P - 3 Q - 1 R - 2

Key Points

- Shift registers are vital for data format conversion (serial-to-parallel or vice-versa).
- Every flip-flop stage in a ripple counter performs a divide-by-2 frequency division.
- Binary decoders convert an n-bit address into 2^n unique selection lines in memory systems.

Q5.04.3 MCQ 2004 2M Ans: C ●●○

The given circuit is a 3-bit ripple counter with outputs labeled C (MSB), B, and A (LSB).

- The counter has active-low clear inputs (CLR, indicated by bubbles on the reset pin). Therefore, the clear operation is triggered when the gate output is 0.
- A modulo-6 counter counts from 0 to 5 (000_2 to 101_2) and resets immediately upon reaching state 6 (110_2).

At the reset state $CBA = 110$:

- The inputs to the 2-input gate are taken from the complemented outputs $\overline{B}$ and $\overline{C}$.
- For $CBA = 110$, we have $C = 1 \implies \overline{C} = 0$ and $B = 1 \implies \overline{B} = 0$.

The gate must output a 0 to clear the flip-flops precisely when both its inputs are 0 ($\overline{B} = 0$ and $\overline{C} = 0$).
Let us test the given gate options for inputs $(0, 0)$:

- **OR gate:** Gives $0 + 0 = 0$. For any normal counting state (0 to 5), at least one input ($\overline{B}$ or $\overline{C}$) is 1, keeping the OR gate output at 1 (inactive). Thus, it works perfectly.
- **AND gate:** Gives $0 \cdot 0 = 0$. However, if either input becomes 0 (which happens during other valid counting states like $CBA = 100$ or 010), the AND gate would prematurely output 0 and reset the counter. Thus, it cannot be used.

Therefore, the 2-input gate is an OR gate.

(C) an OR gate

Key Points

- A modulo-N counter resets on reaching the state corresponding to the value N.
- Active-low reset pins require a logic 0 to clear the state.
- An OR gate with complemented inputs acts equivalent to a NAND gate with uncompled inputs via De Morgan's Law: $\overline{C} + \overline{B} = \overline{C \cdot B}$.

Q5.03.1 MCQ 2003 2M Ans: B ● ● ○

In a synchronous counter, the clock pulse is applied to all the flip-flops simultaneously. Therefore, the total propagation delay is determined solely by the maximum propagation delay of a single flip-flop.

$$S = t_{pd} = 10\,\text{ns}$$

In a ripple (asynchronous) counter, the output of one flip-flop acts as the clock input for the next flip-flop. Thus, the delays accumulate across all stages. For an n-bit ripple counter, the total worst-case propagation delay is the sum of the individual delays.

$$R = n \times t_{pd} = 4 \times 10\,\text{ns} = 40\,\text{ns}$$

(B) $R = 40\,\text{ns}, S = 10\,\text{ns}$

Key Points

- **Synchronous Counter:** Worst-case delay is independent of the number of bits:

$$T_{CLK} \geq (t_{pd})_{FF}$$

- **Ripple Counter:** Worst-case delay accumulates with the number of bits:

$$T_{CLK} \geq n \times (t_{pd})_{FF}$$

Q5.03.2 MCQ 2003 1M Ans: D ● ● ○

A 0 to 6 counter counts up to 6 and needs to reset back to 0 upon hitting the next state (7).
The binary representation for state 7 with 3 flip-flops is:

$$Q_3Q_2Q_1 = 111$$

To force a reset when this state is reached, a decoding logic expression $Y = Q_1 \cdot Q_2 \cdot Q_3$ is required to assert the active-high clear/reset inputs (CLR).
Since only 2-input gates are allowed, a 3-input AND function must be constructed by cascading two separate 2-input AND gates:

$$Y = (Q_1 \cdot Q_2) \cdot Q_3$$

Thus, the combinational circuit consists of two AND gates.

(D) two AND gates

Key Points

- **Truncated Counters:** A counter that resets at state N requires combinational logic to decode state N and trigger the asynchronous reset/clear line.
- **Gate Cascading:** An n-input AND logic function can be realized using $(n - 1)$ numbers of 2-input AND gates.

Q5.02.1 Descriptive 2002 5M Ans: *

To design the binary mod-5 synchronous counter, we first derive the excitation table of the custom AB flip-flop from its given characteristic table.
The transition characteristics are:

Q_n	Q_{n+1}	Required A, B
0	0	11 or 10 $\Longrightarrow$ $A = 1, B = X$
0	1	00 or 01 $\Longrightarrow$ $A = 0, B = X$
1	0	01 or 11 $\Longrightarrow$ $A = X, B = 1$
1	1	00 or 10 $\Longrightarrow$ $A = X, B = 0$

The counter follows the state sequence: $000 \rightarrow 001 \rightarrow 010 \rightarrow 011 \rightarrow 100 \rightarrow 000$. States 101, 110, and 111 are unused, acting as don't cares (X).
The complete state table for the mod-5 synchronous counter is given below:

Present State			**Next State**			**Flip-Flop Inputs**					
Q_2	Q_1	Q_0	$Q_{2(n+1)}$	$Q_{1(n+1)}$	$Q_{0(n+1)}$	A_2	B_2	A_1	B_1	A_0	B_0
0	0	0	0	0	1	1	X	1	X	0	X
0	0	1	0	1	0	1	X	0	X	X	1
0	1	0	0	1	1	1	X	X	0	0	X
0	1	1	1	0	0	0	X	X	1	X	1
1	0	0	0	0	0	X	1	1	X	1	X
1	0	1	X	X	X	X	X	X	X	X	X
1	1	0	X	X	X	X	X	X	X	X	X
1	1	1	X	X	X	X	X	X	X	X	X

Q5.02.2 Descriptive 2002 5M Ans: *

Using the truth table entries and don't care conditions (101, 110, 111) from the mod-5 state table, we minimize the control expressions via 3-variable Karnaugh Maps:

- **For A_2:** Minimizing minterms $\sum m(0, 1, 2) + d(4, 5, 6, 7)$:

$$A_2 = \overline{Q_1} + \overline{Q_0}$$

- **For B_2:** Minimizing minterms $\sum m(4) + d(0, 1, 2, 3, 5, 6, 7)$:

$$B_2 = 1$$

- **For A_1:** Minimizing minterms $\sum m(0, 4) + d(2, 3, 5, 6, 7)$:

$$A_1 = \overline{Q_0}$$

- **For B_1:** Minimizing minterms $\sum m(3) + d(0, 1, 4, 5, 6, 7)$:

$$B_1 = Q_0$$

- **For A_0:** Minimizing minterms $\sum m(4) + d(1, 3, 5, 6, 7)$:

$$A_0 = Q_2$$

- **For B_0:** Minimizing minterms $\sum m(1, 3) + d(0, 2, 4, 5, 6, 7)$:

$$B_0 = 1$$

Q5.02.3 Descriptive 2002 5M Ans: *

To implement the derived SOP expressions using the minimum number of 2-input NAND gates, we use De Morgan's Law ($\overline{X} + \overline{Y} = \overline{X \cdot Y}$):

- $A_2 = \overline{Q_1} + \overline{Q_0} = \overline{Q_1 \cdot Q_0}$ $\longrightarrow$ (Requires exactly one 2-input NAND gate)
- $B_2 = 1$ $\longrightarrow$ (Connected directly to logic HIGH)
- $A_1 = \overline{Q_0}$ $\longrightarrow$ (Requires one 2-input NAND gate configured as an inverter)
- $B_1 = Q_0$ $\longrightarrow$ (Direct connection)
- $A_0 = Q_2$ $\longrightarrow$ (Direct connection)
- $B_0 = 1$ $\longrightarrow$ (Connected directly to logic HIGH)

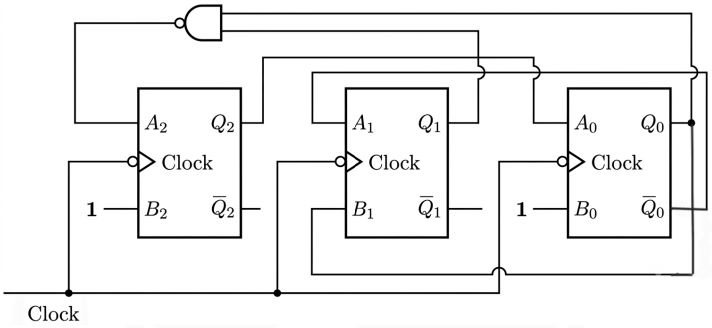


Fig. Solution Q5.02.3

Key Points

- An n-input NAND gate can serve as an inverter by tying all its inputs together.
- De Morgan's identity is directly utilized to implement active-low or inverted SOP configurations with standard NAND components.

Q5.02.4 MCQ 2002 2M Ans: C

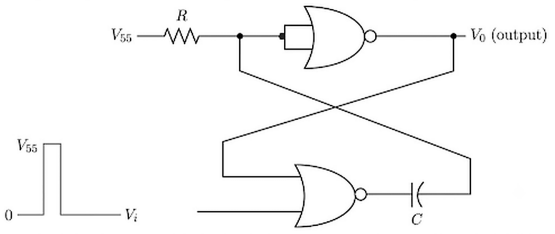


Fig. Solution Q5.02.4

The given logic circuit consists of two cross-coupled CMOS NOR gates with an RC feedback network connected to one of the gates.

- **Gate G_1:** Both inputs are shorted together, so it functions as a **NOT gate (Inverter).**
- **Gate G_2:** Functions as a 2-input **NOR gate**, where one input receives the external trigger pulse V_i and the other receives the feedback via capacitor C.

- **Stable State ($t < 0$):** With no external trigger applied ($V_i = 0\,\text{V}$), the circuit rests in a stable state where the output $V_{o1} = 0\,\text{V}$ (Logic 0) and $V_{o2} = V_{SS}$ (Logic 1). The capacitor voltage is initially relaxed at $V_C(0^-) = 0\,\text{V}$.
- **Quasi-Stable State ($t = 0^+$):** When a high trigger pulse ($V_i = V_{SS}$) is applied to G_2, its output V_{o2} transitions from $1 \rightarrow 0$. Due to the continuity of voltage across the capacitor, the input to G_1 drops, causing V_{o1} to switch from $0 \rightarrow 1$.
- **Recovery:** The circuit remains in this quasi-stable state temporarily while the capacitor C charges through the resistor R. Once the voltage V_p reaches the logic threshold voltage (V_{TL}), gate G_1 switches back, resetting the circuit to its original stable state.

Since the circuit has **one stable state** and **one quasi-stable** state triggered by an external pulse, it functions as a **monostable multivibrator**.

(C) Monostable multivibrator

Key Points

- A cross-coupled latch configuration incorporating a single reactive element (RC coupling) results in a **monostable multivibrator**.
- Astable multivibrators require two reactive (RC) couplings to eliminate all stable states, whereas a standard flip-flop (bistable) has no reactive timing elements in its feedback path.

Q5.01.1 MCQ 2001 2M Ans: C ●●○

Given that the flip-flops are positive edge-triggered, only options (A) and (C) are possible, as options (B) and (D) show negative edge-triggered clocks (indicated by the bubble at the clock input).
Initially, at $t < t_0$, $Q_1 = Q_2 = 0$.

Method 1

Direct Elimination Approach:

- Options (B) and (D) are eliminated as it is given that FF are positive edge-triggered but there is a bubble at the clock terminal.
- For option (A), since $D_1 = 1$ and $D_2 = 1$, once both flip-flops toggle high at the first rising edge t_0, they lock into the high state ($Q_1 = 1, Q_2 = 1$) permanently for all future edges. This contradicts the given pulsated waveform of Y. Thus, only option (C) remains valid.

Method 2

Waveform and State-Table Verification:
For Option (A): Both flip-flops have their data inputs hardwired to high ($D_1 = 1$, $D_2 = 1$). At the first positive edge t_0, Q_1 goes high (1) and Q_2 goes high (1). For subsequent clock edges (t_1, t_2, t_3), they continue to latch 1. The resulting waveform behavior is:

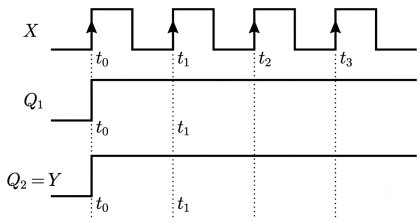


Fig. Solution Q5.01.1

Since output Y does not fall back down at t_1, Option (A) is incorrect.

For Option (C): The given circuit acts as a synchronous counter circuit where $D_1 = 1$ and $D_2 = \overline{Q}_1$. Initially $Q_2 = Q_1 = 0$. Tracking the state evolution at successive rising edges:

CLK	Q_2	Q_1	$D_2 = \overline{Q}_1$	$D_1 = 1$
1	0	0	1	1
2	1	1	0	1
3	0	1	0	1
4	0	1		

The output $Y = Q_2$ stays high exclusively between interval t_0 and t_1, shifting down to 0 from edge t_1 onwards.

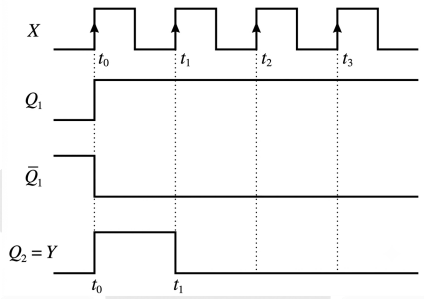


Fig. Solution Q5.01.1

This perfectly matches the objective reference diagram.

(C)

Key Points

- A bubble at the clock input denotes a negative edge-triggered flip-flop, while a simple triangle denotes a positive edge-triggered flip-flop.
- For a D-flip-flop, the next-state equation is $Q_{n+1} = D$.
- In synchronous sequential circuits, changes happen simultaneously at the triggering edge based on the present states.

Q5.01.2 MCQ 2001 1M Ans: C ● ○ ○

Let's analyze both statements given in the question:

- **Statement 1:** An astable multivibrator has no stable states. It continuously switches back and forth between two quasi-stable states on its own. Therefore, it acts as a free-running oscillator and is widely used for generating periodic square waves. Hence, Statement 1 is correct.
- **Statement 2:** A bistable multivibrator has two distinct stable states. It requires an external trigger to switch from one stable state to the other and will remain in that state indefinitely until another trigger is applied. Because it can hold one of two states (0 or 1) permanently without external timing elements, it is used as a basic memory element (like a flip-flop or latch) to store binary information. Hence, Statement 2 is correct.

Since both statements 1 and 2 are correct, the right choice is option (C).

(C) Both the statements 1 and 2 are correct

Key Points

- **Astable Multivibrator:** Zero stable states; functions as a square-wave generator or oscillator.
- **Monostable Multivibrator:** One stable state; functions as a delay element or pulse stretcher.
- **Bistable Multivibrator:** Two stable states; functions as a binary storage cell (latch/flip-flop).

Q5.01.3 MCQ 2001 1M Ans: C ● ○ ○

The given circuit is a 5-stage ring oscillator.

- Number of inverter stages, $N = 5$
- Propagation delay of each inverter, $t_{pd} = 100\,\text{ps} = 100 \times 10^{-12}\,\text{s}$

The time period T of the fundamental frequency of a ring oscillator is given by the formula:

$$T = 2 \cdot N \cdot t_{pd}$$

Substituting the given values:

$$T = 2 \times 5 \times 100\,\text{ps} = 1000\,\text{ps} = 1\,\text{ns}$$

The fundamental oscillation frequency f is the reciprocal of the time period:

$$f = \frac{1}{T} = \frac{1}{1\,\text{ns}} = 1\,\text{GHz}$$

(C) 1 GHz

Key Points

- A ring oscillator must have an odd number of inverting stages (N) to oscillate freely.
- The factor of 2 in the time period equation accounts for a complete cycle consisting of one low-to-high transition propagation and one high-to-low transition propagation through the entire ring chain.

Q5.00.1 MCQ 2000 2M Ans: B ● ● ○

Given circuit is a negative edge-triggered asynchronous counter. Since the clock of each subsequent flip-flop is connected to the Q output of the previous flip-flop, it operates as an **up counter**.
The clear input ($\overline{\text{CLR}}$) is controlled by a NAND gate whose inputs are connected to Q_3 and Q_1. Thus:

$$\overline{\text{CLR}} = \overline{Q_3 Q_1}$$

The counter resets to 0000 immediately when $Q_3 = 1$ and $Q_1 = 1$ (i.e., when the state reaches $1010_2 = 10_{10}$).
Therefore, the valid counting states are from 0000 to 1001 (0 to 9 in decimal), making it a **MOD-10 counter**.
The frequency of the output signal Y (which is Q_3) is given by:

$$f_{out} = \frac{f_{\text{CLK}}}{\text{Modulus of the counter}}$$

$$f_{out} = \frac{10\,\text{kHz}}{10} = 1.0\,\text{kHz}$$

(B) 1.0 kHz

Key Points

- For an asynchronous up-counter using negative edge-triggered flip-flops, the clock input of the next stage is driven by the Q output of the preceding stage.
- The modulus of a counter with a feedback clearing circuit is equal to the decimal equivalent of the state that triggers the clear condition.

Mistakes to be avoided

- Do not confuse the transient state (1010) as a stable counting state. The counter only stays in this state for a negligible propagation delay before resetting.

Q5.00.2 MCQ 2000 2M Ans: D ●●○

From the given combinational logic circuit feeding into the D flip-flop, the boolean expression for the input D can be derived from the logic gates:

- The top AND gate has inputs $\overline{X}$ and Z. Its output is $\overline{X}\,Z$.
- The bottom AND gate has inputs Y and $\overline{Z}$. Its output is $Y\,\overline{Z}$.
- The OR gate combines these two outputs:

$$D = \overline{X}\,Z + Y\,\overline{Z}$$

For a D flip-flop, the next-state characteristic equation is:

$$Z_{n+1} = D$$

Substituting the expression for D:

$$Z_{n+1} = Y\,\overline{Z_n} + \overline{X}\,Z_n$$

The standard characteristic equation of a J-K flip-flop with next state Q_{n+1} and current state Q_n is given by:

$$Q_{n+1} = J\,\overline{Q_n} + \overline{K}\,Q_n$$

By mapping Z to Q, we compare the two equations:

- Comparing the coefficients of $\overline{Z_n}$ (or $\overline{Q_n}$): $J = Y \implies Y = J$
- Comparing the coefficients of Z_n (or Q_n): $\overline{K} = \overline{X} \implies K = X \implies X = K$

Thus, the circuit acts as a J-K flip-flop with inputs $X = K$ and $Y = J$.

(D) $J - K$ Flip-Flop with inputs $X = K$ and $Y = J$

Key Points

- D Flip-Flop Characteristic Equation: $Q_{n+1} = D$
- J-K Flip-Flop Characteristic Equation: $Q_{n+1} = J\overline{Q}_n + \overline{K}Q_n$
- Any flip-flop can be converted into another by finding the required combinational logic expression for its excitation inputs.

Mistakes to be avoided

- Be careful with the inversion bubble at the input of the top AND gate. It is attached to X, resulting in $\overline{X}$, not $\overline{Z}$.
- When comparing with the standard J-K equation, remember that the coefficient of Q_n is $\overline{K}$, so $\overline{X} = \overline{K}$ leads to $X = K$.

Q5.99.1 NAT 1999 Ans: * ●●○

From the given synchronous counter circuit diagram, the input excitation expressions for each J-K flip-flop are:

$$J_A = \overline{Q}_B, \quad K_A = 1$$

$$J_B = Q_A, \quad K_B = 1$$

$$J_C = K_C = \overline{Q}_A Q_B$$

(The feedback line to J_A is taken from the $\overline{Q}$ side of flip-flop B. With $J_A = Q_B$ instead, the counter could never leave the initial state 000.)

The characteristic next-state equation for a J-K flip-flop is:

$$Q_{n+1} = J\overline{Q}_n + \overline{K}Q_n$$

Substituting the excitation expressions, the next-state equations become:

$$Q_{A(n+1)} = \overline{Q}_{B(n)}\,\overline{Q}_{A(n)}$$

$$Q_{B(n+1)} = Q_{A(n)}\,\overline{Q}_{B(n)}$$

$$Q_{C(n+1)} = Q_{C(n)} \oplus \overline{Q}_{A(n)} Q_{B(n)}$$

Starting from the given initial state $Q_{A(n)}Q_{B(n)}Q_{C(n)} = 000$, the state sequence is tabulated below:

Present State			Flip-Flop Inputs						Next State		
$Q_{A(n)}$	$Q_{B(n)}$	$Q_{C(n)}$	J_A	K_A	J_B	K_B	J_C	K_C	$Q_{A(n+1)}$	$Q_{B(n+1)}$	$Q_{C(n+1)}$
0	0	0	1	1	0	1	0	0	1	0	0
1	0	0	1	1	1	1	0	0	0	1	0
0	1	0	0	1	0	1	1	1	0	0	1
0	0	1	1	1	0	1	0	0	1	0	1
1	0	1	1	1	1	1	0	0	0	1	1
0	1	1	0	1	0	1	1	1	0	0	0

The counter goes through six distinct states before returning to 000:

$$000 \rightarrow 100 \rightarrow 010 \rightarrow 001 \rightarrow 101 \rightarrow 011 \rightarrow 000$$

so it is a mod-6 counter.

Mod-6 counter

Key Points

- When $K = 1$, the next-state equation reduces to $Q_{n+1} = J\overline{Q}_n$.
- When $J = K$, the flip-flop toggles if $J = K = 1$ and holds if $J = K = 0$, so $Q_{n+1} = Q_n \oplus J$.

Q5.99.2 NAT 1999 Ans: 6 ●●○

With $J_A = \overline{Q}_B$, $K_A = K_B = 1$, $J_B = Q_A$, and $J_C = K_C = \overline{Q}_A Q_B$, the state sequence in the order (Q_A, Q_B, Q_C) is

$$000 \rightarrow 100 \rightarrow 010 \rightarrow 001 \rightarrow 101 \rightarrow 011 \rightarrow 000.$$

There are six distinct states in the repeating cycle. Hence the counter has modulus

$$\boxed{6}.$$

Key Points

The modulus is the number of distinct states in the repeating cycle.

Q5.99.3 MCQ 1999 2M Ans: D ●●○

To determine the type and modulus of the given asynchronous counter circuit, we analyze its clocking configuration and feedback logic.

1. Determining Counter Direction (Up or Down)

An asynchronous (ripple) counter's direction depends on the clock triggering mechanism (edge sensitivity) and the source connection from the preceding stage:

- The clock inputs of the JK flip-flops lack an inversion bubble, meaning they are **positive edge-triggered**.
- The true output Q of each stage is connected directly to the clock input of the subsequent stage ($Q_A \rightarrow \text{CLK}_B$ and $Q_B \rightarrow \text{CLK}_C$).

When positive edge-triggered flip-flops use the true output Q as the clock signal for the next stage, the counter functions as a **Down Counter**.

2. Analyzing the Feedback Loop and State Sequence

The **Preset** inputs are drawn without bubbles, so they are active-high, and they are driven by the output of an AND gate (no inversion bubble at its output). Let the outputs of the counter stages be denoted as (C, B, A), where C is the Most Significant Bit (MSB) and A is the Least Significant Bit (LSB).
The inputs to the 3-input AND gate are:

- From the complementary output of stage C: $\overline{C}$
- From the true output of stage B: B
- From the complementary output of stage A: $\overline{A}$

Therefore, the output of the AND gate is given by the boolean expression:

$$\text{Gate Output} = \overline{C} \cdot B \cdot \overline{A}$$

The Preset inputs activate when the gate output becomes high (1), which occurs at the specific state:

$$(C, B, A) = (0, 1, 0)_2 = 2_{10}$$

Since a down counter counts backward, let's track the state transitions starting from the maximum state $(1, 1, 1)_2$:

Clock Pulse	C	B	A	Remarks
Initial State	1	1	1	State 7_{10}
1	1	1	0	State 6_{10}
2	1	0	1	State 5_{10}
3	1	0	0	State 4_{10}
4	0	1	1	State 3_{10}
5	0	1	0	Transient State $2_{10} \implies$ Triggers Preset
Immediate Reset	**1**	**1**	**1**	Instantly forced back to 7_{10}

The state sequence contains exactly 5 stable states: $7 \to 6 \to 5 \to 4 \to 3 \to 7 \ldots$ Hence, it is a **MOD-5 down counter**.

(D) mod - 5 down counter

Key Points

Asynchronous Counter Direction Rules:

1. **Up Counter:** Negative edge-triggered with Q as clock, OR Positive edge-triggered with $\overline{Q}$ as clock.
2. **Down Counter:** Positive edge-triggered with Q as clock, OR Negative edge-triggered with $\overline{Q}$ as clock.

Q5.98.1 NAT 1998 Ans: * ●●○

From the given logic diagram, the inputs to the JK flip-flops are:

- Flip-Flop A (Q_0): $J_0 = \overline{Q_2}$, $K_0 = 1$
- Flip-Flop B (Q_1): $J_1 = 1$, $K_1 = 1$
- Flip-Flop C (Q_2): $J_2 = Q_0Q_1$, $K_2 = Q_0$

The characteristic equation for a JK flip-flop is $Q_{n+1} = J\overline{Q_n} + \overline{K}Q_n$. Substituting the inputs, we get the next-state equations:

$$Q_{0(next)} = \overline{Q_2}\,\overline{Q_0}$$

$$Q_{1(next)} = \overline{Q_1}$$

$$Q_{2(next)} = (Q_0Q_1)\overline{Q_2} + \overline{Q_0}Q_2$$

The valid states of the mod-5 counter are $000, 001, 010, 011$, and 100. The unused states are $101, 110$, and 111. Let us analyze the transition from these unused states ($Q_2Q_1Q_0$):

1. **For state** 101**:**

$$Q_{0(next)} = \overline{1}\,\overline{1} = 0$$

$$Q_{1(next)} = \overline{0} = 1$$

$$Q_{2(next)} = (1 \cdot 0)\overline{1} + \overline{1} \cdot 1 = 0$$

Next state is 010 (a valid state).

2. **For state** 110**:**

$$Q_{0(next)} = \overline{1}\,\overline{0} = 0$$

$$Q_{1(next)} = \overline{1} = 0$$

$$Q_{2(next)} = (0 \cdot 1)\overline{1} + \overline{0} \cdot 1 = 1$$

Next state is 100 (a valid state).

3. **For state** 111**:**

$$Q_{0(next)} = \overline{1}\,\overline{1} = 0$$

$$Q_{1(next)} = \overline{1} = 0$$

$$Q_{2(next)} = (1 \cdot 1)\overline{1} + \overline{1} \cdot 1 = 0$$

Next state is 000 (a valid state).

Since all unused states transition into the valid counting sequence, the counter will not enter lockout.

No

Key Points

- Lockout occurs when a digital counter enters an unused state and cycles strictly among unused states without returning to the main valid sequence.
- Checking next-state equations for all unused states determines if a counter is self-starting.

Q5.98.2 NAT 1998 Ans: * ●●○

For satisfactory synchronous operation, the total delay from the clock edge to the settling of the next state's input must be less than or equal to the clock period (T).
The maximum propagation delay path determines the minimum clock period:

- Flip-flop C receives its J_2 input through an AND gate whose inputs come from flip-flops A and B.
- The total delay to stabilize the input at J_2 after a clock edge is the flip-flop propagation delay (t_F) plus the AND gate propagation delay (t_A).

Therefore, the minimum clock period T_{min} is:

$$T_{min} = t_F + t_A$$

The maximum operational frequency (rate) is the reciprocal of T_{min}:

$$f_{max} = \frac{1}{t_F + t_A}$$

$$\boxed{\frac{1}{t_F + t_A}}$$

Key Points

- In a synchronous circuit, the minimum clock period is constrained by the worst-case propagation delay loop: $T \geq t_{pd,\text{FF}} + t_{pd,\text{combinational}} + t_{\text{setup}}$.
- Neglecting setup time, $T_{min} = t_F + t_A$.

Q5.98.3 MCQ 1998 2M Ans: C ●●○

Let the first flip-flop be FF0 with output Q_0 and the second flip-flop be FF1 with output Q_1. From the given logic diagram, the input expressions for the two JK flip-flops are:

- $J_0 = \overline{Q_1}$
- $K_0 = 1$
- $J_1 = Q_0$

- $K_1 = 1$

The characteristic equation of a JK flip-flop is given by:

$$Q_{n+1} = J\overline{Q_n} + \overline{K}Q_n$$

Substituting the values of J and K for each flip-flop, we get the next-state equations:

$$Q_{0(\text{next})} = \overline{Q_1}\,\overline{Q_0}$$

$$Q_{1(\text{next})} = Q_0\overline{Q_1}$$

The state transition behavior can be summarized in the following truth table:

Clock Pulse	Present State		Flip-Flop Inputs		Next State	
	Q_1	Q_0	$J_1 = Q_0$	$J_0 = \overline{Q_1}$	$Q_{1(\text{next})}$	$Q_{0(\text{next})}$
Initial	0	0	0	1	0	1
1	0	1	1	1	1	0
2	1	0	0	0	0	0
3	0	0	*Cycle Repeats*			

The sequence of states is $00 \rightarrow 01 \rightarrow 10 \rightarrow 00$. Since there are 3 distinct states in the counting cycle, the counter is a MOD-3 counter. Therefore, $K = 3$.

(C) 3

Key Points

- The modulus (MOD) of a counter represents the total number of unique states it passes through before repeating its counting cycle.
- For any synchronous sequential circuit, the state sequence is determined by evaluating the next-state equations for each clock transition.

Q5.98.4 MCQ 1998 2M Ans: A

The given circuit is a cross-coupled NAND latch. The output expressions for the gates are:

$$X = \overline{A \cdot Y}$$

$$Y = \overline{B \cdot X}$$

We are given that initially $A = 1$ and $B = 1$. Let us analyze the transition sequence when the input B is replaced by the periodic sequence $101010\ldots$:

1. **Initial Condition** $(A = 1, B = 1)$**:** A cross-coupled NAND latch with inputs 1, 1 is in its **Hold state**. Let us assume the stable operating outputs are settled at some valid complementary state, for instance, $X = 0$ and $Y = 1$.
2. **First bit of the sequence** $(B = 1)$**:** Since B remains 1, the latch stays in the hold state.

$$X = 0, \quad Y = 1$$

3. **Second bit of the sequence** $(B = 0)$**:** When B transitions to 0 while $A = 1$:

$$Y = \overline{0 \cdot X} = 1$$

Substituting $Y = 1$ into the expression for X:

$$X = \overline{1 \cdot 1} = 0$$

Thus, the outputs are explicitly forced to $X = 0$ and $Y = 1$.

4. **Third bit of the sequence ($B = 1$):** When B changes back to 1 while $A = 1$, the circuit enters the **Hold state** again. It retains its immediate previous state values:

$$X = 0, \quad Y = 1$$

Since every $B = 0$ actively forces $X = 0, Y = 1$, and every $B = 1$ maintains the existing state ($X = 0, Y = 1$), the outputs do not fluctuate with the alternating sequence. Instead, they remain constant.
Therefore, the outputs x and y will be fixed at 0 and 1, respectively.

(A) fixed at 0 and 1, respectively

Key Points

- For an active-low SR latch (NAND implementation), the input condition $S = 1, R = 1$ acts as the hold state, meaning it preserves the previous state.
- When an input is pulled to 0, it forces the output of that specific NAND gate to 1, overriding any feedback history.

Q5.97.1 NAT 1997 Ans: * ●●○

From the given synchronous sequential circuit, the input expressions for the three D flip-flops are obtained as:

- $D_0 = \overline{Q_2}$
- $D_1 = \overline{Q_0}$
- $D_2 = Q_0\overline{Q_1}$

Since for a D flip-flop, the next state is equal to the input ($Q_{\text{next}} = D$), the next-state equations are:

$$Q_{0(\text{next})} = \overline{Q_2}$$

$$Q_{1(\text{next})} = \overline{Q_0}$$

$$Q_{2(\text{next})} = Q_0\overline{Q_1}$$

The counter status is initialized to $(Q_0, Q_1, Q_2) = (0, 1, 0)$. Let us track the state transitions at each clock cycle:

Clock Pulse	Present State			Next State		
	Q_2	Q_1	Q_0	$Q_{2(\text{next})}$	$Q_{1(\text{next})}$	$Q_{0(\text{next})}$
Initial	0	1	0	0	1	1
1	0	1	1	0	0	1
2	0	0	1	1	0	1
3	1	0	1	1	0	0
4	1	0	0	0	1	0
5	0	1	0	*Cycle Repeats*		

Extracting the values of Q_0 from the initial state until just before it repeats gives the sequence:

0, 1, 1, 1, 0

Key Points

- For D flip-flops, the next state simply mirrors the present value available at the D input terminal at the triggering edge of the clock signal.

Q5.97.2 NAT 1997 Ans: 4 ●●○

From the circuit, the flip-flop inputs are

$$D_0 = \overline{Q}_2, \qquad D_1 = \overline{Q}_0, \qquad D_2 = Q_0\,\overline{Q}_1$$

Starting from the initial state $(Q_0, Q_1, Q_2) = (0, 1, 0)$, the counter goes through 5 distinct states before returning to its initial condition:

$$(0,1,0) \rightarrow (1,1,0) \rightarrow (1,0,0) \rightarrow (1,0,1) \rightarrow (0,0,1) \rightarrow (0,1,0)$$

Thus, the modulus of the sequence generator (the number of clock cycles per repetition period) is:

$$N = 5$$

Given that the clock period is $T_{\text{clk}} = 50\,\text{ns}$, the total time period (T) required for one complete cycle of the sequence is:

$$T = N \times T_{\text{clk}} = 5 \times 50\,\text{ns} = 250\,\text{ns}$$

The repetition rate (f) of the generated sequence is the reciprocal of the total time period:

$$f = \frac{1}{T} = \frac{1}{250 \times 10^{-9}\,\text{s}} = 4 \times 10^6\,\text{Hz} = 4\,\text{MHz}$$

$$\boxed{4\,\text{MHz}}$$

Key Points

- The total time period of a sequence generator cycle is given by $T = N \cdot T_{\text{clk}}$, where N is the number of distinct states in the loop.
- Frequency or repetition rate is calculated using $f = \frac{1}{T}$.

Q5.97.3 MCQ 1997 2M Ans: D ●○○

The input expressions for the given JK flip-flop are:

- $J = \overline{Q}$
- $K = 1$

The next-state characteristic equation for a JK flip-flop is defined as:

$$Q_{\text{next}} = J\overline{Q} + \overline{K}Q$$

Substituting the given input values into the characteristic equation:

$$Q_{\text{next}} = (\overline{Q})\overline{Q} + (\overline{1})Q = \overline{Q}$$

We are given that the flip-flop is initially cleared ($Q = 0$). The behavior over the next 6 clock pulses can be tracked sequentially in the truth table below:

Clock Pulse	Present State (Q)	$J = \overline{Q}$	$K = 1$	Next State (Q_{next})
Initial	0	1	1	1
1	1	0	1	0
2	0	1	1	1
3	1	0	1	0
4	0	1	1	1
5	1	0	1	0
6	0	1	1	1

Collecting the outputs observed after each of the 6 clock pulses sequentially, the output sequence at Q is 101010.

(D) 010101

Key Points

- When $J = \overline{Q}$ and $K = 1$, the operation behaves exactly like a T flip-flop configured with $T = 1$, which toggles its state on every active clock edge.
- Sequential clocking of a toggling flip-flop results in an alternating periodic square wave waveform.

Q5.96.1 MCQ 1996 1M Ans: A ● ○ ○

The operational characteristics of all four shift register types are outlined below:

- **Serial-In Serial-Out (SISO):** Data is loaded one bit at a time and retrieved sequentially. It introduces a controllable time delay equal to N clock cycles.
- **Serial-In Parallel-Out (SIPO):** Data is loaded serially, but all stored bits are accessible simultaneously on separate output lines. It is primarily used for serial-to-parallel conversion.
- **Parallel-In Serial-Out (PISO):** All stages are loaded simultaneously with data bits in a single clock cycle, and the data is then shifted out sequentially bit by bit. It is used for parallel-to-serial conversion.
- **Parallel-In Parallel-Out (PIPO):** Data is loaded into all stages simultaneously and is immediately available at the parallel outputs. It provides temporary data storage without introducing delay or format conversion.

A Serial-In Serial-Out (SISO) shift register accepts data serially and shifts it through its stages with each clock pulse. For an N-bit shift register, data entered at the input will appear at the output after exactly N clock periods. Therefore, it acts as a digital delay line to delay a pulse train by a finite number of clock cycles.

(A) a serial-in serial-out shift register

Key Points

- **SISO Shift Register:** Primarily used as a delay line or temporary data storage.
- **Delay Duration:** Total delay is given by $T_d = N \cdot T_{clk}$, where N is the number of flip-flops (stages) and T_{clk} is the clock period.

Q5.95.1 MCQ 1995 1M Ans: D ● ○ ○

A switch-tail ring counter (also known as a Johnson counter) is formed by feeding the complemented output of the last flip-flop back into the input of the first flip-flop.
When a single D flip-flop is used, its complemented output $\overline{Q}$ is fed directly back into its own D input:

$$D = \overline{Q}$$

The characteristic equation of a D flip-flop is given by:

$$Q_{next} = D$$

Substituting $D = \overline{Q}$ into the characteristic equation:

$$Q_{next} = \overline{Q}$$

This matches the behavior of a T flip-flop with its toggle input held high ($T = 1$), where the output toggles on every active clock edge:

$$Q_{next} = T \oplus Q = 1 \oplus Q = \overline{Q}$$

Thus, the resulting circuit behaves exactly as a T flip-flop.

(D) T flip-flop

Key Points

- **1-Bit Johnson Counter:** Formed by setting $D = \overline{Q}$, which yields a modulus-2 counter (MOD-2).
- **Frequency Division:** The output frequency is exactly half of the clock frequency ($f_{out} = \frac{f_{clk}}{2}$) with a 50% duty cycle.

Q5.94.1 NAT 1994 1M Ans: * ● ○ ○

A pulse can be stretched (widened) to a predetermined duration using a monostable multivibrator circuit (also known as a one-shot multivibrator).
When triggered by an input pulse of short duration (2 μs), the monostable multivibrator enters its quasi-stable state and remains there for a time period determined by its external resistor (R) and capacitor (C) components. After this time interval (10 ms), it automatically returns to its stable state, effectively stretching the input pulse.

monostable multivibrator

Key Points

- **Monostable Multivibrator:** Has one stable state and one quasi-stable state. It functions as a pulse stretcher or time-delay circuit.
- **Output Pulse Width (T):** Determined by the time constant of the external RC network (e.g., $T \approx 1.1RC$ for a 555 timer in monostable mode).

Q5.94.2 NAT 1994 1M Ans: * ● ○ ○

In synchronous counters, all flip-flops are connected to a common clock signal and trigger simultaneously. Consequently, the propagation delay is equal to the delay of only a single flip-flop plus any combinational logic gating delay.
In contrast, ripple (asynchronous) counters pass the clock signal through the flip-flops sequentially in a chain, causing the individual propagation delays to accumulate ($N \cdot t_{pd}$). Therefore, synchronous counters operate at much higher speeds.

faster

Key Points

- **Synchronous Counters:** Clocked simultaneously; propagation delay does not increase significantly with the number of bits.
- **Ripple Counters:** Clocked sequentially; cumulative delay limits the maximum operating frequency.

Q5.94.3 NAT 1994 1M Ans: 100 ● ○ ○

The oscillation frequency f of a ring oscillator consisting of an odd number of inverters n is given by the formula:

$$f = \frac{1}{2 \cdot n \cdot t_{pd}}$$

Where:

- $n = 5$ is the number of inverters.
- $f = 1.0\,\text{MHz} = 1.0 \times 10^6\,\text{Hz}$ is the frequency of oscillation.
- t_{pd} is the propagation delay per gate.

Rearranging the formula to solve for t_{pd}:

$$t_{pd} = \frac{1}{2 \cdot n \cdot f}$$

Substituting the given values:

$$t_{pd} = \frac{1}{2 \cdot 5 \cdot (1.0 \times 10^6\,\text{Hz})}$$

$$t_{pd} = \frac{1}{10^7}\,\text{s} = 10^{-7}\,\text{s}$$

Converting the time delay into nanoseconds (ns):

$$t_{pd} = 10^{-7} \times 10^9\,\text{ns} = 100\,\text{ns}$$

100

Key Points

- **Ring Oscillator Frequency:** Calculated using $f = \frac{1}{2n \cdot t_{pd}}$ because a complete wave cycle requires the signal to traverse the entire ring twice (once for the rising edge and once for the falling edge).
- **Condition for Oscillation:** The number of inverters n must be an odd integer to maintain unstable feedback and sustain continuous oscillation.

Q5.94.4 MCQ 1994 1M Ans: B ● ○ ○

- **Spatial Code (Parallel Data):** Data where all bits are present simultaneously over multiple parallel lines.
- **Temporal Code (Serial Data):** Data where bits are arranged sequentially and transmitted one after another over a single line over time.
- **Shift Registers** are sequential logic circuits primarily used for parallel-to-serial conversion (spatial to temporal) and serial-to-parallel conversion (temporal to spatial).

(B) shift-registers

Key Points

- PISO (Parallel-In Serial-Out) shift registers convert spatial code to temporal code.
- SIPO (Serial-In Parallel-Out) shift registers convert temporal code to spatial code.

Q5.92.1 MCQ 1992 1M Ans: C ● ● ○

Let the bits of the 4-bit shift register be represented from left to right as $Q_3Q_2Q_1Q_0$.
From the given figure, the feedback to the serial input is the Exclusive-OR (XOR) combination of the last two stages (Q_1 and Q_0):

$$\text{Serial In} = Q_1 \oplus Q_0$$

The register performs a right-shift operation on each clock pulse. The state transitions are tracked step-by-step in the table below:

Clock Pulse	Serial In ($Q_1 \oplus Q_0$)	Register Content ($Q_3Q_2Q_1Q_0$)
Initial State	$1 \oplus 0 = 1$	0110
After 1st Pulse	$1 \oplus 1 = 0$	1011
After 2nd Pulse	$0 \oplus 1 = 1$	0101
After 3rd Pulse	—	1010

After three clock pulses, the content of the shift register is 1010.

(C) 1010

Key Points

- **Linear Feedback Shift Register (LFSR):** A shift register whose input bit is a linear function (usually XOR) of its previous states.
- **Right Shift Operation:** On each clock edge, the bits transition as: $Q_{3(\text{next})}$ = Serial In, $Q_{2(\text{next})} = Q_3$, $Q_{1(\text{next})} = Q_2$, and $Q_{0(\text{next})} = Q_1$.

Q5.91.1 NAT 1991 1M Ans: *

To convert an SR flip-flop into a T flip-flop, we derive the required inputs S and R in terms of the target input T and current state Q.
First, consider the characteristic table of the target T flip-flop:

T	Q	Q_{next}
0	0	0
0	1	1
1	0	1
1	1	0

Next, we map these state transitions onto the excitation table of the SR flip-flop to determine the required inputs:

T	Q	Q_{next}	S	R
0	0	0	0	X
0	1	1	X	0
1	0	1	1	0
1	1	0	0	1

From the conversion table, we can solve for S and Rusing kmaps:

- For S:

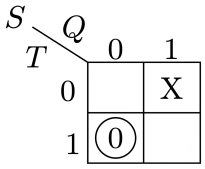


Fig. Solution Q5.91.1

Thus, $S = T\overline{Q}$.

- For R:

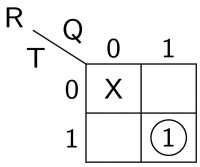


Fig. Solution Q5.91.1

Thus, $R = TQ$.

By substituting these expressions into the question's format:

$$S = T\overline{Q} \implies \text{Connecting } R \text{ to } Q \text{ (since } R = TQ)$$

$$R = TQ \implies \text{Connecting } S \text{ to } \overline{Q} \text{ (since } S = T\overline{Q})$$

Therefore, R is connected to Q through an AND gate with T, and S is connected to $\overline{Q}$ through an AND gate with T.

$\boxed{R \text{ and } S}$

Key Points

- **Flip-Flop Conversion:** Done by combining the characteristic table of the destination flip-flop with the excitation table of the available flip-flop.
- **SR to T Logic:** $S = T\overline{Q}$ and $R = TQ$.

Q5.90.1 MCQ 1990 1M Ans: C ●○○

In an n-bit asynchronous (ripple) counter, the clock signal ripples through the flip-flops sequentially. For proper operation, the total cumulative propagation delay of all flip-flops must be less than or equal to the clock period (T_{clk}).
The minimum clock period required is expressed as:

$$T_{clk} \geq n \cdot t_{pd}$$

Where:

- $n = 4$ is the number of bits (flip-flops).
- $t_{pd} = 50\,\text{ns}$ is the propagation delay of each flip-flop.

Calculating the minimum clock period:

$$T_{clk} \geq 4 \times 50\,\text{ns} = 200\,\text{ns}$$

The maximum clock frequency (f_{max}) is the reciprocal of the minimum clock period:

$$f_{max} = \frac{1}{T_{clk}} = \frac{1}{200 \times 10^{-9}\,\text{s}} = 5 \times 10^{6}\,\text{Hz} = 5\,\text{MHz}$$

$\boxed{\text{(C) 5 MHz}}$

Key Points

- **Ripple Counter Speed Constraint:** The maximum operating frequency decreases as the number of stages (n) increases due to cumulative flip-flop delay: $f_{max} = \frac{1}{n \cdot t_{pd}}$.
- **Synchronous Comparison:** Synchronous counters avoid this limitation since all stages are clocked in parallel, allowing for much higher frequencies independent of n.

Q5.88.1 NAT 1988 8M Ans: * ●●○

For a ripple counter utilizing negative edge-triggered flip-flops, the cascading connection between successive stages determines the direction of counting:

- **Up-Counting Mode ($M = 1$):** To count up with negative edge-triggered flip-flops, the clock input of the next stage must be triggered by the true output (Q) of the preceding stage. Thus, $F = Q$ when $M = 1$.
- **Down-Counting Mode ($M = 0$):** To count down with negative edge-triggered flip-flops, the clock input of the next stage must be triggered by the complemented output ($\overline{Q}$) of the preceding stage. Thus, $F = \overline{Q}$ when $M = 0$.

Combining both conditions using boolean logic, the expression for the intermediate function F is:

$$\boxed{F = MQ + \overline{M}\,\overline{Q} = M \odot Q}$$

Key Points

- **Asynchronous Counter Configuration:**
  - Negative Edge + $Q \implies$ UP Counter
  - Negative Edge + $\overline{Q} \implies$ DOWN Counter

Q5.88.2 NAT 1988 8M Ans: * ●●●

To determine the output voltage V_0 as a function of time, we first analyze the digital state sequence of the 3-bit synchronous counter configuration.

1. **Digital Circuit Analysis:** The connection setup for the three flip-flops (FF_0, FF_1, FF_2) is as follows:
   - For FF_0: $J_0 = \overline{Q}_2$ and $K_0 = Q_2$. This simplifies to a direct connection of $D_0 = \overline{Q}_2$.
   - For FF_1: $J_1 = Q_0$ and $K_1 = \overline{Q}_0$, which corresponds to $D_1 = Q_0$.
   - For FF_2: $J_2 = Q_1$ and $K_2 = \overline{Q}_1$, which corresponds to $D_2 = Q_1$.

   This is a 3-bit **Johnson Counter** (switch-tail ring counter). Since all flip-flops are initially reset to zero ($Q_2Q_1Q_0 = 000$), the sequence of states at each successive clock edge progresses as:

2. **Analog Operational Amplifier Analysis:** The output stage consists of an inverting summing amplifier configured with matching input and feedback resistors (R). Assuming a standard logic high level voltage of $V_H = V_{cc}$ and logic low level of $V_L = 0\,\text{V}$, the output voltage V_0 is given by:

$$V_0 = -\left(\frac{R}{R}Q_0 + \frac{R}{R}Q_1 + \frac{R}{R}Q_2\right) = -(Q_0 + Q_1 + Q_2) \cdot V_{cc}$$

   The circuit functions as a staircase waveform generator with a total period of $6 \cdot T_{clk}$.

Clock State	Q_2	Q_1	Q_0	Output Voltage V_0
Initial (0)	0	0	0	0 V
1	0	0	1	$-V_{cc} \cdot (0 + 0 + 1) = -V_{cc}$
2	0	1	1	$-V_{cc} \cdot (0 + 1 + 1) = -2V_{cc}$
3	1	1	1	$-V_{cc} \cdot (1 + 1 + 1) = -3V_{cc}$
4	1	1	0	$-V_{cc} \cdot (1 + 1 + 0) = -2V_{cc}$
5	1	0	0	$-V_{cc} \cdot (1 + 0 + 0) = -V_{cc}$
6 (Same as 0)	0	0	0	0 V

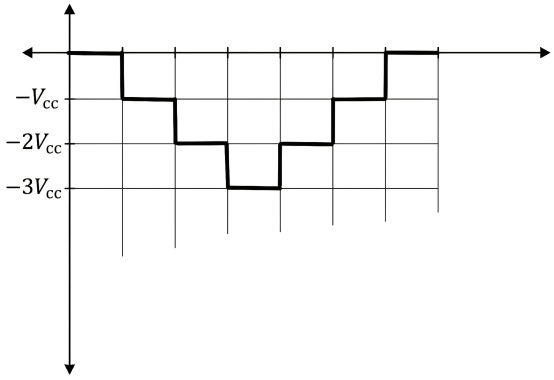


Fig. Solution Q5.88.2

Key Points

- **Johnson Counter Modulus:** A 3-bit switch-tail ring counter produces a sequence length of $2N = 6$ discrete states.
- **Staircase Generator:** Combining a Johnson counter with a summing amplifier acts as a digital-to-analog structure producing a symmetric triangular/staircase voltage waveform.

Q5.88.3 MCQ 1988 1M Ans: C ● ○ ○

The given circuit contains cross-coupled NAND gates at the output stage, which forms the fundamental storage core of an active-low set-reset architecture.

- **Input Stage:** The inputs A and B pass through separate single-input inverting gates to produce $\overline{A}$ and $\overline{B}$ (no inputs are shorted together in the circuit).
- **Cross-Coupled Stage:** These inverted signals are fed directly into a standard cross-coupled NAND gate structure.

When A maps to the Set control function (S) and B maps to the Reset control function (R), the cross-coupled NAND gates receive active-low signals $\overline{S}$ and $\overline{R}$. Hence the external inputs are active-high: $A = B = 0$ holds the previous state, $A = 1, B = 0$ sets Q, $A = 0, B = 1$ resets Q, and $A = B = 1$ is the forbidden combination. This is the behavior of an **R-S latch**.

(C) R-S latch

Key Points

- **Latch vs. Flip-Flop:** A latch is a level-triggered or asynchronous bistable storage element, whereas a flip-flop requires a clock edge transition indicator (which is absent in this circuit diagram).
- **NAND Latch Logic:** A cross-coupled NAND configuration maintains its previous data state when both intermediate inputs are held high (1).

Q5.87.1 NAT 1987 Ans: * ●●○

To design the feedback logic for the synchronous up/down counter, we determine the toggle inputs T_1 and T_0 using a state transition table.

1. **State Transition and Excitation Table:** The desired counting sequence behaves as an up-counter when $M = 1$ ($00 \rightarrow 01 \rightarrow 10 \rightarrow 11$) and a down-counter when $M = 0$ ($00 \rightarrow 11 \rightarrow 10 \rightarrow 01$).

   Using the characteristic behavior of a T flip-flop ($T = 1$ toggles state, $T = 0$ maintains state), we create the excitation table:

M	Q_1	Q_0	$Q_{1(\text{next})}$	$Q_{0(\text{next})}$	T_1	T_0
0	0	0	1	1	1	1
0	0	1	0	0	0	1
0	1	0	0	1	1	1
0	1	1	1	0	0	1
1	0	0	0	1	0	1
1	0	1	1	0	1	1
1	1	0	1	1	0	1
1	1	1	0	0	1	1

2. **Logic Expression Derivation:**
   - **For T_0:** The column is 1 for all combinations. Therefore, $T_0 = 1$.
   - **For T_1:** Let us simplify using a kmap:

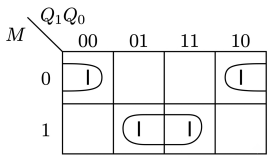


Fig. Solution Q5.87.1

$$T_1 = \overline{M}\,\overline{Q}_0 + MQ_0 = M \odot Q_0$$

$$\boxed{T_1 = M \odot Q_0,\ T_0 = 1}$$

Key Points

- **Synchronous Toggle Condition:** The LSB (Q_0) always toggles on every clock edge, so $T_0 = 1$.
- **Directional Control:** The next stage toggles based on the combination of the mode control M and the current state of Q_0.

Q5.87.2 NAT 1987 Ans: * ●●○

To realize the expressions $T_1 = \overline{M}\,\overline{Q}_0 + MQ_0$ and $T_0 = 1$ using 4-to-1 multiplexers with Q_1Q_0 as selection lines (Q_1 as MSB, Q_0 as LSB):

1. **Multiplexer Implementation for T_1:** We map the function $T_1(Q_1, Q_0)$ by evaluating its required input data lines (I_0, I_1, I_2, I_3) corresponding to the selection inputs Q_1Q_0:

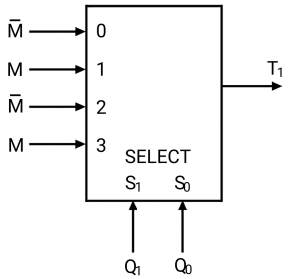


Fig. Solution Q5.87.2

2. **Multiplexer Implementation for T_0:** Since $T_0 = 1$ unconditionally, all four data inputs of its multiplexer are tied directly to logic high level:

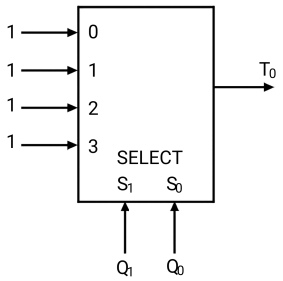


Fig. Solution Q5.87.2

Inputs for $T_1 : \{\overline{M}, M, \overline{M}, M\}$; Inputs for $T_0 : \{1, 1, 1, 1\}$

Key Points

- **4-to-1 MUX Logic:** Implementing a function via MUX mapping avoids routing overhead by directly assigning the remaining single literal (M or $\overline{M}$) to the data terminal inputs.

Q5.87.3 MSQ | 1987 | 1M | Ans: B,D

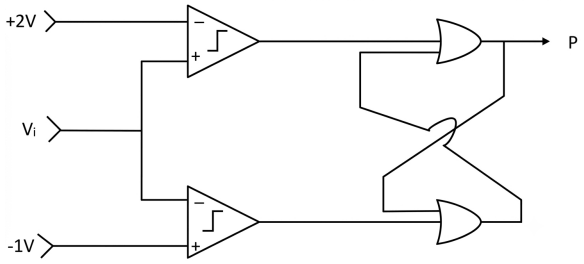


Fig. Solution Q5.87.3

The circuit consists of two voltage comparators driving a cross-coupled NOR latch. Let the output of the top comparator be S and the output of the bottom comparator be R.

- **Top Comparator:** The non-inverting terminal (+) is connected to V_i and the inverting terminal (−) is connected to +2 V.

$$S = \begin{cases} 1 & \text{if } V_i > +2\,\text{V} \\ 0 & \text{if } V_i < +2\,\text{V} \end{cases}$$

- **Bottom Comparator:** The inverting terminal (−) is connected to V_i and the non-inverting terminal (+) is connected to −1 V.

$$R = \begin{cases} 1 & \text{if } V_i < -1\,\text{V} \\ 0 & \text{if } V_i > -1\,\text{V} \end{cases}$$

Let us evaluate all the four options:

1. **Checking Option A:** For $V_i = +3$ V, we have $V_i > +2$ V $\implies S = 1$ and $V_i > -1$ V $\implies R = 0$. For a NOR latch, when $S = 1$ and $R = 0$, the latch resets, forcing $P = 0$ and $Q = 1$. Thus, statement (A) is incorrect.
2. **Checking Option B:** Following the same conditions as Option A ($V_i = +3$ V $\implies S = 1, R = 0$), the latch resets giving $P = 0$. Thus, statement (B) is correct.
3. **Checking Option C:** For $V_i = -2$ V, we have $V_i < +2$ V $\implies S = 0$ and $V_i < -1$ V $\implies R = 1$. When $S = 0$ and $R = 1$, the NOR latch enters the set state, forcing $P = 1$ and $Q = 0$. Therefore, P cannot be 0, making statement (C) incorrect.
4. **Checking Option D:** For $V_i = 0$ V, we have $V_i < +2$ V $\implies S = 0$ and $V_i > -1$ V $\implies R = 0$. When both inputs to the NOR latch are 0, the circuit enters the **Hold State** ($P_{next} = P$). Thus, P depends entirely on its previous state and can be either 0 or 1. Hence, statement (D) is correct.

(B) For $V_i = +3$ V, $P = 0$

(D) For $V_i = 0$ V, P can be either 0 or 1.

Key Points

- **NOR Latch Truth Table:**
  - $S = 0, R = 0 \implies$ Hold state (output maintains its previous state of 0 or 1).
  - $S = 1, R = 0 \implies$ Reset output $P = 0$.
  - $S = 0, R = 1 \implies$ Set output $P = 1$.

Q5.87.4 MCQ | 1987 | 1M | Ans: A | ● ○ ○

The problem requires designing feedback logic to truncate a 3-bit ripple counter so that it follows a Modulo-6 counting sequence:

$$000 \rightarrow 001 \rightarrow 010 \rightarrow 011 \rightarrow 100 \rightarrow 101 \rightarrow (000)$$

1. **Identification of the Reset State:** The counter cycles through 6 valid states (0 to 5 in decimal). The next sequential state that would normally occur is $6_{10} = 110_2$.

   Therefore, when the states reach $Q_2Q_1Q_0 = 110$, the counter must immediately clear all flip-flops back to 000.

2. **Active-Low Reset Analysis:** The problem statement specifies that the flip-flops are cleared to '0' by applying a logic low level ('0') at the active-low reset inputs (R). Thus, the output of the feedback logic F must satisfy:

   - $F = 0$ when $Q_2 = 1, Q_1 = 1, Q_0 = 0$ (the state 110_2).
   - $F = 1$ for all other valid counting states (000 through 101) to allow normal operation.

3. **Formulating the Logic Function:** To generate a 0 uniquely for the minterm corresponding to $Q_2Q_1\overline{Q}_0$, a standard NAND implementation is used:

$$F = \overline{Q_2 \cdot Q_1 \cdot \overline{Q}_0}$$

This matches the expression given in option (A).

$$\boxed{\text{(A) } F = \overline{Q_2Q_1\overline{Q}_0}}$$

Key Points

- **Asynchronous Truncation:** An asynchronous clear mechanism causes a temporary glitch state (here, 110) that is quickly wiped out as soon as the reset logic propagates back to the flip-flop clear terminals.
- **Active-Low Conventions:** NAND gates are naturally well-suited as decode logic for active-low clear systems since they output a 0 only when the target transient combination is matched.

PrepFusion

SOLUTIONS

VI

ADC & DAC

Topics

1. Introduction to DAC — Weighted Resistor and R-2R Ladder Type
2. Introduction to ADC — Counter Type
3. SAR Type ADC
4. Flash Type ADC
5. Single Slope and Dual Slope ADC
6. Sample and Hold Operation in ADC

Unit Snapshot*

Stream: EC	**Questions:** 38	**MCQ:** 21	**MSQ:** 1	**NAT:** 12
1M: 13	**2M:** 17	**Descriptive:** 8	**Avg Weightage** ~1M	**Range:** 0M(2015)–3M(2025)

**See preface for how Avg Weightage and Range are calculated.*

Q6.25.1 MCQ 2025 2M Ans: A ● ○ ○

Given: $n = 10$, $f_s = 1\,\text{MHz}$, Full scale voltage = 3.3 V, and $f_m = 500\,\text{kHz}$. The Signal-to-Noise Ratio (SNR) for an n-bit ADC is given by:

$$\text{SNR} = 1.76 + 6.02n$$

$$\text{SNR} = 1.76 + 6.02 \times 10 = 61.96\,\text{dB}$$

The data rate (R_D) is given by:

$$R_D = nf_s = 10 \times 1\,\text{MHz} = 10\,\text{Mbps}$$

Hence, the correct option is (A).

$$\boxed{(A) 61.96 \text{ and } 10}$$

Key Points

- The Signal-to-Noise Ratio for an ideal ADC is $\text{SNR}_{\text{dB}} \approx 6.02n + 1.76$.
- Data rate R_D in PCM/ADC systems is the product of the number of bits per sample n and the sampling frequency f_s.

Q6.25.2 NAT 2025 1M Ans: 250 ● ○ ○

The given circuit is a 4-bit weighted-resistor DAC with a reference voltage $V_{REF} = 2\,\text{V}$. The inputs are b_3, b_2, b_1, b_0 (MSB to LSB).

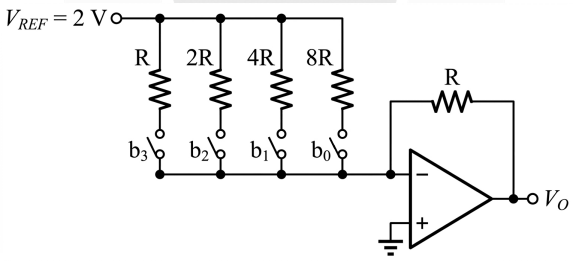


Fig. Solution Q6.25.2

Method 1

The output voltage V_o for a weighted-resistor DAC is given by:

$$V_o = -\left[\frac{b_0}{8R} + \frac{b_1}{4R} + \frac{b_2}{2R} + \frac{b_3}{R}\right] R \times V_{REF}$$

$$V_o = -\left[\frac{b_0}{8} + \frac{b_1}{4} + \frac{b_2}{2} + \frac{b_3}{1}\right] \times 2$$

For binary input 1110 ($b_3 = 1, b_2 = 1, b_1 = 1, b_0 = 0$):

$$V_{o1} = -\left[0 + \frac{1}{4} + \frac{1}{2} + \frac{1}{1}\right] \times 2 = -3.5\,\text{V}$$

For binary input 1101 ($b_3 = 1, b_2 = 1, b_1 = 0, b_0 = 1$):

$$V_{o2} = -\left[\frac{1}{8} + 0 + \frac{1}{2} + \frac{1}{1}\right] \times 2 = -3.25\,\text{V}$$

The magnitude of the change in output voltage is:

$$\Delta V_o = |V_{o1} - V_{o2}| = |-3.5 - (-3.25)| = 0.25\,\text{V} = 250\,\text{mV}$$

Method 2

Binary 1110 corresponds to decimal 14 and binary 1101 corresponds to decimal 13. Since the difference between the two inputs is exactly 1 unit (the LSB), the magnitude of the change in output voltage is equal to the resolution of the DAC.
The resolution is the smallest possible change in output, which occurs when only the LSB (b_0) is high.

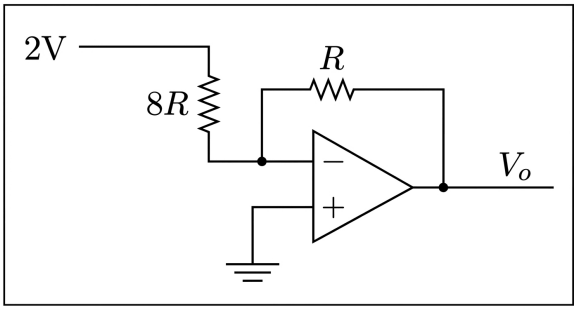


Fig. Solution Q6.25.2

For $b_0 = 1$ and all other bits 0, the inverting amplifier output is:

$$V_o = -\frac{R_f}{R_{in}} \times V_{REF} = -\frac{R}{8R} \times 2\,\text{V} = -0.25\,\text{V}$$

$$\text{Resolution} = |V_o| = 0.25\,\text{V}$$

The magnitude of change for a 1-bit difference is 0.25 V or 250 mV.

$$\boxed{250}$$

Mistakes to be avoided

- Do not swap the MSB and LSB resistor values; the largest resistor ($8R$) always corresponds to the LSB (b_0).
- Pay attention to the requested unit; the answer must be in mV if specified (0.25 V = 250 mV).

Q6.24.1 MCQ 2024 2M Ans: B ●●○

Given:

- Number of bits $n = 10$
- Normalized power = 1 W
- Noise power $N = 10\ \mu\text{W}$

$$SNR = \frac{S}{N} = \frac{1\,\text{W}}{10\,\mu\text{W}} = 10^5$$

$$(SNR)_{dB} = 10\log_{10}(10^5)$$

$$(SNR)_{dB} = 50\,\text{dB}$$

Using the ideal SNR formula for an ADC:

$$1.76 + 6.02n = 50$$

$$6.02n = 48.24$$

$$n = 8$$

$$\boxed{(B)8}$$

Key Points

- SNR is a calculated value that represents the ratio of RMS Signal to RMS Noise.
- If we multiply the $\log_{10}$ of this ratio by 10 for power (or 20 for voltage) we derive SNR in decibels.

Q6.23.1 NAT 2023 1M Ans: 10 ● ○ ○

Given:

- SNR of ADC = 61.96 dB

The signal-to-noise ratio (SNR) of an ideal n-bit ADC is given by the formula:

$$\text{SNR} = 1.76 + 6.02n \text{ dB}$$

Substituting the given value:

$$61.96 = 1.76 + 6.02n$$
$$6.02n = 61.96 - 1.76$$
$$6.02n = 60.2$$
$$n = \frac{60.2}{6.02}$$
$$n = 10$$

$$\boxed{10}$$

Q6.21.1 NAT 2021 1M Ans: 4.5 ● ○ ○

Given data:

- Resolution parameters: $n = 8$ bits, $V_{FS} = 7.68$ V
- Digital input: $(10010110)_2 = 128 + 16 + 4 + 2 = (150)_{10}$

The analog output voltage V_{out} for a unipolar DAC is computed as:

$$V_{out} = \left(\frac{V_{FS}}{2^n - 1}\right) \times \text{Decimal Equivalent}$$

Substituting the values:

$$V_{out} = \left(\frac{7.68}{2^8 - 1}\right) \times 150 = \left(\frac{7.68}{255}\right) \times 150 \approx 4.5176 \text{ V}$$

Rounding off to one decimal place, $V_{out} = 4.5$ V.

$$\boxed{4.5}$$

Key Points

- DAC Resolution is defined as $\frac{V_{FS}}{2^n - 1}$.
- Output voltage scales linearly with the decimal equivalent of the digital input.

Mistakes to be avoided

- Use $2^n - 1$ in the denominator for full-scale reference calculations, not 2^n.

Q6.20.1 NAT 2020 1M Ans: 3.050 to 3.080 ● ○ ○

Given data from the problem statement:

- Number of bits: $n = 10$
- Full-scale voltage range: $V_{\text{FSR}} = 10\,\text{V}$
- Hexadecimal digital input: $(13\text{A})_{16}$

Converting the hexadecimal input to its decimal equivalent:

$$(13\text{A})_{16} = 1 \times 16^2 + 3 \times 16^1 + 10 \times 16^0 = 256 + 48 + 10 = (314)_{10}$$

The step size (resolution) of the 10-bit D/A converter is computed as:

$$\text{Resolution } (R) = \frac{V_{\text{FSR}}}{2^n - 1} = \frac{10\,\text{V}}{2^{10} - 1} = \frac{10}{1023} \approx 9.77517 \times 10^{-3}\,\text{V}$$

The analog output voltage V_o is given by:

$$V_o = \text{Resolution } (R) \times \text{Decimal Equivalent}$$

$$V_o = \left(\frac{10}{1023}\right) \times 314 = \frac{3140}{1023} \approx 3.0694\,\text{V}$$

Rounding off to three decimal places yields the final analog output voltage as 3.069 V.

$$\boxed{3.069}$$

Key Points

- The step size or resolution of an n-bit DAC over a full-scale range is given by $R = \frac{V_{\text{FSR}}}{2^n - 1}$.
- Hexadecimal values must always be completely expanded into base-10 weights before evaluating the final output signal voltage.

Q6.16.1 MCQ 2016 2M Ans: A ● ● ●

An N-bit flash ADC requires $2^N - 1$ comparators. For an 8-bit resolution, the number of comparators is:

$$2^8 - 1 = 255$$

Each comparator has an input capacitance of $C = 8$ pF. Since the analog input voltage V_{in} is fed simultaneously to all comparators, they are connected in parallel. Thus, the total input capacitance C_T seen by the analog source is:

$$C_T = (2^8 - 1) \times C = 255 \times 8\,\text{pF} = 2.04\,\text{nF}$$

The source resistance is given as $R = 75\,\Omega$. This forms a low-pass RC charging circuit with a time constant τ:

$$\tau = R \cdot C_T = 75\,\Omega \times 2.04\,\text{nF} = 153\,\text{ns}$$

Method 1

For the input to settle within an accuracy of $\frac{1}{2}$ LSB during a full-scale step input change, the voltage across the capacitors must settle to its final value within a specific tolerance. Typically, a high-precision settling to within less than 1% error requires about 5 time constants (5τ):

$$\text{Settling Time } (T_s) = 5\tau = 5 \times 153\,\text{ns} = 765\,\text{ns}$$

The maximum sampling rate is limited by this settling time:

$$f_s = \frac{1}{T_s} = \frac{1}{765 \text{ ns}} \approx 1.3 \times 10^6 \text{ samples/sec} \approx 1 \text{ megasamples per second}$$

Method 2

Alternatively, we can compute the exact conversion time t by evaluating the discharging/charging equation. The error voltage must be less than $\frac{1}{2}$ LSB:

$$V_{\text{error}} = V_{\text{ref}} \cdot e^{-t/(R \cdot C_T)} = \frac{\text{LSB}}{2}$$

Since LSB $= \frac{V_{\text{ref}}}{2^8 - 1}$, we have:

$$V_{\text{ref}} \cdot e^{-t/(R \cdot C_T)} = \frac{V_{\text{ref}}}{2(2^8 - 1)}$$

$$e^{t/(R \cdot C_T)} = 2(2^8 - 1) = 2(255) = 510$$

Taking the natural logarithm on both sides:

$$\frac{t}{R \cdot C_T} = \ln(510) \approx 6.234$$

$$t = 6.234 \times \tau = 6.234 \times 153 \text{ ns} \approx 953.8 \text{ ns}$$

The maximum sampling rate corresponding to this minimum conversion time is:

$$f_s = \frac{1}{t} = \frac{1}{953.8 \text{ ns}} \approx 1.048 \text{ megasamples per second}$$

Among the given options, 1 megasamples per second is the closest value.

(A) 1 megasamples per second

Key Points

- An N-bit flash ADC uses $2^N - 1$ parallel comparators, multiplying the individual input capacitance by this factor to find the total capacitance C_T.
- The equivalent RC network dictates the transient settling response time constant: $\tau = R \cdot C_T$.
- Accuracy requirements to within $\frac{1}{2}$ LSB establish the minimum tracking/settling window constraint before digitization can reliably take place.

Mistakes to be avoided

- **Incorrect Capacitance:** Forgetting that all comparator inputs are in parallel, which scales up the total load capacitance by $2^N - 1$ instead of considering just a single 8 pF capacitor.

Q6.15.1 NAT 2015 1M Ans: 0.93 to 0.94 ●○○

Method 1

The step size is the output change for a 1 LSB increment (0001_2). Given:

- $V_{\text{out}}(0000_2) = 0$ V
- $V_{\text{out}}(0001_2) = 0.0625$ V
- To find: V_{out} for 1111_2 that is full scale voltage V_{FSO}

$$\text{Step Size} = 0.0625 \text{ V}$$

For an n-bit converter, the total number of non-zero intervals is $2^n - 1$. For $n = 4$:

$$\text{Total Steps} = 2^4 - 1 = 15$$

The Full-Scale Output (V_{FSO}) for the input 1111_2 is:

$$V_{FSO} = \text{Total Steps} \times \text{Step Size} = 15 \times 0.0625 \text{ V} = 0.9375 \text{ V}$$

Method 2

The resolution is determined by the variation between consecutive digital codes:

$$\text{Step Size} = V_{out}(0001_2) - V_{out}(0000_2) = 0.0625 \text{ V}$$

The voltage output V_o scales linearly with the decimal equivalent of the binary value:

$$(1111)_2 = 2^3 + 2^2 + 2^1 + 2^0 = 15$$

$$V_o = 0.0625 \text{ V} \times 15 = 0.9375 \text{ V}$$

$$\boxed{0.9375}$$

Key Points

- **Resolution:** The smallest analog increment produced by an LSB change.
- **DAC Transfer Function:** V_{out} = Decimal Equivalent × Step Size.

Q6.14.1 NAT 2014 1M Ans: 0.24 to 0.26 ●○○

Given that an analog voltage range from 0 V to 8 V is divided into 16 equal intervals.
The width of each interval (step size or resolution, S) is calculated as:

$$S = \frac{\text{Total Voltage Range}}{\text{Number of Intervals}} = \frac{8 \text{ V} - 0 \text{ V}}{16} = 0.5 \text{ V}$$

For an Analog-to-Digital Converter (ADC), the maximum quantization error occurs at the midpoint of an interval when rounding to the nearest code level:

$$Q_{max} = \pm\frac{S}{2}$$

Substituting the value of step size (S):

$$Q_{max} = \frac{0.5 \text{ V}}{2} = 0.25 \text{ V}$$

$$\boxed{0.25}$$

Key Points

- **Step Size (S):** Defined directly as the voltage span divided by the number of conversion intervals.
- **Quantization Error:** The difference between the actual continuous analog input and its discrete digitized representation, bounded by $\pm\frac{S}{2}$.

Q6.08.1 MCQ 2008 2M Ans: D ●●○

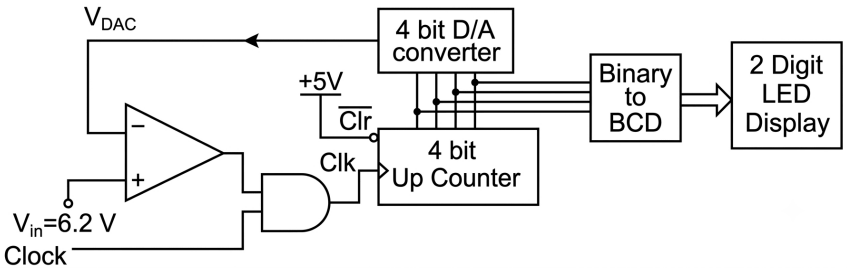


Fig. Solution Q6.08.1

- The circuit represents a counter-ramp ADC. Starting from the clear state $(0000)_2$, the counter increments by 1 on each clock pulse.
- This clock signal passes through an AND gate, which is controlled by the output of a comparator.
- The clock pulses keep reaching the counter as long as the comparator output stays high, that is, $V_{in} > V_{DAC}$.
- As soon as V_{DAC} slightly exceeds V_{in}, the comparator output switches to low, closing the AND gate to freeze the counter at a stable output reading.
- The output voltage of the 4-bit Digital-to-Analog Converter (DAC) is given in the question as:

$$V_{DAC} = \sum_{n=0}^{3} 2^{n-1} b_n = 2^{-1}b_0 + 2^0 b_1 + 2^1 b_2 + 2^2 b_3$$

$$V_{DAC} = 0.5b_0 + 1b_1 + 2b_2 + 4b_3$$

Method 1

Let us track the behavior across successive clock pulses:

b_3	b_2	b_1	b_0	V_{DAC} (V)
0	0	0	0	0
0	0	0	1	0.5
0	0	1	0	1
0	0	1	1	1.5
0	1	0	0	2
0	1	0	1	2.5
0	1	1	0	3
0	1	1	1	3.5

b_3	b_2	b_1	b_0	V_{DAC} (V)
1	0	0	0	4
1	0	0	1	4.5
1	0	1	0	5
1	0	1	1	5.5
1	1	0	0	6
1	1	0	1	6.5
1	1	1	0	7
1	1	1	1	7.5

From the table, we can see as soon as the counter reaches $(1101)_2 = 13$, the DAC output voltage becomes 6.5 V, which is just greater than $V_{in} = 6.2$ V.
This forces the comparator output to 0, disabling the clock signal via the AND gate and freezing the LED display stably at $(1101)_2 = 13$.

Method 2

- **Determine Resolution (Step Size):** Expanding the given equation for the Least Significant Bit (LSB) term (b_0):

$$\text{Resolution} = 2^{-1} = 0.5\,\text{V}$$

This means V_{DAC} increases by exactly 0.5 V with every step.

- **Find the Stopping Condition:** The counter continues to count upward until V_{DAC} just exceeds $V_{in} = 6.2\,\text{V}$ to turn off the AND gate.

$$V_{DAC} = \text{Count} \times \text{Resolution} > 6.2\,\text{V}$$

- **Calculate the Stable Decimal Count:** The closest multiple of 0.5 V that is strictly greater than 6.2 V is 6.5 V:

$$\text{Count} = \frac{6.5\,\text{V}}{0.5\,\text{V}} = 13$$

- **Convert to Digital Output:** Converting the decimal count 13 into its 4-bit binary equivalent yields:

$$(13)_{10} = (1101)_2$$

(D) 13

Key Points

- In a counter-ramp type ADC, the counter continues to count upward until V_{DAC} slightly crosses over the analog value V_{in}.
- Once $V_{DAC} > V_{in}$, the clock input gets gated off, rendering the current digital state stable.

Q6.08.2 MCQ 2008 2M Ans: B ● ○ ○

From the previous sub-question, at steady state, the counter freezes at count 13, making $V_{DAC} = 6.5\,\text{V}$ while $V_{in} = 6.2\,\text{V}$. The magnitude of the error is:

$$\text{Error} = |V_{DAC} - V_{in}| = 6.5\,\text{V} - 6.2\,\text{V} = 0.3\,\text{V}$$

(B) 0.3

Key Points

- The steady-state error in digital ramp ADCs represents the quantization step mismatch at the instant when final output is achieved.

$$\text{Error} = V_{DAC} - V_{in}$$

Q6.07.1 MCQ 2007 2M Ans: B ● ● ○

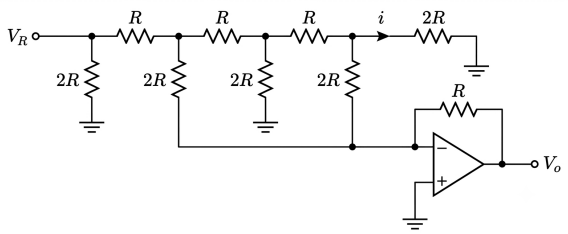


Fig. Solution Q6.07.1

By the concept of virtual ground the ladder circuit can be reduced as follows:

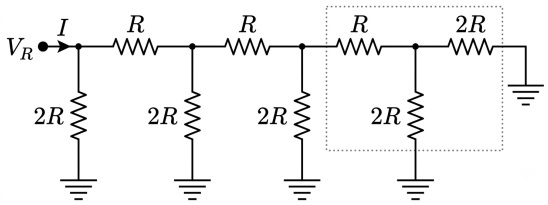
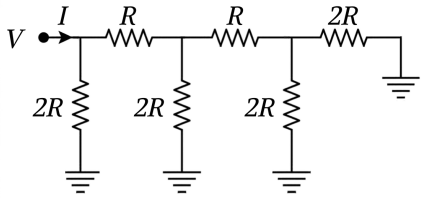
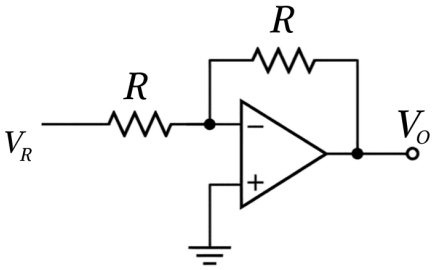


Fig. Solution Q6.07.1

- **Total Ladder Resistance:** We can see the reduction of the ladder step-by-step from right to left:



Step 1: Outermost parallel $2R \parallel 2R = R$, in series with R gives $2R$.



Step 2: Repeated reduction simplifies the entire inner network to R.

- **Total Source Current (I):** Using Ohm's law with $V_R = 10\,\text{V}$ and $R = 10\,\text{k}\Omega$:

$$I = \frac{V_R - 0}{R_{eq}} = \frac{10\,\text{V}}{10\,\text{k}\Omega} = 1\,\text{mA}$$

- **Current Splitting at Nodes:** At each node moving left-to-right along the ladder, the current splits exactly in half:

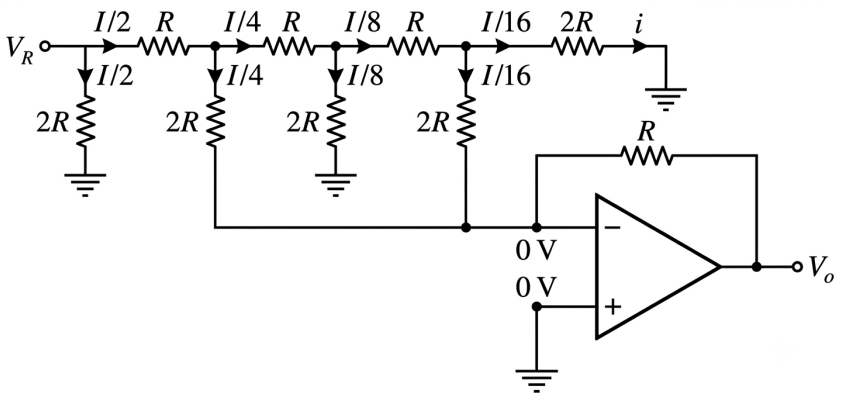


Fig. Solution Q6.07.1

- **Calculating the Target Current (i):** Substituting the value of total current $I = 1\,\text{mA}$:

$$i = \frac{1\,\text{mA}}{16} = 0.0625\,\text{mA} = 62.5\,\mu\text{A}$$

$$\boxed{\text{(B) } 62.5\,\mu\text{A}}$$

Key Points

- At any junction node in a standard R-$2R$ network, the current entering splits into two equal components.
- The final branch current at the n-th bit position of an N-bit ladder scales down precisely by a factor of 2^{-n}.

Q6.07.2 MCQ | 2007 | 2M | Ans: C ● ● ○

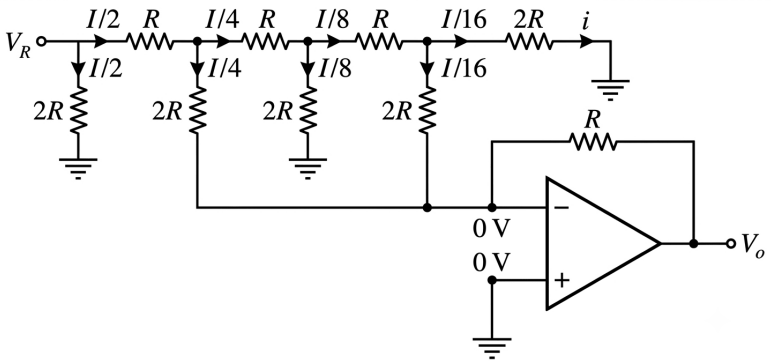


Fig. Solution Q6.07.2

- **Identify Injected Currents:** From the above image, only the shunt branches carrying currents $\frac{I}{4}$ and $\frac{I}{16}$ connect directly to the inverting input of the op-amp, while all other branches are grounded.
- **Total Summing Junction Current (I_{total}):**

$$I_{total} = \frac{I}{4} + \frac{I}{16} = \frac{5I}{16}$$

The ladder current drawn from the reference is $I = \frac{V_R}{R} = \frac{10\,\text{V}}{10\,\text{k}\Omega} = 1\,\text{mA}$ (as in the previous question), so:

$$I_{total} = \frac{5 \times 1\,\text{mA}}{16} = \frac{5}{16}\,\text{mA}$$

- **Calculate the Output Voltage (V_o):**

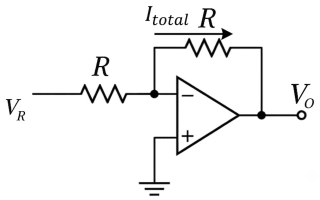


Fig. Solution Q6.07.2

By applying Kirchhoff's Current Law (KCL):

$$0 - V_o = I_{total} \times R$$

$$V_o = -\left(\frac{5}{16}\,\text{mA}\right) \times 10\,\text{k}\Omega = -\frac{50}{16}\,\text{V} = -3.125\,\text{V}$$

$$\boxed{\text{(C)}\ -3.125\,\text{V}}$$

Key Points

- The summing junction of an inverting op-amp adds up all input currents while keeping the voltage at a stable virtual ground.
- Grounded selector lines isolate unwanted branch currents completely from participating in the output voltage conversion.

Q6.06.1 MCQ 2006 2M Ans: B

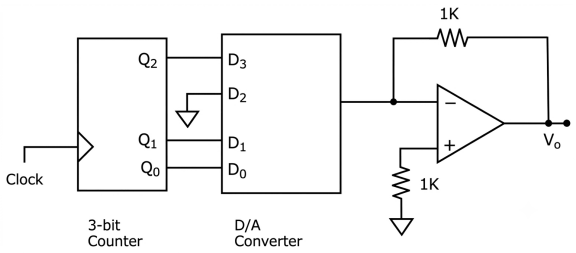


Fig. Solution Q6.06.1

From the connection diagram, the outputs are wired as

$$D_3 = Q_2, D_2 = 0, D_1 = Q_1, D_0 = Q_0$$

State Table Construction

Since the 3-bit UP counter counts from 000_2 to 111_2 (decimal 0 to 7) and continuously recycles, we map out the corresponding DAC states and output steps below:

Clock Pulse	Q_2	Q_1	Q_0	D_3	D_2	D_1	D_0	Relative V_0 Level
0	0	0	0	0	0	0	0	0
1	0	0	1	0	0	0	1	1
2	0	1	0	0	0	1	0	2
3	0	1	1	0	0	1	1	3
4	1	0	0	1	0	0	0	8
5	1	0	1	1	0	0	1	9
6	1	1	0	1	0	1	0	10
7	1	1	1	1	0	1	1	11
8 (Reset)	0	0	0	0	0	0	0	0

Role of the Op-Amp

The operational amplifier setup shown serves several key roles:

- **Current-to-Voltage Conversion:** It functions as a transimpedance amplifier that takes the variable output current from the 4-bit DAC and converts it into a clean analog voltage (V_0).
- **Virtual Ground Maintenance:** It holds the inverting input node at a virtual ground potential (0 V), which is necessary to preserve the linearity and conversion accuracy of the DAC's internal ladder network.
- **Output Buffering:** It acts as a buffer by providing a very low output impedance, ensuring that the analog staircase waveform remains stable and unaffected by external load conditions.

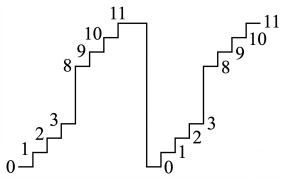


Fig. Solution Q6.06.1

(B)

Mistakes to be avoided

- Do not assume the staircase is perfectly linear. Ignoring the fact that D_2 is tied firmly to ground would lead you to mistake the response for a standard regular ramp shown in option (C).
- Ensure you recognize that the 3-bit UP counter is unidirectional; it only counts upwards and overflows to zero, meaning it cannot produce triangular up-and-down waves like options (A) or (D).

Q6.04.1 MCQ 2004 1M Ans: D ● ○ ○

The problem asks for the minimum number of bits required to represent 100 distinct increments in straight binary encoding. To represent N distinct values or states using binary encoding, the number of bits n required must satisfy the following relation:

$$2^n \geq N$$

Given that the number of increments $N = 100$:

$$2^n \geq 100$$

Let us evaluate the powers of 2:

- For $n = 7$: $2^7 = 128$ (Sufficient, as $128 \geq 100$)

Thus, the minimum number of bits required to encode 100 increments is 7.

(D) 7

Key Points

- The maximum number of unique states that can be represented by n bits in straight binary is 2^n.
- To find the minimum number of bits for N states, use the formula: $n = \lceil \log_2(N) \rceil$.

Q6.03.1 MCQ 2003 2M Ans: A ● ● ●

The output voltage V_o of the given weighted-resistor 4-bit digital-to-analog converter (DAC) is expressed as:

$$V_o = -R_f \left[\frac{d_3 V_R}{R_1} + \frac{d_2 V_R}{R_2} + \frac{d_1 V_R}{R_3} + \frac{d_0 V_R}{R_4} \right]$$

Given the nominal values $R_f = R$, $R_1 = R$, $R_2 = 2R$, $R_3 = 4R$, and $R_4 = 8R$:

$$V_o = -V_R \frac{R_f}{R} \left[d_3 + \frac{d_2}{2} + \frac{d_1}{4} + \frac{d_0}{8} \right]$$

To calculate the absolute worst-case tolerance (maximum percentage error), we determine the true value and the maximum worst-case value for the full-scale output ($d_3 d_2 d_1 d_0 = 1111$).

1. Ideal (True) Output Voltage (V_{o2}): Ignoring all tolerance limits ($V_R = 5\,\text{V}$ and $R_f = R_1 = R$):

$$V_{o2} = 5 \times \frac{R}{R}\left[1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8}\right] = 5 \times \frac{15}{8} = 9.375\,\text{V}$$

2. Maximum Worst-Case Output Voltage (V_{o1}): With a ±10% tolerance, the components can deviate to maximize V_o:

- Maximum reference voltage: $V_{R,\max} = 5 \times 1.1 = 5.5\,\text{V}$
- Maximum feedback resistance: $R_{f,\max} = 1.1R$
- Minimum input scaling resistance: $R_{\min} = 0.9R$

Substituting these maximum values:

$$V_{o1} = 5.5 \times \frac{1.1R}{0.9R}\left[1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8}\right] = 5.5 \times \frac{1.1}{0.9} \times \frac{15}{8} \approx 12.604\,\text{V}$$

3. Tolerance of the DAC (T): The overall percentage deviation is defined by:

$$T = \pm\left[\frac{V_{o1} - V_{o2}}{V_{o2}}\right] \times 100\%$$

$$T = \pm\left[\frac{12.604 - 9.375}{9.375}\right] \times 100\% \approx \pm 34.44\%$$

Rounding to the nearest multiple of 5% gives ±35%.

(A) ± 35%

Key Points

- Worst-case error in a ratio $\frac{X}{Y}$ occurs when the numerator is maximized ($X_{\max}$) and the denominator is minimized ($Y_{\min}$).
- Full-scale input configuration (1111_2) yields the absolute peak magnitude for structural error analysis.

Q6.03.2 MCQ 2003 1M Ans: C

For an n-bit flash (parallel) analog-to-digital converter (ADC), the total number of reference voltage levels required is 2^n. Since the ground or zero level does not require an active comparator comparison, the number of comparators needed to resolve these levels simultaneously is:

$$N = 2^n - 1$$

Given that the number of bits $n = 8$:

$$N = 2^8 - 1 = 256 - 1 = 255$$

(C) 255

Key Points

- An n-bit flash ADC requires $2^n - 1$ comparators, 2^n resistors, and a $2^n \times n$ priority encoder.
- Flash ADC is the fastest type of ADC because all comparisons are performed simultaneously in a single clock cycle.

Mistakes to be avoided

- Do not select 2^n (256); remember that one less comparator is required because the bottom-most level represents zero voltage.

Q6.02.1 MCQ 2002 1M Ans: C ●○○

A comparator-type ADC is also known as a flash type ADC. For an n-bit flash ADC, the number of reference voltage levels required is 2^n. Excluding the ground reference, the number of comparators required is:

$$N = 2^n - 1$$

Given that the number of bits $n = 3$:

$$N = 2^3 - 1 = 8 - 1 = 7$$

(C) 7

Key Points

- A comparator-type or flash ADC requires $2^n - 1$ comparators.
- It is the fastest architecture available because all conversions happen simultaneously in one clock cycle.

Q6.01.1 NAT 2001 1.67M Ans: 0.03125 ●○○

The resolution of an n-bit Analog-to-Digital Converter (ADC) is the width of one quantization interval (1 LSB). An n-bit ADC has 2^n output codes, so it divides the input range from V_{min} to V_{max} into 2^n equal intervals:

$$\text{Resolution} = \frac{V_{max} - V_{min}}{2^n}$$

Given parameters:

- Number of bits (n) = 8
- Full-scale range = 0 V to 8 V

Substituting the values into the equation:

$$\text{Resolution} = \frac{8 - 0}{2^8} = \frac{8}{256} = 0.03125 \text{ V/bit}$$

0.03125

Key Points

- **ADC Resolution:** Resolution defines the smallest change in the analog input voltage that can be distinguished by the digital output code, $\frac{V_{FS}}{2^n}$. Some texts instead use $\frac{V_{FS}}{2^n - 1}$ (here $\frac{8}{255} \approx 0.03137$ V); for large n the two values are almost equal.

Q6.01.2 Descriptive 2001 1.67M Ans: * ●●○

The step size (Δ) corresponds to the resolution of the converter:

$$\Delta = \frac{V_{max} - V_{min}}{2^n - 1} = \frac{8}{255} \text{ V}$$

Assuming a uniformly distributed quantization error between $-\frac{\Delta}{2}$ and $+\frac{\Delta}{2}$, the mean squared quantization error (quantization noise power) is defined as:

$$\text{Mean Squared Quantization Error} = \frac{\Delta^2}{12}$$

Substituting the value of step size (Δ):

$$\text{Mean Squared Quantization Error} = \frac{\left(\frac{8}{255}\right)^2}{12} = \frac{64}{65025 \times 12} = \frac{64}{780300} \approx 8.202 \times 10^{-5}\ \text{V}^2$$

$$\boxed{8.2 \times 10^{-5}}$$

Key Points

- **Quantization Error Variance:** For any ideal quantizer with a uniform distribution of error over one step interval, the variance or mean squared error is consistently given by $\frac{\Delta^2}{12}$.

Q6.01.3 Descriptive 2001 1.67M Ans: * ●●○

In a counter-controlled ADC, the digital equivalent code is reached when the output of the internal Digital-to-Analog Converter (DAC) just equals or exceeds the input voltage ($V_{in} = 1.59$ V).
Let N be the decimal equivalent count step required to reach the target voltage:

$$N \times \text{Resolution} = V_{in}$$

$$N \times \frac{8}{255} = 1.59$$

$$N = \frac{1.59 \times 255}{8} = \frac{405.45}{8} \approx 50.68$$

Since the count value must be an integer step, we round up to the next integer level to match or cross the target voltage:

$$N = 51$$

The time taken by an up-counter running at a clock frequency f_{clk} to complete N clock periods is:

$$t = \frac{N}{f_{clk}}$$

Given clock frequency (f_{clk}) = 1 MHz = 1×10^6 Hz:

$$t = \frac{51}{1 \times 10^6} = 51 \times 10^{-6}\ \text{seconds} = 51\ \mu\text{s}$$

$$\boxed{51 \times 10^{-6}}$$

Key Points

- **Counter ADC Conversion Time:** The conversion time for a simple tracking or counter-controlled ADC is variable and linearly dependent on the magnitude of the input voltage, taking up to a maximum of $2^n \cdot T_{clk}$ cycles.

Q6.00.1 MCQ 2000 2M Ans: B ●●○

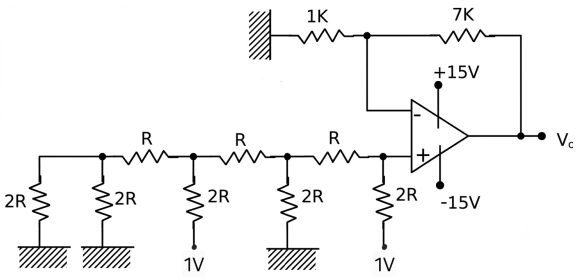


Fig. Solution Q6.00.1

The given circuit consists of a 4-bit R-$2R$ ladder network connected to a non-inverting operational amplifier configuration. **Determine the binary input from the ladder connections** Let the ground terminal represent Logic-0 and the 1 V source represent Logic-1. Looking at the ladder configuration from the most significant bit (MSB) closest to the op-amp non-inverting terminal to the least significant bit (LSB):

- b_3 (MSB, closest to V^+) is connected to 1 V $\rightarrow$ 1
- b_2 is connected to Ground $\rightarrow$ 0
- b_1 is connected to 1 V $\rightarrow$ 1
- b_0 (LSB, farthest from V^+) is connected to Ground $\rightarrow$ 0

Thus, the 4-bit binary input sequence $(b_3b_2b_1b_0)_2 = (1010)_2$. The decimal equivalent of this binary value is:

$$D = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = 10$$

Find the non-inverting node voltage V^+ For a standard R-$2R$ ladder with reference voltage V_{ref}, the voltage V^+ at the non-inverting input terminal is given by:

$$V^+ = \frac{V_{ref}}{2^n} \times D$$

Given $V_{ref} = 1$ V and $n = 4$:

$$V^+ = \frac{1}{2^4} \times 10 = \frac{10}{16} \text{ V}$$

Calculate the output voltage V_o The operational amplifier is in a non-inverting configuration with feedback resistor $R_f = 7\,\text{k}\Omega$ and input resistor $R_1 = 1\,\text{k}\Omega$. The output voltage is expressed as:

$$V_o = \left(1 + \frac{R_f}{R_1}\right) V^+$$

Substituting the given resistor values and V^+:

$$V_o = \left(1 + \frac{7\,\text{k}\Omega}{1\,\text{k}\Omega}\right) \times \frac{10}{16}$$

$$V_o = 8 \times \frac{10}{16} = 5 \text{ V}$$

(B) 5 V

Key Points

- The output expression for an n-bit R-$2R$ ladder DAC connected to a non-inverting amplifier is:

$$V_o = \left(1 + \frac{R_f}{R_1}\right) \times \frac{V_{ref}}{2^n} \times D$$

  where D is the decimal equivalent of the binary pattern applied.
- In an R-$2R$ network, the bit closest to the amplifier terminal always represents the Most Significant Bit (MSB).

Mistakes to be avoided

- Carefully trace the binary inputs starting from the terminal nearest to the Op-Amp as the MSB (b_{n-1}) to avoid reversing the binary sequence to $(0101)_2$.

Q6.00.2 MCQ 2000 1M Ans: C ● ○ ○

Given that the number of bits is $n = 4$.
The number of comparators required for an n-bit flash type Analog-to-Digital Converter (ADC) is given by the formula:

$$N = 2^n - 1$$

Substituting $n = 4$ into the formula:

$$N = 2^4 - 1 = 16 - 1 = 15$$

Thus, the number of comparators required is 15.

(C) 15

Key Points

- For an n-bit flash ADC:
  - Number of comparators = $2^n - 1$
  - Number of resistors = 2^n
  - Number of decoder input lines = $2^n - 1$
- Flash ADC is the fastest type of ADC because conversion takes place simultaneously through parallel comparators.

Mistakes to be avoided

- Do not confuse the number of comparators ($2^n - 1$) with the number of resistors or total resolution states (2^n).

Q6.00.3 MCQ 2000 1M Ans: B ● ○ ○

A Successive Approximation Register (SAR) Analog-to-Digital Converter operates using a binary search algorithm to determine the digital equivalent of an analog input voltage.
The conversion time (T_{conv}) for an n-bit SAR ADC depends exclusively on the total number of bits (n) and the internal clock period (T_{CLK}):

$$T_{conv} = n \cdot T_{CLK}$$

Because the binary search process evaluates each bit position sequentially from the Most Significant Bit (MSB) to the Least Significant Bit (LSB), the total number of clock cycles required remains constant. It does not fluctuate with variations in the magnitude or amplitude of the analog input signal.
Given parameters:

- Conversion time for an analog input of 1 V = 20 μs

Since the conversion mechanism is independent of the input level, the conversion time for an analog input of 2 V remains identically 20 μs.

(B) 20 μs

Key Points

- **SAR Conversion Characteristics:** A key benefit of the SAR architecture over a counter-controlled or dual-slope ADC is its fixed, predictable conversion speed (n clock cycles for an n-bit output) regardless of the input voltage amplitude.

Q6.99.1 MCQ 1999 1M Ans: D ●●○

Given:

- Analog input voltage, $V_{in} = 6.6\,\text{V}$
- Resolution (step size) = 0.5 V

The number of steps N corresponding to the input analog voltage is given by:

$$N = \frac{V_{in}}{\text{Resolution}}$$

$$N = \frac{6.6\,\text{V}}{0.5\,\text{V}} = 13.2$$

Since a counting type ADC counts upwards sequentially step-by-step starting from zero, it will increment its internal counter until the internal DAC output voltage exceeds or equals the analog input voltage. Therefore, the fractional step 13.2 is rounded up to the next higher integer value:

$$N \approx 14$$

Converting the decimal value 14 into its 4-bit binary equivalent:

$$(14)_{10} = (1110)_2$$

Thus, the digital output of the ADC will be 1110.

(D) 1110

Key Points

- **Counting Type ADC:** Continues counting up until the internal staircase voltage becomes greater than or equal to V_{in}. Hence, any fractional step is rounded up to the nearest integer.
- **Successive Approximation Register (SAR) ADC:** Employs a binary search technique based on a fixed threshold comparison at each step. For an input of 13.2 steps, it would approximate downward to 13 steps ($(1101)_2$).
- **Resolution:** Corresponds to the step size of the internal DAC, representing the minimum analog change required to vary the LSB.

Mistakes to be avoided

- Do not apply standard decimal mathematical rounding rules (where 13.2 rounds down to 13). For a counting-type architecture, the comparator keeps the clock gate open until the internal reference strictly reaches or surpasses the input value, forcing a ceiling function.

Q6.98.1 MCQ 1998 2M Ans: B ●○○

The primary advantage of a dual-slope Analog-to-Digital Converter (ADC) in precision measurement instruments like digital voltmeters (DVMs) is its high accuracy and excellent noise rejection capacity.
The dual-slope conversion process consists of two parts:

1. **Integrating phase (Fixed Time T_1):** The unknown analog input voltage V_{in} is integrated for a fixed duration. The integration eliminates high-frequency noise and mains hum (50 Hz or 60 Hz) through averaging.
2. **De-integrating phase (Variable Time T_2):** A known reference voltage V_{ref} of opposite polarity is connected to the integrator, bringing the output back to zero.

Since both integration and de-integration use the exact same resistor (R) and capacitor (C) network over the same clock cycle frequency, variations in component values due to temperature or aging cancel out mathematically:

$$V_{in} = V_{ref} \cdot \frac{T_2}{T_1}$$

Consequently, it delivers the highest accuracy and stability among all ADC architectures, making it ideal for digital voltmeters.

(B) its accuracy is high

Key Points

- **Flash ADC:** Fastest conversion speed because all quantization levels are compared simultaneously in parallel, but requires large power and area ($2^n - 1$ comparators).
- **Successive Approximation Register (SAR) ADC:** Moderate speed with fixed conversion time (n clock cycles) and low power consumption, making it highly popular for general data acquisition.
- **Dual-Slope Integrating ADC:** Highest accuracy and excellent line noise rejection through signal averaging, though it features the slowest conversion speed.
- **Counter Type (Staircase) ADC:** Simple design using a single comparator and counter, but conversion time varies directly with the analog input amplitude.
- **Tracking Type (Delta-Modulated) ADC:** High tracking speed for slowly varying continuous signals due to its up-down counter configuration following the input signal closely.
- **Sigma-Delta (Σ-Δ) ADC:** Exceptionally high resolution achieved via oversampling and noise-shaping techniques, making it ideal for high-fidelity audio and precision instrumentation.

Q6.98.2 MCQ 1998 2M Ans: B ●●○

To determine the current I through the resistance r, we can systematically simplify the network connected to the inverting terminal using Thevenin's theorem from left to right.

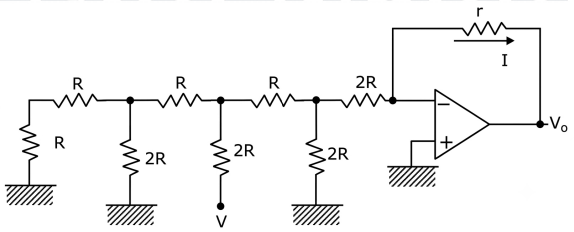


Fig. Solution Q6.98.2

- **Step 1: Initial reduction of the leftmost section**

  Starting from the absolute left of the circuit:

  6. The leftmost vertical 2 R resistors combine in series to give 2R with the adjacent $2R$ resistor, giving a combined resistance of $4R$ for that branch.
  6. This $2R$ equivalent resistance is in parallel with the next vertical $2R$ resistor, which simplifies to:

$$2R \parallel 2R = R$$

  This simplified resistance of R is now in series with the top horizontal R resistor, giving a total equivalent resistance of $2R$ for the first section.

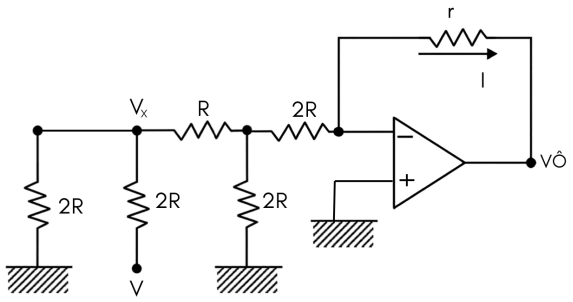


Fig. Solution Q6.98.2

- **Step 2: Finding the Thevenin equivalent at the source node** Looking back from the node where the source V is connected, according to Thevenin's theorem, the open-circuit voltage V_x is:

$$V_x = \frac{V}{2}$$

The equivalent resistance looking into this node is:

$$R_{thx} = 2R \parallel 2R = R$$

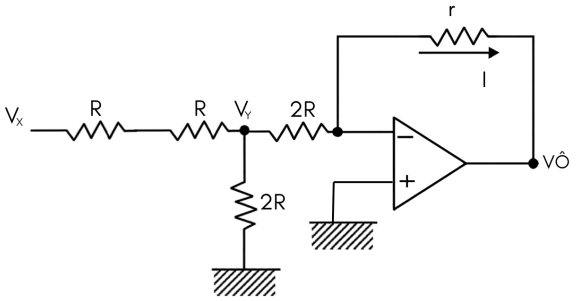


Fig. Solution Q6.98.2

- **Step 3: Repeating the process for the next stage** Similarly, we will repeat the same process for the next node as the circuit conditions are identical. The equivalent voltage V_y at the next node becomes:

$$V_y = \frac{V_x}{2} = \frac{V}{4}$$

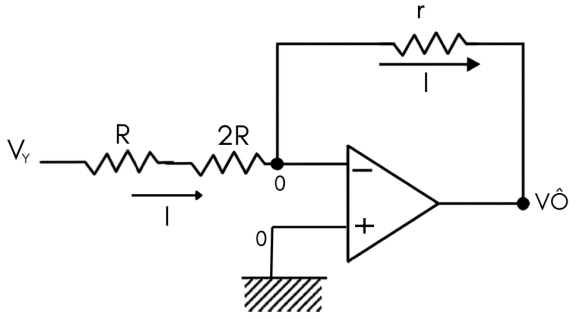


Fig. Solution Q6.98.2

- **Step 4: Final branch current calculation** Since the op-amp is ideal and operates with negative feedback, the inverting input terminal is at virtual ground (0 V).

  The total resistance in the final path leading to the virtual ground consists of the equivalent resistance R in series with the remaining $2R$ resistor.

  Finally, because no current enters the ideal op-amp terminal, the same current that flows through this branch must flow through the upper feedback resistor r:

$$I = \frac{V_y - 0}{R + 2R} = \frac{V_y}{3R} = \frac{\frac{V}{4}}{3R} = \frac{V}{12R}$$

$$\boxed{\text{(B) } \frac{V}{12R}}$$

Key Points

- **Virtual Ground Concept:** For an ideal operational amplifier with negative feedback, the potential at the inverting terminal tracks the non-inverting terminal ($V_- = V_+ = 0$ V).
- **Ideal Op-Amp Input Current:** The current entering the inverting input terminal is strictly zero ($I_{in} = 0$). Therefore, the entire current coming from the ladder network is forced to flow into the feedback resistor r.
- **Systematic Network Reduction:** When dealing with ladder networks, always reduce step-by-step from the side furthest from the source toward the node of interest to avoid errors in equivalent resistance.

Mistakes to be avoided

- **Misidentifying Series/Parallel Branches:** Do not mistake the leftmost vertical resistors as being in parallel immediately; always verify the node connections carefully from left to right.
- **Assuming Current Depends on r:** The current I is entirely determined by the input ladder network and the voltage V_y because the node before r is held at virtual ground. The value of the resistance r does not change the value of I.

Q6.96.1 MCQ 1996 2M Ans: A ●●○

To determine the maximum permissible net temperature coefficient of the ADC, we compare the allowed error with the temperature change from the calibration point.
First, calculate the value of 1 Least Significant Bit (LSB):

$$\text{LSB} = \frac{V_{\text{FS}}}{2^n} = \frac{10.24\text{ V}}{2^{10}} = \frac{10.24\text{ V}}{1024} = 0.01\text{ V} = 10\text{ mV}$$

The accuracy is specified as $\pm\text{LSB}/2$, so the output may drift from its calibrated value by at most:

$$\frac{\text{LSB}}{2} = \frac{10\text{ mV}}{2} = 5\text{ mV}$$

The system is calibrated at 25°C and operates from 0°C to 50°C. The maximum temperature change (ΔT) from the calibration point is:

$$\Delta T = 50^\circ\text{C} - 25^\circ\text{C} = 25^\circ\text{C} - 0^\circ\text{C} = 25^\circ\text{C}$$

The drift over this largest temperature change must stay within the 5 mV limit:

$$|\text{TC}| \leq \frac{\text{LSB}/2}{\Delta T} = \frac{5\text{ mV}}{25^\circ\text{C}} = 0.2\text{ mV}/^\circ\text{C} = 200\ \mu\text{V}/^\circ\text{C}$$

$$\boxed{\text{(A) } \pm 200\ \mu\text{V}/^\circ\text{C}}$$

Key Points

- **LSB Step Value**: For any n-bit converter, 1 LSB = $\frac{V_{FS}}{2^n}$. Here, 10.24 V/1024 = 10 mV.
- **Temperature Coefficient Limit**: The allowed error (±LSB/2 here) is divided by the largest temperature deviation from the calibration temperature, not by the full operating range.

Q6.96.2 MCQ 1996 1M Ans: D ●○○

To determine the type of the given 12-bit ADC, we analyze the relationship between its fixed conversion time ($T_c = 14\ \mu s$) and clock period ($T_{CLK} = 1\ \mu s$).
Since the conversion time is stated as a single fixed value independent of the analog input magnitude, we can eliminate the **Counting** and **Integrating** types, which have variable conversion times depending on the input voltage.
Among constant conversion time architectures:

- **Flash Type**: Requires exactly 1 clock cycle ($1 \times 1\ \mu s = 1\ \mu s$).
- **Successive Approximation (SAR) Type**: Requires approximately n clock cycles ($12 \times 1\ \mu s = 12\ \mu s$).

The observed 14 μs closely matches the fixed 12 μs baseline of the SAR type, with the additional 2 μs accounting for standard internal control-logic overhead (initialization and end-of-conversion framing).
Hence, the correct option is **D**.

(D)successive approximation type

Key Points

- **SAR Characteristics**: Successive approximation ADCs feature a fixed conversion time ($T_c \approx n \cdot T_{CLK}$) that remains constant regardless of the analog input value.
- **Overhead Cycles**: Extra clock periods are utilized by the register hardware for startup, sequencing, and setting the data-ready flag.

Q6.95.1 Descriptive 1995 3M Ans: A-4, B-3, C-2 ●●○

Based on the key characteristics and hardware architectures of different Analog-to-Digital Converters (ADCs):

- **(A) Flash converter:** Also known as a parallel comparator ADC, it operates at very high speeds but requires $2^n - 1$ comparators for an n-bit resolution. This extensive parallel architecture makes it require a very complex hardware layout. → **(4) requires a very complex hardware**
- **(B) Dual slope converter:** As an integrating-type ADC, its continuous integration of the input voltage inherently rejects high-frequency periodic noise, thereby minimizing the effect of power supply interference. → **(3) minimizes the effect of power supply interference**
- **(C) Successive approximation converter:** This feedback-driven architecture utilizes an internal Successive Approximation Register (SAR) and a Digital-to-Analog Converter (DAC) to iteratively converge upon the correct binary representation. → **(2) requires a digital-to-analog converter**

A-4, B-3, C-2

Key Points

- Flash ADC features the maximum speed but exhibits exponential hardware complexity ($2^n - 1$ comparators).
- Dual-Slope ADC offers high accuracy and noise immunity at the cost of slow conversion speed.

- Successive Approximation Register (SAR) ADC incorporates a feedback loop containing a built-in DAC.

Q6.94.1 Descriptive 1994 2M Ans: A-2, B-5, C-1 ● ○ ○

Based on the conversion time characteristics of an n-bit (n = 8) Analog-to-Digital Converter (ADC):

- **(A) Successive approximation:** The conversion requires exactly n clock cycles regardless of the input voltage. For an 8-bit ADC, the maximum conversion time is 8 clock cycles. → **(2) 8**
- **(B) Dual-slope:** The worst-case conversion time occurs when the input voltage is at its maximum value, requiring 2^{n+1} clock cycles. For an 8-bit ADC, this corresponds to 2^{8+1} = 512 clock cycles. → **(5) 512**
- **(C) Parallel Comparator:** Also known as a Flash ADC, it processes all bits simultaneously using parallel comparators, taking only a single clock cycle for conversion. → **(1) 1**

A-2, B-5, C-1

Key Points

- Parallel Comparator (Flash) ADC: Conversion time is independent of resolution and equals 1 clock cycle.
- Successive Approximation Register (SAR) ADC: Conversion time is linearly dependent on resolution, requiring n clock cycles.
- Dual-Slope ADC: Slowest conversion speed, requiring a maximum of 2^{n+1} clock cycles.

Q6.92.1 MSQ 1992 2M Ans: B, C ● ● ○

Analyzing the features of a Dual-slope integration type Analog-to-Digital Converter (ADC):

- **Option A is incorrect:** Dual-slope ADCs are the slowest among major types of ADCs because they require up to 2^{n+1} clock cycles for a single conversion.
- **Option B is correct:** In a dual-slope ADC, any variations or drifts in the values of the integrating resistor (R) and capacitor (C) affect both the charging (integration) phase and discharging (de-integration) phase identically. As a result, the component values cancel out in the final conversion formula, ensuring excellent accuracy without requiring ultra-stable components.
- **Option C is correct:** The continuous integration of the input voltage acts as a low-pass filter, which completely rejects high-frequency noise and effectively eliminates periodic power supply hum (50 Hz or 60 Hz noise) when the integration time is chosen to be an integer multiple of the line period.
- **Option D is incorrect:** The resolution of an ADC is determined strictly by its number of bits (n). For the same number of bits, the theoretical resolution remains identical across different ADC architectures.

(B) very good accuracy without putting extreme requirements on component stability

(C) good rejection of power supply hum

Key Points

- Dual-slope ADCs offer superb noise immunity and high accuracy due to the dual integration principle where component tolerances (R, C, and clock frequency) cancel out.
- They are highly preferred in digital multimeters where accuracy and noise rejection are vital, despite the significantly slower conversion rate.

Q6.91.1 NAT 1991 2M Ans: 3.6 ● ● ○

Given frequency components of the bandpass signal:

$$f_L = 300\,\text{Hz} = 0.3\,\text{kHz}$$

$$f_H = 1.8\,\text{kHz}$$

The bandwidth (BW) of the signal is calculated as:

$$BW = f_H - f_L = 1.8\,\text{kHz} - 0.3\,\text{kHz} = 1.5\,\text{kHz}$$

Using the criteria for bandpass sampling, the parameter m is determined by finding the largest integer less than or equal to $\frac{f_H}{BW}$:

$$m = \left\lfloor \frac{f_H}{BW} \right\rfloor = \left\lfloor \frac{1.8}{1.5} \right\rfloor = \lfloor 1.2 \rfloor = 1$$

The minimum sampling rate (f_s) required for distortion-free reconstruction is given by:

$$f_s = \frac{2f_H}{m} = \frac{2 \times 1.8\,\text{kHz}}{1} = 3.6\,\text{kHz}$$

$$\boxed{3.6}$$

Q6.90.1 MCQ 1990 2M Ans: C ● ● ○

A true R-$2R$ ladder network relies on the property that when looking into any node toward the LSB, the equivalent input resistance must always equal $2R$. Let us test the circuits by setting $V_2 = V_1 = V_0 = 0$ V (grounded) and collapsing the branches from the bottom (LSB side, V_0) up to the output node V_A.

- **Testing Network (iii):**

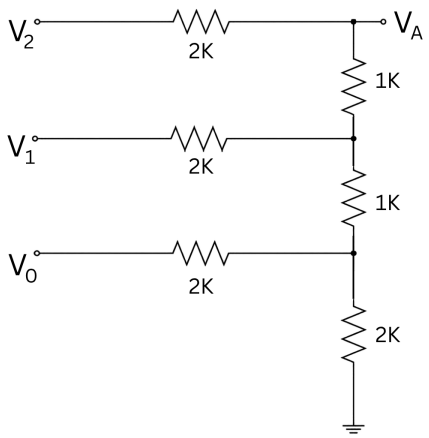


Fig. Solution Q6.90.1

1. At the bottom LSB node (V_0), the input resistor (2 kΩ) is in parallel with the ground termination resistor (2 kΩ):

$$2\,\text{k}\Omega \parallel 2\,\text{k}\Omega = 1\,\text{k}\Omega$$

2. This combination is now in series with the next upward 1 kΩ ladder resistor:

$$1\,\text{k}\Omega + 1\,\text{k}\Omega = 2\,\text{k}\Omega$$

3. At the middle node (V_1), this equivalent 2 kΩ resistance is in parallel with the V_1 input resistor (2 kΩ):

$$2\ \text{k}\Omega \parallel 2\ \text{k}\Omega = 1\ \text{k}\Omega$$

4. This value is in series with the top-most 1 kΩ ladder resistor leading to node V_A:

$$1\ \text{k}\Omega + 1\ \text{k}\Omega = 2\ \text{k}\Omega$$

5. Finally, at the MSB node (V_A), this perfectly balances in parallel with the V_2 input resistor (2 kΩ), collapsing the whole ladder cleanly into a single equivalent output resistance of 1 kΩ.

- **Testing Network (i):**

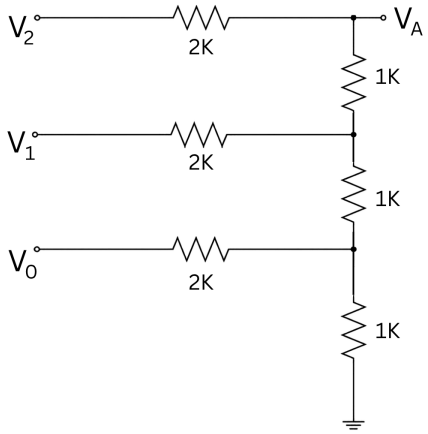


Fig. Solution Q6.90.1

At the bottom node, the input resistor (2 kΩ) is in parallel with a 1 kΩ ground resistor. This yields 2 kΩ || 1 kΩ = 0.67 kΩ, which fails to produce the uniform 1 kΩ step required to continuously repeat the series-parallel combination upwards.

- **Testing Network (ii):**

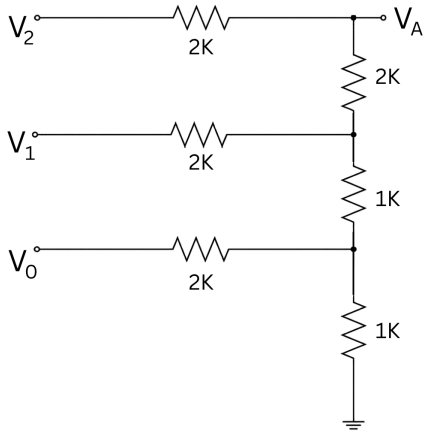


Fig. Solution Q6.90.1

Similary here,at the bottom node, the input resistor (2 kΩ) is in parallel with a 1 kΩ ground resistor. This yields 2 kΩ || 1 kΩ = 0.67 kΩ, which fails to produce the uniform 1 kΩ step required to continuously repeat the series-parallel combination upwards.

Therefore, only network (iii) possesses the structural symmetry to collapse perfectly step-by-step.
Hence, the correct option is **C**.

(C)Only (iii)

Key Points

- **The R-2R Principle**: Grounding all inputs demonstrates that the network acts as a sequence of balanced voltage dividers. Looking in any direction from any junction node always presents an equivalent resistance of exactly $2R$.
- **LSB Termination Requirement**: Without the extra termination resistor of value $2R$ at the LSB node to ground, the ladder cannot initiate its continuous $2R \parallel 2R = R$ collapsing sequence.

Q6.87.1 NAT 1987 Ans: * ●●○

The voltage reference levels at the non-inverting inputs of the comparators are:

$$V_0 = \frac{V_{CC}}{4}, \quad V_1 = \frac{2V_{CC}}{4}, \quad V_2 = \frac{3V_{CC}}{4}$$

The truth table relating the analog input V_i, the comparator outputs, and the natural binary outputs D_1D_0 is:

Analog Input V_i	C_2	C_1	C_0	D_1	D_0
$0 < V_i < \frac{V_{CC}}{4}$	0	0	0	0	0
$\frac{V_{CC}}{4} < V_i < \frac{2V_{CC}}{4}$	0	0	1	0	1
$\frac{2V_{CC}}{4} < V_i < \frac{3V_{CC}}{4}$	0	1	1	1	0
$V_i > \frac{3V_{CC}}{4}$	1	1	1	1	1

The rest of the terms are Don't care terms(X)
The Karnaugh maps for D_1 and D_0 in terms of C_2, C_1, C_0 are shown below:

- For D_1:

D_1 / C_2 \ C_1C_0	00	01	11	10
0	0	0	1	X
1	X	X	1	X

Fig. Solution Q6.87.1

$$D_1 = C_1$$

- For D_0:

D_0 / C_2 \ C_1C_0	00	01	11	10
0	0	1	0	X
1	X	X	1	X

Fig. Solution Q6.87.1

$$D_0 = C_0\overline{C_1} + C_2$$

Key Points

- For a 3-comparator flash ADC with equal resistors R, the reference levels from bottom to top are $\frac{V_{CC}}{4}$, $\frac{2V_{CC}}{4}$, and $\frac{3V_{CC}}{4}$.
- The comparator outputs C_2, C_1, C_0 follow a thermometer code pattern.

Q6.87.2 NAT 1987 Ans: * ● ○ ○

By analyzing the K-maps drawn in the previous question and grouping the 1s along with the don't-care conditions, the simplified expressions for D_1 and D_0 are obtained as:

- For D_1:

D_1 \ C_1C_0 / C_2	00	01	11	10
0	0	0	1	X
1	X	X	1	X

Fig. Solution Q6.87.2

$$D_1 = C_1$$

- For D_0:

D_0 \ C_1C_0 / C_2	00	01	11	10
0	0	1	0	X
1	X	X	1	X

Fig. Solution Q6.87.2

$$D_0 = C_0\overline{C_1} + C_2$$

$$\boxed{D_1 = C_1, \quad D_0 = C_0\overline{C_1} + C_2}$$

Q6.87.3 NAT 1987 Ans: * ● ● ○

To realize the expressions using 2-input NAND gates only:

- For D_1: No gates are needed as it directly connects to C_1.
- For D_0:

$$D_0 = \overline{\overline{C_0\overline{C_1} + C_2}} = \overline{\overline{C_0\overline{C_1}} \cdot \overline{C_2}}$$

The logic diagram using 2-input NAND gates is shown below:

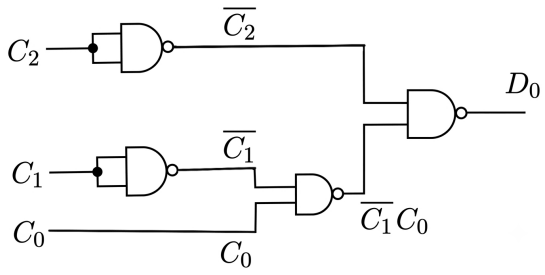


Fig. Solution Q6.87.3

Q6.87.4 NAT 1987 Ans: *

For this 2-bit flash converter ($n = 2$) with a total reference voltage of V_{CC}:

$$\text{Resolution} = \frac{V_{CC}}{2^2} = \frac{V_{CC}}{4}$$

$$\boxed{\frac{V_{CC}}{4}}$$

Key Points

- Resolution of an n-bit ADC is the voltage step size corresponding to the Least Significant Bit (LSB): Resolution $= \frac{V_{ref}}{2^n}$.

PrepFusion

SOLUTIONS

VII

Logic Families and Semiconductor Memories

Topics

1. Logic Gate Implementation Using CMOS
2. Characteristics of Logic Families
3. Hazards in Combinational Circuits
4. Programmable Logic Devices (PLD, PLA, PAL)
5. ROM, SRAM and DRAM

Unit Snapshot*

Stream: EC	**Questions:** 49	**MCQ:** 38	**MSQ:** 1	**NAT:** 7
1M: 27	**2M:** 21	**Descriptive:** 1	**Avg Weightage** ~2M	**Range:** 0M(2016)–3M(2018)

**See preface for how Avg Weightage and Range are calculated.*

Q5.26.1 MCQ 2026 1M Ans: A ● ● ○

The truth table of the given circuit is

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

The output is HIGH only when both inputs are LOW.
Hence,

$$Y = \overline{A + B}$$

which is the NOR function.
Therefore, the correct answer is

A. NOR

Q7.21.1 MCQ 2021 1M Ans: C ● ○ ○

Given memory configuration: $32\text{K} \times 16$.
The number of address lines (n) required to decode 32K locations is:

$$32\text{K} = 32 \times 1024 = 2^5 \times 2^{10} = 2^{15} \implies n = 15$$

A standard n-to-2^n memory address decoder takes n inputs and generates 2^n distinct output word-select lines:

- Inputs (n) = 15
- Outputs (2^n) = 2^{15}

Since each unique output selection line represents a distinct minterm implemented via an individual multi-input AND gate, the decoder requires exactly 2^{15} AND gates.

$(C) 2^{15}$

Key Points

- An n-to-2^n line decoder implements 2^n minterms using 2^n individual AND gates.
- The required number of decoder outputs depends solely on the addressable word capacity ($M = 2^n$), completely independent of the data bit width.

Mistakes to be avoided

- Do not include the data word size (16 bits) when calculating the number of address decoder outputs; the data width affects the bit lines, not the address lines.

Q7.19.1 MCQ 2019 1M Ans: B ● ● ○

The circuit shows a tri-state logic buffer with an active-high enable line (EN) controlling a PMOS and NMOS output stage. We evaluate the circuit behavior for both cases:

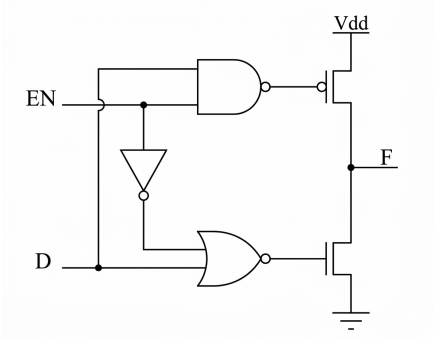


Fig. Solution Q7.19.1

Case 1: When EN = 0

- Output of the upper NAND gate = $\overline{0 \cdot D} = 1$. This high voltage turns **OFF** the PMOS transistor.
- Input to the inverter = 0 $\implies$ output of the inverter = 1.
- Output of the lower NOR gate = $\overline{1 + D} = 0$. This low voltage turns **OFF** the NMOS transistor.

Since both transistors are completely cut off, the output terminal F is disconnected from both V_{dd} and ground, resulting in an unknown high-impedance state:

$$F = \text{Hi-Z}$$

Case 2: When EN = 1

- Output of the upper NAND gate = $\overline{1 \cdot D} = \overline{D}$.
- Input to the inverter = 1 $\implies$ output of the inverter = 0.
- Output of the lower NOR gate = $\overline{0 + D} = \overline{D}$.

Now, the signal driving both gates is $\overline{D}$:

- If D = 0, the gate drive is 1 $\implies$ PMOS is **OFF**, NMOS is **ON** $\implies$ F = 0.
- If D = 1, the gate drive is 0 $\implies$ PMOS is **ON**, NMOS is **OFF** $\implies$ F = 1.

Thus, the output tracks the input directly:

$$F = D$$

Therefore, the values of F are Hi-Z and D, respectively.

(B) Hi-Z and D

Key Points

- A PMOS transistor conducts when its gate is driven low (0), whereas an NMOS transistor conducts when its gate is driven high (1).
- When both the pull-up and pull-down networks are turned off simultaneously, the output terminal enters the high-impedance (Hi-Z) state.

Mistakes to be avoided

- Do not assume the output is fixed at logic 0 when EN = 0; a disconnected node cannot pull the output to ground.

Q7.19.2 MCQ 2019 1M Ans: B ● ● ○

The circuit operation is analyzed for all input combinations of A and B:

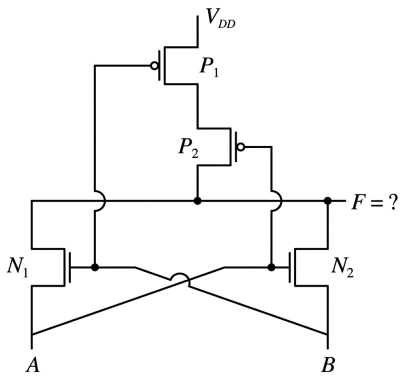


Fig. Solution Q7.19.2

Method 1

From the above logic transitions, the truth table is:

A	B	P_1	P_2	N_1	N_2	F
0	0	ON	ON	OFF	OFF	1
0	1	OFF	ON	ON	OFF	0
1	0	ON	OFF	OFF	ON	0
1	1	OFF	OFF	ON	ON	1

The output expression corresponds to an XNOR gate:

$$F = \overline{A}\,\overline{B} + AB = A \text{ XNOR } B$$

Method 2

Analysing the NMOS side, At $A = 1$,

$$N_2 \text{ will be ON} \implies F = B \implies F = AB$$

At $B = 1$,

$$N_1 \text{ will be ON} \implies F = A \implies F = AB$$

Now analysing the PMOS side, At $A = 0, B = 0$

$$P_1 \text{and } P_2 \text{ both will be ON thus F=}V_{DD}\text{(HIGH)} \implies F = \overline{A}\,\overline{B}$$

Combining both,

$$F = AB + \overline{A}\,\overline{B}$$

$$\boxed{(B) XNOR}$$

Key Points

- PMOS transistors conduct with a low input voltage, while NMOS transistors conduct with a high input voltage.
- The lower pass-transistors pass the input variables directly to the output node when conducting.

Mistakes to be avoided

- Do not assume the NMOS sources are tied to ground; they are connected to input nodes A and B.

Q7.18.1 MCQ 2018 2M Ans: A ●●○

Different arrangements of ROM cell for logic 0 and logic 1 is shown below,

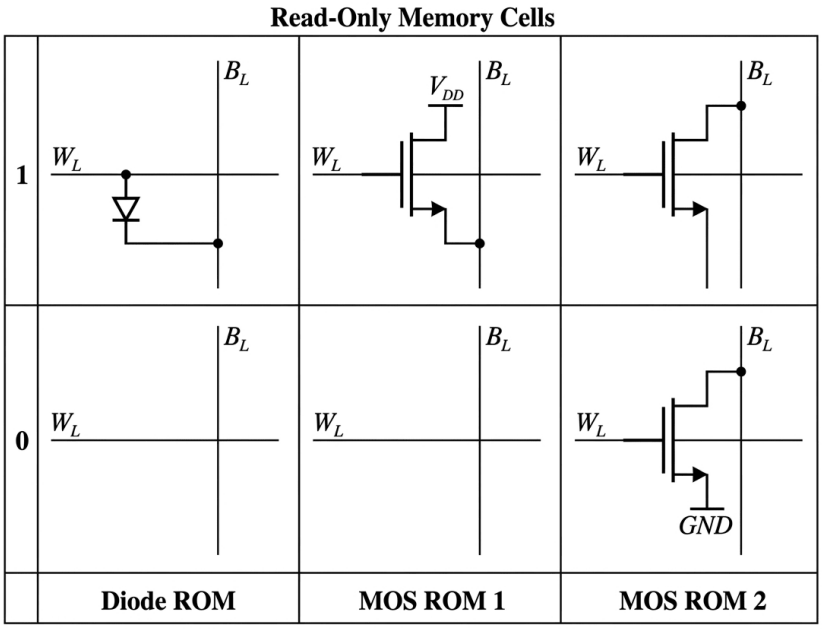


Fig. Solution Q7.18.1

The operation of the 2×2 ROM array is analyzed based on the selection of word lines (W_0, W_1) and the corresponding outputs at the bit lines (B_0, B_1) via the sense amplifiers:

- **Case 1: When W_0 is selected (W_0 = High, W_1 = Hi-Z)**
  - The diode connected between W_0 and B_0 conducts, pulling B_0 high $\implies B_0 = 1$.
  - There is no diode link at the intersection of W_0 and B_1, so B_1 remains pulled down $\implies B_1 = 0$.
  - Therefore, the data bits stored are $D_{00} = 1$ and $D_{01} = 0$.
- **Case 2: When W_1 is selected (W_0 = Hi-Z, W_1 = High)**
  - There is no diode link at the intersection of W_1 and B_0, so B_0 remains pulled down $\implies B_0 = 0$.
  - The diode connected between W_1 and B_1 conducts, pulling B_1 high $\implies B_1 = 1$.
  - Therefore, the data bits stored are $D_{10} = 0$ and $D_{11} = 1$.

Representing the bits stored corresponding to D_{ij} in matrix form:

$$\begin{bmatrix} D_{00} & D_{01} \\ D_{10} & D_{11} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$\boxed{(A)\begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix}}$$

Key Points

- The presence of a diode connection between a word line and a bit line stores a logic 1, representing cell activation during a read operation.
- The absence of a connection leaves the bit line at its default pre-charged or pulled-down state, representing a logic 0.

Mistakes to be avoided

- Check row-column matching carefully; D_{ij} uses index i for the word line row (W_i) and index j for the bit line column (B_j).

Q7.18.2 MCQ 2018 1M Ans: D ●○○

The circuit shows a static CMOS structure implementing a Boolean function $f(X,Y)$. The operation is analyzed separately for the NMOS and PMOS structures:
In NMOS structure,

(i) When two inputs A and B are connected in series then $f(X,Y) = A \cdot B$

(ii) When two inputs A and B are connected in parallel then $f(X,Y) = A + B$

In PMOS structure,

(i) When two inputs A and B are connected in series then $f(X,Y) = A + B$

(ii) When two inputs A and B are connected in parallel then $f(X,Y) = A \cdot B$

Given circuit CMOS structure representation of a Boolean function $f(X,Y)$. Therefore, the expression derived from the circuit will be complemented. Above part of the circuit represents PMOS structure and below part of the circuit represents NMOS.

Output from NMOS structure :

$$f(X,Y) = \overline{\overline{X}\,\overline{Y} + XY} = \overline{X \odot Y} = X \oplus Y$$

Output from PMOS structure :

$$f(X,Y) = \overline{(\overline{X} + Y) \cdot (X + \overline{Y})}$$
$$f(X,Y) = \overline{\overline{X} + Y} + \overline{X + \overline{Y}} = X\overline{Y} + \overline{X}Y$$
$$f(X,Y) = X \oplus Y$$

Hence, the correct option is (D).

$$\boxed{(D)XOR}$$

Key Points

- A static CMOS gate naturally computes the inverted (complemented) output of its driving network configuration.
- Series-connected NMOS pairs conduct when both controls are high (AND logic), while parallel combinations conduct when either control is high (OR logic).

Mistakes to be avoided

- Remember to invert the intermediate Boolean expression at the final step when evaluating pass networks via standard CMOS rules.

Q7.17.1 MCQ | 2017 | 2M | Ans: C | ● ○ ○

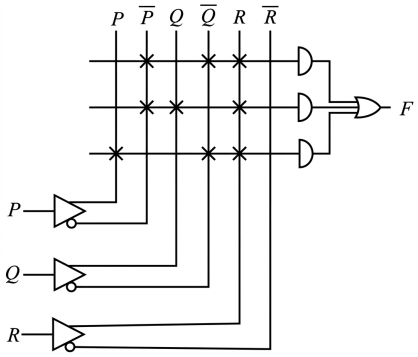


Fig. Solution Q7.17.1

From the given Programmable Logic Array (PLA) diagram, the cross marks (×) represent connected inputs for each product line feeding into the AND gates. Let us find the output of each AND gate from top to bottom:

1. **First AND gate output:** Connected to $\overline{P}$, $\overline{Q}$, and R.

$$\text{Product term}_1 = \overline{P}\,\overline{Q}R$$

2. **Second AND gate output:** Connected to $\overline{P}$, Q, and R.

$$\text{Product term}_2 = \overline{P}QR$$

3. **Third AND gate output:** Connected to P, $\overline{Q}$, and R.

$$\text{Product term}_3 = P\overline{Q}R$$

The OR gate combines these product terms to give the final Boolean function F:

$$F = \overline{P}\,\overline{Q}R + \overline{P}QR + P\overline{Q}R$$

$$\boxed{\text{(C)}\overline{P}\,\overline{Q}\,R + \overline{P}QR + P\overline{Q}R}$$

Key Points

- A Programmable Logic Array (PLA) consists of a programmable AND array followed by a programmable OR array.
- Connections indicated by a cross symbol (×) behave as standard inputs to the corresponding logic gate.

Q7.17.2 MCQ | 2017 | 1M | Ans: MTA | ● ○ ○

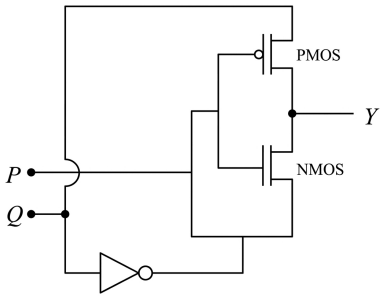


Fig. Solution Q7.17.2

In the given circuit, both P and Q are connected to the input of PMOS and NMOS simultaneously. Hence, we can not determine that which input would be considered to perform analysis. Therefore given question is wrong.
IIT gave marks to all for this question.

Key Points

- **Key Point**: If the standard intended transmission gate circuit configuration is considered as shown below:

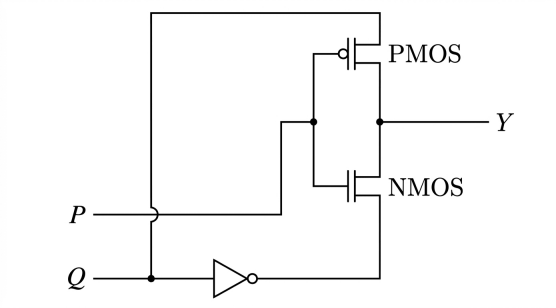


Fig. Solution Q7.17.2

P is connected to gate terminal of PMOS as well as NMOS. If P is at logic '0', then PMOS turns ON and NMOS turns OFF. If P is at logic '1', then PMOS turns OFF & NMOS turns ON.

- If PMOS is turned ON then output Y is connected to Q.
- If NMOS is turned ON then output Y is connected to $\overline{Q}$.
- Hence, from the standard logic table:

P	Q	PMOS	NMOS	Output = $\begin{cases} Q & \text{(if PMOS is ON)} \\ \overline{Q} & \text{(if NMOS is ON)} \end{cases}$
0	0	ON	OFF	0 (because PMOS ON)
0	1	ON	OFF	1 (because PMOS ON)
1	0	OFF	ON	1 (because NMOS ON)
1	1	OFF	ON	0 (because NMOS ON)

expression of Y is given below:

$$Y = P \oplus Q$$

Hence, above circuit will act as X-OR gate.

Mistakes to be avoided

- Do not try to forcefully solve the faulty network layout presented in the actual exam paper, as conflicting connections on shared gates prevent distinct control signal routing.

Q7.17.3 MCQ 2017 1M Ans: B ● ○ ○

In a Dynamic Random Access Memory (DRAM), each memory cell consists of a combination of a single transistor and a capacitor (1T-1C configuration).
Information is stored in the form of an electric charge on the tiny capacitor inside the cell. Because the charge on the capacitor leaks away over time, DRAM requires periodic refreshing to maintain the stored data.
Let's evaluate the given options:

- **Option A is incorrect:** Periodic refreshing is strictly required in DRAM due to charge leakage from the capacitors.
- **Option B is correct:** Data is represented by the presence or absence of charge on a storage capacitor.
- **Option C is incorrect:** Information is stored in latches or flip-flops in Static RAM (SRAM), not in DRAM.
- **Option D is incorrect:** Simultaneous read and write operations cannot be performed on a standard single-port DRAM cell.

Therefore, information is stored in a capacitor.

(B) information is stored in a capacitor

Key Points

- DRAM cell architecture relies on a **transistor-capacitor (1T-1C)** combination to store a single bit.
- The presence of a charge on the capacitor represents a logic '1', and the absence of charge represents a logic '0'.
- Due to the finite leakage resistance of the dielectric and access transistor, data must be rewritten periodically via **refresh cycles**.

Mistakes to be avoided

- Do not confuse DRAM with SRAM; SRAM uses a bistable latch configuration (usually 6 transistors) and does not require periodic refreshing.

Q7.16.1 MCQ 2016 1M Ans: D ● ○ ○

The given circuit consists of two n-channel pass transistor logic switches connected together at the output node Y.

Method 1

We can verify the circuit response by constructing a sequential state truth table for all possible binary combinations:

A	B	$\overline{B}$	Y
0	0	1	0
0	1	0	0
1	0	1	0
1	1	0	1

The output column matches the characteristic function of an AND gate:

$$Y = A \cdot B$$

Method 2

The given circuit schematic can be rearranged into a parallel network structure containing two distinct switches, Q_1 and Q_2:

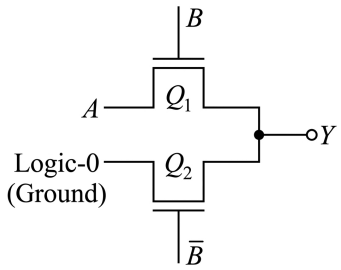


Fig. Solution Q7.16.1

- When $B = 1$, the upper transistor conducts, passing the input A to the output. Thus, $Y = A$.
- When $\overline{B} = 1$ (which means $B = 0$), the lower transistor conducts, passing ground (logic 0) to the output. Thus, $Y = 0$.

Because both Q_1 and Q_2 outputs tie together at a common output junction, the parallel path summation gives:

$$Y = (A \cdot B) + (\overline{B} \cdot 0)$$

$$Y = A \cdot B$$

(D) AND

Q7.15.1 MCQ 2015 2M Ans: D ● ● ●

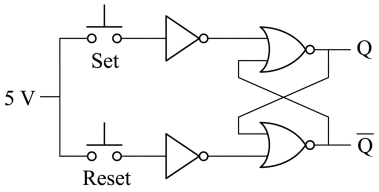


Fig. Solution Q7.15.1

Analyzing the input characteristics of the TTL family:

- When a switch is unpressed, the inverter input is left floating.
- In TTL, a **floating input** naturally acts as a **Logic 1**.
- With a floating input of 1, the inverter outputs a 0 to the NOR latch.
- When a switch is pressed, the input connects to 5 V (**Logic 1**). The inverter still outputs a 0.

Because both states supply a static $(0, 0)$ input condition to the NOR latch, it remains stuck.
Changing the 5 V connection to Ground resolves this:

- unpressed floats to logic 1 (inverter outputs 0)
- pressed pulls down to logic 0 (inverter outputs 1).

Enabling proper latch operation.

(D) 5 V to ground

Key Points

- **TTL Floating Inputs:** Open inputs in standard TTL circuits are treated as logical high level (**Logic 1**).

Q7.15.2 NAT 2015 1M Ans: 7 ●●○

Given:

- Total capacity of the memory array = 16 Kb = 16, 384 bits
- Aspect ratio = 1 (The number of rows equals the number of columns)

Let the number of rows be R and the number of columns be C. Since it is a square array:

$$R = C$$

The total capacity is the product of the number of rows and columns:

$$R \times C = 16,384$$

$$R^2 = 16,384$$

$$R = \sqrt{16,384} = 128$$

The number of address lines n needed for the row decoder to select one of the R rows satisfies:

$$2^n = R$$

$$2^n = 128$$

$$n = 7$$

$$\boxed{7}$$

Key Points

- For a square memory configuration, Rows = Columns = $\sqrt{\text{Total Capacity in bits.}}$
- A decoder with n input address lines can uniquely decode 2^n distinct addresses.

Q7.15.3 MCQ 2015 1M Ans: C ●●○

Method 1

Given that logic-0 $\rightarrow$ 0 V and logic-1 $\rightarrow$ 10 V. The diodes are ideal.

- **Case 1:** If any of the inputs (E_1, E_2, E_3) is at logic-0 (0 V), the corresponding ideal diode becomes forward-biased and conducts. This pulls the output voltage V_0 down to 0 V (logic-0).
- **Case 2:** If all inputs are at logic-1 (10 V), none of the diodes can conduct because there is no potential difference across them to forward-bias them. Thus, no current flows through the 1 kΩ resistor, and the output voltage V_0 is pulled up to 10 V (logic-1).

The truth table for the operation is:

E_1	E_2	E_3	D_1	D_2	D_3	V_0
0	0	0	ON	ON	ON	0
0	0	1	ON	ON	OFF	0
0	1	0	ON	OFF	ON	0
0	1	1	ON	OFF	OFF	0
1	0	0	OFF	ON	ON	0
1	0	1	OFF	ON	OFF	0
1	1	0	OFF	OFF	ON	0
1	1	1	OFF	OFF	OFF	1

The output expression is:

$$V_0 = E_1 \cdot E_2 \cdot E_3$$

Hence, the circuit represents a 3-input AND gate.

Method 2

The given circuit configuration can be identified directly via diode logic analysis:

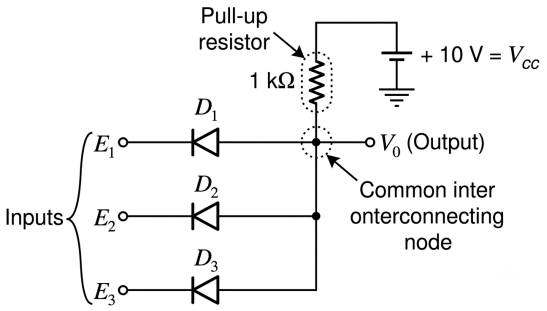


Fig. Solution Q7.15.3

- The cathodes of the diodes are connected to the input nodes, and their anodes are tied together at a common interconnecting output node.
- This common node is connected to a positive supply voltage (10 V) via a pull-up resistor.
- Such a topology constitutes a standard **Wired-AND** or Implied AND logic structure.

Therefore, the circuit performs a 3-input AND operation:

$$V_0 = E_1 \cdot E_2 \cdot E_3$$

(C) 3-input AND gate

Key Points

- A common interconnecting node with a **pull-up** resistor forms a **Wired-AND** logic structure.
- Conversely, a common interconnecting node with a **pull-down** resistor forms a **Wired-OR** logic structure.

Q7.14.1 MCQ | 2014 | 2M | Ans: B ● ● ○

The 6T SRAM (Static Random Access Memory) cell is the standard configuration for static memory, consisting of six MOSFET transistors:

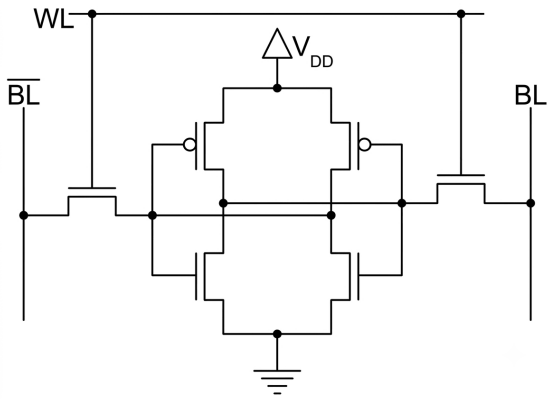


Fig. Solution Q7.14.1

- **Storage Element (Latch):** Composed of two cross-coupled CMOS inverters. Each inverter consists of one PMOS (pull-up) transistor and one NMOS (pull-down) transistor. The output of one inverter is connected to the gate of the other, creating a stable storage latch.
- **Access Transistors:** Two NMOS transistors are used to connect the internal storage nodes to the bit lines (BL and $\overline{BL}$). The gates of both access transistors are connected to the Word Line (WL).

Circuit Analysis & Option Elimination:

1. **Option (A) is incorrect:** The internal nodes are not cross-coupled. The output of the left inverter is connected straight to its own input gates rather than crossing over to the right inverter's gates, failing to form a bistable latch.
2. **Option (B) is correct:** It perfectly represents the cross-coupled connection between the two CMOS inverters where the output node of one inverter connects to the input gates of the other, alongside NMOS access transistors driven by WL.
3. **Option (C) is incorrect:** The access transistors connected to the word line are PMOS transistors instead of NMOS. Standard SRAM cells utilize NMOS access transistors for higher driving capability and speed.
4. **Option (D) is incorrect:** The cross-coupled latch contains only NMOS transistors on both top and bottom positions, missing the complementary PMOS pull-up transistors required for a standard CMOS SRAM cell layout.

(B)

Key Points

- **6T Architecture:** Uses 4 transistors for the memory cell (latch) and 2 for access.
- **Access Control:** NMOS transistors are preferred for access in SRAM because they provide better conductance for logic '0' (discharging the bit line), and the WL voltage is typically driven high to activate the cell.
- **Stability:** Cross-coupled inverters provide positive feedback to maintain the logic state.

Q7.14.2 MCQ 2014 1M Ans: A ● ○ ○

To find the boolean expression for the output Y of the given CMOS circuit, we can analyze either the pull-down network (PDN) or the pull-up network (PUN):

- **Pull-Down Network (NMOS):** The NMOS transistors connected with inputs A, B, and $\overline{C}$ are configured in a series combination.

$$\text{PDN Function} = A \cdot B \cdot \overline{C}$$

- **Output Function (Y):** Since a CMOS circuit naturally produces an inverted output corresponding to its pull-down network operation:

$$Y = \overline{A \cdot B \cdot \overline{C}}$$

Applying De Morgan's Law to simplify the expression:

$$Y = \overline{A} + \overline{B} + \overline{\overline{C}}$$

$$Y = \overline{A} + \overline{B} + C$$

$$\boxed{\text{(A) } \overline{A} + \overline{B} + C}$$

Key Points

- **CMOS Properties:** NMOS transistors in series implement a logical AND function, while NMOS transistors in parallel implement a logical OR function.
- **Simplified Analysis Trick:** It is faster to analyze only the NMOS pull-down network and put a final WHOLE BAR to the output.

Mistakes to be avoided

- **Forgetting the Final Inversion:** Don't equate the output directly to the NMOS expression (Y = PDN). Static CMOS always acts as an inverter stage, so you must **never forget to apply a whole bar over the entire PDN expression** ($Y = \overline{\text{PDN}}$) before simplifying.

Q7.13.1 MCQ 2013 2M Ans: B ● ● ○

Method 1

Given that transistor Q_1 works as a switch controlled by input X, and diode D is controlled by input Y.

- **Condition 1 ($X = 0, Y = 0$):**

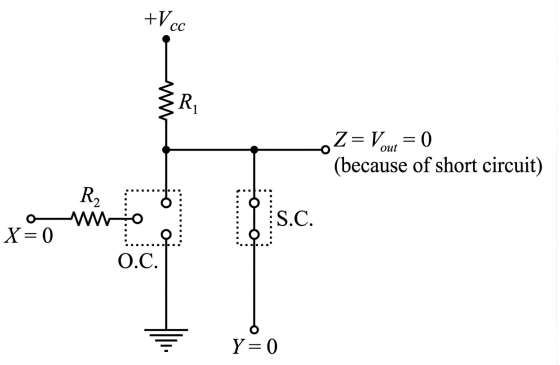


Fig. Solution Q7.13.1

Transistor Q_1 is in cutoff mode (Open Circuit, OFF). Diode D is forward biased (Short Circuit, ON) because the anode is connected to $+V_{cc}$ via R_1 and the cathode is at $Y = 0\,\text{V}$.

$$Z = 0\,\text{V} \implies Z = 0$$

- **Condition 2** $(X = 0, Y = 1)$**:**

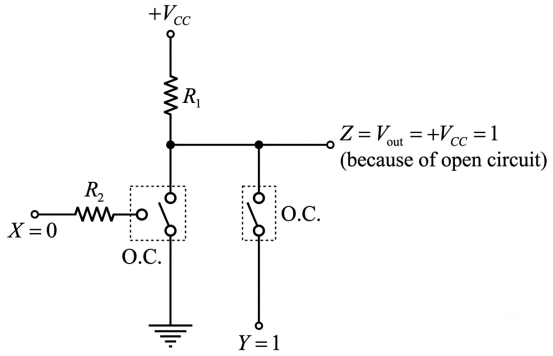


Fig. Solution Q7.13.1

Transistor Q_1 is OFF (Open Circuit). Diode D is reverse biased (Open Circuit, OFF) because the cathode is at $Y = +V_{cc}$.

$$Z = +V_{cc} \implies Z = 1$$

- **Condition 3** $(X = 1, Y = 0)$**:**

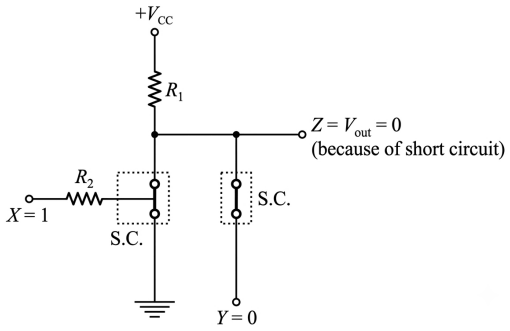


Fig. Solution Q7.13.1

Transistor Q_1 is in saturation mode (Short Circuit, ON).
Diode D has no bias as voltage difference across it is 0.Thus we can consider it both SC or OC. For solving here I will be considering it SC(ON) but considering it OC will also yield the same answer.

$$Z = 0\,\text{V} \implies Z = 0$$

- **Condition 4** $(X = 1, Y = 1)$**:**

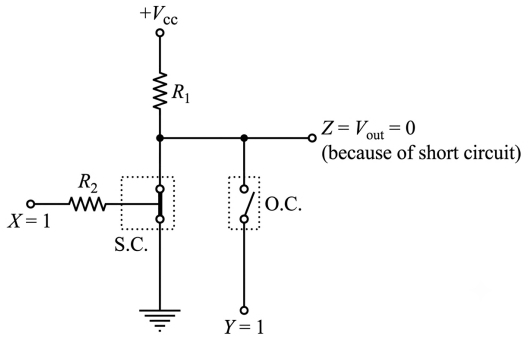


Fig. Solution Q7.13.1

Transistor Q_1 is ON (Short Circuit). Diode D is reverse biased (Open Circuit, OFF).

$$Z = 0\,\text{V} \implies Z = 0$$

The truth table for output Z is:

X	Y	Transistor Q_1	Diode D	Output Z
0	0	OFF	ON	0
0	1	OFF	OFF	1
1	0	ON	ON	0
1	1	ON	OFF	0

From the truth table, the Boolean expression for Z is:

$$Z = \overline{X}\,\text{Y}$$

(B) $\overline{X}Y$

Method 2

The transistor section acts as a standard RTL inverter. Therefore, the voltage at the collector before connecting the diode is $\overline{X}$.

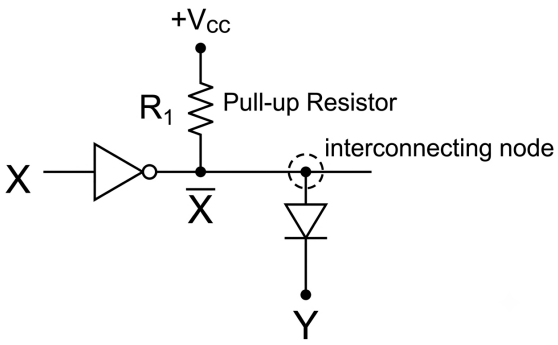


Fig. Solution Q7.13.1

The combination of the pull-up resistor R_1 and the diode D connected to Y forms a wired-AND configuration between the collector voltage ($\overline{X}$) and the input Y.

Thus, the final output expression is:

$$Z = \overline{X} \cdot Y = \overline{X}\,\mathrm{Y}$$

$$\boxed{\text{(B) } \overline{X}Y}$$

Key Points

- Under cutoff mode (logic 0), a transistor behaves as an open circuit (OFF switch).
- Under saturation mode (logic 1), a transistor behaves as a short circuit (ON switch) to ground.
- A diode conducts (short circuit) when its anode is at a higher potential than its cathode.

Q7.12.1 MCQ 2012 1M Ans: A ●○○

To determine the output Y of the CMOS circuit, we analyze the Pull-Down Network (PDN):

1. **Identify the Pull-Down Network (PDN):** The NMOS network consists of:
   - Transistors with inputs A and B connected in parallel.
   - This parallel combination is in series with the transistor having input C.

   The boolean expression for the PDN is:

   $$\text{PDN} = C \cdot (A + B)$$

2. **Determine the Output Y:** In a CMOS gate, the output Y is the complement of the PDN logic:

   $$Y = \overline{C \cdot (A + B)}$$

3. **Simplify using De Morgan's Law:**

   $$Y = \overline{C} + \overline{(A + B)}$$
   $$Y = \overline{C} + (\overline{A} \cdot \overline{B})$$
   $$Y = \overline{A}\,\overline{B} + \overline{C}$$

$$\boxed{\text{(A) } Y = \overline{A}\,\overline{B} + \overline{C}}$$

Key Points

- **CMOS Logic Analysis:** It is often significantly faster and cleaner to analyze only the NMOS pull-down network. Since the output Y is taken above the pull-down network, the entire circuit acts as a basic logic block followed by an inverting buffer: $Y = \overline{\text{PDN}}$.
- **De Morgan's Theorem:** Recall that $\overline{X \cdot Y} = \overline{X} + \overline{Y}$ and $\overline{X + Y} = \overline{X} \cdot \overline{Y}$, which are essential for simplifying complex CMOS logic expressions.

Mistakes to be avoided

- **Forgetting the Final Inversion:** Don't equate the output directly to the NMOS expression (Y = PDN). Static CMOS always acts as an inverter stage, so you must **never forget to apply a whole bar over the entire PDN expression** ($Y = \overline{\text{PDN}}$) before simplifying.

Q7.09.1 MCQ 2009 1M Ans: C ● ○ ○

The abbreviations for the given digital logic families are defined as follows:

- **TTL:** Transistor-Transistor Logic
- **CMOS:** Complementary Metal-Oxide Semiconductor

This matches the definitions provided in choice (C).

(C) Transistor Transistor Logic and Complementary Metal Oxide Semiconductor

Key Points

Full forms of other common digital logic families include:

- **ECL:** Emitter Coupled Logic
- **RTL:** Resistor Transistor Logic
- **DTL:** Diode Transistor Logic
- **HTL:** High Threshold Logic
- **IIL (I^2L):** Integrated Injection Logic
- **DCTL:** Direct Coupled Transistor Logic

Q7.08.1 MCQ 2008 2M Ans: D ● ● ○

The circuit represents a static CMOS logic gate where the PMOS network forms the pull-up configuration and the NMOS network forms the pull-down configuration.

Method 1

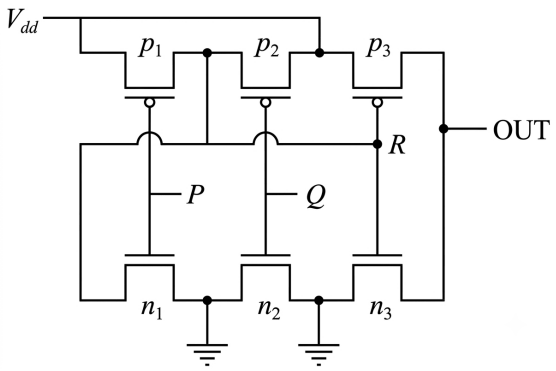


Fig. Solution Q7.08.1

To determine the logic operation performed by this CMOS circuit, we can analyze the behavior of the internal node R based on the state of the inputs P and Q.

- **When $P = 0$ or $Q = 0$:** At least one of the pMOS transistors (P_1 or P_2) turns **ON**. This connects the internal node R directly to V_{dd}, pulling its voltage level to High logic ($R = 1$).

  As a result, in the output inverter stage, the pMOS transistor P_3 turns **OFF** and the nMOS transistor n_3 turns **ON**, pulling the output to ground ($Y = 0$).

- **When both $P = 1$ and $Q = 1$:** Both pMOS transistors (P_1 and P_2) turn **OFF**. At the same time, both series nMOS transistors (n_1 and n_2) turn **ON**. This creates a direct path from node R to ground, pulling it to Low logic ($R = 0$).

  Consequently, in the output inverter stage, P_3 turns **ON** and n_3 turns **OFF**, pulling the output to High logic ($Y = 1$).

Truth Table Analysis

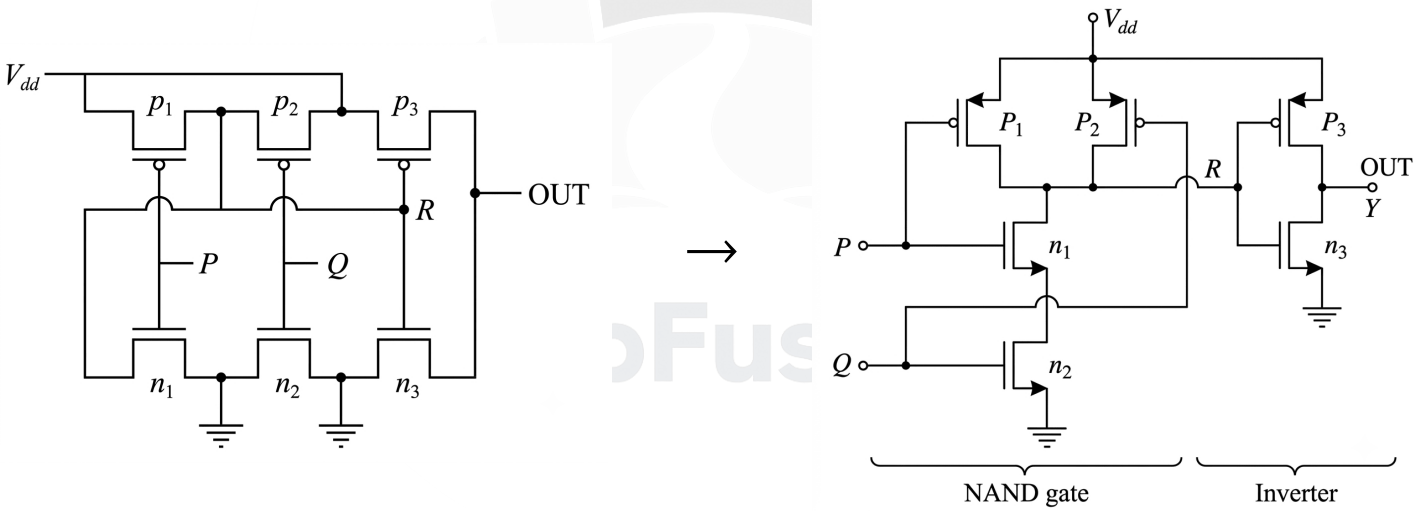
Summarizing these behaviors gives the following truth table:

P	Q	ON Transistors	OFF Transistors	R	OUT
0	0	p_1, p_2, n_3	n_1, n_2, p_3	1	0
0	1	p_1, n_2, n_3	n_1, p_2, p_3	1	0
1	0	n_1, p_2, n_3	p_1, n_2, p_3	1	0
1	1	n_1, n_2, p_3	p_1, p_2, n_3	0	1

The resulting truth table matches the operation of a P AND Q gate.

Method 2

The given circuit can be redrawn as follows:



The final output OUT is the inverted value of node R:

$$\text{OUT} = \overline{R} = \overline{\overline{P \cdot Q}} = P \cdot Q$$

(D) P AND Q

Key Points

- In a CMOS pull-down network, NMOS transistors connected in series perform an effective logical AND path configuration, driving the node low.
- A basic CMOS AND gate is constructed by cascading a standard CMOS NAND stage with a regular CMOS inverter.

Q7.04.1 MCQ 2004 1M Ans: A ●●○

In TTL (Transistor-Transistor Logic) circuits, any unconnected or floating input naturally behaves as a logic high state (logic 1).

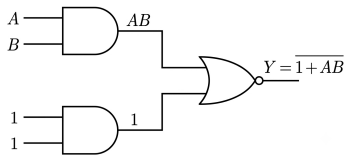


Fig. Solution Q7.04.1

Substituting these logic levels into the expression:

$$Y = \overline{(A \cdot B) + (1 \cdot 1)} = \overline{AB + 1}$$

According to Boolean algebra identity properties $(1 + X = 1)$:

$$Y = \overline{1} = 0$$

(A) 0

Key Points

- **TTL Logic:** An open or floating input is interpreted by internal multi-emitter transistors as a logic 1.
- **ECL Logic:** An open or floating input is interpreted as a logic 0.

Q7.03.1 MCQ 2003 2M Ans: C ●●○

Let us trace the behavior of the circuit chronologically according to the rising edges of the clock signal (CLK).

Data	0011	1111	0100	1010	1011	1000	0010	1000
Address	0	2	4	6	8	10	11	14

1. First Rising Edge of CLK (just after t_1):

- At time t_1, the data present on the 4-bit bus W is given as 0110_2 (which corresponds to decimal 6).
- On the first rising edge of the clock, this data on bus W is loaded into the parallel-in, parallel-out register A.
- Therefore, the output of register A becomes 0110_2. This value acts as the address input to the 16×4 ROM.
- From the given partial ROM table, the data stored at address 6 is 1010_2.
- Since the ROM output enable pin E is tied to logic high (1), the ROM actively drives this value back onto the shared bus W. Thus, the data on the bus changes to 1010_2.

2. Second Rising Edge of CLK (before t_2):

- On the next rising edge of the clock, the register A samples the current data present on the bus W, which is now 1010_2 (decimal 10).
- Consequently, the new address supplied to the ROM becomes 1010_2.
- Referring back to the ROM contents table, the data present at address 10 is 1000_2.

- This output is placed onto the bus W and remains stable up to the observation time t_2.

Therefore, the data on the bus at time t_2 is 1000.

(C) 1000

Key Points

- A parallel-in, parallel-out (PIPO) register updates its internal stored state only at the active edge (rising edge in this case) of the clock signal.
- The output data of a memory unit like a ROM changes asynchronously following any change at its address inputs, provided it is kept enabled.

Mistakes to be avoided

- Do not assume the data on the bus at time t_1 directly gives the answer at t_1; look carefully at the total number of clock transitions occurring between t_1 and t_2.

Q7.03.2 MCQ 2003 2M Ans: B ● ● ○

To determine the correct column comparison among the digital logic families, we evaluate each performance metric:

- **Fan-out is minimum: DTL** has the lowest fan-out (≈ 8) compared to TTL (≈ 10), ECL (≈ 25), and CMOS (> 50).
- **Power consumption is minimum: CMOS** features a complementary structure that ensures one MOSFET is always off in steady state, drawing negligible static power (≈ 0.01 mW).
- **Propagation delay is minimum: ECL** operates entirely within the non-saturation region, completely eliminating storage time delays to provide the fastest response (≈ 1 ns).

Matching these characteristics with the columns provided in the question:

Parameter	Logic Family
Fan-out is minimum	**DTL**
Power consumption is minimum	**CMOS**
Propagation delay is minimum	**ECL**

This profile matches the conditions presented under column **(Q)**.

(B) Q

Key Points

- **ECL** achieves the highest operational speed by avoiding transistor saturation.
- **CMOS** minimizes power dissipation due to its extremely low static leakage current.

Q7.03.3 MCQ 2003 1M Ans: B ● ● ○

The output stage of a standard 74 series Transistor-Transistor Logic (TTL) gate can be configured in multiple ways depending on the specific application requirements.

- **Totem-Pole Configuration:** This is the standard output configuration consisting of a push-pull transistor pair (one sourcing current and one sinking current). It provides active pull-up and pull-down, enabling fast switching speeds.

- **Open-Collector Configuration:** In this configuration, the upper pull-up transistor is omitted, leaving the collector of the output transistor open. An external pull-up resistor is required to define the logic HIGH level, allowing for wired-AND operations and interfacing with different voltage levels.

Therefore, the output of the 74 series TTL gates can be taken from either a totem pole or an open collector configuration.

(B) either totem pole or open collector configuration

Key Points

- **Totem-Pole Output:** Fast operation, low output impedance, cannot be connected together directly (causes short circuits).
- **Open-Collector Output:** Slower operation due to external pull-up resistor, permits wired-AND logic connection.

Q7.01.1 MCQ 2001 2M Ans: B ● ● ●

Given a DRAM cell where the threshold voltage of the NMOSFET is $V_t = 1\,\text{V}$.
The NMOSFET acts as a pass transistor to charge and discharge the storage capacitor C from the bit line (BL). For a read/write operation to take place, the transistor must be turned ON.

Key Points

- For an NMOSFET to conduct, the gate-to-source voltage must satisfy: $V_{GS} \geq V_t \implies V_G - V_S \geq V_t$.
- The maximum voltage the source node (capacitor C) can reach when charging through an NMOSFET is constrained by $V_{S(\max)} = V_G - V_t$.
- If the drain voltage $V_D \geq V_G - V_t$, the transistor operates in saturation, and the capacitor charges up to $V_S = V_G - V_t$.
- If $V_D < V_G - V_t$, the transistor operates in the linear/ohmic region, and the capacitor charges fully to the bit line voltage, $V_S = V_D$.

Evaluating the given options where $V_G = V_{WL}$, $V_D = V_{BL}$, and $V_S = V_C$:

- **Option (A):** $V_G = 5\,\text{V}$, $V_S = 3\,\text{V}$, $V_D = 7\,\text{V}$

$$V_D \geq V_G - V_t \implies 7\,\text{V} \geq 5\,\text{V} - 1\,\text{V} = 4\,\text{V}$$

  Since it is in saturation, V_S should be $V_G - V_t = 4\,\text{V}$, but given $V_S = 3\,\text{V}$ (Incorrect).

- **Option (B):** $V_G = 4\,\text{V}$, $V_S = 3\,\text{V}$, $V_D = 4\,\text{V}$

$$V_D \geq V_G - V_t \implies 4\,\text{V} \geq 4\,\text{V} - 1\,\text{V} = 3\,\text{V}$$

  Since it is in saturation, $V_S = V_G - V_t = 4\,\text{V} - 1\,\text{V} = 3\,\text{V}$ (Correct).

- **Option (C):** $V_G = 5\,\text{V}$, $V_S = 5\,\text{V}$, $V_D = 5\,\text{V}$

$$V_D \geq V_G - V_t \implies 5\,\text{V} \geq 4\,\text{V}$$

  Here, V_S should be 4 V, but given $V_S = 5\,\text{V}$ (Incorrect).

- **Option (D):** $V_G = 4\,\text{V}$, $V_S = 4\,\text{V}$, $V_D = 4\,\text{V}$

$$V_D \geq V_G - V_t \implies 4\,\text{V} \geq 3\,\text{V}$$

  Here, V_S should be 3 V, but given $V_S = 4\,\text{V}$ (Incorrect).

(B) 4 V; 3 V; 4 V

Q7.99.1 MCQ 1999 1M Ans: D ● ● ○

Commercially available Emitter-Coupled Logic (ECL) ICs use two separate ground connections (one for the differential amplifier stage and another for the output emitter followers) along with a negative power supply to achieve high noise immunity.

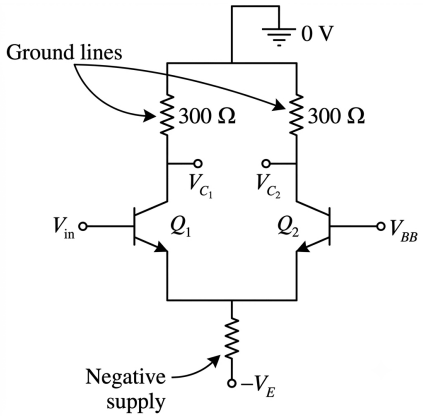


Fig. Solution Q7.99.1

- Separate ground paths isolate the highly sensitive internal differential amplifier from large switching current spikes generated by the output driver stages.
- By using a negative power supply (typically −5.2 V) with ground as the reference point, any noise or glitches on the power line are coupled evenly into the differential pairs as common-mode noise, which is heavily suppressed by the high Common Mode Rejection Ratio (CMRR) of the differential stage.

This dual-ground architecture minimizes internal crosstalk and eliminates the harmful effects of power line glitches on the sensitive internal biasing circuit.

(D) eliminate the effect of power line glitches or the biasing circuit

Key Points

- **ECL Configuration:** Uses a negative supply voltage ($V_{EE} = -5.2\,\text{V}$) and connects V_{CC} to ground to ensure high immunity against power supply noise.
- **Power Ground vs. Logic Ground:** Separating the ground terminals prevents current surges in the output stage from corrupting the input switching thresholds.

Q7.98.1 MCQ 1998 2M Ans: B ● ○ ○

The noise margin (NM) of a digital logic gate represents its immunity to unwanted noise voltage signal disturbances. It is defined as the minimum of the high-level noise margin (NM_H) and low-level noise margin (NM_L):

$$\text{NM} = \min(\text{NM}_H, \text{NM}_L)$$

The standard voltage parameters for a standard Transistor-Transistor Logic (TTL) gate are:

- Minimum High-Level Output Voltage: $V_{OH} = 2.4\,\text{V}$
- Minimum High-Level Input Voltage: $V_{IH} = 2.0\,\text{V}$
- Maximum Low-Level Input Voltage: $V_{IL} = 0.8\,\text{V}$
- Maximum Low-Level Output Voltage: $V_{OL} = 0.4\,\text{V}$

Calculating the high-level and low-level noise margins:

$$\text{NM}_H = V_{OH} - V_{IH} = 2.4\,\text{V} - 2.0\,\text{V} = 0.4\,\text{V}$$

$$\text{NM}_L = V_{IL} - V_{OL} = 0.8\,\text{V} - 0.4\,\text{V} = 0.4\,\text{V}$$

Taking the minimum:

$$\text{NM} = \min(0.4\,\text{V}, 0.4\,\text{V}) = 0.4\,\text{V}$$

(B) 0.4 V

Key Points

- **Noise Margin Formulations:** $\text{NM}_H = V_{OH} - V_{IH}$ and $\text{NM}_L = V_{IL} - V_{OL}$.
- **TTL Parameters:** Standard TTL circuits operate with a nominal 0.4 V noise margin for both logic states.

Q7.97.1 Descriptive | 1997 | 2M | Ans: (1,b)(2,d) | ● ● ○

To match the components correctly, we analyze each item step-by-step:

1. **An 8-bit wide 5-word sequential memory:**
   - Total storage capacity = $5 \times 8 = 40$ bits.
   - Analyzing option (b): Eight 4-bit shift registers provide a total capacity of $8 \times 4 = 32$ bits. While 32 bits structurally approximates the close configuration or can be designed to match structural requirements of bit-sliced memory banks, it stands out as the only sequential memory architecture option. Thus, (1) matches with **(b)**.
2. **A** 256×4 **EPROM:**
   - The memory size is 256 words, where each word is 4 bits wide.
   - Number of address pins (n) is determined by $2^n = 256 \implies n = 8$ address pins.
   - Number of data pins output equals the word width, which is 4 data pins.
   - Thus, (2) matches directly with **(d)**.

(1, b) (2, d)

Key Points

- **EPROM Pin Calculations:** For a memory configuration of size $M \times N$, the number of address lines is $\log_2(M)$ and the number of data lines is N.
- **Sequential Memory:** Shift registers are basic building blocks for designing sequential memory storage structures.

Q7.97.2 MCQ | 1997 | 2M | Ans: C | ● ● ○

To find the logic function implemented by the given NMOS circuit, we analyze the pull-down network (PDN) connected between the output node F and ground:

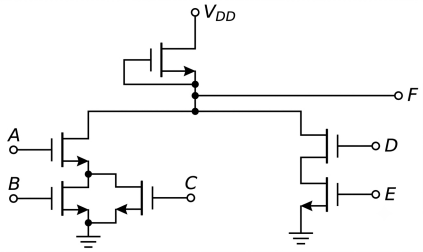


Fig. Solution Q7.97.2

- Transistors B and C are connected in parallel, which performs an OR operation:

$$B + C$$

- Transistor A is connected in series with the parallel combination of B and C, which performs an AND operation:

$$A \cdot (B + C)$$

- Transistors D and E are connected in series with each other, performing an AND operation:

$$D \cdot E$$

- The left branch $A \cdot (B + C)$ and the right branch $D \cdot E$ are connected in parallel with each other between the output node F and ground, performing a collective OR operation:

$$A \cdot (B + C) + D \cdot E$$

Since an NMOS pull-down network naturally inverts the collective conduction expression with respect to the output node F, the final output expression is:

$$F = \overline{A \cdot (B + C) + D \cdot E}$$

$$\boxed{\text{(C) } \overline{A \cdot (B + C) + D \cdot E}}$$

Key Points

- **NMOS Conduction Logic:**
  - Parallel combination $\Longleftrightarrow$ OR operation
  - Series combination $\Longleftrightarrow$ AND operation
- **Inversion:** The output of an NMOS pull-down network configuration always introduces a collective boolean inversion (NOT).

Q7.97.3 MCQ 1997 1M Ans: C ● ○ ○

In a standard Transistor-Transistor Logic (TTL) circuit, the internal architecture can be broadly divided into three main operational stages:

1. **Input Stage:** Utilizes a multi-emitter transistor to implement input logic functions.
2. **Phase Splitter Stage:** Controls and splits the phase signals to drive the final stage transistors alternatively.
3. **Output Stage / Buffer:** Configured as a **totem pole output stage**, consisting of two vertically stacked transistors where only one conducts at a given time.

The totem pole configuration acts as an efficient active pull-up and pull-down output buffer. It provides a very low output impedance during both logic transitions, significantly increasing the operational switching speed and driving capacity (fan-out) of the logic gate.

(C) the output buffer

Key Points

- **Totem Pole Structure:** Uses an active pull-up transistor configuration instead of a passive pull-up resistor.
- **Impedance Characteristics:** Offers low output impedance in both HIGH and LOW states, enabling rapid charging and discharging of parasitic load capacitances.

Q7.97.4 MCQ 1997 1M Ans: A

A standard static Random Access Memory (SRAM) cell utilizes a volatile bistable latch configuration to store each bit of data.

Unlike a dynamic RAM (DRAM) cell which relies on a single transistor and a storage capacitor (1T1C configuration) that demands continuous periodic refreshing, a standard SRAM cell is constructed using a cross-coupled inverter structure.

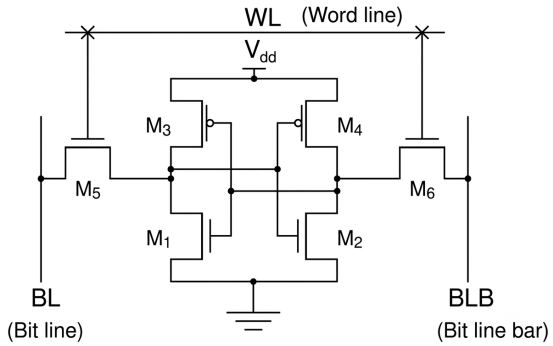


Fig. Solution Q7.97.4

- **Cross-Coupled Latch Core:** Two CMOS inverters are connected back-to-back to form a stable latch mechanism. Each CMOS inverter contains 1 PMOS transistor and 1 NMOS transistor, summing up to 4 MOS transistors (2 PMOS and 2 NMOS).
- **Access Control Path:** Two additional NMOS pass-transistors act as access switches connecting the internal storage nodes to the complementary bit lines (BL and $\overline{BL}$). These gates are simultaneously activated by the Word Line (WL).

Summing these up: 4 (Latch Core) + 2 (Access Switches) = 6 MOS transistors. No capacitors are utilized in the storage mechanism of a classic SRAM cell.

(A) 6 MOS transistors

Key Points

- **SRAM Configuration:** A standard full-CMOS SRAM cell is universally designated as a **6T cell** because it utilizes exactly 6 MOS transistors and 0 passive capacitors.
- **DRAM Contrast:** For comparison, a standard DRAM cell consists of 1 MOS transistor and 1 capacitor (**1T1C structure**), introducing higher storage density but requiring continuous fresh cycles.

Q7.97.5 MCQ 1997 1M Ans: B ● ● ○

The given parameters for the 74 ALS04 inverter are:

$$I_{OH(max)} = -0.4\,\text{mA}, \quad I_{OL(max)} = 8\,\text{mA}$$

$$I_{IH(max)} = 20\,\mu\text{A}, \quad I_{IL(max)} = -0.1\,\text{mA}$$

To find the total fan-out of the logic gate, we determine the maximum load it can handle safely in both the High and Low logic states.

1. **High-Level Fan-out (Fan-out$_H$):**

$$(\text{Fan-out})_H = \left|\frac{I_{OH(max)}}{I_{IH(max)}}\right| = \frac{0.4 \times 10^{-3}\,\text{A}}{20 \times 10^{-6}\,\text{A}} = \frac{400}{20} = 20$$

2. **Low-Level Fan-out (Fan-out$_L$):**

$$(\text{Fan-out})_L = \left|\frac{I_{OL(max)}}{I_{IL(max)}}\right| = \frac{8 \times 10^{-3}\,\text{A}}{0.1 \times 10^{-3}\,\text{A}} = 80$$

The absolute fan-out of the logic gate is the lower constraint of the two states:

$$\text{Fan-out} = \min\left[(\text{Fan-out})_H,\ (\text{Fan-out})_L\right] = \min(20,\ 80) = 20$$

(B) 20

Key Points

- **Fan-out Definition:** Fan-out represents the maximum number of similar logic gate inputs that a single driving gate's output can reliably control without degrading the voltage levels below specification thresholds.
- **Limiting Condition:** The overall fan-out capability is strictly bounded by the minimum of the high-state current margin and low-state current margin ($\min[(\text{Fan-out})_H, (\text{Fan-out})_L]$) to prevent thermal overload and preserve noise immunity margins.

Q7.96.1 MCQ 1996 2M Ans: C ● ● ○

To design the required memory system, we can analyze the depth and width expansion of the memory configuration:

1. **Required Memory System Specifications:**
   - Total capacity = 16 K bytes = 16 K × 8 bits
   - Word capacity (depth) = 16 K
   - Word size (width) = 8 bits
2. **Individual Memory Chip Specifications:**
   - Address lines = 12 $\implies$ Word capacity (depth) = 2^{12} = 4 K
   - Data lines (width) = 4 bits
   - Individual chip size = 4 K × 4 bits
3. **Calculation of Required Chips:**

- **Number of chips required for depth expansion** (to go from 4 K to 16 K):

$$\text{Depth ratio} = \frac{16\,\text{K}}{4\,\text{K}} = 4$$

- **Number of chips required for width expansion** (to go from 4 bits to 8 bits):

$$\text{Width ratio} = \frac{8\,\text{bits}}{4\,\text{bits}} = 2$$

- **Total number of chips required**:

$$\text{Total chips} = 4 \times 2 = 8$$

Alternatively, using total memory capacity:

$$\text{Total chips} = \frac{\text{Total capacity required}}{\text{Capacity of single chip}} = \frac{16\,\text{K} \times 8\,\text{bits}}{4\,\text{K} \times 4\,\text{bits}} = 4 \times 2 = 8$$

(C) 8

Key Points

- **Memory Capacity:** The total storage capacity is the product of the number of words (determined by address lines) and the word length (determined by data lines).
- **Grid Arrangement:** To expand both address capacity and word size, memory chips are arranged in a matrix of $M \times N$ chips, where M is the depth expansion factor and N is the width expansion factor.

Q7.96.2 MCQ 1996 2M Ans: D ●●○

To find the storage capacitance of the dynamic RAM (DRAM) cell, we use the relationship between capacitance, discharge current, voltage drop, and time:

1. **Given Data:**
   - Initial voltage of the cell, $V = 5\,\text{V}$
   - Maximum allowable voltage fall, $\Delta V = 0.5\,\text{V}$
   - Refresh time interval, $\Delta t = 20\,\text{ms} = 20 \times 10^{-3}\,\text{s}$
   - Constant discharge current, $I = 0.1\,\text{pA} = 0.1 \times 10^{-12}\,\text{A} = 10^{-13}\,\text{A}$
2. **Capacitance Calculation:** The formula relating the constant discharge current to the change in voltage over a time period is:

$$I = C\frac{\Delta V}{\Delta t}$$

Rearranging the formula to solve for the storage capacitance (C):

$$C = \frac{I \cdot \Delta t}{\Delta V}$$

Substituting the given values:

$$C = \frac{10^{-13}\,\text{A} \times 20 \times 10^{-3}\,\text{s}}{0.5\,\text{V}}$$

$$C = \frac{2 \times 10^{-15}}{0.5}\,\text{F}$$

$$C = 4 \times 10^{-15}\,\text{F}$$

(D) 4×10^{-15} F

Key Points

- **DRAM Cell Storage:** A dynamic RAM cell stores binary information in the form of an electric charge on a small MOS capacitor.
- **Charge Leakage:** Because the charge leaks over time due to leakage currents, the cell must be periodically refreshed to maintain the stored state.

Q7.95.1 Descriptive | 1995 | 3M | Ans: A-1, B-4, C-3 | ● ○ ○

Matching standard TTL gate specifications with the given parameters:

- (A) $V_{OH}(\text{min}) = 2.4\,\text{V} \longrightarrow (1)$
- (B) $V_{IH}(\text{min}) = 2.0\,\text{V} \longrightarrow (4)$
- (C) $V_{OL}(\text{max}) = 0.4\,\text{V} \longrightarrow (3)$

(A)-1, (B)-4, (C)-3

Key Points

- Standard TTL logic voltage levels: $V_{OH}(\text{min}) = 2.4\,\text{V}$, $V_{IH}(\text{min}) = 2.0\,\text{V}$, $V_{IL}(\text{max}) = 0.8\,\text{V}$, and $V_{OL}(\text{max}) = 0.4\,\text{V}$.

Q7.95.2 MCQ | 1995 | 1M | Ans: A | ● ● ○

To determine the minimum number of MOS transistors required to design a Dynamic RAM (DRAM) cell:

1. **DRAM Cell Construction:** A standard dynamic RAM (DRAM) cell stores a single bit of data as an electric charge on a storage capacitor (C_s). To read from or write to this capacitor, an access transistor is connected in series with it.
2. **The 1T1C Structure:** The most widely used and basic DRAM configuration is the **1T1C (1-Transistor, 1-Capacitor)** cell:
   - **1 MOS Transistor (Access Transistor):** Acts as a switch controlled by the Word Line (WL) to connect the storage capacitor to the Bit Line (BL).
   - **1 Capacitor (Storage Capacitor):** Stores the electric charge representing the binary state (0 or 1).
3. **Comparison with Static RAM (SRAM):** Unlike SRAM, which requires a minimum of 6 transistors (6T) or 4 transistors (4T) with resistors to maintain bistable latch states, DRAM utilizes the simple 1T1C structure. This minimal transistor requirement makes DRAM significantly denser and cheaper per bit than SRAM.

Thus, the minimum number of MOS transistors required to make a dynamic RAM cell is 1.

(A) 1

Key Points

- **DRAM Cell:** Formed by 1 MOS transistor and 1 capacitor (1T1C).
- **Refresh Cycle:** Due to charge leakage in the capacitor over time, a DRAM cell must be refreshed periodically to preserve data.

Q7.94.1 Descriptive 1994 2M Ans: FALSE ● ● ○

To analyze the statement regarding the output stage of a standard TTL logic gate:

1. **Structure of the TTL Totem-Pole Output Stage:** The standard TTL output stage (often called the totem-pole configuration) consists of:
   - A pull-up transistor (Q_3) whose emitter is connected to the anode of a diode (D).
   - A pull-down transistor (Q_4) whose collector is connected to the cathode of the diode (D) and also serves as the output terminal (Y).
   - The diode (D) situated between the emitter of the pull-up transistor Q_3 and the collector of the pull-down transistor Q_4.
2. **Actual Purpose of the Diode (D):**
   - When the output is low (logic 0), the phase-splitter transistor (Q_2) is saturated, forcing the base of the pull-up transistor Q_3 to approximately $V_{BE4,\text{sat}} + V_{CE2,\text{sat}} \approx 0.7\,\text{V} + 0.2\,\text{V} = 0.9\,\text{V}$.
   - At the same time, the pull-down transistor Q_4 is saturated, pulling the output node (Y) down to $V_{CE4,\text{sat}} \approx 0.2\,\text{V}$.
   - Without the diode, the base-to-emitter voltage of Q_3 would be:

     $$V_{BE3} = V_{B3} - V_Y \approx 0.9\,\text{V} - 0.2\,\text{V} = 0.7\,\text{V}$$

     This potential is high enough to turn Q_3 partially ON, causing both the pull-up (Q_3) and pull-down (Q_4) transistors to conduct simultaneously, leading to heavy current draw from the power supply.
   - Inserting the diode D adds an extra 0.7 V drop in the path. For Q_3 to conduct, the base voltage V_{B3} must overcome both the V_{BE3} of the transistor and the forward voltage drop V_D of the diode:

     $$V_{B3,\text{req}} = V_Y + V_D + V_{BE3} \approx 0.2\,\text{V} + 0.7\,\text{V} + 0.7\,\text{V} = 1.6\,\text{V}$$

     Since V_{B3} is only 0.9 V, the diode successfully keeps the pull-up transistor Q_3 **completely cut off** when the pull-down transistor Q_4 is ON.
3. **Conclusion:** The primary purpose of the diode is to keep the pull-up transistor turned off while the pull-down transistor is turned on. It is **not** to isolate the output node from the power supply V_{CC} (or V_α in the question text).

Thus, the statement is **False**.

False

Key Points

- **TTL Totem-Pole Diode:** Placed in the output path to prevent simultaneous conduction of both pull-up and pull-down transistors.
- **Off-State Maintenance:** The diode shifts the required turn-on voltage of the upper transistor up by approximately 0.7 V, ensuring it remains reliably in cut-off when the output is low.

Q7.94.2 MCQ 1994 1M Ans: D ● ● ○

To determine the correct application of a PLA (Programmable Logic Array):

1. **Understanding PLA Structure:** A PLA is a type of Programmable Logic Device (PLD) consisting of:
   - A **programmable AND array** (to generate product terms of the inputs).
   - A **programmable OR array** (to sum the product terms to generate outputs).

2. **Functionality:** Because it consists strictly of arrays of AND and OR logic gates without any inherent storage/memory elements (like flip-flops or registers) in its standard design, it directly implements sum-of-products boolean functions. Therefore, its primary and fundamental purpose is **to realize combinational logic**.

3. **Analysis of other options:**

   - **Option A (as a microprocessor):** A microprocessor is a highly complex sequential and control system containing ALUs, registers, and control units; a simple PLA cannot act as a microprocessor.
   - **Option B (as a dynamic memory):** DRAM is built using capacitors and access transistors to store data charges; a PLA is logic-based and cannot be used as dynamic memory.
   - **Option C (to realize sequential logic):** Standard PLAs do not contain memory elements (flip-flops). While some registered PLAs (which include flip-flops on the outputs) can realize sequential logic, the standard, fundamental PLA is designed specifically for combinational logic.

(D) to realise a combinational logic

Key Points

- **PLA Features:** Both the AND array and the OR array are programmable, making it highly flexible for implementing combinational sum-of-products expressions.
- **Comparison with PAL/ROM:**

Device	AND Array	OR Array
PROM	Fixed	Programmable
PAL	Programmable	Fixed
PLA	Programmable	Programmable

Q7.94.3 MCQ 1994 1M Ans: C ●●○

To determine the basic hardware architecture of a dynamic RAM (DRAM) cell:

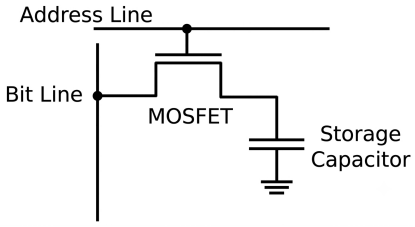


Fig. Solution Q7.94.3

1. **Basic Component Design:** A basic dynamic RAM (DRAM) cell stores a single bit of binary data as an electric charge on a small integrated capacitor. To control the access to this capacitor for reading or writing data, a single MOS transistor is placed in series with it acting as a switch.

2. **The 1T1C Cell Configuration:** This standard architecture is widely known as the **1T1C cell** configuration:

   - **1 Transistor (Access Transistor):** The gate of this NMOS/PMOS transistor is connected to the Word Line (WL), while its channel connects the Bit Line (BL) directly to the storage element.
   - **1 Capacitor (Storage Capacitor):** Accumulates or releases charge to establish the logical state (1 or 0).

3. **Contrast with Alternative Configurations:**

- **6 Transistors (Option A):** Typical structure required to design a standard Static RAM (SRAM) cell utilizing cross-coupled inverters to form a bistable latch.
- **2 Transistors and 2 Capacitors (Option B) / 2 Capacitors only (Option D):** Do not describe the fundamental, elementary layout of modern industrial DRAM arrays.

(C) 1 transistor and 1 capacitor

Key Points

- **High Packing Density:** The simple 1T1C architecture takes up very little silicon area, allowing DRAM to achieve significantly higher data storage capacities per unit chip area compared to SRAM.
- **Volatile Nature & Leaks:** Since the charge on the isolated capacitor leaks away naturally over time, dynamic RAM relies on external peripheral circuits to periodically perform a "refresh cycle."

Q7.92.1 MCQ 1992 2M Ans: C,D ● ● ○

To determine the correct statements, we analyze the structural configuration of different Programmable Logic Devices (PLDs):

1. **PROM (Programmable Read-Only Memory):** A PROM consists of a **fixed AND array** acting as a full decoder (which generates all possible minterms for the given inputs) followed by a **programmable OR array** that sums selected minterms to realize the desired boolean functions.
   - Therefore, Statement C is **correct** and Statement A is incorrect.
2. **PLA (Programmable Logic Array):** A PLA offers the highest flexibility among basic PLDs because it consists of a **programmable AND array** as well as a **programmable OR array**. This allows any product term to be generated and shared among multiple outputs.
   - Therefore, Statement D is **correct** and Statement B is incorrect.

(C) PROM contains a fixed AND array and a programmable OR array.
(D) PLA contains a programmable AND array and a programmable OR array.

Key Points

Here is a comprehensive comparison of standard PLD structures:

Device Type	AND Array	OR Array
PROM / ROM	Fixed	Programmable
PAL	Programmable	Fixed
PLA	Programmable	Programmable

Q7.92.2 MCQ 1992 2M Ans: D ● ● ○

To identify the logic function represented by the given Resistor-Transistor Logic (RTL) circuit, we can analyze the operation of the BJTs (Q_1, Q_2, and Q_3) under different input logic combinations. Let the inputs be V_{i1} and V_{i2} and the intermediate node voltage at the collectors of Q_1 and Q_2 be y.

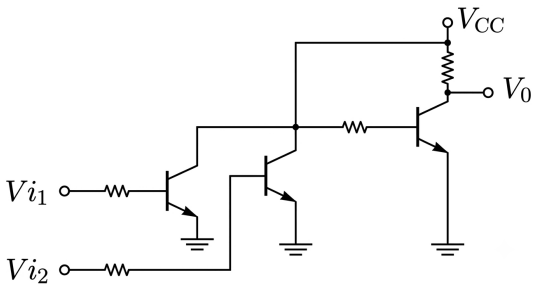


Fig. Solution Q7.92.2

Method 1

We construct the truth table by analyzing the conduction state of each transistor:

7. **When $V_{i1} = 0\,\text{V}$ and $V_{i2} = 0\,\text{V}$ (Logic 0):** Both transistors Q_1 and Q_2 are turned **OFF**. The intermediate node y is pulled up to V_{CC} (Logic 1). This high voltage turns Q_3 **ON** (into saturation), pulling the output node to ground:

$$V_0 \approx 0.2\,\text{V} \quad \text{(Logic 0)}$$

7. **When $V_{i1} = 5\,\text{V}$ (Logic 1) and/or $V_{i2} = 5\,\text{V}$ (Logic 1):** At least one of the transistors Q_1 or Q_2 is turned **ON** (into saturation). This pulls the intermediate node y down to ground (Logic 0). Since y is low, the base-emitter junction of Q_3 is reverse-biased, turning Q_3 **OFF**. Consequently, the output V_0 is pulled up to V_{CC} through the collector resistor:

$$V_0 = V_{CC} \quad \text{(Logic 1)}$$

V_{i1}	V_{i2}	Q_1	Q_2	Q_3	V_0
0	0	OFF	OFF	ON	0
0	1	OFF	ON	OFF	1
1	0	ON	OFF	OFF	1
1	1	ON	ON	OFF	1

Since the output is high if any input is high, the circuit represents an **OR** gate.

Method 2

We partition the schematic into cascading stages:

1. **First Stage (Q_1 and Q_2):** The transistors Q_1 and Q_2 are connected in parallel to a common pull-up resistor. This structure represents a basic **RTL NOR gate**:

$$y = \overline{V_{i1} + V_{i2}}$$

2. **Second Stage (Q_3):** The transistor Q_3 is a simple common-emitter switch, which acts as a **NOT gate**:

$$V_0 = \overline{y}$$

Combining the two stages gives:

$$V_0 = \overline{\overline{V_{i1} + V_{i2}}} = V_{i1} + V_{i2}$$

This matches the Boolean expression of an **OR** gate.

(D) OR

Key Points

- **Historical Perspective:** Resistor-Transistor Logic (RTL) was widely used before IC technology matured because transistors were far more expensive than resistors.
- **Disadvantages of RTL:** RTL circuits suffer from low speed, poor noise margins, and high current/heat dissipation. These limitations led to its replacement by Transistor-Transistor Logic (TTL).

Q7.91.1 NAT 1991 2M Ans: 32 ●●○

1. Architecture and Addressing Scheme

For a 256×8-bit ROM:

- Total capacity = 256 words $\times$ 8 bits/word = 2048 bits.
- Total address lines required = $\log_2(256) = 8$ lines (A_7 to A_0).

Using a two-dimensional (row/column) coincident addressing scheme with **8-to-1 selectors** (multiplexers):

- The 8-to-1 selectors require $\log_2(8) = 3$ select lines (A_2, A_1, A_0).
- The remaining $8 - 3 = 5$ address lines (A_7, A_6, A_5, A_4, A_3) are used as inputs to a row decoder to select one of $2^5 = 32$ rows.

2. Row Decoder Construction using NAND Gates

To decode 5 address lines into 32 distinct row select lines:

- We require one 5-input NAND gate for each of the 32 output rows.
- This directly requires **32 NAND gates** (assuming inverted address lines are freely available or generated externally).

32

Key Points

- **2D Memory Addressing:** Organizes memory cells into an X-Y matrix to significantly reduce the size and complexity of the decoding circuitry compared to 1D linear addressing.
- **Decoder Complexity:** A n-to-2^n active-low decoder implemented with basic logic gates requires 2^n individual n-input NAND gates at the output stage.

Q7.91.2 NAT 1991 2M Ans: * ●●○

To convert the given nMOS logic gate into its full CMOS equivalent circuit, we can analyze the structural duality of the pull-down (nMOS) and pull-up (pMOS) networks.
A standard static CMOS logic gate consists of a pull-down network (PDN) made of nMOS transistors and a pull-up network (PUN) made of pMOS transistors. The PUN is the topological dual of the PDN:

- **Series connections** in the PDN become **parallel connections** in the PUN.
- **Parallel connections** in the PDN become **series connections** in the PUN.

1. Analyzing the nMOS Pull-Down Network (PDN):

Looking at the given circuit:

- Transistor Q_3 (controlled by B) and transistor Q_4 (controlled by C) are connected in **series** with each other: $(B \cdot C)$.
- Transistor Q_2 (controlled by A) is connected in **parallel** with the entire series combination of Q_3 and Q_4: $A + (B \cdot C)$.

Therefore, the boolean function realized at the output node is:

$$f(A, B, C) = \overline{A + (B \cdot C)}$$

2. Constructing the pMOS Pull-Up Network (PUN):

Applying the principle of duality to find the pMOS network structure:

- Since B and C are in **series** in the PDN, their corresponding pMOS transistors (Q_6 and Q_7) must be connected in **parallel**: $(B \parallel C)$.
- Since A is in **parallel** with the branch containing B and C in the PDN, its corresponding pMOS transistor (Q_5) must be connected in **series** with the parallel combination of B and C: A in series with $(B \parallel C)$.

Connecting this constructed PUN between V_{DD} and the output node, and retaining the original PDN connected between the output node and ground, yields the complete CMOS gate structure shown below.

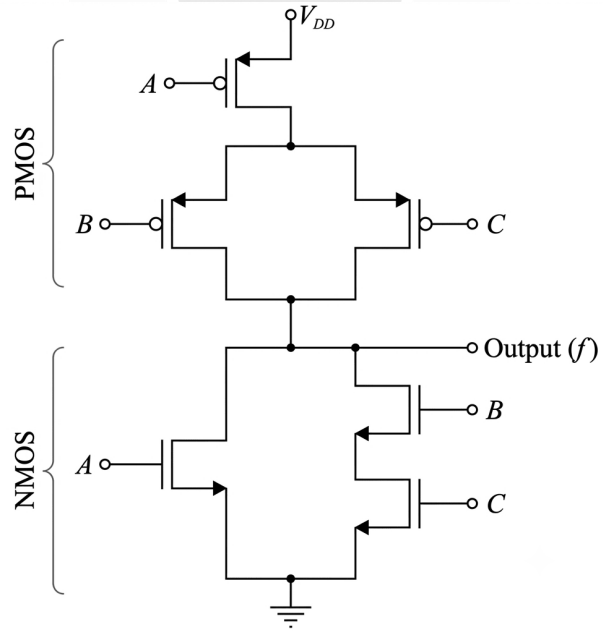


Fig. Solution Q7.91.2

Key Points

- **Static CMOS Composition:** Static CMOS circuits are fully complementary, requiring exactly N nMOS transistors for the PDN and N pMOS transistors for the PUN to implement an N-input logic function.
- **Pseudo-nMOS vs CMOS:** The original circuit uses a depletion-load/diode-connected nMOS device (Q_1) as a static load. Converting it to CMOS eliminates static power dissipation because there is never a direct active path from V_{DD} to ground in steady state.

Q7.91.3 NAT | 1991 | 2M | Ans: * | ●●○

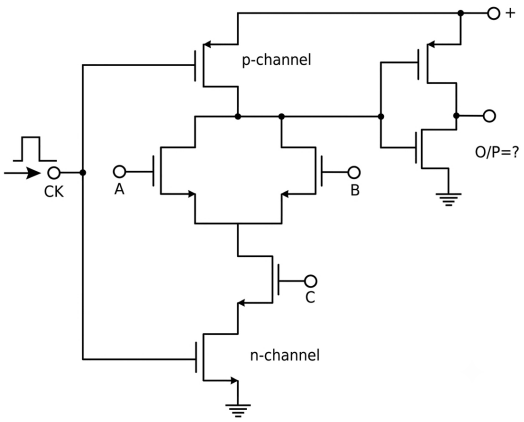


Fig. Solution Q7.91.3

To determine the Boolean expression for the given circuit when the clock signal CK is high, we can analyze the behavior of the dynamic logic structure stage by stage.

1. Circuit Configuration and Operation Phases

The circuit shown is a dynamic CMOS logic gate followed by a static CMOS inverter buffer (to ensure proper signal levels and prevent degradation). Let Y' be the intermediate node at the input of the inverter, and Y be the final output node (O/P). The clock signal CK controls the precharge and evaluation phases:

- **Precharge Phase (CK = 0):** The top p-channel transistor turns **ON**, pulling node Y' up to $+V_{DD}$ (Logic 1). The bottom n-channel evaluation transistor is **OFF**. The output Y becomes $\overline{Y'}$ = Logic 0.
- **Evaluation Phase (CK = 1):** The top p-channel precharge transistor turns **OFF**, and the bottom n-channel transistor turns **ON**, providing a conditional path to ground for node Y'.

2. Boolean Expression Analysis during Evaluation (CK = 1)

When CK is high, the intermediate node Y' will discharge to ground (Logic 0) if and only if there is a conducting path through the nMOS pull-down network:

- Transistors A and B are connected in parallel, creating a logic path represented by $(A + B)$.
- Transistor C is in series with the parallel combination of A and B.
- The bottom evaluation transistor controlled by CK is active (conducting).

Therefore, the condition for node Y' to be pulled low to ground (Logic 0) is:

$$Y' = \overline{(A + B) \cdot C}$$

The final output stage is a standard static CMOS inverter, which complements the intermediate node state:

$$Y = \overline{Y'} = \overline{\overline{(A + B) \cdot C}} = (A + B) \cdot C$$

$$\boxed{(A + B)C}$$

Key Points

- **Dynamic Logic Advantages:** Dynamic logic networks require fewer transistors than standard complementary CMOS (only $N + 2$ transistors for an N-input gate instead of $2N$), leading to smaller silicon area and reduced parasitic capacitance.
- **High-Impedance State Note:** During the evaluation phase (CK = 1), if the pull-down condition $(A + B)C = 0$ is met, node Y' enters a floating high-impedance state, relying temporarily on internal node capacitance to maintain its stored precharge voltage.

Q7.91.4 NAT 1991 1M Ans: * ● ○ ○

A **FAMOS** (Floating-gate Avalanche-injection Metal-Oxide Semiconductor) transistor is the core technology used to implement UV-erasable EPROMs (Erasable Programmable Read-Only Memories).

- **Programming:** High-voltage avalanche injection forces electrons to penetrate the oxide layer and become trapped on the isolated floating gate, raising the threshold voltage to represent a programmed logic state.
- **Erasing:** Because the floating gate is completely surrounded by high-quality silicon dioxide insulation, these trapped charges cannot escape under normal operating conditions. To clear the memory cell, the device must be exposed to high-energy **ultraviolet (UV) light**.
- **Mechanism:** The UV photons provide enough energy to the trapped electrons to overcome the energy barrier of the silicon dioxide layer, allowing them to flow back into the substrate, discharging the gate and resetting the cell status.

Thus, a bit stored in a FAMOS device can be erased by exposing it to ultraviolet light.

Ultraviolet light

Key Points

- **EPROM Window:** EPROM chips feature a transparent quartz window directly over the semiconductor die to permit the penetration of UV light during the erasure process.
- **Bulk Erasure:** Ultraviolet exposure clears all memory cells simultaneously rather than allowing byte-addressable erasing.

Q7.89.1 MCQ 1989 1M Ans: D ● ● ○

To determine the total noise margin (NM) of a logic family, we need to calculate both the high-level noise margin ($(NM)_H$) and the low-level noise margin ($(NM)_L$), and then select the minimum of the two values.
Given parameters from the problem statement:

- Threshold voltage, $V_T = 2\,\text{V}$
- Minimum guaranteed output high voltage, $V_{OH} = 4\,\text{V}$
- Minimum accepted input high voltage, $V_{IH} = 3\,\text{V}$
- Maximum guaranteed output low voltage, $V_{OL} = 1\,\text{V}$
- Maximum accepted input low voltage, $V_{IL} = 1.5\,\text{V}$

The formulations for the noise margins are defined as:

1. **High-Level Noise Margin ($(NM)_H$):**

$$(NM)_H = V_{OH} - V_{IH}$$

$$(NM)_H = 4\,\text{V} - 3\,\text{V} = 1\,\text{V}$$

2. **Low-Level Noise Margin ($(NM)_L$):**

$$(NM)_L = V_{IL} - V_{OL}$$

$$(NM)_L = 1.5\,\text{V} - 1\,\text{V} = 0.5\,\text{V}$$

The overall noise margin is the worst-case configuration, which corresponds to the minimum value:

$$NM = \min\left[(NM)_H, (NM)_L\right]$$

$$NM = \min(1\,\text{V}, 0.5\,\text{V}) = 0.5\,\text{V}$$

(D) 0.5 V

Key Points

- **Physical Definition:** Noise margin acts as a quantitative measure of the noise immunity of a digital circuit. It represents the maximum spurious noise voltage amplitude that can be added to an input signal without corrupting the intended logic level.
- **Design Rule Constraints:** For stable, noise-tolerant hardware operation, the voltage bounds must strictly adhere to $V_{OH} > V_{IH} > V_{IL} > V_{OL}$.

Q7.89.2 MSQ 1989 1M Ans: A,C ●●○

To evaluate the correct statements among the given digital IC logic families (**ECL**, **TTL**, and **CMOS**), we analyze the characteristic parameters of each option.
Let us evaluate each statement individually based on standard semiconductor properties:

- **Statement A: ECL has the least propagation delay. Correct.** Emitter Coupled Logic (ECL) is a non-saturating logic family where the transistors operate strictly within the active and cut-off regions. Because the transistors never enter saturation, there is no storage-time delay during switching transitions. This makes ECL the fastest logic family with the smallest propagation delay (≈ 1 ns).
- **Statement B: TTL has the largest fan-out. Incorrect.** CMOS features an extremely high input impedance due to the insulated SiO_2 gate structures of MOSFETs. Under static (DC) conditions, a CMOS gate draws negligible input current, allowing it to drive an exceptionally high number of similar gates. Hence, CMOS (not TTL) offers the highest static fan-out (≥ 50).
- **Statement C: CMOS has the biggest noise margin. Correct.** The noise margin of a logic family is proportional to its logic voltage swing. CMOS logic levels swing fully from 0 V to V_{DD}. The typical noise margin for CMOS is roughly $0.3V_{DD}$ to $0.45V_{DD}$ (about 1.5 V to 2.25 V for a 5 V system), which is far superior to TTL (≈ 0.4 V) and ECL (≈ 0.25 V).
- **Statement D: TTL has the lowest power consumption. Incorrect.** CMOS technology exhibits the lowest static power consumption because one of the complementary transistors (nMOS or pMOS) is always turned off in steady-state, blocking any direct path from V_{DD} to ground.

Thus, both statements **A** and **C** are technically accurate.

(A)ECL has the least propagation delay
(C)CMOS has the biggest noise margin

Key Points

Parameter	TTL	CMOS	ECL
Propagation Delay	Medium (≈ 10 ns)	Medium (≈ 15 ns)	**Lowest** (≈ 1 ns)
Power Dissipation	Medium (≈ 10 mW)	**Lowest** (Static ≈ nW)	Highest (≈ 25 mW)
Noise Margin	Medium (≈ 0.4 V)	**Highest** (≈ 1.5 V)	Lowest (≈ 0.25 V)
Fan-out	10	**Highest** (> 50)	25

Q7.87.1 MCQ 1987 2M Ans: B ●●○

To determine the correct relationship between the input and output voltage levels of a valid digital logic family, we analyze the definition of the critical voltage parameters necessary to ensure reliable data transmission and positive noise margins.
A properly functioning logic family must satisfy conditions that allow a driving gate's output to be cleanly interpreted by a receiving gate's input, even in the presence of signal degradation or noise. The voltage specifications are defined as follows:

- V_{OH}: The minimum guaranteed output high-level voltage from a driving gate.
- V_{IH}: The minimum accepted input high-level voltage recognized by a receiving gate as logic 1.
- V_{IL}: The maximum accepted input low-level voltage recognized by a receiving gate as logic 0.
- V_{OL}: The maximum guaranteed output low-level voltage from a driving gate.

For a stable digital system with non-zero (positive) noise margins:

1. **High-Level Noise Immunity:** The output high voltage must exceed the required input high voltage to accommodate noise drops:
$$V_{OH} > V_{IH} \implies (NM)_H = V_{OH} - V_{IH} > 0$$
2. **Low-Level Noise Immunity:** The input low voltage threshold must be higher than the maximum output low voltage to ensure safety against noise spikes:
$$V_{IL} > V_{OL} \implies (NM)_L = V_{IL} - V_{OL} > 0$$

Combining these relations with the absolute voltage separation between logic high and logic low levels ($V_{IH} > V_{IL}$), we establish the complete hierarchical sequence:

$$V_{OH} > V_{IH} > V_{IL} > V_{OL}$$

$$\boxed{\text{(B) } V_{OH} > V_{IH} > V_{IL} > V_{OL}}$$

Key Points

Parameter	Operational Requirement
High-Level Noise Margin	$(NM)_H = V_{OH} - V_{IH}$ (Must be > 0)
Low-Level Noise Margin	$(NM)_L = V_{IL} - V_{OL}$ (Must be > 0)

- **Invalid Regions:** The region between V_{IH} and V_{IL} represents an undefined or transition region where the logic state cannot be reliably determined.
- **Noise Immunity Implications:** If a logic family violates this hierarchical ordering (e.g., $V_{OH} < V_{IH}$), it results in a negative noise margin, causing systemic data corruption during standard operations.

Q7.87.2 NAT 1987 1M Ans: *

To fill in the blanks for the given common data statements concerning various digital logic families, we evaluate their power-delay product metrics and supply voltage requirements.
Among the different configurations within the Transistor-Transistor Logic (TTL) sub-families:

- Standard TTL (74XX) features a balanced speed but suffers from relatively high power dissipation.
- Low-power TTL (74LXX) drastically decreases power usage, but at the expense of significantly increasing the propagation delay (making it much slower).
- Low-power Schottky TTL (74LSXX) incorporates Schottky diode clamping across base-collector junctions to prevent transistor saturation. This design element yields a sub-family that requires considerably lower power dissipation (≈ 2 mW per gate compared to 10 mW for standard TTL) while preserving a comparable propagation delay (≈ 9 ns).

Therefore, the missing term is

74LS

Q7.87.3 NAT 1987 1M Ans: *

Considering the operational requirements for supply voltages across the listed logic structures:

- Bipolar-based families like TTL and ECL demand tightly controlled supply levels. For instance, standard TTL gates operate reliably only within a narrow toleration window of 5 V ± 5% (4.75 V to 5.25 V).
- Conversely, Complementary Metal-Oxide-Semiconductor (CMOS) configurations function independently of a highly precise supply voltage because their complementary switching behavior relies primarily on voltage ratios rather than fixed forward-bias diode voltages. Standard CMOS integrated circuits can seamlessly operate over a broad range of supply levels, typically spanning from 3 V to 15 V.

Therefore, the missing term is

CMOS.

SOLUTIONS

VIII

Computer Organization and Architecture

Topics

1. Machine Instructions and Addressing Modes
2. ALU, Data-Path and Control Unit
3. Instruction Pipelining

Unit Snapshot*

Stream: EC	**Questions:** 74	**MCQ:** 55	**MSQ:** 2	**NAT:** 11
1M: 24	**2M:** 39	**Descriptive:** 11	**Avg Weightage** ~2M	**Range:** 1M(2024)–3M(2013)

**See preface for how Avg Weightage and Range are calculated.*

Q8.26.1 MCQ | 2026 | 2M | Ans: C | ● ● ●

Given:

- Memory Size = 256 KB
- First Location Address (Starting Address) = $(2500)_H$

Step 1: Determine the total number of locations in the memory. Since the memory size is 256 KB and memory is typically byte-addressable:

$$\text{Total Locations} = 256 \times 1024 = 262,144 \text{ locations}$$

Step 2: Convert the total number of locations into hexadecimal to find the address range.

$$256\,\text{K} = 2^8 \times 2^{10} = 2^{18}$$

In hexadecimal, 2^{18} is calculated as:

$$2^{18} = 2^2 \times 2^{16} = 4 \times (16)^4 = (40000)_H$$

The total number of addresses (offset) starting from 0 would be from $(00000)_H$ to $(2^{18} - 1)_H$.

$$\text{Offset} = (40000)_H - (1)_H = (3FFFF)_H$$

Step 3: Calculate the Last Location Address. The last address is found by adding the offset to the starting address:

$$\text{Last Address} = \text{Starting Address} + \text{Offset}$$

$$\text{Last Address} = (02500)_H + (3FFFF)_H$$

Performing the hexadecimal addition:

$$\begin{array}{cccccc} & 3 & F & F & F & F \\ + & 0 & 2 & 5 & 0 & 0 \\ \hline & 4 & 2 & 4 & F & F \end{array}$$

The last location address is $(424FF)_H$.

$$\boxed{(C)(424\text{FF})_{\text{H}}}$$

Key Points

- **Memory Range**: For a memory of size N, the addresses range from S to $S + (N - 1)$, where S is the starting address.
- **Hexadecimal Conversion**: $1\,\text{K} = 2^{10} = (400)_H$. Therefore, $256\,\text{K} = 256 \times 2^{10} = (100)_H \times (400)_H = (40000)_H$.

Mistakes to be avoided

- **Off-by-one error**: Always remember to add $(N - 1)$ to the starting address, not N. Adding N gives the address of the location immediately following the memory block.
- **Hex Addition**: Ensure carries are handled correctly in base-16 ($F + 5 = 14$ in decimal, which is 1 carry and 4 in hex).

Q8.24.1 NAT | 2024 | 1M | Ans: 2047 | ● ● ●

Given:

- Architecture: 32-bit
- Instruction Length: 1 word = 32 bits
- Number of Registers: 24
- Instruction Set Size: 40

Step 1: Calculate bits required for the Register Identifier fields. There are 24 registers. The number of bits n required to uniquely identify 24 registers is:

$$2^n \geq 24 \implies n = \lceil \log_2(24) \rceil = 5 \text{ bits}$$

Since the instruction has two source register identifiers and one destination register identifier, each requires 5 bits. **Step 2:** Calculate bits required for the Op-code field. The instruction set size is 40. The number of bits n required for the op-code is:

$$2^n \geq 40 \implies n = \lceil \log_2(40) \rceil = 6 \text{ bits}$$

Step 3: Calculate bits remaining for the Immediate Value field. The total instruction length is 32 bits. The fields are distributed as follows:

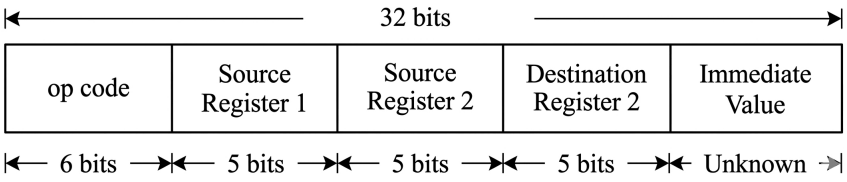


Fig. Solution Q8.24.1

$$\text{Bits}_{\text{Immediate}} = 32 - (\text{Op-code} + \text{Source 1} + \text{Source 2} + \text{Destination})$$

$$\text{Bits}_{\text{Immediate}} = 32 - (6 + 5 + 5 + 5) = 32 - 21 = 11 \text{ bits}$$

Step 4: Find the maximum value of the unsigned immediate operand. For an n-bit unsigned integer, the range is 0 to $2^n - 1$. With 11 bits:

$$\text{Max Value} = 2^{11} - 1 = 2048 - 1 = 2047$$

Hence, the maximum value of the unsigned integer in the immediate value field is **2047**.

2047

Key Points

- The number of bits required to address N distinct items is $\lceil \log_2 N \rceil$.
- The total bits in an instruction must equal the sum of the bits in its individual fields (opcode, registers, immediate values, etc.).
- The range for an n-bit unsigned operand is $[0, 2^n - 1]$.

Mistakes to be avoided

- Do not use $n = \log_2 N$ if it results in a non-integer; always round up (ceiling) to the next whole bit.
- Ensure you account for *all* register fields mentioned in the question (in this case, three: two source and one destination).

Q8.21.1 MCQ | 2021 | 2M | Ans: A | ● ● ○

Given the initial values of the registers:

$$R_1 = 25\text{H}, \quad R_2 = 30\text{H}, \quad R_3 = 40\text{H}$$

A stack operates on a Last-In, First-Out (LIFO) basis, meaning the last value pushed onto the stack will be the first one to be popped out.

1. **Push Operations:**
   - PUSH $\{R_1\}$: Stores 25H at the bottom position of the stack.
   - PUSH $\{R_2\}$: Stores 30H on top of R_1.
   - PUSH $\{R_3\}$: Stores 40H at the top of the stack.

   The stack configuration from top to bottom is now: [40H, 30H, 25H].
2. **Pop Operations:**
   - POP $\{R_1\}$: Retrieves the top value (40H) and stores it into R_1.$R_1 = 40\text{H}$
   - POP R_2: Retrieves the next value (30H) and stores it into R_2. $R_2 = 30\text{H}$
   - POP $\{R_3\}$: Retrieves the final value (25H) and stores it into R_3. $R_3 = 25\text{H}$

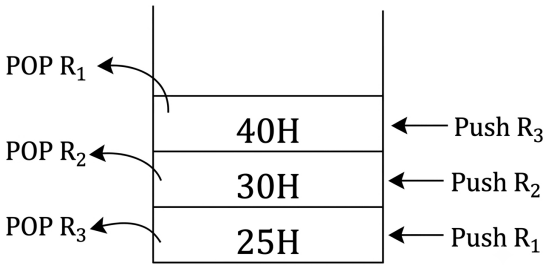


Fig. Solution Q8.21.1

Thus, the final register contents are $R_1 = 40\text{H}$, $R_2 = 30\text{H}$, and $R_3 = 25\text{H}$.

$$\boxed{(A) R_1 = 40\text{H}, \quad R_2 = 30\text{H}, \quad R_3 = 25\text{H}}$$

Key Points

- A stack architecture implicitly enforces a **Last-In, First-Out (LIFO)** order for storage and retrieval, completely reversing the order of sequence sequence if popped in the exact same sequence.

Q8.20.1 NAT | 2020 | 1M | Ans: 14 | ● ○ ○

The memory size to be accessed is given as 16 K bytes.
The relationship between memory capacity and the number of address lines (n) required is:

$$\text{Memory Size} = 2^n \text{ bytes}$$

Expressing 16 K as a power of 2:

$$16\text{ K} = 16 \times 1024 = 2^4 \times 2^{10} = 2^{14}$$

Equating the exponents:

$$2^n = 2^{14} \implies n = 14$$

Thus, the number of address lines required is 14.

$$\boxed{14}$$

Key Points

- 1 K (Kilo) in digital memory conversion is equal to $2^{10} = 1024$.
- n address lines can uniquely address 2^n distinct memory locations.

Q8.17.1 MCQ 2017 2M Ans: B ● ● ●

Let us trace the step-by-step execution of the given 8085 microprocessor instructions:

1. `MVI A, 33H`
   Loads the accumulator (Register A) with the immediate value 33H.

$$A = 33H = 0011\ 0011_2$$

2. `MVI B, 78H`
   Loads Register B with the immediate value 78H.

$$B = 78H = 0111\ 1000_2$$

3. `ADD B`
   Adds the content of Register B to the Accumulator ($A \leftarrow A + B$).

$$\begin{array}{rll} & 0011\ 0011_2 & (33H) \\ + & 0111\ 1000_2 & (78H) \\ \hline & 1010\ 1011_2 & (ABH) \end{array}$$

   Now, A = ABH.

4. `CMA`
   Complements each bit of the Accumulator (1's complement).

$$A = \overline{1010\ 1011}_2 = 0101\ 0100_2 = 54H$$

5. `ANI 32H`
   Performs a bitwise AND operation between the Accumulator and the immediate value 32H ($A \leftarrow A \cdot 32H$).

$$\begin{array}{rll} & 0101\ 0100_2 & (54H) \\ \cdot & 0011\ 0010_2 & (32H) \\ \hline & 0001\ 0000_2 & (10H) \end{array}$$

Therefore, the final value in the Accumulator immediately after execution is 10H.

$$\boxed{\text{(B)10H}}$$

Key Points

- `MVI` performs an immediate load into the specified register.
- `ADD` performs binary addition and updates accumulator along with flags.
- `CMA` acts as a bitwise NOT operation exclusively on the accumulator.
- `ANI` executes a bitwise logical AND operation, resetting the Carry Flag (CY) and setting the Auxiliary Carry Flag (AC).

Mistakes to be avoided

- Make sure to perform a bitwise logical **AND** operation for the `ANI` instruction instead of mistakenly performing arithmetic **ADDition** between the two hex values.

Q8.17.2 MCQ 2017 1M Ans: C ● ○ ○

Given parameters from the problem statement:

$$\text{Clock frequency } (f) = 5\,\text{MHz} = 5 \times 10^6\,\text{Hz}$$
$$\text{Execution time } (t) = 1.4\,\mu\text{s} = 1.4 \times 10^{-6}\,\text{s}$$

The duration of a single T-state (T) corresponds to one clock period:

$$T = \frac{1}{f} = \frac{1}{5 \times 10^6\,\text{Hz}} = 0.2 \times 10^{-6}\,\text{s} = 0.2\,\mu\text{s}$$

The total instruction execution time is given by the product of the number of T-states (N) and the duration of one T-state:

$$t = N \times T$$

Substituting the known values to find N:

$$1.4\,\mu\text{s} = N \times 0.2\,\mu\text{s}$$
$$N = \frac{1.4\,\mu\text{s}}{0.2\,\mu\text{s}} = 7$$

Hence, the number of T-states needed for executing the instruction is 7.

(C)7

Key Points

- A T-state represents the basic timing unit of an 8085 microprocessor, equal to one complete clock cycle.
- Execution Time = (Total Number of T-states) × (Clock Period).

Q8.16.1 MCQ 2016 2M Ans: A ● ● ●

The address range 1000 H to 2FFF H determines the chip select logic for the 8 KB ROM. Since the chip select $\overline{CS}$ is active-low, it must output 0 within this specific address range and 1 otherwise.

1. **Address Range Mapping ($A_{15} - A_{12}$):** Converting the hex boundaries to binary:
   - $1000\,\text{H} \rightarrow 0001\ 0000\ 0000\ 0000_2$
   - $2\text{FFF}\,\text{H} \rightarrow 0010\ 1111\ 1111\ 1111_2$

   The four MSBs ($A_{15}A_{14}A_{13}A_{12}$) for the valid range are 0001 or 0010.

2. **Logic Requirements:** For $\overline{CS}$ to be active (logic 0):
   - A_{15} and A_{14} must be 0. If either is 1, the address is outside the range.
   - The bits $A_{13}A_{12}$ must be 01 or 10.
3. **Deriving the Expression:** The signal $\overline{CS}$ is 1 (inactive) if the address is invalid. The address is invalid if $A_{15} = 1$ OR $A_{14} = 1$ OR the combination $A_{13}A_{12}$ is 00 or 11. The combination ($A_{13}A_{12} = 00$ or 11) is equivalent to the XNOR operation:

$$(A_{13} \odot A_{12}) = (A_{13} \cdot A_{12} + \overline{A_{13}} \cdot \overline{A_{12}})$$

Combining these, the active-low logic expression is:

$$\overline{CS} = A_{15} + A_{14} + (A_{13} \cdot A_{12} + \overline{A_{13}} \cdot \overline{A_{12}})$$

$$\boxed{\text{(A) } A_{15} + A_{14} + (A_{13} \cdot A_{12} + \overline{A_{13}} \cdot \overline{A_{12}})}$$

Key Points

- **Active Low Logic:** For active-low inputs ($\overline{CS}$), the logic expression evaluates to 0 when the selection condition is met. This requires summing (ORing) the "invalid" address conditions.

Mistakes to be avoided

- **Active State Confusion:** Remember that for active-low signals, the chip is enabled when the logic equals 0. Never mistake the chip select expression for a standard active-high logic gate output.

Q8.16.2 MCQ 2016 2M Ans: C ●●○

To understand why a PUSH operation requires more clock cycles than a POP operation in the 8085 microprocessor, let us analyze the step-by-step execution of both instructions:

- **PUSH Operation:**
  1. The Stack Pointer (SP) is pre-decremented: SP ← SP − 1.
  2. The higher-order byte of the register pair is written to the memory location pointed to by SP.
  3. The Stack Pointer (SP) is decremented again: SP ← SP − 1.
  4. The lower-order byte of the register pair is written to the memory location pointed to by SP.
- **POP Operation:**
  1. The lower-order byte is read from the memory location currently pointed to by SP.
  2. The Stack Pointer (SP) is post-incremented: SP ← SP + 1.
  3. The higher-order byte is read from the memory location pointed to by SP.
  4. The Stack Pointer (SP) is incremented again: SP ← SP + 1.

Thus, the PUSH operation requires the stack pointer to be pre-decremented before any data write occurs, whereas the POP operation immediately reads data from the address already present in the stack pointer.

(C) The stack pointer needs to be pre-decremented before writing registers in a PUSH, whereas a POP operation uses the address

Key Points

- **PUSH Cycle (12 T-states):** Fetch (4T) + Memory Write (3T) + Memory Write (3T) + 2 extra T-states for the internal SP decrement pre-processing.
- **POP Cycle (10 T-states):** Fetch (4T) + Memory Read (3T) + Memory Read (3T).

Q8.16.3 MCQ 2016 1M Ans: D ●○○

To understand the execution of the RLC instruction, let us break down how the 8085 microprocessor processes data rotation:

- **Initial Conditions:**
  - Accumulator contents: A = A7H = $(1010\,0111)_2$
  - Carry Flag: CY = 0
- **Operation of RLC (Rotate Left Accumulator):** Unlike instructions that rotate data *through* the carry flag (such as RAL), RLC rotates the 8 bits of the accumulator circularly to the left. The most significant bit (D_7) has a **dual destination:**
  1. It is shifted directly into the least significant bit position (D_0).
  2. It overwrites the current value of the Carry Flag (CY).

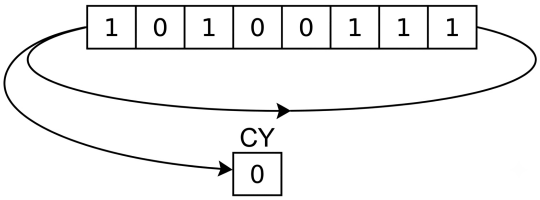


Fig. Solution Q8.16.3

Bit Position:	D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
Final Value:	0	1	0	0	1	1	1	**1**

CY: 1

Grouping the final binary string into 4-bit nibbles to determine the hexadecimal value:

- Higher nibble: $(0100)_2 = 4_H$
- Lower nibble: $(1111)_2 = F_H$

Therefore, the final accumulator content is $(4F)_H$ and the carry flag CY is 1.

(D) $4F$ and 1

Key Points

- **RLC vs RAL:** RLC moves D_7 directly to D_0 and CY. RAL moves D_7 to CY, and the old value of CY goes to D_0.
- The initial state of the carry flag (CY = 0) has no influence on the final result of an RLC command.

Q8.15.1 MCQ 2015 1M Ans: B ●○○

To determine which registers store the result of an arithmetic operation and its associated status flags in an 8085 microprocessor, let us look at the architecture of its Arithmetic Logic Unit (ALU):

- **Accumulator (Register A):**
  - The accumulator is an 8-bit register that is part of the ALU.
  - It is used to store 8-bit data and is always one of the default operands for any multi-operand arithmetic or logical instruction (such as addition or subtraction).
  - Crucially, after the ALU processes the operation, the final result is automatically routed and saved back into the **Accumulator (A)**.
- **Flag Register (Register F):**
  - The flag register is an 8-bit status register containing 5 active flip-flops acting as conditional flags.
  - These flags reflect the status conditions of the most recent data result generated by the ALU.
  - Any overflow bit resulting from an arithmetic addition sets or resets the Carry Flag (CY) located inside this **Flag Register (F)**.

Therefore, the registers that store the result of an addition and the overflow status bit are **A** and **F**, respectively.

(B) A and F

Key Points

- **8085 Flag Register Layout:** The 5 flags inside the 8-bit flag register (F) are structured as:

D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
S	Z	×	AC	×	P	×	CY

  Where S = Sign Flag, Z = Zero Flag, AC = Auxiliary Carry Flag, P = Parity Flag, and CY = Carry Flag (which tracks the final overflow/carry bit out of D_7).
- **Implicit Register Usage:** In instructions like `ADD B`, the operation performed is implicitly $A \leftarrow A + B$, showing that the accumulator serves as both an input operand source and the destination repository for the output.

Q8.15.2 MCQ 2015 1M Ans: D ●○○

To determine which instruction changes the contents of the accumulator, let us analyze the functionality of each given 8085 instruction:

- **A. `MOV B, M`:** This instruction moves an 8-bit data byte from the memory location pointed to by the HL register pair into register B. The contents of the accumulator are completely unaffected.
- **B. `PCHL`:** This instruction copies the contents of the HL register pair into the Program Counter (PC). Specifically, $PC_H \leftarrow H$ and $PC_L \leftarrow L$. It alters the program execution flow but does not modify the accumulator.
- **C. `RNZ`:** This is a conditional control transfer instruction (Return if No Zero). It checks the status of the Zero Flag (Z). If Z = 0, the program pops the return address from the stack into the Program Counter (PC). This instruction changes only the execution path and does not impact the accumulator data.
- **D. `SBI BEH`:** This instruction stands for **Subtract Immediate from Accumulator with Borrow**. It performs an arithmetic operation on the accumulator register:

$$A \leftarrow A - \text{BEH} - \text{CY}$$

Since the result of this subtraction is automatically updated and stored back in the accumulator, it directly changes the contents of the accumulator.

(D) SBI BEH

Key Points

- **Accumulator-centric Instructions:** Almost all immediate arithmetic/logical instructions like `ADI`, `SUI`, `ANI`, `XRI`, and `SBI` target the accumulator implicitly as both an operand source and destination.
- `SBI <data>` explicitly modifies the accumulator along with the flags (S, Z, AC, P, CY) according to the result of the subtraction operation.

Q8.15.3 MCQ 2015 1M Ans: C ● ● ●

The problem requires multiplying two 8-bit numbers stored in registers B and C using the successive addition method. In this method, the content of register C is added to the accumulator repeatedly, while register B acts as a down-counter to track the number of additions remaining.

- **Analysis of Option A:**

```
        MVI A, 00H    ; Load immediate value 00H into the Accumulator (A = 00H)
        JNZ LOOP      ; Jump to LOOP if Zero flag Z = 0 (Unpredictable behavior as flags are not yet initialized)
        CMP C         ; Compare Accumulator with Register C
LOOP:   DCR B         ; Decrement Register B by 1 (B = B − 1)
        HLT           ; Halt the processor execution
```

Explanation: This program contains a conditional jump before any arithmetic operations affect the status flags, lacks an addition step, and directly halts on the first iteration sequence.

- **Analysis of Option B:**

```
        MVI A, 00H    ; Initialize the Accumulator with value 00H
        CMP C         ; Compare Accumulator with Register C (Updates flags based on A − C)
LOOP:   DCR B         ; Decrement the loop counter in Register B
        JNZ LOOP      ; Jump back to LOOP if B ≠ 0
        HLT           ; Halt the processor
```

Explanation: Though the loop structure correctly decrements register B down to zero, the program only performs comparisons (‘CMP C‘) and completely omits any addition sequence. It does not calculate a product.

- **Analysis of Option C:**

```
        MVI A, 00H    ; Clear Accumulator (A = 00H) to store the running sum
LOOP:   ADD C         ; Accumulator ← Accumulator + Register C
        DCR B         ; Decrement Register B by 1. Updates the Zero flag (Z)
        JNZ LOOP      ; If B ≠ 0 (Zero flag Z = 0), jump back to LOOP
        HLT           ; Terminate the program when B = 0
```

Explanation: This program accurately implements the successive addition algorithm. It initializes the product tracking register (A) to zero, updates the product with the multiplicand (C), decrements the multiplier loop counter (B), and loops continuously until the counter runs down to zero.

- **Analysis of Option D:**

```
        MVI A, 00H      ; Initialize Accumulator A = 00H
        ADD C           ; Add Register C to Accumulator (A = A + C)
        JNZ LOOP        ; Jump to LOOP if Zero flag Z = 0
LOOP:   INR B           ; Increment Register B by 1 (B = B + 1)
        HLT             ; Halt the processor
```

Explanation: This program increments the counter variable ('INR B') instead of decrementing it towards zero. Furthermore, the 'HLT' instruction is positioned immediately following 'INR B', meaning the loop structure terminates prematurely on the first jump execution block.

```
        MVI A, 00H
LOOP    ADD C
(C)     DCR B
        JNZ LOOP
        HLT
```

Key Points

- **MVI r, data**: Moves an 8-bit immediate value into the designated register r.
- **ADD r**: Adds the contents of register r to the accumulator and stores the result back in the accumulator.
- **DCR r**: Decrements the designated register r by 1. Crucially, it updates the Zero flag based on the result.
- **JNZ label**: Jumps to the specified memory location label if the Zero flag is reset ($Z = 0$), which corresponds to a non-zero computational result.

Mistakes to be avoided

- **Flag dependencies**: Always ensure that conditional jump instructions directly follow the specific arithmetic or logical instructions meant to update the control flags.
- **Loop Direction**: Confirm that loop counters for deterministic loop executions run toward zero ('DCR') rather than away from it ('INR'), unless checking for specific overflow boundary conditions.

Q8.14.1 MCQ 2014 2M Ans: D ●●○

To determine the correct microprocessor instruction for data transfer, we must find the interfacing mechanism and the memory address or I/O port address mapped to the external device.

- **Step 1: Identify Interfacing Scheme (I/O Mapped vs. Memory Mapped)**
  - The signal IO/$\overline{M}$ from the 8085 microprocessor is connected directly to the active-low enable pin $\overline{G}_{2A}$ of the 3-to-8 decoder.
  - For the decoder to be enabled, $\overline{G}_{2A}$ must be active low (0). Therefore, IO/$\overline{M}$ = 0.
  - When IO/$\overline{M}$ = 0, the microprocessor executes memory operations. This indicates that the circuit uses a **Memory-Mapped I/O** configuration.
  - Consequently, memory instructions like 'LDA' or 'STA' must be used instead of peripheral I/O instructions like 'IN' or 'OUT'.

- **Step 2: Determine the Address Line Requirements** For the I/O device to be selected and read, its enabling conditions must be satisfied:

– **Decoder Active Condition:** The first 5-input AND gate must output a high signal (1) to satisfy the active-high enable pin G_1 of the decoder. Thus:

$$A_7A_6A_5A_4A_3 = 11111$$

– **Decoder Output Selection:** The line $\overline{DS_1}$ of the I/O device is connected to output 0 of the decoder. This output is active when the binary select inputs (C, B, A) match 000:

$$A_2A_1A_0 = 000$$

– **Device Second Chip Select (DS_2):** The second 8-input AND gate must output 1 to satisfy the active-high select pin DS_2. Accounting for the active-low inverters on A_8 and A_9:

$$\overline{A}_{15}\overline{A}_{14}\overline{A}_{13}\overline{A}_{12}\overline{A}_{11}A_{10}\overline{A}_9\overline{A}_8 = 1 \implies A_{15}A_{14}A_{13}A_{12}A_{11}A_{10}A_9A_8 = 11111000$$

- **Step 3: Map the Full 16-bit Hexadecimal Address** Combining all the address bits derived above into groups of 4 from MSB to LSB:

A_{15}	A_{14}	A_{13}	A_{12}	A_{11}	A_{10}	A_9	A_8	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0
1	1	1	1	1	0	0	0	1	1	1	1	1	0	0	0
F				**8**				**F**				**8**			

The exact memory address is F8F8H. Since the operation inputs data from the external device into the microprocessor, the correct instruction is 'LDA F8F8H'.

(D) LDA F8F8H

Key Points

- **Memory-Mapped I/O**: Uses the standard 16-bit memory address space, and peripherals are accessed using standard memory reference group instructions like 'LDA' or 'STA'.
- **Decoder Logic**: For an active-low output pin Y_n to deliver a logic 0, the overall chip enables must be fully asserted, and the selection inputs must match the binary value of n.

Mistakes to be avoided

- Do not mistakenly choose 'IN F8H'. Although the configuration transfers digital inputs, the control connection path ties to IO/$\overline{M}$ = 0, explicitly disqualifying peripheral I/O instructions.

Q8.14.2 MCQ 2014 2M Ans: A ●●○

The instruction 'STA 1234H' is a 3-byte absolute address storage instruction that transfers the content of the Accumulator to memory location 1234H. The operation requires 4 machine cycles (M_1 to M_4) consisting of a total of 13 states (13 T).

- **Analysis of Machine Cycles and Higher-Order Address Bits ($A_{15} - A_8$):**

1. **Opcode Fetch (M_1):** The microprocessor fetches the operational instruction code from the starting execution address location 1FFEH.

   Higher-order address byte $(A_{15} - A_8)$ = 1FH

2. **Operand 1 Read / Lower-Byte Address Read (M_2):** The microprocessor increments its internal program counter to 1FFFH to read the lower byte of the target memory parameter space (34H).

   Higher-order address byte $(A_{15} - A_8)$ = 1FH

3. **Operand 2 Read / Higher-Byte Address Read (M_3):** The program counter increments to 2000H to read the higher-order byte of the target memory parameter space (12H).

$$\text{Higher-order address byte } (A_{15} - A_8) = 20\text{H}$$

4. **Memory Write Operation (M_4):** The microprocessor control subsystem writes the architectural configuration content stored inside the internal Accumulator register out onto the evaluated target absolute destination memory slot 1234H.

$$\text{Higher-order address byte } (A_{15} - A_8) = 12\text{H}$$

- **Verification of Options:**
  - **Option A (1FH, 1FH, 20H, 12H):** Correctly matches the higher-order address sequence generated across all four consecutive bus operations.
  - **Option B:** Includes incorrect state counts and intermediate address values.
  - **Option C:** Fails to account for the tracking rollover state when transitioning from memory location 1FFFH to 2000H.
  - **Option D:** Incorrectly shifts the machine state execution alignment sequences.

(A) 1FH, 1FH, 20H, 12H

Key Points

- During any memory tracking step, the 16-bit address pins ($A_{15} - A_0$) are split. Pins $A_{15} - A_8$ retain the higher byte of the current address for the duration of that specific machine cycle.
- Program execution sequentially advances address pointers via automatic increments (1FFEH → 1FFFH → 2000H) when pulling sequential Multi-byte instruction streams.

Mistakes to be avoided

- Do not overlook address line modifications during parameter fetches. Moving from 1FFFH to 2000H modifies the upper byte from 1FH to 20H.

Q8.13.1 MCQ 2013 2M Ans: D ● ● ●

Each memory chip has a storage capacity of 1024 Bytes = 2^{10} Bytes. This indicates that 10 address lines ($A_9 - A_0$) are directly routed to the internal address ports of all four chips for byte decoding inside the chips.

- **Analysis of Chip Selection Enable Logic:**
  - For any of the RAM chips to become operational, the 4-input AND gate must evaluate to a logic high (1) to feed the active-high enable input pin of the 2-to-4 demultiplexer/decoder.
  - Examining the logical connections with inverters linked to the AND gate input terminals:

$$\overline{A}_{15} \cdot \overline{A}_{14} \cdot A_{11} \cdot \overline{A}_{10} = 1$$

  - This forces the explicit structural conditions:

$$A_{15} = 0, \quad A_{14} = 0, \quad A_{11} = 1, \quad A_{10} = 0$$

- **Analysis of Decoder Selection Lines (S_1, S_0):**

– The routing connections for the selection inputs of the demultiplexer are mapped as:

$$S_1 = A_{13}, \quad S_0 = A_{12}$$

– The demultiplexer selectively paths the enabled active high routing line based on the binary pairing values of $(A_{13}A_{12})$:
  * If $A_{13}A_{12} = 00$, the decoder output line 00 triggers RAM#1.
  * If $A_{13}A_{12} = 01$, the decoder output line 01 triggers RAM#2.
  * If $A_{13}A_{12} = 10$, the decoder output line 10 triggers RAM#3.
  * If $A_{13}A_{12} = 11$, the decoder output line 11 triggers RAM#4.

- **Address Boundary Conversions to Hexadecimal:**

1. **Memory Slot Allocation for RAM#1 ($A_{13}A_{12} = 00$):**
   – Starting Range Address ($A_9 - A_0 = 0000000000_2$):

   $$0000\ 1000\ 0000\ 0000_2 = 0800H$$

   – Ending Range Address ($A_9 - A_0 = 1111111111_2$):

   $$0000\ 1011\ 1111\ 1111_2 = 0BFFH$$

2. **Memory Slot Allocation for RAM#2 ($A_{13}A_{12} = 01$):**
   – Starting Range Address ($A_9 - A_0 = 0000000000_2$):

   $$0001\ 1000\ 0000\ 0000_2 = 1800H$$

   – Ending Range Address ($A_9 - A_0 = 1111111111_2$):

   $$0001\ 1011\ 1111\ 1111_2 = 1BFFH$$

3. **Memory Slot Allocation for RAM#3 ($A_{13}A_{12} = 10$):**
   – Starting Range Address ($A_9 - A_0 = 0000000000_2$):

   $$0010\ 1000\ 0000\ 0000_2 = 2800H$$

   – Ending Range Address ($A_9 - A_0 = 1111111111_2$):

   $$0010\ 1011\ 1111\ 1111_2 = 2BFFH$$

4. **Memory Slot Allocation for RAM#4 ($A_{13}A_{12} = 11$):**
   – Starting Range Address ($A_9 - A_0 = 0000000000_2$):

   $$0011\ 1000\ 0000\ 0000_2 = 3800H$$

   – Ending Range Address ($A_9 - A_0 = 1111111111_2$):

   $$0011\ 1011\ 1111\ 1111_2 = 3BFFH$$

(D) 0800H-0BFFH, 1800H-1BFFH, 2800H-2BFFH, 3800H-3BFFH

Key Points

- Address lines that form connections to chip logic enables define the base sector offsets, while individual internal address lines ($A_9 - A_0$) map out the contiguous range blocks inside the chip boundaries.
- Interfacing configuration decoders decode non-contiguous structural bit blocks like $A_{13}A_{12}$ to distinguish sequential page blocks.

Mistakes to be avoided

- Be careful with address lines when constructing the bit groups for hexadecimal conversions; check whether lines are inverted or held constant before computing boundaries.

Q8.13.2 MCQ 2013 1M Ans: A ●●○

To determine the final contents of the accumulator, we trace the step-by-step execution of the given 8085 assembly program.

- **Step 1: Initialization**
  - 'MVI A, 05H': Loads the immediate hexadecimal value 05H into the Accumulator register (A = 05H).
  - 'MVI B, 05H': Loads the immediate hexadecimal value 05H into register B (B = 05H), which serves as the loop counter.

- **Step 2: Loop Execution (Successive Addition)** The loop starts at label 'PTR' and continues adding the changing contents of register B to the accumulator until the value in register B becomes zero.
  - **Iteration 1:**
    - 'PTR: ADD B' $\implies$ $A = A + B$ = 05H + 05H = 0AH
    - 'DCR B' $\implies$ B = 05H – 1 = 04H
    - 'JNZ PTR' $\implies$ Since $B \neq 0$ (Zero Flag $Z = 0$), the program jumps back to 'PTR'.
  - **Iteration 2:**
    - 'ADD B' $\implies$ A = 0AH + 04H = 0EH
    - 'DCR B' $\implies$ B = 03H
    - 'JNZ PTR' $\implies$ Loops back since $B \neq 0$.
  - **Iteration 3:**
    - 'ADD B' $\implies$ A = 0EH + 03H = 11H
    - 'DCR B' $\implies$ B = 02H
    - 'JNZ PTR' $\implies$ Loops back since $B \neq 0$.
  - **Iteration 4:**
    - 'ADD B' $\implies$ A = 11H + 02H = 13H
    - 'DCR B' $\implies$ B = 01H
    - 'JNZ PTR' $\implies$ Loops back since $B \neq 0$.
  - **Iteration 5:**
    - 'ADD B' $\implies$ A = 13H + 01H = 14H
    - 'DCR B' $\implies$ B = 00H
    - 'JNZ PTR' $\implies$ Since $B = 0$ (Zero Flag $Z = 1$), the loop terminates and control moves to the next instruction.

- **Step 3: Final Arithmetic Operation**

– 'ADI 03H': Adds the immediate value 03H to the current content of the accumulator.

$$A = 14H + 03H = 17H$$

– 'HLT': Halts the execution of the microprocessor.

(A) 17H

Key Points

- **ADD r**: Adds the contents of register r to the accumulator and updates the flags based on the sum.
- **DCR r**: Decrements register r by 1 and directly modifies the Zero flag (Z).
- **Hexadecimal Addition**: Ensure all math additions follow base-16 rules (e.g., 05H + 05H = 0AH).

Mistakes to be avoided

- Do not treat register B as a fixed constant inside the loop; it is decremented during each pass, altering the value added to the accumulator in subsequent iterations.

Q8.11.1 MCQ 2011 2M Ans: C ●●○

The program is executed instruction by instruction starting with the initial conditions. The carry flag (CY) is initially 0.

1. **`MVI A, 07H`** This moves the hexadecimal value 07H immediately into the Accumulator (Register A). In 8-bit binary:

$$A = 0000\,0111_2$$

2. **`RLC`** This instruction stands for **Rotate Accumulator Left**. Every single bit in Register A shifts left by one position. The leftmost bit (D_7) leaves its position and loops around to fill the rightmost empty spot (D_0), and it also copies itself into the Carry flag (CY).

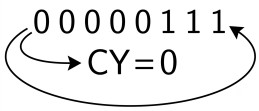


Fig. Solution Q8.11.1

$$A = 0000\,1110_2 = 0E\text{ H}, \quad CY = 0$$

3. **`MOV B, A`** This copies the current contents of Register A straight into Register B.

$$B = 0000\,1110_2 = 0E\text{ H}$$

4. **`RLC, RLC`** We rotate the bits of Register A left twice.

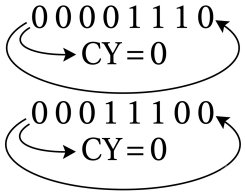


Fig. Solution Q8.11.1

$$A = 0011\,1000_2 = 38H, \quad CY = 0$$

5. **`ADD B`** This adds the binary value inside Register B to the current value inside Register A and stores the final result right back in Register A ($A = A + B$):

$$\begin{array}{rl} 0011\,1000_2 & (38H) \\ +\ 0000\,1110_2 & (0E\,H) \\ \hline 0100\,0110_2 & (46H) \end{array}$$

   Now, $A = 0100\,1110_2$.

6. **`RRC`** This stands for **Rotate Accumulator Right**. Every bit in Register A moves right by one position. The rightmost bit (D_0) leaves its spot, loops around to fill the leftmost open space (D_7), and copies itself into the Carry flag (CY).

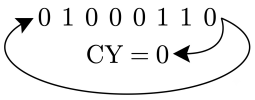


Fig. Solution Q8.11.1

$$A = \mathbf{0}010\,0011_2, \quad CY = 0$$

Converting the final binary sequence $0010\,0111_2$ into standard hex notation gives:

$$0010_2 = 2, \quad 0111_2 = 7 \implies A = 23H$$

(C) 23H

Key Points

- **`RLC` (Rotate Left):** Moves every bit left. D_7 goes to D_0 and also copies into the Carry flag (CY).
- **`RRC` (Rotate Right):** Moves every bit right. D_0 goes to D_7 and also copies into the Carry flag (CY).
- **`ADD r`:** Executes binary addition between the target register and the accumulator ($A = A + r$).

Q8.10.1 MCQ 2010 2M Ans: C ●●○

The program executes line by line, modifying the data inside the 8-bit micro-storage units called registers. Let's track the values step by step:

1. **`3000 MVI A, 45H`** This loads the hexadecimal value 45H directly into the main register, the Accumulator (Register A). In 8-bit binary representation:

$$A = 0100\,0101_2$$

2. **`3002 MOV B, A`** This copies the contents of the accumulator A into Register B.

$$B = 0100\,0101_2$$

3. **`3003 STC`** This stands for **Set Carry**. It sets Carry flag (CY) to 1.

$$CY = 1$$

4. **3004 CMC** This stands for **Complement Carry**. It flips the current state of the Carry flag to its opposite value.

$$\text{CY} = 0$$

5. **3005 RAR** The RAR instruction shifts every bit in the Accumulator one position to the right through a 9-bit loop. The rightmost bit (D_0) moves into the Carry flag (CY), while the old value of CY fills the leftmost slot (D_7).

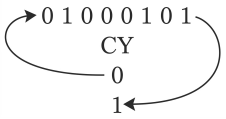


Fig. Solution Q8.10.1

$$A = \mathbf{0}010\,0010_2, \quad \text{CY} = 1$$

6. **3006 XRA B** This performs a bit-by-bit logical **XOR** comparison between the current elements in Register A and Register B ($A = A \oplus B$).

$$\begin{array}{ll} 0010\,0010_2 & \text{(Current } A\text{)} \\ \oplus\ 0100\,0101_2 & \text{(Stored } B\text{)} \\ \hline 0110\,0111_2 & \text{(New value in } A\text{)} \end{array}$$

Converting the final binary sequence back to hex yields $0110_2 = 6$ and $0111_2 = 7$, resulting in $A = 67\text{H}$. Thus, the final contents of the accumulator are 67H.

(C) 67H

Key Points

- **RAR:** Rotates the bits rightwards, treating the Carry flag as a 9th bit in the shifting loop.
- **XRA B Operation:** XORs the content of the accumulator with the contents in register B.

Q8.10.2 MCQ 2010 1M Ans: B ●●○

To determine the address range for the device connected to the active-low output $\overline{Y_5}$, we need to analyze the enabling conditions of the 74LS138 3-to-8 decoder along with its select lines.

1. Decoder Enable Inputs

For the 74LS138 decoder to function, all of its enable pins must be active simultaneously:

- **Enable G_1 (Active-High):** The input signal IO/$\overline{\text{M}}$ passes through an inverter before reaching G_1. Thus, to get $G_1 = 1$, we require:

$$\text{IO}/\overline{\text{M}} = 0 \implies \text{Memory mapped operation}$$

- **Enable $\overline{G_{2A}}$ (Active-Low):** This pin is permanently tied to ground in the circuit layout, meaning it is unconditionally enabled (0).
- **Enable $\overline{G_{2B}}$ (Active-Low):** This pin must receive a low signal (0) from the output of the 5-input logic gate. The gate is an active-low input NAND configuration acting as a NOR match logic:

$$\overline{A_{15}} \cdot A_{14} \cdot \overline{A_{13}} \cdot A_{12} \cdot \overline{A_{11}} = 1 \implies A_{15}A_{14}A_{13}A_{12}A_{11} = 00101_2$$

2. Decoder Select Lines

To select the specific output pin $\overline{Y_5}$, the binary select inputs CBA must represent the decimal value 5 (101_2):

$$C = A_{10} = 1, \quad B = A_9 = 0, \quad A = A_8 = 1 \implies A_{10}A_9A_8 = 101_2$$

3. Complete Address Range Calculation

Combining the fixed upper address bits with the unmapped lower lines (A_7 to A_0) that can vary freely from all-0s to all-1s, we form the total 16-bit address block:

Boundary	A_{15}	A_{14}	A_{13}	A_{12}	A_{11}	A_{10}	A_9	A_8	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0	Hex Value
Lower Limit	0	0	1	0	1	1	0	1	0	0	0	0	0	0	0	0	2D00H
Upper Limit	0	0	1	0	1	1	0	1	1	1	1	1	1	1	1	1	2DFFH

Grouping the bits into nibbles reveals the valid range as 2D00H – 2DFFH, matching option (B).

(B) 2D00 – 2DFF

Key Points

- **74LS138 Decoder Function:** Maps a 3-bit binary code on input pins CBA to one of 8 active-low outputs, provided $G_1 = 1$, $\overline{G_{2A}} = 0$, and $\overline{G_{2B}} = 0$.
- **Address Mapping Procedure:** Secure the enable conditions first, set the destination code line bits, and fill remaining lower-order lines to capture full boundary extremes.

Q8.09.1 MCQ | 2009 | 1M | Ans: D | ●●○

To identify the type of interrupt described, let's analyze its characteristics based on how it handles addresses and execution controls:

1. **Vectored vs. Non-Vectored (Address Location):**
   - The problem states that the service routine starts from a **fixed location of memory** which cannot be externally altered.
   - When an interrupt has a predefined, hardwired memory address for its execution routine, it is classified as a **vectored interrupt**.
2. **Maskable vs. Non-Maskable (Delay or Rejection Control):**
   - The problem states that the interrupt **can be delayed or rejected** by the processor.
   - If the processor has the authority to ignore, disable, or postpone an incoming request, the interrupt is classified as a **maskable interrupt**.

Combining both properties, the interrupt is both **maskable and vectored**, which aligns with option (D).

(D) maskable and vectored

Key Points

- **Vectored Interrupt:** Automatically points the Program Counter to a fixed vector address table entry without requiring external address inputs.
- **Maskable Interrupt:** Can be enabled or disabled via soft-setting internal status register bits (like the Interrupt Enable

flag).

Q8.08.1 MCQ 2008 2M Ans: C ●●○

To determine the final contents of the Program Counter (PC) and the HL register pair, we track the assembly instructions execution line by line:

1. **`2710 LXI H, 30A0H`** This instruction stands for **Load Register Pair Immediate**. It loads the 16-bit hexadecimal constant 30A0H directly into the HL register pair.

$$HL = 30A0H$$

2. **`2713 DAD H`** This instruction stands for **Double Add Accumulator/Register-pair**. It adds the 16-bit contents of the specified register pair (HL) to the HL register pair itself (HL = HL + HL). Performing hexadecimal addition:

Hexadecimal:	**3**	**0**	**A**	**0**	H
Binary Grouping:	(0011)	(0000)	(1010)	(0000)	
	0011	0000	1010	0000	(Initial HL)
+	0011	0000	1010	0000	(Initial HL)
Binary Sum:	0110	0001	0100	0000	
Hex Conversion:	↓	↓	↓	↓	
Result:	**6**	**1**	**4**	**0**	H

Thus, the contents of the register pair become HL = 6140H.

3. **`2714 PCHL`** This instruction copies or transfers the 16-bit contents of the HL register pair straight into the Program Counter (PC ← HL).
   - The value inside HL remains unchanged by this transfer.
   - Therefore, both registers now share the exact same value:

$$PC = 6140H, \quad HL = 6140H$$

This matches the values stated in option (C).

$$\boxed{\text{(C) } PC = 6140H, HL = 6140H}$$

Key Points

- **`DAD rp`:** Executes 16-bit addition with the HL register pair as the destination target.
- **`PCHL`:** Performs an unconditional structural branch by overwriting the PC address tracking register with the current data held in HL.

Q8.07.1 MCQ 2007 2M Ans: B ● ● ○

Given the 8085 assembly language program, we trace the execution line by line:

- **Line 1:** `MVI A, B5H` → Loads the accumulator A with B5H.

$$A = \text{B5H} = 1011\,0101_2$$

- **Line 2:** `MVI B, 0EH` → Loads register B with 0EH.

$$B = \text{0EH} = 0000\,1110_2$$

- **Line 3:** `XRI 69H` → Performs Exclusive-OR operation between the contents of A and 69H ($0110\,1001_2$).

$$\begin{array}{rcl} 1011\,0101_2 & \rightarrow & \text{B5H} \\ \oplus\, 0110\,1001_2 & \rightarrow & \text{69H} \\ \hline 1101\,1100_2 & \rightarrow & \text{DCH} \end{array}$$

- **Line 4:** `ADD B` → Adds the contents of register B to the accumulator A.

$$\begin{array}{rcl} 1101\,1100_2 & \rightarrow & \text{DCH} \\ +\, 0000\,1110_2 & \rightarrow & \text{0EH} \\ \hline 1110\,1010_2 & \rightarrow & \text{EAH} \end{array}$$

Thus, the contents of the accumulator just after the execution of the `ADD` instruction in line 4 is EAH.

(B) EAH

Key Points

- `XRI` updates the Zero (Z), Sign (S), and Parity (P) flags based on the result, while Carry (CY) and Auxiliary Carry (AC) are cleared (0).
- `ADD` affects all conditional status flags based on the arithmetic sum.

Q8.07.2 MCQ 2007 2M Ans: C ● ● ○

Continuing the execution from line 4 where $A = \text{EAH} = 1110\,1010_2$:

- **Line 5:** `ANI 9BH` → Performs a bitwise AND operation between A and 9BH ($1001\,1011_2$).

$$\begin{array}{rcl} 1110\,1010_2 & \rightarrow & \text{EAH} \\ \cdot\, 1001\,1011_2 & \rightarrow & \text{9BH} \\ \hline 1000\,1010_2 & \rightarrow & \text{8AH} \end{array}$$

- **Line 6:** `CPI 9FH` → Compares the accumulator A (8AH) with the immediate data 9FH by performing an internal subtraction ($A - 9\text{FH}$).

$$\text{Since } A = \text{8AH} < \text{9FH}$$

- For a comparison instruction (`CPI` or `CMP`) when $A <$ data:
  - The Carry flag is set: $CY = 1$.

– The Zero flag is cleared: $Z = 0$ (since the results are not equal).

- **Line 7:** `STA 3010H` → Stores the content of A (8AH) into memory location 3010H. This data transfer instruction does not affect any flags.

Therefore, the status of the flags after line 7 is $CY = 1$ and $Z = 0$.

$$\boxed{\text{(C) } CY = 1, Z = 0}$$

Key Points

- `CPI` flag conditions:
  - If $A <$ data $\rightarrow CY = 1, Z = 0$
  - If $A =$ data $\rightarrow CY = 0, Z = 1$
  - If $A >$ data $\rightarrow CY = 0, Z = 0$
- Data transfer instructions like `STA` do not alter the status flags.

Mistakes to be avoided

- **Incorrect Flag Update:** Do not change the flag status during the execution of `STA 3010H`, as move and store operations in the 8085 preserve previous flag states.

Q8.07.3 MCQ 2007 2M Ans: C ●●○

To decode the address range for which the 8255 PPI chip gets selected, we look at the chip select ($\overline{\text{CS}}$) logic.

Method 1

The chip select input $\overline{\text{CS}}$ is active-low ($\overline{\text{CS}} = 0$). For the output of the NAND gate to be 0, all of its inputs must be at logic 1:

$$A_7 = 1,\ A_6 = 1,\ A_5 = 1,\ A_4 = 1,\ A_3 = 1,\ \text{and IO}/\overline{\text{M}} = 1$$

The address range can be determined by varying the remaining lines as shown below:

A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0		
1	1	1	1	1	0	0	0	→	F8H
1	1	1	1	1	1	1	1	→	FFH

Thus, the total address range for chip selection is F8H – FFH.

Method 2

The address lines are assigned distinct functions within the interface configuration:

- A_1 and A_0 are tied directly to the 8255 for internal port selection.
- A_2 is unused and acts as a don't care ($\times$) bit.
- A_3 to A_7 are routed to the NAND gate for chip selection.

Evaluating the address mapping based on the state of the don't care bit A_2:

$IO/\overline{M}$	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0
1	1	1	1	1	1	×	0	0
1	1	1	1	1	1	×	0	1
1	1	1	1	1	1	×	1	0
1	1	1	1	1	1	×	1	1

- If $A_2 = 0$, the address range spans from F8H to FBH.
- If $A_2 = 1$, the address range spans from FCH to FFH.

Combining both states, the overall chip selection range is F8H to FFH.

(C) F8H – FFH

Key Points

- In an I/O mapped I/O scheme, the 8085 microprocessor uses an 8-bit address line replicated on both upper ($A_{15} - A_8$) and lower ($A_7 - A_0$) address buses.
- Unused lines within the decoding boundary function as don't cares (×), causing foldback or mirror memory spaces.

Mistakes to be avoided

- **Ignoring the Don't Care Line:** Do not assume $A_2 = 0$ permanently. Since it is completely missing from the NAND gate decoder input list, it must be evaluated for both 0 and 1 boundaries.

Q8.06.1 MCQ 2006 2M Ans: B ●●○

The given address range for the I/O device is D4 H to D7 H. Converting these hexadecimal addresses into their 8-bit binary representation (A_7 to A_0):

$$\text{D4 H} = 1101\ 0100_2$$
$$\text{D7 H} = 1101\ 0111_2$$

Let us analyze the bit pattern of the address range:

A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0
1	1	0	1	0	1	0	0
1	1	0	1	0	1	1	1

From the decoder circuit diagram, the inputs to the 3-to-8 decoder are connected as:

- MSB = A_4
- Middle Bit = A_3
- LSB = A_2

For the given address range, the values of these specific address lines remain constant at:

$$A_4A_3A_2 = 101_2 = 5_{10}$$

Thus, the input combination corresponds to selection line 5. When the decoder is enabled by the logic gate output at the $\overline{EN}$ terminal ($A_7 = 1, A_6 = 1, A_5 = 0 \implies$ AND gate output $= 1 \implies \overline{EN} = 0$), output line 5 becomes active. Therefore, the chip-select ($\overline{CS}$) line of the peripheral device must be connected to output 5.

(B) output 5

Key Points

- In I/O interfacing, the fixed address bits determine the decoder selection and chip enable signals, while the varying lower bits define individual registers within the peripheral.
- A standard 3-to-8 decoder decodes the binary value of its select lines (MSB A_4, A_3, LSB A_2) to activate the corresponding decimal output lines.

Q8.06.2 MCQ 2006 2M Ans: B ●●○

Let us trace the program execution block step-by-step and monitor the state of the Stack Pointer (SP) and relevant registers:

1. **`LXI SP, EFFF H`**: Initializes the stack pointer.

   SP = EFFF H

2. **`CALL 3000 H`**: Pushes the 16-bit return address (address of the next sequential instruction in the main program) onto the stack and decrements SP by 2.

   SP = EFFF H − 2 = EFFD H

3. **`3000 H LXI H, 3CF4 H`**: Loads the 16-bit value 3CF4 H into the HL register pair.

   HL = 3CF4 H $\implies$ H = 3C H, L = F4 H

4. **`PUSH PSW`**: Pushes the Program Status Word (A register and Flags) onto the stack and decrements SP by 2.

   SP = EFFD H − 2 = EFFB H

5. **`SPHL`**: Copies the contents of the HL register pair directly into the stack pointer.

   SP = HL = 3CF4 H

6. **`POP PSW`**: Pops the top 2 bytes from the stack into the PSW register pair and increments SP by 2.

   SP = 3CF4 H + 2 = 3CF6 H

7. **`RET`**: Pops the 16-bit return address from the top of the stack into the Program Counter (PC) to return to the main program, incrementing SP by 2.

   SP = 3CF6 H + 2 = 3CF8 H

Thus, on completion of the `RET` instruction execution, the content of SP is 3CF8 H.

(B) 3CF8 H

Key Points

- **`PUSH`** and **`CALL`** operations decrement the stack pointer (SP = SP − 2).
- **`POP`** and **`RET`** operations increment the stack pointer (SP = SP + 2).
- **`SPHL`** is a register transfer instruction that completely overwrites the existing SP value with the contents of the HL pair.

Q8.05.1 MCQ 2005 2M Ans: D ●●○

Each memory chip has a capacity of 256 Bytes = 2^8 Bytes. Therefore, 8 address lines (A_7 to A_0) are connected directly to both chips to address individual bytes internally.
Let us find the chip enable conditions for both chips based on the logic gates provided in the circuit:

- **For Chip #1:** The chip select is driven by the output of an AND gate with inputs A_8 and $\overline{A_9}$. For active-high selection, the enable signal is 1 when:
$$A_8 = 1 \quad \text{and} \quad A_9 = 0 \implies A_9A_8 = 01_2$$
- **For Chip #2:** The chip select is driven by the output of an AND gate with inputs A_9 and $\overline{A_8}$. For active-high selection, the enable signal is 1 when:
$$A_9 = 1 \quad \text{and} \quad A_8 = 0 \implies A_9A_8 = 10_2$$

Thus, for any memory address range to be represented by either Chip #1 or Chip #2, the bit combination A_9A_8 must be either 01_2 or 10_2. Let us verify the choices by converting the hexadecimal address ranges to binary to examine the status of the A_9 and A_8 bits:

- **Option (A) [0100 H - 02FF H]:**
$$0100\,\text{H} = 0000\ 0000\ \mathbf{1}000\ 0000_2 \implies A_9A_8 = 01_2$$
$$02FF\,\text{H} = 0000\ 0001\ \mathbf{0}111\ 1111_2 \implies A_9A_8 = 10_2$$
This range is valid since it maps to the chips.
- **Option (B) [1500 H - 16FF H]:**
$$1500\,\text{H} = 0001\ 0100\ \mathbf{1}000\ 0000_2 \implies A_9A_8 = 01_2$$
$$16FF\,\text{H} = 0001\ 0110\ \mathbf{1}111\ 1111_2 \implies A_9A_8 = 10_2$$
This range is valid since it maps to the chips.
- **Option (C) [F900 H - FAFF H]:**
$$F900\,\text{H} = 1111\ 1001\ \mathbf{0}000\ 0000_2 \implies A_9A_8 = 10_2$$
$$FAFF\,\text{H} = 1111\ 1010\ \mathbf{1}111\ 1111_2 \implies A_9A_8 = 01_2$$
This range is valid since it maps to the chips.
- **Option (D) [F800 H - F9FF H]:**
$$F800\,\text{H} = 1111\ 1000\ \mathbf{0}000\ 0000_2 \implies A_9A_8 = 00_2$$
Since $A_9A_8 = 00_2$, neither chip is selected at the beginning of this range.

Hence, the address range F800 H – F9FF H is NOT represented by Chip #1 and Chip #2.

(D) F800 - F9FF

Key Points

- The total number of internal addressable locations inside a memory chip determines the number of lower-order address lines routed to it (2^n = size in bytes).
- The remaining higher-order address lines are decoded using combinational logic gates to generate specific chip select ($\overline{CS}$ or CS) signals.

Q8.05.2 MCQ 2005 2M Ans: C ● ● ○

Let us trace the program step-by-step from memory address 0100 H onwards:

1. **`0100 H LXI SP, 00FF H`**: Loads the stack pointer register with 00FF H.
2. **`0103 H LXI H, 0107 H`**: Loads the 16-bit address 0107 H into the HL register pair.

$$\text{HL} = 0107\ \text{H} \implies \text{Memory pointer } M = [0107\ \text{H}]$$

3. **`0106 H MVI A, 20H`**: Loads the accumulator (A) with the immediate value 20 H.

$$\text{A} = 20\ \text{H}$$

4. **`0108 H SUB M`**: Subtracts the contents of the memory location pointed by HL (which is address 0107 H) from the accumulator.

   Looking closely at the program bytes layout, the instruction `LXI H, 0107 H` occupies 3 bytes (0103 H, 0104 H, 0105 H), where the low-order byte 07 H is stored at 0104 H and the high-order byte 01 H is stored at 0105 H.

   The instruction `MVI A, 20H` occupies 2 bytes (0106 H and 0107 H). Therefore, the memory location 0107 H holds the data byte value 20 H.

$$\text{Content of } M = [0107\ \text{H}] = 20\ \text{H}$$

   Executing the subtraction:

$$\text{A} = \text{A} - M = 20\ \text{H} - 20\ \text{H} = 00\ \text{H}$$

When the program counter reaches 0109 H, the execution of `SUB M` is complete, and the content of the accumulator is 00 H.

(C) 00H

Key Points

- In the 8085 microprocessor, M refers to the memory location whose address is held by the HL register pair.
- Multi-byte instructions store immediate data or addresses in subsequent memory locations, which can overlap with memory pointers if targeted directly by the program logic.

Q8.05.3 MCQ 2005 2M Ans: C ● ● ○

Continuing the execution trace from memory location 0109 H with the current accumulator value A = 00 H and HL = 0107 H:

1. **`0109 H ORI 40H`**: Performs a bitwise logical OR operation between the accumulator contents and the immediate data 40 H.

$$\text{A} = 00\ \text{H} \vee 40\ \text{H} = 40\ \text{H}$$

2. **`010B H ADD M`**: Adds the content of the memory location pointed by HL (address 0107 H) to the accumulator.

   As verified in the preceding block, the value at memory address 0107 H is the immediate data byte 20 H from the earlier `MVI A, 20H` instruction.

$$M = [0107\ \text{H}] = 20\ \text{H}$$

   Executing the addition:

$$\text{A} = \text{A} + M = 40\ \text{H} + 20\ \text{H} = 60\ \text{H}$$

Thus, the final value remaining in the accumulator after the last instruction executes is 60 H.

(C) 60H

Key Points

- `ORI` modifies the accumulator state through bit-by-bit logical disjunction.
- The value of the memory operand *M* remains unchanged unless an explicit instruction writes to the memory space pointed to by HL.

Q8.04.1 MCQ 2004 2M Ans: D ●●○

Let us analyze the operating modes of the 8255 Programmable Peripheral Interface (PPI) for the given statements:

I. **Statement I:** The conversion is initiated by a signal from Port C, and a handshake control signal on Port C causes data to be strobed into Port A. This configuration describes a strobed input operation where control/handshaking lines of Port C support the data transfer on Port A. This corresponds exactly to **Mode 1** (Strobed Input/Output mode).

II. **Statement II:** Two computers exchange data using a pair of 8255s where Port A works as a bidirectional data port supported by appropriate handshaking signals. The bidirectional bus I/O operation is uniquely supported only by Port A in **Mode 2** (Strobed Bidirectional Bus I/O mode).

Therefore, the appropriate modes of operation for I and II are Mode 1 and Mode 2 respectively.

(D) Mode 1 for I and Mode 2 for II

Key Points

- **Mode 0:** Simple Input/Output mode with no handshaking lines.
- **Mode 1:** Strobed Input/Output mode where Ports A and B transfer data utilizing dedicated pins of Port C for handshaking signals.
- **Mode 2:** Strobed Bidirectional Bus I/O mode which allows Port A to act as an 8-bit bidirectional data bus using 5 lines of Port C for control.

Q8.04.2 MCQ 2004 2M Ans: B ●●○

Let us break down each 8085 instruction into its constituent machine cycles:

I. `LDA 3000H` (Load Accumulator Direct): This is a 3-byte instruction that loads the content of memory location 3000 H directly into the accumulator. It requires the following **4 machine cycles**:

- **Opcode Fetch** (4 T-states) – Fetches the operational code for `LDA`.
- **Memory Read** (3 T-states) – Fetches the lower order address byte (00 H).
- **Memory Read** (3 T-states) – Fetches the higher order address byte (30 H).
- **Memory Read** (3 T-states) – Reads the data byte located at memory address 3000 H into the Accumulator.

Total memory cycles for instruction I = **4**.

II. `LXI D, F0F1H` (Load Register Pair Immediate): This is a 3-byte instruction that loads the 16-bit immediate data F0F1 H into the DE register pair. It requires the following **3 machine cycles**:

- **Opcode Fetch** (4 T-states) – Fetches the operational code for `LXI D`.
- **Memory Read** (3 T-states) – Fetches the lower order data byte (F1 H) into register E.
- **Memory Read** (3 T-states) – Fetches the higher order data byte (F0 H) into register D.

Total memory cycles for instruction II = **3**.

Thus, the number of memory cycles required are 4 for I and 3 for II.

(B) 4 for I and 3 for II

Key Points

- A machine/memory cycle is defined as the time required by the microprocessor to complete one access operation with memory or an I/O peripheral.
- **LDA** requires an extra data-transfer memory cycle because it performs a memory access at execution time to fetch the operand byte from the 16-bit target location.
- **LXI** completes execution immediately upon loading the immediate address/constant payload during instruction bytes reading.

Q8.04.3 MCQ 2004 2M Ans: D ● ● ○

To multiply two numbers using repeated addition, we add the multiplicand to the accumulator a number of times equal to the multiplier.
Here, register B holds the multiplicand (0A H) and register C holds the multiplier (0B H). The accumulator is initially cleared using `MVI A, 00H`.
The correct sequence of instructions inside the loop must be:

1. **ADD B**: Adds the content of register B to the accumulator ($A \leftarrow A + B$).
2. **DCR C**: Decrements the multiplier in register C by 1 ($C \leftarrow C - 1$).
3. **JNZ LOOP**: Checks the Zero flag. If $C \neq 0$, it jumps back to `LOOP` to repeat the addition.

Thus, the complete sequence is **ADD B, DCR C, JNZ LOOP**.

(D) ADD B, DCR C, JNZ LOOP

Key Points

- **MVI A, 00H** clears the accumulator to store the sum.
- **ADD B** performs the repeated addition.
- **DCR C** updates the loop counter and automatically sets the Zero flag when $C = 0$.
- **JNZ LOOP** loops back as long as the counter has not reached zero.

Q8.04.4 MCQ 2004 2M Ans: A ● ○ ○

Let's analyze the execution of the given 8085 assembly instructions step by step:

1. **LXI H, 9258H**: Loads the 16-bit immediate data 9258 H into the HL register pair. Thus, the memory pointer M points to the memory location 9258 H.
2. **MOV A, M**: Copies the data byte from the memory location pointed to by the HL pair (i.e., location 9258 H) into the Accumulator (A).
3. **CMA**: Complements (1's complement) the contents of the Accumulator.
4. **MOV M, A**: Copies the modified contents of the Accumulator back into the memory location pointed to by the HL pair (i.e., location 9258 H).

The complete operation complements the contents of memory location 9258 H and stores the result back in location 9258 H. Looking closely at the given options, option A states: "contents of location 9258 are moved to the accumulator". While this describes only the second instruction rather than the whole sequence, options B, C, and D contain incorrect memory locations (8529 H and 5892 H) due to reversed or scrambled bytes. Therefore, by elimination of completely false statements, option A is selected as the correct choice.

(A) contents of location 9258 are moved to the accumulator

Key Points

- **M** represents the memory location whose address is stored in the **HL** register pair.
- **CMA** is an implicit addressing mode instruction that performs a bitwise NOT operation on the accumulator content.

Q8.03.1 MCQ 2003 2M Ans: A ● ○ ○

The `CMP B` instruction compares the contents of register B with the contents of the accumulator (A) by internally performing a subtraction operation: $(A - B)$. The contents of both registers remain unchanged, but the status flags are updated based on the result:

- **Case 1 ($A > B$):** The result of subtraction is positive. No borrow is needed, and the result is non-zero.

  Carry Flag (CY) = 0, Zero Flag (Z) = 0

- **Case 2 ($A = B$):** The result of subtraction is zero. No borrow is needed.

  Carry Flag (CY) = 0, Zero Flag (Z) = 1

- **Case 3 ($A < B$):** A borrow is required to perform the subtraction, making the result negative and non-zero.

  Carry Flag (CY) = 1, Zero Flag (Z) = 0

Since the problem states that the content of the accumulator is less than that of register B ($A < B$), the Carry flag will be set (CY = 1) and the Zero flag will be reset (Z = 0).

(A) Carry flag will be set but Zero flag will be reset

Key Points

- `CMP` instructions affect the Carry flag (CY) and Zero flag (Z) specifically to assist with conditional jumps like `JC`, `JNC`, `JZ`, and `JNZ`.
- The comparison always subtracts the specified register or memory location *from* the accumulator.

Q8.02.1 NAT 2002 5M Ans: 13 ● ● ○

To determine the total number of machine cycles required to execute the routine once up to the first execution of the `JMP` instruction, we break down each instruction into its corresponding machine cycles:

1. **MOV A, B**: Requires 1 machine cycle (Opcode Fetch).
2. **OUT 55H**: Requires 3 machine cycles (Opcode Fetch, Memory Read for the 8-bit port address, and I/O Write to send data to the port).
3. **DCR B**: Requires 1 machine cycle (Opcode Fetch).

4. **STA FFF8H**: Requires 4 machine cycles (Opcode Fetch, Memory Read for lower byte of address, Memory Read for higher byte of address, and Memory Write to store the accumulator data).
5. **JMP START**: Requires 3 machine cycles (Opcode Fetch, Memory Read for lower byte of target address, and Memory Read for higher byte of target address).

Summing the machine cycles for all instructions:

$$\text{Total Machine Cycles} = 1 + 3 + 1 + 4 + 3 = 13$$

13

Q8.02.2 NAT 2002 5M Ans: 8.2 ● ● ●

To find the time interval between two consecutive memory write ($\overline{\text{MEMW}}$) signals, we analyze the execution timeline of the loop cycle. A $\overline{\text{MEMW}}$ signal is generated only during the Memory Write machine cycle of the `STA FFF8H` instruction. Therefore, the time interval between two consecutive $\overline{\text{MEMW}}$ signals is equal to the total number of clock cycles (T-states) executed between the end of one Memory Write cycle and the end of the next Memory Write cycle in the subsequent loop iteration.
Let's look at the T-states required for each instruction:

- **MOV A, B**: 4 T-states
- **OUT 55H**: 10 T-states (4 + 3 + 3)
- **DCR B**: 4 T-states
- **STA FFF8H**: 13 T-states (4 + 3 + 3 + 3). The last cycle is Memory Write (3 T).
- **JMP START**: 10 T-states (4 + 3 + 3)

The total clock cycles executed in one full loop iteration is:

$$\text{Total } T\text{-states} = 4 + 10 + 4 + 13 + 10 = 41\,T$$

Given operating clock frequency:

$$f = 5\,\text{MHz} \implies T = \frac{1}{5\,\text{MHz}} = 0.2\,\mu\text{s}$$

The time interval between two consecutive $\overline{\text{MEMW}}$ signals is:

$$t = 41 \times 0.2\,\mu\text{s} = 8.2\,\mu\text{s}$$

8.2

Q8.02.3 NAT 2002 2M Ans: 8.8 ● ● ●

When three WAIT states are introduced into the I/O WRITE machine cycle of the `OUT 55H` instruction, each WAIT state adds exactly 1 T-state to the execution time of that instruction.
The updated T-states for the instructions in the loop are:

- **OUT 55H**: Original 10 T + 3 WAIT states = 13 T
- All other instructions (`MOV A, B`, `DCR B`, `STA FFF8H`, and `JMP START`) remain unaffected.

The updated total number of clock cycles for one complete loop iteration becomes:

$$\text{Total } T\text{-states} = 4 + 13 + 4 + 13 + 10 = 44\,T$$

With clock cycle period $T = 0.2\,\mu$s, the updated time interval between two consecutive $\overline{\text{MEMW}}$ signals is:

$$t = 44 \times 0.2\,\mu\text{s} = 8.8\,\mu\text{s}$$

$$\boxed{8.8}$$

Q8.02.4 MCQ 2002 2M Ans: B ●●○

Let us trace the program step by step to determine the state of the registers and flags:

1. **MVI B, 87H**: Loads register B with 87 H.
$$B = 87\,\text{H} = 1000\,0111_2$$
2. **MOV A, B**: Copies the content of register B into the Accumulator (A).
$$A = 87\,\text{H}$$
3. **START: JMP NEXT**: Unconditionally jumps to the label `NEXT`.
4. **NEXT: XRA B**: Performs an exclusive-OR (XOR) operation between the contents of the accumulator A and register B. The result is stored back in the accumulator.
$$A \leftarrow A \oplus B = 87\,\text{H} \oplus 87\,\text{H} = 00\,\text{H}$$
Since the result is 00 H, its most significant bit (MSB) is 0. Therefore, the Sign flag (S) is cleared (S = 0, indicating a positive value).
5. **JP START**: The instruction `JP` (Jump if Positive) checks the Sign flag. It jumps to `START` if S = 0. Since S = 0, the jump condition is satisfied, and control transfers to `START`.
6. **START: JMP NEXT**: Again, unconditionally jumps back to `NEXT`.
7. **NEXT: XRA B**: The XOR operation is performed again with the current contents ($A = 00\,\text{H}$ and $B = 87\,\text{H}$):
$$A \leftarrow A \oplus B = 00\,\text{H} \oplus 87\,\text{H} = 87\,\text{H}$$
Now, the accumulator contains $87\,\text{H} = 1000\,0111_2$. The MSB of the result is 1, which sets the Sign flag (S = 1, indicating a negative value).
8. **JP START**: Checks the Sign flag. Since S = 1, the jump condition is not satisfied. The program does not jump and instead proceeds sequentially to the next instruction.
9. **OUT PORT2**: Sends the current content of the accumulator (87 H) to `PORT2`.
10. **HLT**: Halts the execution of the processor.

Thus, the execution results in an output of 87 H at `PORT2`.

$$\boxed{\text{(B) an output of 87H at PORT2}}$$

Key Points

- **XRA B** modifies status flags based on the result. XORing an identical value clears the register and resets the Sign flag.
- **JP** checks the Sign flag (S). It executes the jump only when S = 0.

Q8.01.1 Descriptive 2001 Ans: * ● ● ○

Initially, all flags are **RESET** (CY = 0, Z = 0, S = 0).

1. **FF00 H:** `MVI A, FFH` → Accumulator A = FFH.
2. **FF02 H:** `INR A` → A = FFH + 01H = 00H. Zero flag sets (Z = 1). Carry flag CY is unaffected by `INR` and remains 0.
3. **FF03 H:** `JC FF0CH` → Since CY = 0, branch is **not taken**.
4. **FF06 H:** `ORI A8H` → A = 00H ∨ A8H = A8H. Logical operations clear carry (CY = 0). Sign flag sets (S = 1) because MSB is 1.
5. **FF08 H:** `JM FF15H` → Since S = 1, branch to **FF15 H** is **taken**.
6. **FF15 H:** `MVI A, FFH` → A = FFH.
7. **FF17 H:** `ADI 02H` → A = FFH + 02H = 01H. Carry is generated (CY = 1, Z = 0).
8. **FF19 H:** `RAL` → Rotate A left through carry. Initial $A = 0000\,0001_2$, CY = 1. After rotation, $A = 0000\,0011_2$ = 03H and CY = 0 (MSB of A was 0).
9. **FF1A H:** `JZ FF23H` → Since A = 03H (Z = 0), branch is **not taken**.
10. **FF1D H:** `JC FF10H` → Since CY = 0, branch is **not taken**.
11. **FF20 H:** `JNC FF12H` → Since CY = 0, branch to **FF12 H** is **taken**.
12. **FF12 H:** `OUT PORT 2` → 03H is written to **PORT 2**.
13. **FF14 H:** `HLT` → Execution halts.

MVI A, FFH → INR A → JC FF0CH → ORI A8H → JM FF15H → MVI A, FFH
→ ADI 02H → RAL → JZ FF23H → JC FF10H → JNC FF12H
→ OUT PORT 2 → HLT

Key Points

- The `INR` instruction does not alter the Carry flag.
- Logical instructions (`ORI`, `XRA`, etc.) always clear the Carry flag.

Q8.01.2 Descriptive 2001 Ans: * ● ● ○

From the step-by-step program execution trace:

- The program successfully branches to location **FF12 H** via the `JNC FF12H` condition since the carry flag (CY) is 0 after the `RAL` instruction.
- At location **FF12 H**, the instruction executed is `OUT PORT 2`.
- The content of the Accumulator at this step is 03H ($0000\,0011_2$).

PORT 2 is loaded with data pattern 03H

Mistakes to be avoided

- Ensure to include the incoming carry flag status into the LSB during the execution of `RAL`.

Q8.01.3 MCQ | 2001 | 1M | Ans: C | ● ● ○

Method 1

The size of the memory is given as 4K × 8-bit.

$$\text{Memory size} = 4\text{K} = 4 \times 2^{10} = 2^2 \times 2^{10} = 2^{12} \text{ bytes}$$

This implies that 12 address lines ($A_0 - A_{11}$) are needed to reference internal memory locations within the chip. The range of offset addresses within a 4K memory block spans from all 0s to all 1s on these 12 internal lines:

$$\text{Minimum Offset} = 0000\,0000\,0000_2 = 000\text{H}$$

$$\text{Maximum Offset} = 1111\,1111\,1111_2 = \text{FFFH}$$

The address of the last byte is calculated by adding the maximum offset to the starting address:

$$\text{Last Address} = \text{Starting Address} + \text{Maximum Offset}$$

$$\text{Last Address} = \text{AA00H} + \text{0FFFH}$$

Performing hexadecimal addition:

AA00H:	1010	1010	0000	0000
+ 0FFFH:	0000	1111	1111	1111
B9FFH:	1011	1001	1111	1111

Therefore, the address of the last byte in the RAM is B9FF H.

Method 2

Alternatively, using the absolute number of bytes:

$$\text{Total memory locations} = 4\text{K} = 4 \times 1024 = 4096 \text{ locations}$$

Converting the decimal value 4096_{10} to hexadecimal:

$$4096_{10} = 1000\text{H}$$

The relationship between the final address, starting address, and total size is:

$$\text{Final Address} = \text{Starting Address} + \text{Total Locations} - 1$$

$$\text{Final Address} = \text{AA00H} + 1000\text{H} - 1 = \text{BA00H} - 1 = \text{B9FFH}$$

(C) B9FF H

Key Points

- A memory unit of size 2^n bytes has internal offsets ranging from 0 to $2^n - 1$.
- The final boundary address is consistently determined using the structural relation: End Address = Start Address + Size − 1.

Q8.00.1 NAT 2000 Ans: * ●●○

Program Mapping and Initialization Table:
To understand execution starting from **8000 H**, we map out the memory layout based on the program starting at **7FFF H**:

Address (H)	Machine Code (H)	Instruction Mapped	Comments
7FFF	3E	MVI A, C3H	Opcode for `MVI A`
8000	**C3**	**JMP 8000H** (Partial Opcode Interpretation)	Data byte for `MVI A`
8001	00	NOP	Opcode for `NOP`
8002	80	ADD B	Opcode for `ADD B`
8003	3D	DEC A	Opcode for `DEC A`
8004	C2	JNZ 800A H	Opcode for `JNZ`
8005	0A		Lower-order address
8006	80		Higher-order address
8007	C3	JMP 800C H	Opcode for `JMP`
8008	0C		Lower-order address
8009	80		Higher-order address
800A	D3	OUT 10H	Opcode for `OUT`
800B	10		Device Address
800C	76	HLT	Opcode for `HLT`

Execution Trace from 8000 H:

1. When the Program Counter (PC) starts at **8000 H**, the processor reads the byte `C3` as an instruction opcode.
2. `C3` is the opcode for an unconditional jump instruction (`JMP`). Therefore, the microprocessor interprets the next two successive memory bytes as a target address.
3. The bytes at **8001 H** (`00`) and **8002 H** (`80`) are read as the target address, forming **8000 H** (stored in Little-Endian format: `8000 H`).
4. This forces the program counter back to **8000 H**, creating a continuous loop at this address block.

The microprocessor enters into an infinite loop at memory locations 8000H – 8002H

Key Points

- Any raw data byte processed when fetched during an opcode fetch cycle is strictly interpreted as an instruction opcode.
- `C3` is the 3-byte unconditional jump instruction (`JMP`) code in the 8085 instruction set architecture.

Q8.00.2 NAT 2000 Ans: * ●●○

Memory Operation Identification:
A memory operation refers to either a Memory Read (MR) cycle or a Memory Write (MW) cycle. We calculate the total cycles required during instruction execution:

1. **`MVI A, C3H`** (2 bytes):
   - Opcode Fetch (1 Memory Read)
   - Memory Read to fetch data byte `C3H` (1 Memory Read)

   Total = 2 Memory Operations.

2. **`NOP`** (1 byte):
   - Opcode Fetch (1 Memory Read)

   Total = 1 Memory Operation.

3. **`ADD B`** (1 byte):
   - Opcode Fetch (1 Memory Read)

   Total = 1 Memory Operation.

4. **`DEC A`** (1 byte):
   - Opcode Fetch (1 Memory Read)

   Total = 1 Memory Operation.

5. **`JNZ 800AH`** (3 bytes):
   - Opcode Fetch (1 Memory Read)
   - Memory Read for lower address byte `0AH` (1 Memory Read)
   - Memory Read for higher address byte `80H` (1 Memory Read)

   Total = 3 Memory Operations.

6. **`JMP 800CH`** (3 bytes):
   - Opcode Fetch (1 Memory Read)
   - Memory Read for lower address byte `0CH` (1 Memory Read)
   - Memory Read for higher address byte `80H` (1 Memory Read)

   Total = 3 Memory Operations.

7. **`OUT 10`** (2 bytes):
   - Opcode Fetch (1 Memory Read)
   - Memory Read for port address `10H` (1 Memory Read)
   - *Note:* The actual I/O write (I/OW) to output data to the port is an I/O operation, not a memory operation.

   Total = 2 Memory Operations.

8. **`HLT`** (1 byte):
   - Opcode Fetch (1 Memory Read)

Total = 1 Memory Operation.

$$\text{Total Memory Operations} = 2 + 1 + 1 + 1 + 3 + 3 + 2 + 1 = 14$$

14

Q8.00.3 NAT 2000 Ans: * ● ○ ○

Instruction Selection and Timing Analysis:
To clear the accumulator (set $A = 00H$) with minimum execution time, we compare common alternative approaches:

1. **`XRA A`** (Exclusive OR accumulator with itself):
   - Size: 1 Byte
   - Machine Cycles: 1 (Opcode Fetch)
   - T-states: 4 T-states
2. **`SUB A`** (Subtract accumulator from itself):
   - Size: 1 Byte
   - Machine Cycles: 1 (Opcode Fetch)
   - T-states: 4 T-states
3. **`MVI A, 00H`** (Move immediate value 00H to accumulator):
   - Size: 2 Bytes
   - Machine Cycles: 2 (Opcode Fetch + Memory Read)
   - T-states: 7 T-states (4 + 3)

Both `XRA A` and `SUB A` execute in the absolute minimum possible time of 4 T-states.

XRA A (or SUB A)

Key Points

- Register-to-register logical or arithmetic instructions targeting the accumulator require only 1 machine cycle (4 T-states) because no supplementary memory operands are read.

Q8.00.4 MCQ 2000 2M Ans: A ● ● ○

The execution of the `SUB B` instruction performs subtraction by adding the 2's complement of register B's content to the accumulator (A):

$$A \leftarrow A - B = A + (\text{2's complement of } B)$$

Given values:

- $A = 3AH = 0011\ 1010_2$
- $B = 49H = 0100\ 1001_2$

Step 1: Compute the 2's complement of B

$$\text{1's complement of } B = 1011\ 0110_2$$

$$\text{2's complement of } B = 1011\ 0110_2 + 1 = 1011\ 0111_2$$

Step 2: Add to Accumulator

$$\begin{array}{rcl} A: & & 0011\,1010_2 \\ +\ 2\text{'s complement of } B: & & 1011\,0111_2 \\ \hline \text{Result:} & \mathbf{0} & 1111\,0001_2 \end{array}$$

Step 3: Flag Analysis

- **Accumulator (A):** The binary result $1111\,0001_2$ corresponds to F1H.
- **Carry Flag (CY):** During addition using 2's complement, an internal carry out of MSB is 0. In the 8085 architecture, for subtraction operations, the carry flag is the complement of this addition carry:

$$\text{CY} = \overline{\text{Addition Carry}} = \overline{0} = 1$$

  This indicates that a borrow was required because $A < B$.
- **Sign Flag (S):** The most significant bit (MSB) of the result is 1, so S = 1 (negative result).

$$\boxed{\text{(A) } A = \text{F1}, \text{CY} = 1, S = 1}$$

Key Points

- For subtraction (`SUB`, `SUI`), the 8085 complements the final carry bit of the adder hardware to represent the true borrow state.

Q8.00.5 MCQ 2000 1M Ans: A ● ○ ○

The restart instruction `RST n` is a 1-byte software interrupt instruction in the 8085 microprocessor. It transfers the program execution to a specific vector address located in the lower memory area.
The vector address for any restart instruction `RST n` (where n ranges from 0 to 7) is calculated by multiplying the interrupt vector number n by 8 bytes:

$$\text{Vector Address (Decimal)} = n \times 8$$

Given $n = 6$ for the `RST 6` instruction:

$$\text{Vector Address (Decimal)} = 6 \times 8 = 48_{10}$$

Now, converting the decimal address 48_{10} to its 16-bit hexadecimal equivalent:

$$48_{10} = 3 \times 16 + 0 = 30\text{H}$$

Thus, the execution transfers to the memory location 0030H (written as 30H).

$$\boxed{\text{(A) 30 H}}$$

Key Points

- Each software interrupt vector location is separated by exactly 8 bytes in memory because historical interrupt service routines (ISR) could use a jump instruction to redirect elsewhere.

Q8.00.6 MCQ 2000 1M Ans: C ●○○

The 8085 microprocessor consists of physical pins dedicated to receiving external signaling requests to interrupt regular program execution. These are classified structurally as hardware interrupts.
There are exactly 5 hardware interrupt pins on the 8085 microprocessor package, listed below in decreasing order of their operational priority:

1. **TRAP:** Non-maskable highest priority edge-and-level triggered interrupt.
2. **RST 7.5:** Maskable edge-triggered interrupt.
3. **RST 6.5:** Maskable level-triggered interrupt.
4. **RST 5.5:** Maskable level-triggered interrupt.
5. **INTR:** Maskable lowest priority level-triggered general-purpose interrupt.

Therefore, the total number of hardware interrupts present is 5.

(C) 5

Key Points

- Software interrupts (`RST 0` to `RST 7`) are initiated by instructions written within code, whereas hardware interrupts are triggered by external peripheral signals applied directly to the CPU pins.

Q8.99.1 NAT 1999 Ans: * ●●●

Program Functional Analysis:
To understand what the program does, let us track the flow and the registers step-by-step. Note that there is a slight typo in the problem text label: `JNZ LOOP1` references the start of the core comparison block beginning at `LOOP: INX H`.

1. **Initialization:**
   - `MVI C, 03H` → Counter register C = 03H (Total number of elements to process).
   - `LXI H, 2000H` → Pointer register pair HL = 2000H.
   - `MOV A, M` → Load the first memory element into the accumulator: A = [2000H] = 18H.
   - `DCR C` → Decrement count: C = 02H.
2. **Loop Execution Details:**
   - **Iteration 1** (C = 02**H**)**:**
     - `INX H` → Pointer advances to HL = 2001H.
     - `MOV B, M` → Load next element: B = [2001H] = 10H.
     - `CMP B` → Compare A (18H) with B (10H). Since $A > B$, no borrow occurs → Carry flag CY = 0.
     - `JNC LOOP2` → Since CY = 0, branch to `LOOP2` is **taken**, skipping `MOV A, B`. Thus, A retains the value 18H.
     - `LOOP2: DCR C` → Decrement count: C = 01H.
     - `JNZ LOOP` → Since $C \neq 0$, jump back to `LOOP`.
   - **Iteration 2** (C = 01**H**)**:**
     - `INX H` → Pointer advances to HL = 2002H.
     - `MOV B, M` → Load next element: B = [2002H] = 2BH.
     - `CMP B` → Compare A (18H) with B (2BH). Since $A < B$, a borrow occurs → Carry flag CY = 1.
     - `JNC LOOP2` → Since CY = 1, branch is **not taken**.
     - `MOV A, B` → Updates accumulator with the larger value: $A = B$ = 2BH.

- `DCR C` → Decrement count: $C = 00H$.
- `JNZ LOOP` → Since $C = 0$, the loop terminates.

3. **Storage and Termination:**

- `STA 2100H` → Stores the final content of A (2BH) into memory location 2100H.

As observed, the accumulator constantly retains the maximum value encountered in the array by updating whenever a larger element is found.

The program finds the maximum value from an array of 3 numbers stored at memory locations 2000H to 2002H and saves the maximum value into memory location 2100H.

Key Points

- `CMP r` modifies the Carry flag (CY) based on subtraction ($A - r$). If $A < r$, CY = 1; if $A \geq r$, CY = 0.

Q8.99.2 NAT 1999 Ans: *

Final State Trace Values:
Following the step-by-step trace detailed in the prior problem solution, we compile the final architectural states at the time of `HLT`:

(i) **Contents of the registers A, B, C, H and L:**

- **A:** Retains the maximum value found = 2BH.
- **B:** Loaded with the last element from 2002H = 2BH.
- **C:** Decremented down to 00H.
- **H, L:** Holds the final pointer value after the last increment = 20H and 02H respectively.

$$A = 2BH,\ B = 2BH,\ C = 00H,\ H = 20H,\ L = 02H$$

(ii) **Condition of the carry and zero flags:**

- The last flag-altering instruction executed before exit was `DCR C` at `LOOP2` where C changed from 01H → 00H.
- Since the result of `DCR C` is zero, the Zero flag is set (Z = 1).
- `DCR` does not alter the Carry flag. The Carry flag preserves its state from the last `CMP B` instruction where $A = 18H$ and $B = 2BH$ ($A < B$), so CY = 1.

$$CY = 1,\ Z = 1$$

(iii) **Contents of the memory locations 2000 H, 2001 H, 2002 H and 2100 H:**

- Locations 2000H to 2002H are read-only in this program and hold initial values.
- Location 2100H receives the final stored value of A.

$$[2000H] = 18H,\ [2001H] = 10H,\ [2002H] = 2BH,\ [2100H] = 2BH$$

(i) $A = 2BH, B = 2BH, C = 00H, H = 20H, L = 02H$
(ii) $CY = 1, Z = 1$
(iii) $[2000H] = 18H, [2001H] = 10H, [2002H] = 2BH, [2100H] = 2BH$

Mistakes to be avoided

- Remember that DCR instructions do not impact the Carry flag. The final CY is set exclusively by the earlier CMP comparison.

Q8.99.3 MCQ 1999 2M Ans: D ●●○

A 4 K RAM requires 12 address lines ($A_0 - A_{11}$), because $4\,\text{K} = 2^{12}$ bytes.
The chip select logic is given by CS = $\overline{A_{15}}A_{14}A_{13}$. For the RAM chip to be selected, CS must be high (CS = 1), which dictates the following fixed states for the highest address bits:

$$A_{15} = 0, \quad A_{14} = 1, \quad A_{13} = 1$$

Since address line A_{12} is neither used for selecting locations inside the 4 K RAM nor included in the chip select logic, it acts as a don't-care bit (A_{12} = X). This creates two distinct foldback memory ranges:

1. **When $A_{12} = 0$:**
   - Starting Address: $0110\ 0000\ 0000\ 0000_2 = 6000\,\text{H}$
   - Ending Address: $0110\ 1111\ 1111\ 1111_2 = 6\text{FFF}\,\text{H}$
2. **When $A_{12} = 1$:**
   - Starting Address: $0111\ 0000\ 0000\ 0000_2 = 7000\,\text{H}$
   - Ending Address: $0111\ 1111\ 1111\ 1111_2 = 7\text{FFF}\,\text{H}$

Thus, the overall memory range consists of 6000 H – 6FFF H and 7000 H – 7FFF H.

(D) 6000 H – 6FFF H and 7000 H – 7FFF H

Key Points

- **Memory Size vs Address Lines:** Number of lines n required for memory size N is found via $N = 2^n$.
- **Foldback / Mirror Memory:** Unused address lines that are not part of decoding create mirrored memory blocks across the address space.

Q8.98.1 NAT 1998 5M Ans: * ●●●

To store the hexadecimal value 5DH (01011101_2) into the flag register of the 8085 microprocessor without using any arithmetic instructions and ensuring no other registers are altered, we must manipulate the processor status word (PSW) using the stack.
The PSW register pair consists of the Accumulator (A, high byte) and the Flag register (F, low byte).

```
PUSH B    ; Save original BC pair onto the stack
PUSH PSW  ; Save original Accumulator and Flags
POP B     ; Move original A into B, and original F into C
MVI C, 5DH ; Load desired flag value 5DH into register C
PUSH B    ; Push original A (now in B) and new F (in C) as PSW
POP PSW   ; Pop the values into Accumulator and Flag register
POP B     ; Restore the original BC pair from the stack
```

Key Points

- The PSW pair is organized as *A* (MSB) and *F* (LSB).
- Utilizing an alternate register pair like *BC* allows temporary retention of the Accumulator's data so it remains unaltered after execution.

Mistakes to be avoided

- Do not use instructions like `MVI A, 5DH` directly without preserving and restoring the original contents of the Accumulator.

Q8.98.2 MCQ 1998 2M Ans: C ●○○

An instruction that explicitly manipulates processor status flags (such as setting or clearing the carry flag, e.g., `STC` or `CLC` in 8085/x86 architectures) is classified under **logical instructions** or processor control operations that form a part of the logical instruction group.
Unlike arithmetic operations that modify flags as a side-effect of data manipulation, these instructions perform direct boolean/bit-level manipulation of the status flags.
Therefore, the instruction is classified as a logical instruction.

(C) logical

Key Points

- **Logical Instructions:** Perform bitwise operations (AND, OR, XOR) and flag manipulation operations like `STC` (Set Carry) and `CMC` (Complement Carry).
- **Data Transfer:** Moves data between registers or memory without modifying any flags.
- **Arithmetic:** Modifies flags depending upon the result of arithmetic calculations (e.g., `ADD`, `SUB`).

Q8.98.3 MCQ 1998 2M Ans: B ●○○

An Input-Output Processor (IOP) is a specialized processor with direct memory access (DMA) capabilities that independently manages I/O operations. Its primary function is to transfer data directly between the **main memory** and **I/O devices** without continuous intervention from the central processing unit (CPU).
By handling these background data transfers independently, the IOP prevents the CPU from being bogged down by slow peripheral operations.

(B) main memory and I/O devices

Key Points

- **I/O Processor (IOP):** Acts as an independent processor containing its own instruction set specifically designed for managing data transfer interfaces.
- **Data Path:** The data transmission occurs directly between peripheral I/O devices and the system's main memory.

Q8.97.1 Descriptive 1997 5M Ans: * ●●●

To determine the total execution time of the delay subroutine, we must calculate the operating frequency and sum up the T-states (clock cycles) for each instruction.

1. System Operating Frequency

The internal operating frequency (f_{sys}) of the 8085 microprocessor is half of its crystal frequency:

$$f_{sys} = \frac{2\,\text{MHz}}{2} = 1\,\text{MHz}$$

Thus, the time period of one clock cycle (T) is:

$$T = \frac{1}{f_{sys}} = \frac{1}{1\,\text{MHz}} = 1\,\mu\text{s}$$

2. Clock Cycles for Memory-Access Based Instructions

The problem states that for standard instructions, the number of clock cycles is given by $(3n + 1)$, where n is the number of memory accesses:

- **MVI A, 64H:** Requires $n = 2$ memory accesses (1 opcode fetch + 1 data read).

$$\text{T-states} = 3(2) + 1 = 7\,\text{T}$$

- **NOP:** Requires $n = 1$ memory access (opcode fetch).

$$\text{T-states} = 3(1) + 1 = 4\,\text{T}$$

- **DCR A:** Requires $n = 1$ memory access (opcode fetch).

$$\text{T-states} = 3(1) + 1 = 4\,\text{T}$$

- **POP PSW:** Requires $n = 3$ memory accesses (1 opcode fetch + 2 stack reads).

$$\text{T-states} = 3(3) + 1 = 10\,\text{T}$$

- **RET:** Requires $n = 3$ memory accesses (1 opcode fetch + 2 stack reads to pop PC).

$$\text{T-states} = 3(3) + 1 = 10\,\text{T}$$

3. Loop Execution Analysis

The loop counter is loaded with $64\text{H} = 100_{10}$. The loop consists of `NOP`, `DCR A`, and `JNZ LOOP`.

- For the first 99 iterations, the jump is **taken** (10 T):

$$T_{taken} = 99 \times (4_{\text{NOP}} + 4_{\text{DCR}} + 10_{\text{JNZ}}) = 99 \times 18 = 1782\,\text{T}$$

- For the 100th iteration, the jump is **not taken** (7 T):

$$T_{not_taken} = 1 \times (4_{\text{NOP}} + 4_{\text{DCR}} + 7_{\text{JNZ}}) = 15\,\text{T}$$

Total T-states inside the loop:

$$T_{loop} = 1782 + 15 = 1797\,\text{T}$$

4. Total Instruction T-states Calculation

Now, we add the overhead outside the loop (including `CALL`):

Instruction	Frequency/Condition	T-states
CALL	Executed 1 time	18
PUSH PSW	Executed 1 time	12
MVI A, 64H	Executed 1 time	7
Loop Body	Executed 100 times	1797
POP PSW	Executed 1 time	10
RET	Executed 1 time	10
Total		**1854** T

5. Total Time Taken

$$\text{Total Time} = 1854 \times 1\,\mu\text{s} = 1854\,\mu\text{s} = 1.854\,\text{ms}$$

$$\boxed{1.854\,\text{ms}}$$

Key Points

- Always ensure the operating frequency is correctly determined from the crystal input ($f_{sys} = f_{crystal}/2$ for 8085).
- Conditional jump instructions (`JNZ`) consume different clock cycles depending on whether the branch is taken or falls through.

Q8.97.2 Descriptive | 1997 | 2M | Ans: (1, c) (2, b) | ●●○

To understand how data moves between the registers of the 8085 microprocessor and the stack, we examine the operations of the stack pointer (SP):

1. **PUSH Instruction:** The 8085 stack grows downward in memory (towards lower addresses). When a `PUSH` instruction is executed:
   - The SP is decremented first, and the higher-order byte of the register pair is copied to the memory location pointed to by SP.
   - The SP is decremented again, and the lower-order byte of the register pair is copied to the memory location pointed to by SP.

   Thus, a `PUSH` instruction **pre-decrements** the stack pointer. This matches item **(c)**.

2. **POP Instruction:** When a `POP` instruction is executed:
   - The lower-order byte is retrieved from the memory location currently pointed to by SP, and then SP is incremented.
   - The higher-order byte is retrieved from the next memory location, and then SP is incremented again.

   Thus, a `POP` instruction retrieves the data and then increments the stack pointer, which is described as a **post-increment** operation. This matches item **(b)**.

Therefore, the correct matching is $(1, \text{c})$ and $(2, \text{b})$.

$$\boxed{(1, \text{c})\ (2, \text{b})}$$

Key Points

- **Stack Growth:** In 8085, the stack grows towards lower memory addresses during a PUSH operation.
- **PUSH Sequence:** SP ← SP – 1 → Store High Byte → SP ← SP – 1 → Store Low Byte.
- **POP Sequence:** Read Low Byte → SP ← SP + 1 → Read High Byte → SP ← SP + 1.

Q8.97.3 MCQ 1997 2M Ans: C ●●○

To find the address from which the next instruction will be fetched, let us trace the execution of the program step-by-step:

1. **6010: LXI H, 8A79H** Loads the 16-bit value 8A79H into the HL register pair.

$$H = 8\text{AH}, \quad L = 79\text{H}$$

2. **6013: MOV A, L** Copies the content of register L into the accumulator (A).

$$A = 79\text{H}$$

3. **6015: ADD H** Adds the contents of register H to the accumulator:

$$\begin{array}{rrcll} & A = 79\text{H} & = & 0111 & 1001_2 \\ + & H = 8\text{AH} & = & 1000 & 1010_2 \\ \hline & A = 03\text{H} & = & 0000 & 0011_2 \end{array}$$

During this addition:

- A carry is generated from lower nibble to upper nibble ($9 + A = 13_{10} = \text{D}$, which exceeds 15), so Auxiliary Carry (AC) = 1.
- A carry is generated from the most significant bit ($7 + 8 = 15_{10} + 1_{\text{carry}} = 16_{10}$), so Carry (CY) = 1.

4. **6016: DAA** (Decimal Adjust Accumulator) Since CY = 1 and AC = 1, the DAA instruction adds 66H ($0110\ 0110_2$) to the accumulator:

$$\begin{array}{rrcll} & A = 03\text{H} & = & 0000 & 0011_2 \\ + & 66\text{H} & = & 0110 & 0110_2 \\ \hline & A = 69\text{H} & = & 0110 & 1001_2 \end{array}$$

Thus, after DAA, $A = 69\text{H}$.

5. **6017: MOV H, A** Copies the adjusted value from accumulator A into register H.

$$H = 69\text{H}$$

The HL pair now holds:

$$HL = 6979\text{H}$$

6. **6018: PCHL** Copies the contents of the HL register pair directly into the Program Counter (PC).

$$\text{PC} \leftarrow HL = 6979\text{H}$$

Since the Program Counter contains the address of the next instruction to be executed, the next instruction will be fetched from address 6979H.

(C) 6979

Key Points

- **DAA Logic:** Adds 6 to the lower nibble if it exceeds 9 or if AC = 1. Adds 6 to the upper nibble if it exceeds 9 or if CY = 1.
- **PCHL Function:** Performs an unconditional jump by directly modifying the PC register with the 16-bit address held in the HL pair.

Q8.97.4 MCQ 1997 2M Ans: A ●●○

To determine the valid I/O address range for the peripheral, the active low chip select signal ($\overline{\text{CHIP SELECT}}$) must be equal to 0.

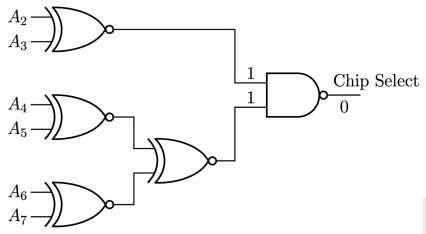


Fig. Solution Q8.97.4

1. Logic Gate Analysis

1. **NAND Gate Output:** For $\overline{\text{CHIP SELECT}} = 0$, both inputs to the final NAND gate must be equal to 1.
2. **Top XNOR Gate (A_2, A_3):** The output of the top XNOR gate must be 1:

$$A_2 \odot A_3 = 1 \implies A_2 = A_3$$

   Thus, A_2 and A_3 must have identical logic values (both 0 or both 1).

3. **Bottom NOR Gate Network (A_4, A_5, A_6, A_7):** The output of the NOR gate combining the lower stages must be 1. For a NOR gate to produce a 1, both of its inputs must be 0:
   - Output of XNOR gate for A_4, A_5 must be 0:

$$A_4 \odot A_5 = 0 \implies A_4 \neq A_5$$

   - Output of XNOR gate for A_6, A_7 must be 0:

$$A_6 \odot A_7 = 0 \implies A_6 \neq A_7$$

2. Testing the Options via Address Bit Decoding

Since the I/O address uses lines A_7 to A_0, and lines A_1, A_0 are not connected to the decoding logic, they are don't cares (X) representing the range limits from 00_2 to 11_2. Let us construct a truth table validating the constraints:

Option Address Range	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0
(A) 60H to 63H	0	1	1	0	0	0	0 → 1	0 → 1
(B) A4H to A7H	1	0	1	0	0	1	0 → 1	0 → 1
(C) 30H to 33H	0	0	1	1	0	0	0 → 1	0 → 1
(D) 70H to 73H	0	1	1	1	0	0	0 → 1	0 → 1

3. Verification of Constraints for Option (A)

Substituting the bits of option (A) (60H = 0110 0000_2):

- $A_2 = A_3 = 0 \implies A_2 \odot A_3 = 1$ (Satisfied)
- $A_4 = 0, A_5 = 1 \implies A_4 \odot A_5 = 0$ (Satisfied)
- $A_6 = 1, A_7 = 0 \implies A_6 \odot A_7 = 0$ (Satisfied)

All logic constraints are satisfied, yielding an active low output.
Conversely, for option (D) (70H), $A_4 = 1$ and $A_5 = 1 \implies A_4 \odot A_5 = 1$, which violates the conditions and leaves the chip unselected.

(A) 60 H to 63 H

Key Points

- An active low chip select ($\overline{CS}$) requires a logical 0 state to activate the device.
- XNOR outputs 1 when inputs match, whereas NOR outputs 1 only when all inputs are 0.

Q8.97.5 MCQ 1997 1M Ans: C ● ○ ○

The Restart (RST) instructions in the 8085 microprocessor are software interrupts. Like all maskable hardware interrupts and software-initiated vector interrupts, their execution depends on the status of the internal interrupt enable flip-flop.
Upon a system reset or after handling a prior interrupt, the 8085 automatically disables maskable interrupts. Therefore, to execute any software interrupt routine successfully, interrupts must be explicitly enabled beforehand using the Enable Interrupt (EI) instruction.

(C) only if interrupts have been enabled by an EI instruction

Key Points

- **Software Interrupts:** 8085 has 8 software interrupts (`RST 0` to `RST 7`) which act as one-byte call instructions to fixed vector memory locations.
- **`EI` Instruction:** Sets the internal interrupt enable flip-flop, allowing the processor to respond to incoming or programmatic maskable interrupt conditions.

Q8.96.1 MCQ 1996 2M Ans: C ● ● ○

The execution trace of the instructions is as follows:

1. **`1000 LXI SP, 27FF H`**: Loads the stack pointer with 27FF H.

   SP = 27FF H

2. **`1003 CALL 1006 H`**: A `CALL` instruction is a 3-byte instruction. It pushes the return address (1000 H + 3 = 1003 H) onto the stack and decrements SP by 2.

   SP = 27FF H − 2 = 27FD H

   The return address 1003 H is stored at locations 27FE H and 27FD H.

3. **`1006 POP H`**: Pops the top two bytes of the stack into the HL register pair and increments SP by 2.

   HL = 1003 H

   SP = 27FD H + 2 = 27FF H

Thus, on completion, SP = 27FF H and HL = 1003 H.

(C) SP = 27 FF, HL = 1003

Key Points

- `CALL` decrements SP by 2 and pushes the next instruction's address onto the stack.
- `POP` restores data from the stack into a register pair and increments SP by 2.

Q8.96.2 MCQ 1996 1M Ans: D ● ○ ○

The instruction `LDA 2003` is a 3-byte instruction that loads the content of memory location 2003 H into the accumulator. The execution of this instruction involves the following machine cycles:

1. **Opcode Fetch**: Fetching the opcode for `LDA` from the memory (1 memory access).
2. **Memory Read (Lower Byte)**: Reading the lower 8 bits of the address (03 H) from the memory (1 memory access).
3. **Memory Read (Higher Byte)**: Reading the higher 8 bits of the address (20 H) from the memory (1 memory access).
4. **Memory Read (Data)**: Reading the data byte stored at the memory location 2003 H into the accumulator (1 memory access).

Total memory accesses = 1 + 1 + 1 + 1 = 4

(D) 4

Key Points

- Every byte of an instruction requires one memory access to be fetched.
- The total memory accesses consist of both instruction fetching (3 bytes) and data reading/writing (1 byte).

Q8.96.3 MCQ 1996 1M Ans: D ● ● ○

Given a 4K RAM connected to an 8085 microprocessor system (which has a 16-bit address bus A_{15}–A_0).

1. **Determine Internal Address Lines:** The size of the RAM is 4K = $2^2 \times 2^{10} = 2^{12}$ bytes. Thus, 12 lower-order address lines (A_{11}–A_0) are connected directly to the memory chip for internal mapping.
2. **Analyze Chip Select Logic:** The active-high chip select logic is given by:

   $$CS = \overline{A}_{15} A_{14} A_{13}$$

   For the chip to be enabled ($CS = 1$), the fixed bits must be:

   $$A_{15} = 0, \quad A_{14} = 1, \quad A_{13} = 1$$

3. **Identify Don't Care Line:** The remaining address bit A_{12} is unassigned in both the internal mapping and the chip select logic, which means it acts as a don't-care bit (×).

Now, let us determine the range by considering the two cases for A_{12}:

- **Case 1: When** $A_{12} = 0$

A_{15}	A_{14}	A_{13}	A_{12}	A_{11}	A_{10}	A_9	A_8	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0
0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0
0	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1

This gives the range: 6000 H – 6FFF H.

- **Case 2: When** $A_{12} = 1$

A_{15}	A_{14}	A_{13}	A_{12}	A_{11}	A_{10}	A_9	A_8	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0
0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0
0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

This gives the range: 7000 H – 7FFF H.

Therefore, the complete memory range consists of both 6000 H – 6FFF H and 7000 H – 7FFF H due to foldback/mirroring.

(D) 6000 H - 6FFF H and 7000 H - 7FFF H

Key Points

- Address lines not decoded by the memory chip size or the chip select logic result in multiple valid address ranges (aliasing/mirroring).
- Total addressable locations = 2^n, where n is the number of internal address lines.

Q8.95.1 MCQ 1995 1M Ans: C ● ○ ○

An **Assembler** is a software tool or program used in microprocessor systems to bridge the gap between low-level symbolic code and executable binary code.

- It takes low-level **assembly language** programs (written using human-readable mnemonics like `MOV`, `ADD`, `SUB`) as the input source code.
- It translates these instruction mnemonics into their corresponding binary operational codes (opcodes) and data patterns, converting it into **machine language** (object code) that the microprocessor can directly execute.

(C) translation of a programm from assembly language to machine language

Key Points

- **Assembler:** Converts Assembly Language → Machine Language.
- **Compiler/Interpreter:** Converts Higher-Level Language → Machine/Intermediate Language.

Q8.95.2 MCQ 1995 1M Ans: B ● ○ ○

Direct Memory Access (DMA) is a technique used in computer systems that allows high-speed input/output (I/O) devices to transfer data directly to or from the system memory.

- Ordinarily, the microprocessor (μP) acts as an intermediary, fetching data from an I/O device into an internal register and then writing it to memory (or vice versa), which creates a significant bottleneck.
- During a DMA transfer, the DMA controller requests control of the system buses via the `HOLD` signal. Once the microprocessor relinquishes control (`HLDA`), data is transferred directly between the memory and the I/O peripheral **without involving or utilizing the microprocessor's internal processing circuitry**.

Therefore, option (B) is the correct statement.

(B) direct transfer of data between memory and I/O devices without the use of μp

Key Points

- DMA bypasses the CPU completely to maximize data transfer rates between fast peripheral devices and RAM.
- Control signals like `HOLD` and `HLDA` in the 8085 microprocessor are dedicated exclusively to coordinating DMA operations.

Q8.95.3 MCQ 1995 1M Ans: B ● ● ○

When a CPU accepts an interrupt request, it goes through the following general sequence:

1. **Complete Current Instruction:** The CPU first finishes executing the instruction currently in progress.
2. **Acknowledge and Save State:** The CPU acknowledges the interrupt and saves the return address (the program counter) on the stack.
3. **Branch to the ISR:** Control is transferred (branched) to the Interrupt Service Routine (ISR), a subroutine that services the interrupting device. At the end of the ISR, a return instruction brings the CPU back to the interrupted program.

Therefore, the CPU acknowledges the interrupt and branches off to a subroutine.

(B) acknowledges interrupt and branches off to a subroutine

Key Points

- Servicing an interrupt is similar to a subroutine call: the return address is saved, the ISR is executed, and the CPU then returns to the interrupted program.
- In the 8085, for the non-vectored `INTR` input, the device places an instruction (such as `RST n`) on the data bus during the $\overline{\text{INTA}}$ cycle. This is the specific mechanism one processor uses to reach the ISR; it is described by option (D) but is not the general action of a CPU on every interrupt.

Q8.93.1 MCQ 1993 2M Ans: D ● ○ ○

Wait states are extra clock cycles added to a microprocessor's machine cycle to synchronize its operation with slower external devices.

- Microprocessors generally operate at much higher speeds compared to memory chips or peripheral I/O devices.
- When the processor requests a data transfer from a slow peripheral, the device may not be able to present or accept valid data within the standard memory access time.
- To prevent data loss or bus conflicts, the slow peripheral pulls the processor's `READY` line low, forcing the processor to insert one or more **wait states** (T-states) until the peripheral is ready.

Therefore, wait states are explicitly used to interface slow peripherals with a fast processor.

(D) interface slow peripherals to the processor

Key Points

- The READY input signal is sampled by the microprocessor during specific T-states to determine if a wait state needs to be introduced.
- Introducing wait states delays the bus cycle without restarting or resetting the system, matching execution to hardware capabilities.

Q8.93.2 MCQ 1993 2M Ans: B ● ○ ○

In a microprocessor architecture, special-purpose registers are used to manage instruction execution:

- **Program Counter (PC):** This is a 16-bit register that always holds the memory address of the next instruction sequence or byte to be fetched from memory. As soon as a byte is fetched, the PC automatically increments to point to the next address.
- **Instruction Register (IR):** This register holds the opcode of the *current* instruction while it is being decoded and executed.
- **Accumulator:** A primary data register used to store the operands and results of arithmetic and logical operations.
- **Stack Pointer (SP):** A 16-bit register that tracks the top memory address of the stack segment.

Therefore, the register containing the address of the next instruction to be fetched is the Program Counter.

(B) Program Counter

Key Points

- The Program Counter keeps the CPU oriented in the program memory space during sequentially executed instructions.
- Branching instructions (like JMP, CALL) modify the contents of the Program Counter directly to redirect program flow.

Q8.92.1 Descriptive 1992 2M Ans: * ● ● ○

The execution trace of the program is analyzed line-by-line below:

1. **2000 LXI SP, 1000 H**: Initializes the stack pointer (SP) to 1000 H.

$$SP = 1000\,H$$

2. **2003 PUSH H**: Decrements SP by 2 to store the HL register pair content.

$$SP = 1000\,H - 2 = 0FFE\,H$$

3. **2004 PUSH D**: Decrements SP by 2 to store the DE register pair content.

$$SP = 0FFE\,H - 2 = 0FFC\,H$$

4. **2005 CALL 2050 H**: Pushes the 16-bit return address (2005 H + 3 = 2008 H) onto the stack, decrementing SP by 2, and changes the program counter (PC) to the target address 2050 H.

$$SP = 0FFC\,H - 2 = 0FFA\,H$$

$$PC = 2050\,H$$

Once the subroutine executes its RET instruction at the end:

5. **RET** (from subroutine): Pops the return address (2008 H) back into PC and increments SP by 2.

$$PC = 2008\,H$$

$$SP = 0FFA\,H + 2 = 0FFC\,H$$

6. **2008 POP H**: Pops the top of the stack into HL and increments SP by 2.

$$SP = 0FFC\,H + 2 = 0FFE\,H$$

7. **2009 HLT**: Halts the microprocessor execution. Since the HLT instruction is a 1-byte instruction, the Program Counter fetches it and increments to point to the next consecutive memory address.

$$PC = 2009\,H + 1 = 200A\,H$$

Thus, at the complete termination/halt state, the values are:

- Program Counter (PC) = 200A H
- Stack Pointer (SP) = 0FFE H

200A H and 0FFE H

Key Points

- PUSH and CALL instructions decrement the stack pointer (SP) by 2.
- POP and RET instructions increment the stack pointer (SP) by 2.
- Careful hexadecimal subtraction is required: 1000 H − 2 = 0FFE H.

Q8.92.2 MSQ 1992 2M Ans: A,D ● ● ○

In memory-mapped I/O, the I/O devices are treated exactly like memory locations:

- **16-bit Addresses:** Since they share the memory address space, I/O devices are assigned 16-bit addresses (same as memory). Hence, **Option A** is correct.
- **Instructions:** Memory-related instructions like MOV, ADD, ANA, etc., are used instead of IN and OUT instructions (which are only used in I/O-mapped I/O). Hence, **Option B** is incorrect.
- **Device Limit:** The number of I/O devices is not restricted to 256 inputs and 256 outputs, as the entire 64 KB memory address space can be shared. Hence, **Option C** is incorrect.
- **Direct Operations:** Since memory instructions can be used directly with I/O registers, arithmetic and logical operations can be performed directly on the I/O data (e.g., ADD M or ANA M). Hence, **Option D** is correct.

(A) I/O devices have 16-bit addresses

(D) arithmetic and logic operations can be directly performed with the I/O data.

Key Points

- **Memory-Mapped I/O:** I/O devices share the 16-bit memory address space (64 KB) with standard memory.
- **I/O-Mapped I/O:** Uses an 8-bit address space, requiring dedicated IN and OUT instructions.

Q8.91.1 Descriptive | 1991 | 2M | Ans: * | ● ● ○

To determine the register contents after execution halts, we trace the program step-by-step from location `2000H`:

1. **`2000 LXI SP, 1000H`**: Initialized stack pointer:

$$\text{SP} = 1000\text{H}$$

2. **`LXI H, 2F37H`**: Load 16-bit data into HL register pair:

$$H = 2\text{FH}, \quad L = 37\text{H} \quad (\text{HL} = 2\text{F37H})$$

3. **`XRA A`**: Exclusive-OR Accumulator with itself:

$$A = 00\text{H}, \quad \text{Zero Flag (Z)} = 1$$

4. **`MOV A, H`**: Move contents of H to accumulator A:

$$A = 2\text{FH} \quad (\text{Flags remain unchanged; } Z = 1)$$

5. **`INX H`**: Increment HL pair:

$$\text{HL} = 2\text{F38H} \implies H = 2\text{FH}, \quad L = 38\text{H}$$

6. **`PUSH H`**: Push HL pair onto stack:

$$\text{SP} = 0\text{FFFH} \leftarrow H\,(2\text{FH}), \quad \text{SP} = 0\text{FFEH} \leftarrow L\,(38\text{H})$$

New stack pointer:

$$\text{SP} = 0\text{FFEH}$$

7. **`CZ 20FFH`**: Call if Zero. Since $Z = 1$ from the `XRA A` instruction (register movement instructions like `MOV` do not affect flags):

- The next instruction address (`JMP 3000H` at `200E H`) is pushed onto the stack:

$$\text{SP} = 0\text{FFDH} \leftarrow 20\text{H}, \quad \text{SP} = 0\text{FFCH} \leftarrow 0\text{EH}$$

New stack pointer:

$$\text{SP} = 0\text{FFCH}$$

- Program jumps to `20FFH`.

8. **`20FF ADD H`**: Add register H to accumulator A:

$$A = A + H = 2\text{FH} + 2\text{FH} = 5\text{EH}$$

Since the result is non-zero, the Zero Flag resets:

$$Z = 0$$

9. **`RZ`**: Return if Zero. Since $Z = 0$, the return is not executed. Program continues to the next instruction.

10. **`POP B`**: Pop the top of the stack into BC register pair. Top of stack currently has the return address `200EH`:

$$C = 0\text{EH}, \quad B = 20\text{H} \quad (\text{BC} = 200\text{EH})$$

New stack pointer:

$$\text{SP} = 0\text{FFEH}$$

11. **PUSH B**: Push BC pair back onto the stack:

$$\text{SP} = 0\text{FFDH} \leftarrow B\,(20\text{H}), \quad \text{SP} = 0\text{FFCH} \leftarrow C\,(0\text{EH})$$

New stack pointer:

$$\text{SP} = 0\text{FFCH}$$

12. **RMZ**: Return if Minus or Zero. Since $Z = 0$ and MSB of accumulator ($A = 5\text{EH} = 01011110_2$) is 0 (Sign Flag $S = 0$), the return is not executed.

13. **HLT**: The microprocessor execution stops here. The Program Counter points to the address immediately following the HLT instruction:

$$\text{PC} = 2108\text{H}$$

Thus, the final values of the registers are:

$$\text{PC} = 2108\text{H}$$
$$\text{SP} = 0\text{FFCH}$$
$$B = 20\text{H}$$
$$C = 0\text{EH}$$
$$H = 2\text{FH}$$
$$L = 38\text{H}$$

$$\boxed{\text{PC} = 2108\text{H},\ \text{SP} = 0\text{FFCH},\ B = 20\text{H},\ C = 0\text{EH},\ H = 2\text{FH},\ L = 38\text{H}}$$

Key Points

- **Data Transfer Instructions:** Instructions like MOV and LXI do not affect the status flags.
- **Logical Instructions:** Operations like XRA A clear the accumulator and set the Zero flag ($Z = 1$).
- **Stack Operations:** PUSH decrements the stack pointer by 2, whereas POP increments it by 2.

Q8.89.1 MCQ 1989 1M Ans: A ●○○

The stack is a reserved portion of the main memory (RAM) used for temporary storage of binary data during program execution.

- When a subroutine jump instruction (such as CALL) is executed, the microprocessor automatically pushes the contents of the Program Counter (PC)—which represents the program return address—onto the stack.
- Upon executing a return instruction (such as RET), this return address is popped back into the PC to resume normal program flow.

$$\boxed{\text{(A) storing the program return address whenever a subroutine jump instruction is executed.}}$$

Key Points

- **Subroutine Call:** Saves the return address (PC contents) onto the stack automatically.
- **Register Storage:** Although registers can be saved using PUSH instructions, the CPU does not automatically save all important register contents during an interrupt unless explicitly programmed.

Q8.88.1 MSQ 1988 1M Ans: D ●●○

To identify which statement is **NOT** true for an I/O-mapped I/O scheme:

- **Memory Space is Greater (Statement A):** In I/O-mapped I/O, the memory space utilizes 16-bit addresses (2^{16} = 64 KB), whereas the I/O space utilizes 8-bit addresses (2^8 = 256 locations). Thus, the memory space is significantly larger than the I/O space. This statement is **true**.
- **Data Transfer Instructions (Statement B):** Only dedicated instructions like `IN` and `OUT` are available for I/O transfers, meaning standard memory data transfer instructions (such as `MOV`, `LDA`, `STA`) cannot be used for I/O devices. This statement is **true**.
- **Distinct Address Spaces (Statement C):** The memory address space and the I/O address space are completely separate and isolated from each other. This statement is **true**.
- **I/O Address Space is Greater (Statement D):** Since the I/O address space is only 8-bit (256 ports) compared to the 16-bit memory address space (64 KB), the I/O address space is much smaller, not greater. This statement is **false**.

Hence, the statement that is **NOT** true is Option D.

(D) I/O address space is greater.

Key Points

- **I/O-Mapped I/O:** Features independent memory and I/O maps with a maximum of 256 input and 256 output ports (using 8-bit direct addressing).
- **Control Signals:** Uses separate control signals ($\overline{\text{I/O R}}$ and $\overline{\text{I/O W}}$) to differentiate I/O operations from memory operations ($\overline{\text{MEMR}}$ and $\overline{\text{MEMW}}$).

Q8.88.2 MCQ 1988 1M Ans: A ●○○

In computer architecture, the interpretation of the term **register index addressing mode** (often referred to strictly in early textbooks and specific older architectures as distinct from indexed addressing) is defined as follows:

- **Register Index Addressing Mode:** In this mode, the effective address of the operand is held directly inside an index register. Therefore, the effective address (EA) is given simply by the content of the index register itself:

$$\text{EA} = [\text{Index Register}]$$

  Hence, **Option A** is the correct statement.

- **Indexed Addressing Mode:** In contrast, in standard *indexed addressing mode*, the effective address is computed by adding a displacement (operand/offset) to the content of the index register:

$$\text{EA} = [\text{Index Register}] + \text{displacement}$$

  This corresponds to Option B, which refers to the broader "Indexed" scheme rather than pure "Register Index" addressing.

(A) The index register Value.

Key Points

- **Register Index Mode:** The index register acts as a pointer containing the complete address (Effective Address = Index Register Value).

- **Indexed Mode:** Effective Address = Index Register Value + Offset/Operand.

Q8.88.3 MCQ 1988 1M Ans: C ● ● ○

To determine the address range for the 1 KB memory chip, we analyze the interfacing connections:

1. **Chip Address Lines:** The memory size is 1 KB = 2^{10} bytes. Thus, 10 address lines (A_9 to A_0) are connected directly to the memory chip to select internal locations. These lines vary from all 0s to all 1s:

$$A_9\text{—}A_0 = 00\,0000\,0000_2 \quad \text{to} \quad 11\,1111\,1111_2$$

2. **Decoder Select Lines:** The 3-to-8 line decoder has select inputs connected to A_{12}, A_{11}, and A_{10}. The chip select ($\overline{\text{CS}}$) of the RAM is active-low and is connected to output 4 of the decoder. For output 4 to be active, the select input combination must be:

$$A_{12}A_{11}A_{10} = 100_2$$

3. **Decoder Enable:** The active-high chip select input (CS) of the decoder is connected to the output of an AND gate. To enable the decoder, the output of this AND gate must be high (1). Therefore:

$$A_{15}A_{14}A_{13} = 111_2$$

Combining these requirements, we map out the complete 16-bit address space (A_{15} to A_0):

Address	A_{15}	A_{14}	A_{13}	A_{12}	A_{11}	A_{10}	A_9	A_8	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0	Hex Value
Start	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	`F000H`
End	1	1	1	1	0	0	1	1	1	1	1	1	1	1	1	1	`F3FFH`

Therefore, the address range for the memory chip is `F000H`—`F3FFH`.

(C) F000H - F3FFH

Key Points

- **Memory Size Address Lines:** A memory of size 2^N words requires N address lines connected directly to its address pins.
- **Decoder Output Selection:** For an active-low output Y_i of a 3-to-8 decoder to be active, the binary input sequence ($S_2S_1S_0$) must equal the decimal value i.

Q8.88.4 MCQ 1988 1M Ans: B ● ● ○

To determine the maximum addressable memory space using linear selection:

1. **Linear Address Allocation:** In a linear memory selection configuration, individual address lines are connected directly to the chip select (CS) inputs of each memory chip without using a decoder. Since there are 4 memory chips, exactly 4 address lines are reserved exclusively for chip selection (e.g., A_{15}, A_{14}, A_{13}, A_{12}).

2. **Remaining Address Lines:** The total number of available address lines is 16. After dedicating 4 lines for chip selection, the number of remaining address lines available to address the internal locations of a chip is:

$$16 - 4 = 12 \text{ lines} \quad (A_0\text{—}A_{11})$$

3. **Maximum Capacity Calculation:** The maximum addressable size of an individual memory chip using 12 lines is:

$$\text{Size per chip} = 2^{12} = 2^2 \times 2^{10} = 4\,\text{KB}$$

Since there are 4 distinct chips in total, the maximum addressable memory space across the system is:

$$\text{Total Memory Space} = 4 \times 4\,\text{KB} = 16\,\text{KB}$$

(B) 16 K

Key Points

- **Linear Selection:** Avoids hardware decoding latency and cost but reduces the maximum addressable space because address lines are consumed directly as chip enables (1 line per chip).
- **Decoding Efficiency:** For N address lines and M chips, linear addressing gives a total size of $M \times 2^{N-M}$, whereas absolute decoding using a decoder can utilize up to 2^N locations.

PrepFusion

Also in this Series

Every PrepFusion GATE title is built the same way: the real previous-year papers, nothing paraphrased or reconstructed, arranged so the pattern the exam repeats is visible on the page. Each questions volume has a companion solutions volume that reuses the same question identifiers, so you can move between the two without a lookup table.

NexusX	Network Theory, Analog Electronics, Digital Electronics, Control Systems and Signals & Systems — EC, EE, IN
SiliconX	Electronic Devices, Electromagnetic Field Theory and Communication Systems — EC
PowerX	Electrical Machines, Power Systems, Power Electronics, Measurements and Electromagnetic Field Theory — EE, IN
Engineering Mathematics	Linear Algebra, Calculus, Differential Equations, Complex Variables, Probability & Statistics, Numerical Methods, Transform Theory and Vector Calculus — all branches
General Aptitude	Verbal and quantitative aptitude, arranged year-wise and session-wise — all branches

Spotted a wrong answer key, a typo or a question we have mis-tagged? Report it on the PYQ portal at `pyq.prepfusion.in` — corrections go into the next printing, and you can request a video solution for any question from the same place.

PrepFusion